

UNCOVERING THE LIMITS OF MACHINE LEARNING FOR SOFTWARE SECURITY

BY NIKLAS RISSE



MAX PLANCK INSTITUTE FOR SECURITY AND PRIVACY
RUHR UNIVERSITY BOCHUM

A DISSERTATION SUBMITTED FOR THE DEGREE OF
DOCTOR OF NATURAL SCIENCES (DR. RER. NAT.).

Thesis Committee

Chair: Prof. Dr. Kevin Borgolte
Ruhr University Bochum

Thesis Advisor: Dr. Marcel Böhme
Max Planck Institute for Security and Privacy

Thesis Reviewers: Prof. Dr. Andreas Zeller
CISPA Helmholtz Center for Information Security / Saarland University
Prof. Dr. sc. Michael Pradel
CISPA Helmholtz Center for Information Security / University of Stuttgart

Protocol: Dr. Jana Hofmann
Max Planck Institute for Security and Privacy

Date of Defense: August 20, 2026

Copyright © 2026 Niklas Risse

I have no special talents. I am only passionately curious.

— ALBERT EINSTEIN

ABSTRACT

Machine learning (ML) techniques have become extremely popular in software security research and are frequently used to achieve state-of-the-art results on popular benchmarks. However, far less effort has been devoted to exploring the *limits* of these techniques. Do we actually understand what these techniques can and cannot do? Can we trust the high benchmark scores reported in the literature? In this thesis, I aim to *uncover* the limits of machine learning for software security, demonstrate their impact, and open new ways to overcome or circumvent them.

In our first contribution, we study vulnerability detection, where the dominant benchmarking paradigm from 2020 to 2024 was to evaluate techniques using function-level vulnerability datasets: given an isolated function, does it contain a security flaw? We asked whether ML techniques actually understand these isolated functions and introduced a set of modifications to test them adversarially. Surprisingly, we found that semantics-preserving modifications (e.g., renaming identifiers) significantly degraded performance, and that techniques could not distinguish vulnerable functions from their patched versions. These results demonstrate that ML techniques do not truly understand the code they classify.

But can we even determine a function’s security status without its surrounding context? Unsurprisingly, the answer is no. In our second contribution, we examined benchmark datasets containing functions labeled as vulnerable or secure. We manually reviewed 300 cases and found that none could be classified without the surrounding context. Our findings reveal that the dominant evaluation approach for ML-based vulnerability detection is fundamentally flawed. In doing so, we effectively closed down the dominant branch of ML4VD research just as it was gaining momentum.

Are these problems specific to vulnerability detection, or are they symptoms of a broader validity problem in ML for security research? In our third contribution, we reviewed 72 papers from top software engineering and security venues (2023–2024) and identified nine methodological pitfalls that can compromise research validity beyond vulnerability detection.

Finally, in our fourth contribution, we explore how we might overcome the limitations of previous ML-based techniques through the use of agents. On the problem of identifying bug-introducing commits from fix commits, we demonstrate how a relatively simple agent can make more progress on a 20-year-old open problem than the cumulative advances of the previous two decades. With agents, the boundary of what ML-based security techniques can achieve is no longer determined solely by the learned model, but also by the information, tools, time, and computational resources available to the agent. In this sense, the limits of ML techniques become the limits of computation itself.

LIST OF PUBLICATIONS

This thesis is based on the following four publications.

- [RB24] Niklas Risse and Marcel Böhme. “Uncovering the Limits of Machine Learning for Automatic Vulnerability Detection”. In: *Proceedings of the 33rd USENIX Security Symposium (USENIX Security)*. 2024. URL: <https://www.usenix.org/system/files/usenixsecurity24-risse.pdf>.
- [RLB25] Niklas Risse, Jing Liu, and Marcel Böhme. “Top Score on the Wrong Exam: On Benchmarking in Machine Learning for Vulnerability Detection”. In: *Proceedings of the ACM on Software Engineering (ISSTA 2025)*. **ACM SIGSOFT Distinguished Paper Award**. DOI: [10.1145/3728887](https://doi.org/10.1145/3728887).
- [Eve+26] Jonathan Evertz*, Niklas Risse*, Nicolai Neuer, Andreas Müller, Philipp Normann, Gaetano Sapia, Srishti Gupta, David Pape, Soumya Shaw, Devansh Srivastav, Christian Wressnegger, Erwin Quiring, Thorsten Eisenhofer, Daniel Arp, and Lea Schönherr. “Chasing Shadows: Pitfalls in LLM Security Research”. In: *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. * **Equal Contribution**. 2026. DOI: [10.14722/ndss.2026.241749](https://doi.org/10.14722/ndss.2026.241749).
- [RB26] Niklas Risse and Marcel Böhme. “How and Why Agents Can Identify Bug-Introducing Commits”. In: *arXiv preprint* (2026). **Accepted at the 41st IEEE/ACM International Conference on Automated Software Engineering 2026 (ASE’26)**. DOI: [10.48550/arXiv.2603.29378](https://doi.org/10.48550/arXiv.2603.29378).

ACKNOWLEDGEMENTS

Every PhD starts with the choice of an advisor, and I could not have made a better one. Thank you, Marcel, for showing me how to do research, for believing in and supporting me throughout, for the countless discussions about research, life, and the universe, and for being an amazing role model and human.

Next, I would like to thank my wife, Lea. Thank you for encouraging me to follow my dreams, for picking me up in moments of doubt, for always motivating me, and, in this final phase, for taking such good care of our son Henry. And to Henry, now eight months old: thank you for bringing so much joy into our lives and for making me smile during every break from writing this thesis.

I am also grateful to all of my research collaborators. I worked with so many remarkable people and learned a great deal from each of you. Thank you! In alphabetical order: Andreas Müller, Christian Wressnegger, Daniel Arp, David Pape, Devansh Srivastav, Eleonora Losiouk, Erwin Quiring, Gaetano Sapia, Jing Liu, Jonathan Evertz, Lea Schönherr, Marcel Böhme, Nicholas Miazza, Nicolai Neuer, Philipp Normann, Soumya Shaw, Srishti Gupta, and Thorsten Eisenhofer.

Thank you to the past and present members of the Software Security Group. I will miss our lunch breaks, dinners, group meetings, and discussions about research and life. You all made my PhD such a special time, and I am grateful for the time we shared. In alphabetical order: Ardi Madadi, Gaetano Sapia, Hannah Freudenberger, Jing Liu, Junming Liu, Marcel Böhme, Octavio Galland, Prabith Gupta, Rafaila Galanopoulou, Seongmin Lee, and Yasamin Golzar.

I would also like to thank my other colleagues at MPI-SP. I always experienced MPI-SP as a happy place full of wonderful people, and I will miss the MC/2 lunch culture.

Finally, I would like to thank Prof. Dr. Andreas Zeller, Prof. Dr. sc. Michael Pradel, and Marcel Böhme for taking the time to review my thesis, and Prof. Dr. Kevin Borgolte and Dr. Jana Hofmann for serving as chair and protocol of my defense. I am grateful to all of them for being part of my thesis committee.

CONTENTS

Abstract	v
List of Publications	vii
Acknowledgements	ix
Contents	xi

I Preliminaries 1

1 Introduction	3
1.1 Motivation	3
1.2 Our Research Contributions	4
1.2.1 Testing ML for Vulnerability Detection	4
1.2.2 Top Score on the Wrong Exam	4
1.2.3 Pitfalls in ML Security Research	5
1.2.4 Agents for Identifying Bug-Introducing Commits	6
1.3 Structure of this Thesis	6
2 Background	9
2.1 Software	9
2.1.1 Fundamental Limits	10
2.2 Software Security	13
2.2.1 Software Security Analysis	14
2.3 Machine Learning	17
2.3.1 Artificial Intelligence	17
2.3.2 Machine Learning	18
2.3.3 Deep Learning	20
2.3.4 Large Language Models	22
2.3.5 Agents	25
2.3.6 Fundamental Challenges	26
2.4 Application Domains	28
2.4.1 Machine Learning for Vulnerability Detection .	28
2.4.2 Bug-Introducing Commit Identification	37

CONTENTS

II Publications	39
3 Uncovering the Limits of Machine Learning for Automatic Vulnerability Detection	41
4 Top Score on the Wrong Exam: On Benchmarking in Machine Learning for Vulnerability Detection	81
5 Chasing Shadows: Pitfalls in LLM Security Research	115
6 How and Why Agents Can Identify Bug-Introducing Commits	181
III Discussion and Conclusions	207
7 Discussion and Conclusions	209
7.1 Overview	209
7.2 Implications and Impact	210
7.2.1 USENIX'24: Adversarial Testing of ML4VD	210
7.2.2 ISSTA'25: The End of Function-Level VD	211
7.2.3 NDSS'26: A Broader Critique of LLMs for Security	213
7.2.4 SZZ-Agent: A New Horizon for Software Security	214
7.3 Future Work	214
7.3.1 ML4VD Benchmarking	214
7.3.2 Agentic Software Security	216
7.4 Conclusions	218
Appendix	219
Bibliography	221
Statement on the Usage of AI Tools	239

PART I

PRELIMINARIES

INTRODUCTION

If you don't take risks, you can't create a future.

— MONKEY D. LUFFY

In this chapter, I explain my motivation for conducting the research presented in this thesis, provide an overview of our research contributions, and offer guidance on how to read this thesis.

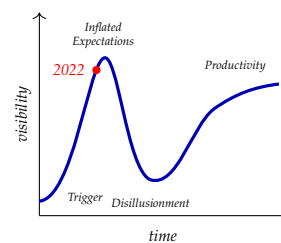
1.1 MOTIVATION

When I started my PhD in November 2022, the research area of machine learning for vulnerability detection (ML4VD) was experiencing dramatic growth in popularity [RLB25]. This growth was driven by the rise of deep learning, which had revolutionized fields such as computer vision in the mid-2010s [KSH12] and began to be applied to vulnerability detection in the late 2010s with high expectations [Li+18; Rus+18; Zho+19]. Early results were highly promising, leading to a flood of papers reporting impressive scores on the latest benchmark datasets [HM22; FT22]. In 2022, however, critical perspectives were difficult to find, amounting to only a handful of papers [Cha+21; Li+21].

I entered the field with a background in adversarial robustness for deep learning models, which I developed as a student research assistant during my Bachelor's and Master's studies at Bielefeld University [RGG21]. Research on adversarial robustness investigates how susceptible deep learning models are to small perturbations in their inputs [GSS14]. Because of this background, I was already aware of several fundamental limitations of deep learning models that the ML4VD community had largely overlooked at the time. Combined with the state of the field and my natural tendency to question the status quo, this made for a highly motivated first-year PhD student eager to challenge established assumptions in ML4VD.

However, uncovering the limits that the field overlooked at the time was not my only motivation. I was equally fascinated by what ML could achieve within those limits. Limits not only show what cannot be done; they also show us how much room there is to make progress. From a certain perspective, every computer science researcher is still just exploring what is possible within the fundamental limits of computation laid out by Turing in 1936 [Tur37]. This perspective motivated me not only to study the limits of machine learning for software security, but also to push the boundaries of what ML systems can contribute to these tasks.

Ultimately, my motivation has always been both: to identify the fundamental limits of machine learning systems, and to use this understanding to realize their full potential in addressing humanity's



The Gartner hype cycle [LF03], a model describing how the public visibility of an emerging technology evolves over time. The red dot marks the start of my PhD in 2022, near the peak of inflated expectations for ML4VD.

INTRODUCTION

greatest challenges. I hope that the research presented in this thesis contributes, even in a very small way, toward that goal.

1.2 OUR RESEARCH CONTRIBUTIONS

This section provides an overview of the research contributions presented in this thesis.

1.2.1 *Testing ML for Vulnerability Detection*

Our first contribution [RB24] investigates the degree to which machine-learning techniques for vulnerability detection actually learn to recognize vulnerabilities, rather than relying on superficial features in existing benchmark datasets. We focus on the standard function-level setting, where a model receives a single function and predicts whether it is vulnerable. To test the robustness of such models, we introduce a benchmarking methodology based on semantics-preserving transformations and vulnerability-patch pairs. We evaluate six state-of-the-art ML4VD techniques, including top-performing models from the CodeXGLUE leaderboard [Lu+21], and apply 11 semantics-preserving transformations such as identifier renaming and insertion of unreachable statements.

Our experiments show that the models are highly sensitive to changes that should not affect the vulnerability label, e.g., changing identifier names. Applying the same changes during training through data augmentation helps only when the identical transformation is also used during testing, partially recovering the lost performance on both evaluated datasets. However, this improvement does not generalize: when training and testing use different transformations, performance drops again substantially. This indicates that training data augmentation does not make the models robust in a general sense, but instead causes them to overfit to the specific transformations seen during training.

We further introduce *VulnPatchPairs*, a dataset of 26.2k C functions derived from CodeXGLUE/Devign [Lu+21; Zho+19], where exactly half of the functions are vulnerable, and the other half are their corresponding patched versions. When trained in the standard vulnerability-detection setting, all six evaluated techniques fail to distinguish vulnerable functions from their patches; on average, their accuracy is worse than random guessing. These results show that high benchmark scores do not imply that models understand the vulnerabilities they classify, and instead suggest that they exploit dataset-specific shortcuts.

1.2.2 *Top Score on the Wrong Exam*

Our second contribution [RLB25] asks whether the dominant function-level problem statement for ML4VD is well-defined in the first place.

We survey 81 ML4VD papers published between January 2020 and December 2024 at top software engineering and security conferences and journals, and find that 88% define vulnerability detection as function-level binary classification: given a function in isolation, decide whether it is vulnerable. This formulation assumes that a function’s security status can usually be determined without its surrounding program context.

We test this assumption empirically on three widely used ML4VD datasets: Devign [Zho+19], BigVul [Fan+20], and DiverseVul [Che+23]. For each dataset, we manually inspect 100 functions labeled as vulnerable and first check whether the function actually contains the labeled vulnerability. Only 39% of the BigVul functions, 47% of the Devign functions, and 65% of the DiverseVul functions are actually vulnerable. For the remaining 151 actually vulnerable functions, we then ask whether the vulnerability can be determined from the function alone. The answer is no: all 151 cases are context-dependent.

This result exposes a fundamental flaw in the dominant evaluation paradigm for ML4VD. If the information required to determine whether a function is vulnerable is absent from the input itself, then high benchmark scores cannot be interpreted as evidence of genuine vulnerability detection. To understand how such performance is nevertheless achievable, we train simple models based solely on word counts and show that they attain results comparable to state-of-the-art ML4VD approaches. This demonstrates that current benchmarks primarily reward the exploitation of spurious dataset correlations rather than any meaningful understanding of software vulnerabilities. Consequently, our findings call into question the validity of the dominant line of ML4VD research built around function-level classification benchmarks.

1.2.3 *Pitfalls in ML Security Research*

Our third contribution [Eve+26] broadens the analysis from vulnerability detection to LLM-based security research more generally. We identify nine recurring methodological pitfalls across the LLM pipeline: data poisoning, LLM-generated label inaccuracy, data leakage, model collapse, spurious correlations, context truncation, prompt sensitivity, surrogate fallacy, and model ambiguity. These pitfalls affect different stages of research, from data collection and pre-training to prompting and evaluation.

We conduct a prevalence study of all 72 peer-reviewed LLM-centric security papers published between January 2023 and December 2024 at leading security and software engineering venues. Surprisingly, every paper in the study contains at least one pitfall, and each of the nine pitfalls appears in multiple papers. Yet only 15.7% of present pitfalls are explicitly discussed, which suggests that many validity threats remain unrecognized or underreported. The most frequent pitfalls include data leakage, context truncation, prompt sensitivity, surrogate fallacy, and model ambiguity, each appearing in more than 20% of the analyzed papers.

INTRODUCTION

To demonstrate their practical impact, we conduct four empirical case studies. We show that model ambiguity can change robustness and precision/recall, that leaking only 20% of the test data into training can increase F1-score by about 0.08–0.11, that 49% of vulnerable functions exceed a 512-token context window and 29% exceed 1024 tokens, and that recursive self-training on generated code can degrade model quality over generations. Based on these findings, we provide concrete recommendations for more rigorous, reproducible, and trustworthy LLM security research.

1.2.4 Agents for Identifying Bug-Introducing Commits

Together, our first three contributions reveal the limitations of current ML-based security research. Our fourth contribution [RB26] explores a potential remedy: can agentic systems overcome some of these limitations by actively gathering and leveraging additional context? We study the long-standing problem of identifying bug-introducing commits from fix commits. Since the original SZZ algorithm in 2005 [SZZ05], progress on this task has been incremental: on the most popular Linux kernel dataset, the original SZZ method achieves an F1-score of 0.54, while the 2025 state of the art reaches only 0.64 [Tan+25].

We introduce *SZZ-Agent*, an agentic workflow that combines traditional SZZ-style candidate generation with LLM-based repository exploration. The agent first evaluates SZZ candidates derived from deleted lines in the fix commit. If this fails, it searches through file histories and uses a binary-search-like procedure over source-code versions to determine when the bug first appeared. In our evaluation, *SZZ-Agent* outperforms all previous approaches on three established C/C++ and Java datasets, reaching F1-scores of 0.77 on DS_LINUX, 0.75 on DS_GITHUB-c, and 0.67 on DS_GITHUB-j.

Through ablation studies, we discover an even simpler and more effective variant, *Simple-SZZ-Agent*. Instead of performing binary search, it directly selects from the candidate set. It reaches 0.81 F1-score on DS_LINUX, 0.75 on DS_GITHUB-c, and 0.75 on DS_GITHUB-j. We find that the agent succeeds by compressing the fix commit message and diff into short, greppable patterns, searching the candidate commits for these patterns, and reading only the most relevant results. This shows how agents can shift the limits of ML-based security techniques: performance is no longer determined only by the learned model, but also by the tools, information, and computation available to the agent.

1.3 STRUCTURE OF THIS THESIS

This thesis is divided into three parts: Part I: Preliminaries, Part II: Publications, and Part III: Discussion and Conclusions.

Part I: Preliminaries. I begin in Chapter 1 by explaining the motivation behind the research presented in this thesis, providing an overview of the research contributions, and outlining the structure of the thesis.

Then, in Chapter 2, I introduce the background knowledge necessary to understand the research contributions presented throughout this work. Starting from fundamental definitions and their historical context, I gradually narrow the discussion to the specific research areas relevant to our contributions.

Part II: Publications. Chapter 3, Chapter 4, Chapter 5, and Chapter 6 present the four main research contributions of this thesis. Each chapter begins with a one-page preamble describing the author contributions and providing supplemental information, followed by the full original text of the corresponding paper, including references and appendices. The included texts are identical to the original publications, with only minor stylistic adjustments to match the formatting of this thesis.

Part III: Discussion and Conclusions. In Chapter 7, I discuss the implications and impact of our research contributions and draw conclusions from them. I also highlight open problems that remain unsolved and outline promising directions for future research from my perspective.

How to Read this Thesis. The background chapter (Chapter 2) begins with fundamental concepts and progressively introduces the more specialized research areas relevant to this thesis. Depending on the reader's expertise and interests, some sections may therefore be skipped. For the publication chapters, each preamble contains a link to the original paper. I recommend reading the original versions where possible, as the papers were specifically formatted and optimized for their respective conference proceedings. Readers with limited time may prefer to focus on the introduction (Chapter 1) and the discussion chapter (Chapter 7). I hope you enjoy reading this thesis!

BACKGROUND

If I have seen further, it is by standing on the shoulders of giants.

— ISAAC NEWTON

In this chapter, I introduce the background knowledge required to understand the research contributions of this thesis, beginning with foundational concepts and historical context before moving toward the specific research areas addressed in this work.

2.1 SOFTWARE

Before we can examine software security and the limits of machine learning in this context, we need a shared understanding of what software is. And before we can define software, we must first look at the very foundations on which it is built: the concept of *computation*.

Computation. The term *computation* was formalized by Alan Turing in 1936 [Tur37]. For the purposes of this thesis, I summarize the underlying idea as follows.

Definition 2.1. *Computation is a stepwise process in which a state is iteratively transformed according to a finite set of mechanically applicable rules.*

Turing also introduced the *Turing machine*, a mathematical model that formalizes this notion of computation [Tur37]. A Turing machine consists of an infinitely long tape divided into cells, where each cell can store a symbol such as 0, 1, or a blank symbol. It also includes a read/write head that can read the symbol currently under it, write a new symbol, and move one cell to the left or right. At any moment, the machine is in a particular internal state, and its behavior is determined by a finite set of rules. Formally, a Turing machine can be defined as a tuple

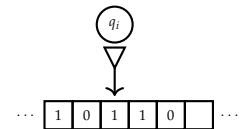
$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}}),$$

where Q is the set of states, Σ is the input alphabet, Γ is the tape alphabet with $\Sigma \subseteq \Gamma$, and

$$\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$$

is the transition function. [Sip13, p. 168] Furthermore, $q_0 \in Q$ is the initial state, while $q_{\text{accept}}, q_{\text{reject}} \in Q$ are the accepting and rejecting halting states, respectively. At each computation step, the transition function determines the symbol to be written, the direction in which the head moves, and the next internal state of the machine.

Universal Turing Machine. Turing also had the insight that there exists a single Turing machine that can simulate any other Turing



Schematic visualization of a Turing machine: a read/write head, currently in state q_i , scans one cell of an infinite tape.

machine, given an *encoding* of that machine along with its input. This machine is called the *universal Turing machine* (UTM). A UTM can therefore compute anything that is Turing-computable. The Church–Turing thesis further states that Turing-computability captures exactly what a human could in principle calculate by following a mechanical procedure [Chu36; Tur37].

Computers. Turing laid the foundation for computing, though the Turing machine was only a mathematical model. The first working programmable computer was Konrad Zuse’s Z3 in 1941, programmed with punch cards and Turing-complete in principle [Roj98]. Modern computers are still largely based on the *von Neumann architecture* proposed in 1945 [Neu45], which consists of a CPU, memory storing both data and instructions, input/output, and a shared bus connecting CPU and memory.

Program. In order to get a computer to compute something we desire, we need to specify what to compute. This is what *programs* are for. In terms of my previous definitions, a program is a finite specification of mechanically applicable rules that, when executed by a computer, transform inputs and machine state through computation to potentially produce outputs. Within the Turing machine model, these rules correspond to the transition function δ , which determines at each step the symbol written, the head movement, and the next state. A program can thus be understood as a specification of such a transition function. I can also express this in simple terms.

Definition 2.2. *A program is a finite set of instructions that tells a computer what to compute.*

Software. To fulfill the requirements of stakeholders, often more is needed than just a single program. *Software* is a broader term we use to describe the non-physical components that direct the behavior of a computer system. Beyond programs, this includes documentation, data, configuration files, and databases, for example.

Definition 2.3. *Software is the collection of non-physical components that direct the behavior of a computer system.*

2.1.1 Fundamental Limits

Since the goal of this thesis is to uncover the limits of machine learning for software security, I begin by narrowing the scope and identifying the fundamental limits of software itself.

Halting Problem. Turing himself identified in his 1936 paper [Tur37] that there are fundamental limits to computation. He identified the *halting problem*, which states that there is no general algorithm that can always determine whether another program will eventually stop running or continue forever. Problems for which such an algorithm exists are called *decidable*; problems for which no such algorithm can

```

1 def halts(program, input):
2     # Assume this oracle exists: it returns True
3     # if program(input) halts, and False otherwise.
4     ...
5
6 def contradiction(program):
7     if halts(program, program):
8         while True: # loop forever
9             pass
10    else:
11        return # halt immediately
12
13 contradiction(contradiction)

```

Figure 2.1: Turing’s contradiction proof of the halting problem in Python. Assuming a halting oracle `halts` exists, the call `contradiction(contradiction)` can neither halt nor run forever, so `halts` cannot exist.

exist are called *undecidable*. The halting problem is therefore undecidable. Turing proved the undecidability of the halting problem by contradiction. He assumed that such a general algorithm exists and then constructed a program whose behavior is logically impossible. Figure 2.1 shows the idea in Python. By identifying the halting problem, Turing proved that there are fundamental limits to computation and that no algorithm can perfectly predict the behavior of all other algorithms.

Gödel’s Incompleteness Theorems. Turing’s halting problem is strongly related to the incompleteness theorems of Kurt Gödel, published in 1931 [Göd31]. Gödel showed that if a formal system is consistent (it never proves contradictions) and powerful enough to describe ordinary arithmetic, then there will always be true statements that the system cannot prove, and the system also cannot prove its own consistency. Essentially, Gödel’s theorems already allow us to draw two conclusions that are similar in structure to what was later proved by Turing for computation: Formal systems have fundamental limits, and no powerful system can perfectly reason about itself.

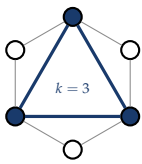
Rice’s Theorem. Rice’s theorem generalizes the halting problem to any interesting question about what a program computes. It states that any nontrivial semantic property of computer programs is undecidable. A semantic property is a property about the behavior, output, or function of a program; nontrivial means that the property is true for some programs and false for others [Ric53]. Suppose we ask: “Does this program ever crash?” Rice’s theorem says that there can be no algorithm that can answer this question correctly for all programs. This has fundamental consequences for automatically *analyzing* software: A perfect algorithm that answers any interesting question about all programs is mathematically impossible.

Undecidability of Execution-State Properties. Rice’s theorem tells us that interesting questions about *what* a program computes are undecid-

able. Yet undecidability is not confined to a program's results: *how* a program computes them, its execution state at a syntactic location, can be equally beyond reach. An example of this is *aliasing*. Landi showed that determining whether two pointers *may* refer to the same storage location, and are thus *aliases*, is undecidable [Lan92; Lan95].

Time Complexity. Now that we have established fundamental limits to what can be decided, we can begin to identify limits to what can *efficiently* be computed. Complexity theory asks: Assuming that a problem is decidable, how much time and space does solving it require? For time complexity, there are two important classes of decidable problems: P and NP .

Class P problems are problems that are always solvable in polynomial time. An example of a problem solvable in polynomial time is the path problem: Given a directed graph, is there a path between two nodes? [Sip13, pp. 284–290] Class NP problems are problems where a proposed solution can always be verified in polynomial time. An example of a class NP problem is the k -clique problem: Does a graph contain a clique (subgraph where every two nodes are connected) of size k ? [Sip13, pp. 295–296] We currently do not know whether all problems that are verifiable in polynomial time are also solvable in polynomial time ($P \stackrel{?}{=} NP$). There are many problems, such as the k -clique problem, for which we *currently* do not know a polynomial-time algorithm. Many scientists believe that no such algorithm will ever be found. However, we also do not have proof that such an algorithm cannot exist. Finally, the class $EXPTIME$ contains decidable problems that can be solved by a deterministic algorithm in exponential time.



An instance of the k -clique problem. The graph contains a clique (highlighted) of size $k = 3$: a subgraph in which every two nodes are connected.

Space Complexity. The second important computational resource next to time is space, e. g., memory. $PSPACE$ is the class of decidable problems solvable using a polynomial amount of memory (space), regardless of how long the computation takes. The canonical example of this problem class is the True Quantified Boolean Formula Problem (TQBF): The problem of deciding whether a fully quantified Boolean formula, such as $\forall x, \exists y, [(x \vee y) \wedge \neg x]$, is true. [Sip13, pp. 338–341]

Based on our understanding of these complexity classes, we can note the following containment relationship [Sip13, p. 336]:

$$P \subseteq NP \subseteq PSPACE \subseteq EXPTIME$$

Practical Perspective. Decidability tells us which problems can be computed at all, and complexity theory which ones can be computed efficiently. In practice, many decidable problems can still become computationally infeasible, depending on the specific problem instance and the available computational resources. However, even for problems in NP , $PSPACE$, or $EXPTIME$, efficient approximation algorithms or heuristics may exist that provide practically useful solutions in polynomial time, although these solutions may not always satisfy the accuracy or correctness requirements of a given application. Nevertheless, we must remain aware of these fundamental limits when exploring what a given technology can and cannot do.

2.2 SOFTWARE SECURITY

Security. Most definitions of software security hinge on the concept of *security*. Therefore, we need to define the concept of security first. One of the most widely cited definitions of security comes from U.S. law and was popularized by the National Institute of Standards and Technology of the U.S. (NIST). In *An Introduction to Information Security*, NIST defines security as “The protection of information and information systems from unauthorized access, use, disclosure, disruption, modification, or destruction in order to provide confidentiality, integrity, and availability.” [NDP17] Another important perspective comes from Bishop’s textbook *Computer Security: Art and Science*, where he makes the point that *secure* is always relative to a *policy*, which he defines as “a statement of what is, and what is not, allowed.” [Bis18] Further, he defines a *security mechanism* as “a method, tool, or procedure for enforcing a security policy.” [Bis18] Because these two definitions complement each other, this thesis adopts a combined version of them.

Definition 2.4. *Security is the protection of information and information systems with respect to security policies in order to provide confidentiality, integrity, and availability.*

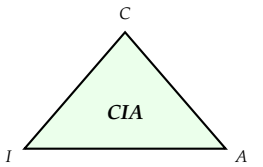
Computer Security. *Computer security* as a discipline was first studied in the early 1970s, although the earliest papers were classified by the government and were only released to the public later, e. g., *the Ware report* [War70], which was created in 1970 and declassified in 1979. Two of the earliest publicly available works were Lampson’s *Protection* [Lam74], which abstractly formulated the problem of access control for the first time, and Anderson’s *Computer Security Technology Planning Study* [And72], which laid the earliest foundation for kernel security research with the *reference monitor* concept, an abstract mechanism that mediates every access of a subject (e. g., user, process, or program) to an object (e. g., file, memory, or device).

Software Security. The first work that would today be considered *software security* is arguably Karger and Schell’s *Multics Security Evaluation: Vulnerability Analysis*, which evaluated the security of the Multics operating system [KS74]. The first taxonomy of software vulnerabilities was later published by Abbott et al. [Abb+76]. However, the term *software security* itself emerged much later. In 2000, Gary McGraw and John Viega published *Building Secure Software* [VM01], arguably the first book dedicated to the topic. McGraw defined software security as “engineering software so that it continues to function correctly under malicious attack” [McG04], and later as “the process of designing, building, and testing software for security” [McG04]. However, even the sharpened version is missing the concept of software *analysis*, i. e., inspecting software to decide whether it should be considered secure. Therefore, for this thesis, I define software security as:

Definition 2.5. *Software security is the process of designing, building, analyzing, and testing software for security.*

2.2.1 Software Security Analysis

Software security can be approached from different perspectives; we can design, build, analyze, and test software for security (see Definition 2.5). For the research contributions presented in this thesis, the main area of interest is *software security analysis*.



The CIA triad. Confidentiality (C), Integrity (I), and Availability (A) are examples of well-known security properties that many systems are required to have.

Security Properties. In software security analysis, the goal is to distinguish secure from insecure software by analyzing the software for *security properties*, which are specific abstract objectives that a system must maintain to be considered secure. While some security properties can almost universally be applied to any software system (e.g., confidentiality, integrity, and availability), others only apply to specific classes of systems. For example, voting systems must be *receipt-free*, which means a voter cannot produce a receipt proving how they voted, in order to prevent vote-buying.

However, determining which security properties are relevant for a system is not always straightforward. Beyond known and explicitly specified properties, there is also the problem of *unknown unknowns* [Böh24]: systems may fail in ways that expose previously unrecognized security requirements. A famous example, also used by Böhme [Böh24] to illustrate the problem of *unknown unknowns*, is the Spectre vulnerability (CVE-2017-5753). Spectre revealed that speculative execution in CPUs could leak sensitive data through microarchitectural side channels, breaking memory isolation between processes. The abstract property—memory isolation—was well known, but the requirement that it must hold even for the side effects of speculatively executed instructions was previously unrecognized.

Definition 2.6. A security property is an abstract objective that a system must maintain to be considered secure, specified independently of any particular mechanism or implementation.

Security Policies. A security property states *what* security means, but not *how* a system must behave to achieve it. To check whether a property holds, we therefore have to operationalize it as one or more security policies that concretely specify what a system can and cannot do. For example, if we want user data to be confidential, we have to define *who* can and cannot do *what* with the user data in the system. Again, we have the problem of unknown unknowns: How can we be sure that we have defined all security policies for a given security property? For example, the Log4Shell vulnerability (CVE-2021-44228) revealed that the widely used Log4j logging library for Java would interpret logged strings as instructions, enabling remote code execution to violate availability, confidentiality, and integrity. Before its discovery, the security policy that a logging framework must not interpret logged data as executable instructions was unknown, as logging was regarded as a benign activity.

Furthermore, there is also the problem of how to find the right level of concreteness so that a security policy can be effectively operationalized. In the Log4Shell example, we may have previously defined

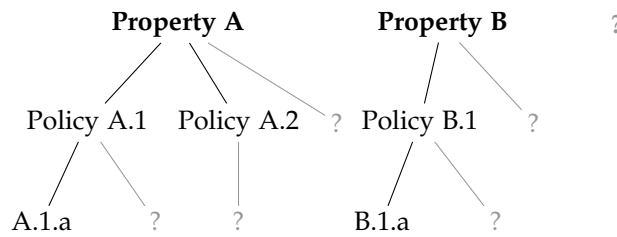


Figure 2.2: Hierarchical relationship between security properties and security policies. Question marks represent unknown unknowns at each level of the hierarchy.

a security policy that disallows remote code execution, but Log4Shell might have still avoided our detection mechanisms because the policy was specified too abstractly to be enforced against this concrete behavior. Figure 2.2 visualizes the relationship between security properties, security policies, and their unknown unknowns.

Definition 2.7. *A security policy is a concrete, enforceable specification of what a system must and must not do in order to satisfy a security property.*

Security Mechanisms. A security policy specifies how a system must behave, but it does not by itself make the system behave that way. The policy needs to be implemented. A security mechanism either aims to *prevent* undesired behavior by constraining the system’s behavior to what the policy permits, or *detect* cases where the system’s behavior deviates from the policy. For example, if we define a security policy that user-supplied input must not be rendered as executable code in a web application (to maintain the integrity and confidentiality of the system), a corresponding preventative *security mechanism* is an input sanitizer that strips or escapes potentially dangerous characters, such as `<script>` tags, from user input before it is included in a web page. This mechanism enforces the policy by preventing cross-site scripting (XSS) attacks. However, the effectiveness of a security mechanism depends on how well it covers the security policy it is meant to enforce. A sanitizer that only escapes `<script>` tags but ignores other injection vectors, such as event handlers (`onerror`, `onload`), fails to fully enforce the policy despite appearing to address it.

Definition 2.8. *A security mechanism is an implementation that enforces a security policy.*

Oracles. Security properties, policies, and mechanisms also relate to the concept of *oracles*, which is frequently used in software testing. In software testing, an oracle is a procedure that determines whether a program’s behavior on a given input is correct or not [Wey82]. Security policies serve as specification oracles by defining what the system may and may not do, providing the specification against which behavior is judged. Detective security mechanisms act as implemented oracles that observe the system and flag deviations from the policy. The analogy to software testing also reveals its most important limitation: The *oracle problem* [Wey82], where no reliable oracle exists to judge correctness.

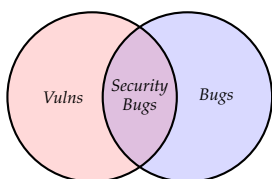
This problem manifests in two ways in the security context. First, an oracle may be entirely absent: For some security properties, no adequate policy may exist that fully captures what it means for the system to be secure, and for some policies, no mechanism may exist that can reliably enforce or detect all violations. Second, even when oracles do exist, they may be incomplete: A security policy may fail to specify all behaviors that violate a security property, and a security mechanism may fail to prevent or detect all violations of a security policy. In both cases, without a perfect oracle at every level, we cannot guarantee security.

Security Vulnerabilities and Bugs. For the research contributions in this thesis, it is important to distinguish *security vulnerabilities* from *bugs*. The *Internet Security Glossary* defines a security vulnerability as a “weakness in a system’s design, implementation, or operation and management that could be exploited to violate the system’s security policy.” [Shi07] I adopt this definition but state it with respect to security *properties* rather than policies: because a policy may be absent or incomplete, a weakness can threaten a security property even when no policy yet forbids the behavior that exploits it. The pre-discovery cases of Spectre and Log4Shell are exactly such weaknesses—vulnerabilities with respect to a property, before any policy named them.

Definition 2.9. *A security vulnerability is a weakness of a system that could be exploited to violate one of the system’s security properties.*

The IEEE Standard 610.12 defines a bug as an “incorrect step, process, or data definition in a computer program” [IEE90]. Zeller provides a more nuanced perspective in *The Debugging Book* [Zel24], distinguishing between (1) *mistakes*, “a human act or decision resulting in an error”, (2) *defects/bugs*, “an error in the program code”, (3) *faults/infections*, “an error in the program state”, and (4) *failures*, “an externally visible error in the program behavior”, leaving us with a cause-effect chain of “Mistake → Defect → Fault → ... → Fault → Failure” [Zel24]. In this thesis, we follow Zeller’s terminology and use the term “bug” to refer specifically to errors in the program code, as this definition clearly separates code defects from their causes and runtime manifestations.

Definition 2.10. *A bug is an error in the program code.*



Security vulnerabilities and bugs overlap in security bugs.

Based on these definitions, we can now distinguish between security vulnerabilities and bugs: Security vulnerabilities are any weaknesses in software, hardware, configuration, protocols, processes, or even human behavior that can be exploited to violate a security property, whether or not a security policy has been defined that the exploit would also violate. Bugs are just errors in the program code, which may or may not lead to security property-violating failures. Not all security vulnerabilities are bugs, and not all bugs are security vulnerabilities, but there is an intersection, which I call *security bugs*: defects in the code that can be exploited to violate a security property.

2.3 MACHINE LEARNING

Machine learning is the technology at the center of this thesis. However, it is not a self-contained idea: it is a subfield of the broader endeavor of artificial intelligence, which in turn rests on the concept of intelligence itself. To understand what machine learning is, what it aims to achieve, and where its boundaries lie, I therefore first clarify these surrounding concepts before narrowing my focus to machine learning.

2.3.1 Artificial Intelligence

Intelligence. There is no universally agreed-upon definition for the term *intelligence*. “Looked at in one way, everyone knows what intelligence is; looked at in another way, no one does.” [Ste00] Legg and Hutter collected 70 definitions of intelligence from the scientific literature and summarized them into a single sentence: “Intelligence measures an agent’s ability to achieve goals in a wide range of environments.” [LH07] However, this definition later received criticism [Cho19], because it does not take into account the efficiency with which the goal-achieving ability is learned. Is a system really intelligent if it can fulfill a task after having seen billions of examples of how to fulfill this task? If an agent can fulfill a task after having seen orders of magnitude fewer examples of it than another agent, do they possess the same level of intelligence? This discrepancy was noticed by François Chollet in his seminal paper *On the measure of intelligence*. He proposed an alternative definition that focuses on skill-acquisition efficiency: “The intelligence of a system is a measure of its skill-acquisition efficiency over a scope of tasks with respect to priors, experience, and generalization difficulty.” [Cho19] This thesis uses Chollet’s definition.

Definition 2.11. *The intelligence of a system is a measure of its skill-acquisition efficiency over a scope of tasks with respect to priors, experience, and generalization difficulty.*

Artificial Intelligence. In *A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence*, which arguably initiated the artificial intelligence field, McCarthy et al. define artificial intelligence in the following way: “The study is to proceed on the basis of the conjecture that every aspect of learning or any other feature of intelligence can in principle be so precisely described that a machine can be made to simulate it.” [McC+06] McCarthy later provided a more accessible version of this definition, defining AI as the “science and engineering of making intelligent machines” [McC07].

Definition 2.12. *Artificial intelligence is the science and engineering of making intelligent machines.*

The Dartmouth paper also became iconic for its optimism regarding AI research speed. In the first paragraph of the paper, the researchers state that a “2 month, 10 man study” should be sufficient to find out “how to make machines use language, form abstractions and concepts, solve kinds of problems now reserved for humans, and improve themselves.” [McC+06]

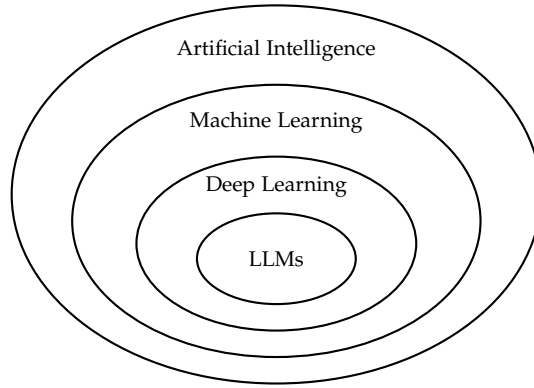


Figure 2.3: Nested subfields of artificial intelligence.

2.3.2 Machine Learning

Machine learning is a subfield of artificial intelligence (see Figure 2.3) that aims to give computers the ability to learn without being explicitly programmed [Sam59]. Mitchell defines learning as follows: “A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .” [Mit97] This definition is closely related to Chollet’s definition of intelligence (see Definition 2.11), which can essentially be interpreted as skill-learning efficiency based on Mitchell’s definition of learning. Nevertheless, artificial intelligence and machine learning are not equivalent fields of study. Some research directions within artificial intelligence do not align with Chollet’s definition of intelligence and therefore do not rely on learning, such as knowledge-based systems [RN21].

Definition 2.13. *A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .*

The following explanations of supervised learning, classification, metrics, and models are summarized from Risse et al. [Ris21, pp. 3–6].

Supervised Learning. Supervised learning is a branch of machine learning where the goal is to learn a function that maps inputs to outputs using labeled examples. Given a training dataset

$$D_{\text{train}} := \{(x_1, y_1), (x_2, y_2) \dots, (x_n, y_n)\}$$

where $x_i \in X_{\text{train}} \subset X$ and $y_i \in Y_{\text{train}} \subset Y$. Typically, X is a space of real-valued vectors, while Y contains either continuous values or discrete categories. We assume the data follows an underlying relationship linking inputs to outputs. The goal is to learn a function

$$f : X \rightarrow Y$$

that approximates this relationship so it can accurately predict outputs for new, unseen inputs. For a concrete dataset, the learned function f is called a *model*.

Classification. Classification is a specific type of supervised learning where the output space Y consists of a finite set of discrete categories or classes. These classes are typically represented by natural numbers, such as $Y := \{0, 1\}$. For example, if $X := \mathbb{R}^3$, a function $h : X \rightarrow Y$ could represent a learned model for a two-class problem. Models built for classification tasks are called *classifiers*. When the task involves exactly two classes, it is known as *binary classification*.

Evaluation and Metrics. Models are typically evaluated on a separate test dataset $D_{\text{test}} := \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$. This dataset contains the same type of input-output pairs as the training data and is assumed to come from the same underlying distribution. If a dedicated test set is not provided, we usually split our original data into training and testing subsets before model selection. How well a model performs on the unseen data is called its *generalization* ability.

To quantify performance, we use metrics defined as functions $M : \mathcal{F} \times \mathcal{P}(X, Y) \rightarrow \mathbb{R}$, where $\mathcal{F} := \{f : X \rightarrow \{0, 1\}\}$ is the set of all possible binary classifiers and $\mathcal{P}(X, Y)$ represents a dataset of input-output pairs. The metric maps a classifier and a dataset to a single real-valued score, where either higher or lower values indicate better performance. Since binary classification is the most relevant setting for this thesis, I introduce the metrics most commonly used in that setting.

Accuracy is one of the most standard evaluation metrics for binary classification. It measures the proportion of correct predictions out of all predictions:

$$\text{Accuracy} := \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

Here, TP (True Positives) and TN (True Negatives) count the correct predictions for true classes 1 and 0, respectively. Conversely, FP (False Positives) counts incorrect predictions where the true class is 0, and FN (False Negatives) counts incorrect predictions where the true class is 1. While accuracy is intuitive and easy to compute, it can be misleading when the dataset has significant class imbalance (i. e., one class outnumbered the other).

The *F1-score* is an alternative metric that is more suitable for imbalanced classification problems. It is defined as the harmonic mean of precision and recall.

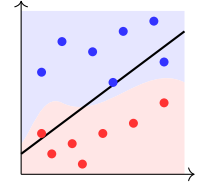
$$\text{F1-score} := 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Here, precision measures the proportion of positive predictions that are actually correct:

$$\text{Precision} := \frac{\text{TP}}{\text{TP} + \text{FP}}$$

while recall measures the proportion of actual positive instances that are correctly identified:

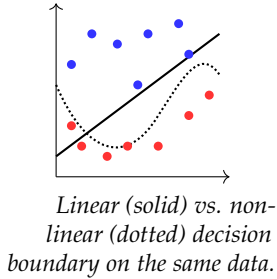
$$\text{Recall} := \frac{\text{TP}}{\text{TP} + \text{FN}}$$



Schematic visualization of a 2-D classification problem. A linear classifier (black) approximates the underlying relationship of red vs. blue classes.

On a dataset with 99% class red and 1% class blue, a trivial classifier that always predicts class red would get 99% accuracy.

Unlike accuracy, the F1-score remains sensitive to both false positives and false negatives even when one class heavily outnumbers the other. Consequently, it provides a more informative evaluation for imbalanced classification tasks.



Model Classes. In binary classification, the goal is to correctly map inputs to one of two classes. To achieve this, deterministic binary classifiers divide the input space into two non-overlapping decision regions [Bis06, pp. 179–180], one for each class. The line separating these regions is called the *decision boundary*.

For *linear models*, the decision boundary is a straight line (or hyper-plane) defined by a linear function of the inputs. A dataset is called linearly separable if its two classes can be perfectly divided by such a linear boundary. If a linear classifier is evaluated on data that is not linearly separable, it will inevitably make errors. When a model relies on overly simplistic assumptions and fails to capture the true complexity of the underlying relationship, this is known as *underfitting*.

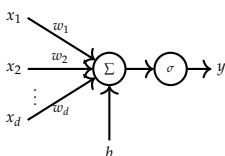
For *non-linear models*, the decision boundary is a curve defined by a non-linear function of the inputs. While they can underfit, they are more prone to *overfitting*: they may achieve near-perfect accuracy on training data by memorizing noise or specific patterns, but fail to generalize to unseen test data. Widely used non-linear models include artificial neural networks [Bis06, pp. 225–290] and maximum margin classifiers [Bis06, pp. 326–344].

Training and Loss Functions. To train a parametrized model in a supervised way, its internal parameters need to be adjusted so that the learned function f minimizes prediction errors on the training data. These prediction errors on the training data can be formalized through a *loss function* $\mathcal{L} : Y_{\text{pred}} \times Y_{\text{true}} \rightarrow \mathbb{R}_{\geq 0}$, which assigns a non-negative penalty to each individual prediction [Bis06, p. 41]. Training can then be seen as an optimization problem over the parameter space Θ :

$$\hat{\theta} = \arg \min_{\theta \in \Theta} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(f(x_i; \theta), y_i)$$

where $f(\cdot; \theta)$ denotes the parametric model and $\hat{\theta}$ represents the optimal parameters. Iterative optimization algorithms, most commonly variants of gradient descent [GBC16, pp. 271–325], update θ step-by-step to reduce the empirical loss on D_{train} . Since minimizing training loss alone often leads to overfitting, regularization terms are typically added to the loss function to constrain model complexity and improve generalization.

2.3.3 Deep Learning



Schematic visualization of an artificial neuron with inputs x_i , weights w_i , bias b , activation σ , and output y .

In 1943, McCulloch and Pitts introduced the idea of a mathematical model of a neuron [MP43]. However, their model was binary and had no learning; the weights were fixed. For the learnable artificial neuron, Rosenblatt introduced the *Perceptron* in 1956:

$$y = \sigma\left(\sum_{i=1}^d w_i x_i + b\right) = \sigma(w^\top x + b)$$

where $x \in \mathbb{R}^d$ is the input, $w \in \mathbb{R}^d$ are learnable weights, $b \in \mathbb{R}$ is a bias, and σ is an activation function. Rosenblatt himself already conceived combining multiple perceptrons into an artificial neural network (multi-layer perceptrons) in the late 1950s. The extension from a single artificial neuron to a network of artificial neurons with multiple layers can be formalized as:

$$f(x) = f_L \circ f_{L-1} \circ \cdots \circ f_1(x), \quad f_l(z) = \sigma(W_l z + b_l)$$

where f_l denotes the transformation at layer l , $W_l \in \mathbb{R}^{d_l \times d_{l-1}}$ is the weight matrix, $b_l \in \mathbb{R}^{d_l}$ is the bias vector, z is the input to that layer (i. e., the output of the previous layer), σ is a non-linear activation function, and L is the total number of layers. The problem that hindered further progress back in the 1960s was that only the output layer could be trained; there was no method to update weights in hidden layers.

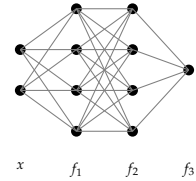
This changed in 1986, when Rumelhart, Hinton, and Williams demonstrated that *backpropagation* can be used to train multi-layer neural networks effectively [RHW86], which made learning neural networks with many layers (i. e., *deep learning*) feasible. It is just the chain rule applied through the composition:

$$\frac{\partial \mathcal{L}}{\partial W_l} = \frac{\partial \mathcal{L}}{\partial f_L} \cdot \frac{\partial f_L}{\partial f_{L-1}} \cdots \frac{\partial f_{l+1}}{\partial f_l} \cdot \frac{\partial f_l}{\partial W_l}$$

Backpropagation allowed us to compute how much each weight in the network contributed to the final error by propagating gradients backwards through the layers. This unlocked research into many deep neural network architectures and learning algorithms that are still used today, e. g., convolutional neural networks (CNN) [LeC+98] or long short-term memory (LSTM) [HS97], and turned it into the most popular research area within machine learning (see Figure 2.3).

However, it was not until the 2010s that several research groups [Cir+10; RMN09; KSH12] demonstrated that deep neural networks can be trained on Graphics Processing Units (GPUs) to achieve breakthrough results in different tasks, most notably with AlexNet [KSH12], which won the ImageNet image classification competition in 2012. These results led to an explosion in deep learning research and laid the foundations for *large language models*.

Universal Approximation Theorem. Interestingly, even a neural network with a single hidden layer can approximate any continuous function arbitrarily well on a compact domain [Cyb89], provided it has sufficiently many hidden units and a suitable activation function. However, this does not imply that such networks can compute anything that is decidable, nor does it establish Turing completeness. Turing completeness for neural networks typically requires additional assumptions, such as recurrence, unbounded computation time, and,



A neural network with input x and three layers f_1, f_2, f_3 . Each connection represents a learnable weight.

BACKGROUND

in some theoretical settings, infinite or arbitrary precision. For example, certain recurrent neural network architectures can simulate Turing machines under idealized assumptions. This does not make them automatically practical: A small RNN trained with gradient descent will not necessarily learn arbitrary algorithms.

2.3.4 Large Language Models

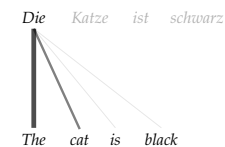
Attention. In 2014, Bahdanau et al. introduced the *attention* mechanism to deep learning [BCB15], which became the basis for today's *large language models* (LLMs). Attention can be defined as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

where, given an input $Z \in \mathbb{R}^{n \times d}$ (e. g., a sequence of n vectors of dimension d), the queries, keys, and values are linear projections:

$$Q = ZW_Q, \quad K = ZW_K, \quad V = ZW_V$$

with learnable weight matrices $W_Q \in \mathbb{R}^{d \times d_k}$, $W_K \in \mathbb{R}^{d \times d_k}$, $W_V \in \mathbb{R}^{d \times d_v}$. For language tasks, the input Z could be a sequence of vectors, each representing one *token*, which is a basic unit of text, typically a character, subword, or word.



Schematic visualization of attention when translating a sentence from English to German.

Advantages. Before the attention mechanism was introduced, sequence-based machine learning methods (e. g., LSTMs [HS97]) compressed the entire sequence prior to the current token into a single vectorial representation. For long sequences, this caused the models to overlook long-range dependencies. The attention mechanism overcame this bottleneck by dynamically attending to all elements of the input sequence at every step of the sequential task. Other advantages were better parallelization, which allowed for efficient GPU processing, and no inductive biases (unlike CNNs [LeC+98], for example), which allowed the models to focus on learning the relationships between inputs and outputs from the data.

Transformers. After realizing that attention mechanisms can overcome many limitations of earlier neural architectures, Vaswani et al. [Vas+17] introduced the *transformer* architecture, which remains the foundation of modern LLMs. Figure 2.4 illustrates the architecture, consisting of an encoder (left) and a decoder (right). The encoder embeds the input sequence into vectors and processes them through stacked layers of multi-head attention and feed-forward networks. The decoder processes the previously generated output tokens using masked self-attention and combines them with the encoder representation through cross-attention. Finally, a linear layer and softmax produce output token probabilities.

GPT. The success of the transformer architecture led to many follow-up studies that proposed variations of the original architecture. Two of the

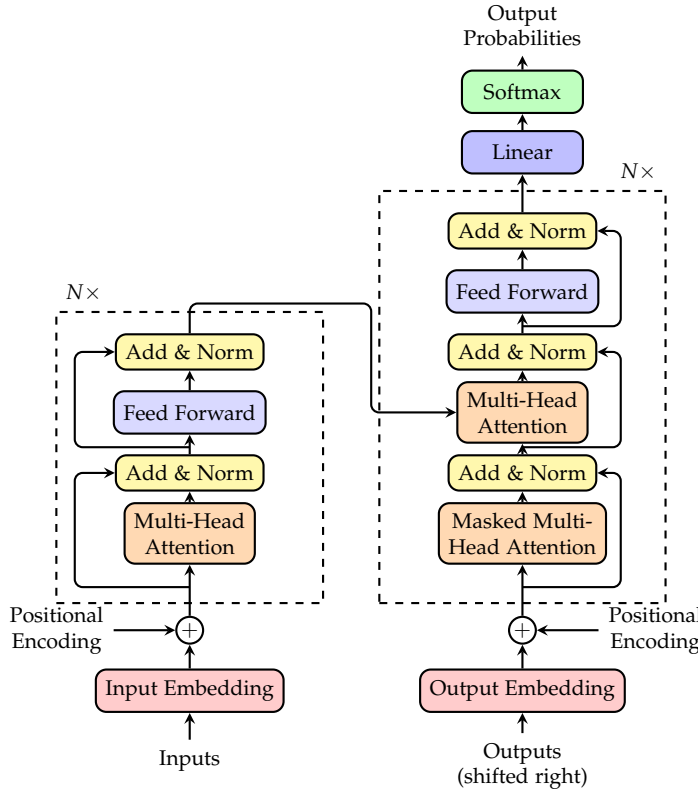


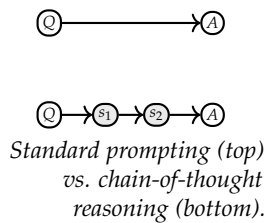
Figure 2.4: The transformer architecture [Vas+17].

most important variations for this thesis are the *Generative Pretrained Transformer* (GPT) [Rad+18] and *Bidirectional Encoder Representations from Transformers* (BERT) [Dev+19]. BERT is encoder-only, meaning it only uses the encoder part (left) of the transformer architecture, and is trained by hiding tokens from the input sequence and trying to guess them based on the surrounding context. In contrast, GPT is decoder-only: it operates on a single sequence, which contains the input sequence and the output sequence up until the current token, and then always tries to predict the next token (autoregressive).

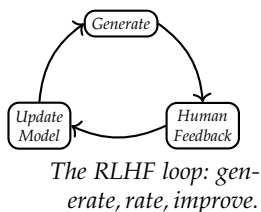
Pre-Training. The technical report for the GPT architecture [Rad+18] demonstrated that large performance gains in natural language tasks can be realized by *unsupervised pre-training* on large corpora of unlabeled text. The concept of unsupervised pre-training was already invented in the early 1990s by Schmidhuber [Sch91], but the GPT technical report demonstrated that this technique could still yield performance gains at a relatively large scale. Subsequent iterations of the GPT model (GPT-2 [Rad+19] and GPT-3 [Bro+20]) further demonstrated that these performance gains follow a power-law relationship with three variables: the number of model parameters (N), the size of the dataset (D), and the amount of compute (C) used for training [Kap+20]. These power-law relationships are also called the *scaling laws* of LLMs: Performance improves predictably as you increase scale.

Supervised Fine-Tuning. To realize the performance gains achieved by unsupervised pre-training, LLMs are usually *post-trained* on labeled examples for specific tasks after the unsupervised pre-training phase. An example of a supervised fine-tuning dataset for training is *LIMA* [Zho+23], which contains carefully curated instructions and responses for chat assistants. Using supervised fine-tuning, significantly better performance on well-specified tasks can be achieved.

In-Context Learning. Learning can also be facilitated by providing examples of instruction-response pairs to the LLM within the input text passed to the model, commonly called the *prompt*. While in-context learning can be very effective for certain tasks [Bro+20], it is limited by the maximum number of tokens the LLM can process in a single pass, the context size (or *window*). Since the attention mechanism connects every token to every other token in the prompt, its computational requirements scale quadratically with the context size. This is why even today’s (April 29, 2026) largest LLMs have no more than one million tokens of context size. However, the context size is not the only factor limiting the effectiveness of in-context learning, as researchers have shown that the effectiveness of in-context learning decreases with the length of the context [Li+24; Liu+24].



Reasoning. In 2022, Wei et al. showed that prompting LLMs with *step-by-step reasoning* examples “dramatically improves performance on arithmetic, commonsense, and symbolic reasoning tasks” [Wei+22]. Instead of just showing the answer, they included the intermediate reasoning steps in the examples. For example, instead of writing “Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now? A: The answer is 11.”, they wrote “A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.” [Wei+22]. This insight enabled the discovery of another scaling law: Allocating more compute at inference time (when a user is prompting the model to solve a task) leads to better performance [Sne+24]. These inference-time reasoning capabilities proved especially impactful for software-related tasks, where step-by-step problem decomposition significantly increased LLM effectiveness [Ope24].



RLHF. With the release of OpenAI’s *ChatGPT*, it became increasingly important to train LLMs so that humans are satisfied with their responses. The most important technique to achieve this is *Reinforcement Learning from Human Feedback* (RLHF) [Chr+17]. In reinforcement learning (RL), an agent learns which actions to take by performing them in an environment to maximize a numerical reward signal [SB18]. In RLHF, the numerical reward signal is derived from human feedback. In chat applications, this can be derived by asking users to rate the responses of the LLM, for example. Beyond aligning model outputs with user preferences, RLHF was also used to internalize reasoning capabilities by training on human-generated reasoning chains, enabling models to perform step-by-step problem solving without explicit prompting [Ope24]. However, the reliance on human feedback

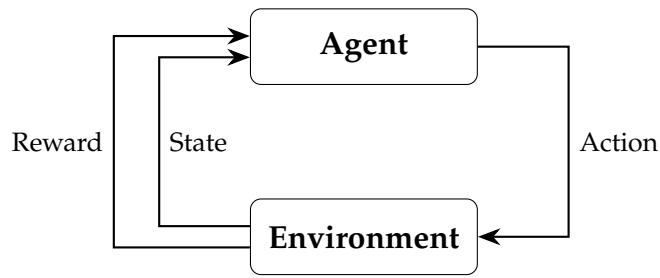


Figure 2.5: The agent–environment interaction loop in reinforcement learning [SB18, p. 54].

also introduces one of RLHF’s key limitations: The difficulty of creating environments that produce a sufficient amount of high-quality and diverse training data [Cas+23].

RLVR. Because collecting large amounts of high-quality human feedback data is difficult, the field also explored reward signals that are verifiable in a cheap and automatic way, such as mathematical equations, logic puzzles, or code compilation. This training method is called *Reinforcement Learning from Verifiable Rewards* (RLVR), and was popularized by Guo et al. (*DeepSeek-R1*) [Guo+25], who found that reasoning behaviors like self-reflection and verification emerged spontaneously without being taught from examples, just by providing the verifiable reward after completing the task. The success of RLVR was enabled by *Group Relative Policy Optimization* (GRPO), a reinforcement learning method that eliminated the need for training a separate critic network, instead estimating baselines from groups of sampled outputs [Sha+24]. RLVR led to further improvements in software-related tasks, where verifiable rewards are relatively easy to obtain (e. g., compilation, tests, sanitizers, errors, states) [Wei+25b; Nor+26].

“Wait, wait. Wait. That’s an aha moment I can flag here.” [Guo+25] DeepSeek-R1-Zero learns to rethink from verifiable rewards.

2.3.5 Agents

In the reinforcement learning framework, an agent is a system that, at each time step $t = 1, 2, 3, \dots$ observes a state $S_t \in \mathcal{S}$, selects an action $A_t \in \mathcal{A}(S_t)$ according to a policy π , and receives a scalar reward $R_{t+1} \in \mathbb{R}$ from the environment (see Figure 2.5).

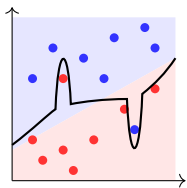
LLM-based Agents. In 2023, Yao et al. explored having LLMs generate both reasoning traces and task-specific actions, showing that “reasoning traces help the model induce, track, and update action plans, while actions allow it to interface with and gather additional information from external sources” [Yao+23]. They called this way of prompting the *ReAct* framework, and laid the foundation for LLM-based agent frameworks. Subsequently, effective tool use became central to research for LLM-based agents, e. g., by teaching models when to call which tools [Sch+23] or by introducing structured formats for invoking tools [Ope23].

Software Engineering Agents. As LLM-based agents became more capable of using tools and learning from verifiable rewards, researchers recognized that the same approach could be applied to software engineering, where outputs are often verifiable through tests, builds, and execution. One of the most impactful research projects in this direction is *SWE-bench* [Jim+24], a benchmark framework that tests LLM-based agents on real software engineering tasks collected from GitHub issues.

Today, SE-agents [Yan+24; Wan+25; Zha+24; Che+21; Ant25] are widely used in research and industry and can essentially use any tools a human software engineer would use, including terminal access, browser use, search, and the ability to compile and run code. However, SE-agents also still face limitations: They struggle with long-horizon tasks requiring hours to days for a professional engineer [Den+25], long-context code understanding [Li+25a], and following complex multi-step plans [BCP26].

2.3.6 Fundamental Challenges

One of the central goals in AI is *generalization*: Learning models that perform well on unseen data. Several fundamental challenges make this difficult.



Schematic visualization of overfitting. The true class boundary is linear (background), but the learned classifier (black) contorts itself to fit two noise points perfectly.

Overfitting. The first is *overfitting*, when a model performs well on its training data but poorly on unseen data from the same distribution. While many countermeasures exist, such as regularization through additional loss terms or constrained parameter counts [Bis06, pp. 256–269], overfitting is fundamentally hard to measure, because any decision made before evaluation can influence test performance. For example, a researcher could evaluate a model repeatedly on the test set with different hyperparameters and report only the best result. Even without such malicious intent, awareness of a popular benchmark can subconsciously guide modeling decisions toward better performance on that specific dataset [Dwo+15; Rec+19]. Furthermore, datasets often have *spurious correlations* between input data and labels, which allows models to achieve high performance by exploiting these correlations rather than learning the intended capability. A well-known example is an image classifier that distinguishes wolves from dogs not by their appearance, but by the presence of snow in the background [RSG16].

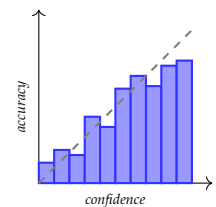
Robustness. A highly related challenge is *robustness*: How stable a model’s predictions are under small input perturbations. *Adversarial examples* demonstrate that carefully crafted changes to an input, sometimes imperceptible to humans, can cause drastically different model outputs [GSS14; RGG21]. In the context of LLMs, adversarial examples exist in the form of *adversarial prompts*, where small modifications to the input text can cause the model to produce unintended or harmful outputs [Zou+23].

Out-of-distribution Generalization. Another related challenge is *out-of-distribution generalization* (OOD generalization): how well a model’s

performance transfers to data drawn from a different distribution than its training data. For instance, a model trained on general code may not perform equally well on vulnerability detection tasks. Like overfitting, OOD generalization is inherently difficult to evaluate, because any exposure to the target distribution during development can obscure the true degree of distributional shift a model is able to handle.

The Need for Data. The effectiveness of machine learning models is inherently tied to the quality and quantity of their training data. As researchers discovered that LLM performance continues to improve with more training data even at larger scales (see Section 2.3.4), they assembled increasingly larger datasets to train even bigger models. This trend introduced several problems. First, verifying that public evaluation datasets were not included in training became increasingly difficult, a problem known as *data leakage*. Second, larger datasets are harder to curate and thus more vulnerable to *data poisoning*, where malicious or low-quality samples degrade model behavior. Third, in domains where real-world data is scarce, such as vulnerability detection, researchers often turn to LLM-generated data, yet training on LLM-generated data can cause *model collapse*, progressively reducing output diversity and quality [Shu+24]. Finally, models learn the patterns present in their data, meaning that any biases in the training set are reproduced in the model’s outputs.

Calibration and Hallucinations. Another fundamental challenge in AI models is *calibration*: Does a model’s expressed confidence align with its actual accuracy? In other words, do models know what they do not know? Guo et al. showed that modern neural networks are poorly calibrated and tend to be overconfident [Guo+17], a finding that extends to today’s LLMs, where verbalized uncertainty estimates remain overconfident across tasks and model families [GV24]. Poor calibration is closely linked to *hallucinations*, where models generate factually incorrect content [Ji+23]. While techniques such as retrieval-augmented generation (RAG) [Lew+20] and reinforcement learning with calibration-aware rewards [Wu+25] have led to improvements, hallucinations remain a challenge.



Reliability diagram of a miscalibrated model. Across all confidence buckets, accuracy lies below or above the diagonal of perfect calibration.

Interpretability. Due to the high complexity of deep learning models, they are also inherently hard to interpret [Lip18]. Why did a model make a particular decision? In the context of vulnerability detection, did the model actually understand the code or just make the decision based on artifacts in the code? Even if a model explains its decision, this explanation can be decoupled from the actual basis of its decision: Rudin [Rud19] argued that post-hoc explanations of black-box models are inherently unfaithful, and Turpin et al. [Tur+23] showed that LLM chain-of-thought explanations can systematically misrepresent the true reasons for a model’s prediction.

BACKGROUND

2.4 APPLICATION DOMAINS

The research contributions of this thesis address two specific application domains within software security: *machine learning for vulnerability detection* and *bug-introducing commit identification*. Both domains build on the foundations introduced in the previous sections, but require additional, more specific background. The following two subsections provide this background.

2.4.1 Machine Learning for Vulnerability Detection

In *machine learning for vulnerability detection* (ML4VD), the goal is to detect security vulnerabilities using machine learning. ML4VD can be formalized into different problem statements that need to be distinguished to understand and categorize the different approaches.

2.4.1.1 Problem Statement

ML4VD as a Binary Classification Problem. The most widely used formalization of ML4VD is as a binary classification problem [RLB25]. In this setting, the input space $\mathcal{X}$ consists of code samples (e. g., functions, files, slices, or commits), which can be represented in textual form (e. g., as tokens), tree-based form (e. g., as Abstract Syntax Trees), graph-based form (e. g., as Data Flow Graphs), or other forms. The output space consists of binary labels $\mathcal{Y} := \{0, 1\}$, where 1 indicates that the sample contains a security vulnerability and 0 indicates that it does not. Given a labeled training dataset $D_{\text{train}} := \{(x_1, y_1), \dots, (x_n, y_n)\}$ with $x_i \in \mathcal{X}$ and $y_i \in \mathcal{Y}$, the goal is to learn a classifier $f : \mathcal{X} \rightarrow \mathcal{Y}$ that approximates the underlying mapping from code to vulnerability labels and generalizes to unseen code samples drawn from the same distribution. Performance is then evaluated on a held-out test set D_{test} using binary classification metrics such as accuracy or the F1-score.

ML4VD as a Multi-Class Classification Problem. The classification-based formalization can also be extended to multiple classes, with $\mathcal{Y} := \{0, 1, \dots, N\}$, where $N + 1$ denotes the number of classes. Typically, each class corresponds to a vulnerability type (e. g., based on the Common Weakness Enumeration (CWE)), extending the task from vulnerability detection to vulnerability detection *and* categorization. Other class semantics are also possible (e. g., vulnerable, secure, and unsure), although they are less common.

Context-conditional Classification. The previous two formalizations largely ignore the context of code samples within their surrounding programs. In Chapter 4, I will present our investigation into whether this assumption is fundamentally valid. An alternative formulation is to classify code samples in the context of their program, essentially asking: Given a code sample from a program, does it make the program vulnerable? Formally, the inputs become $x_i, p_i \in \mathcal{X}$, where x_i denotes the code sample and p_i the program, under the condition that $x_i \subset p_i$.

If a technique classifies functions, I refer to this as “function-level classification”.

In practice, this introduces additional challenges, since programs are often relatively large and it is not always clear how to effectively incorporate them into the classification decision.

Proof of Vulnerability. An orthogonal problem statement to ML4VD as a classification task is vulnerability detection via *proof of vulnerability* (PoV). Instead of discriminating between vulnerable and secure code, this formulation aims to generate conditions under which a program becomes vulnerable, for example in the form of a *vulnerability-triggering* test case. Formally, the input becomes a program $p \in \mathcal{P}$ and the output a test case $t \in \mathcal{T}$. To validate whether a test case triggers a vulnerability, this formulation requires an oracle: a detective security mechanism $\mathcal{O} : \mathcal{P} \times \mathcal{T} \rightarrow \{0,1\}$, where $\mathcal{O}(p,t) = 1$ indicates that executing p on test case t violates the security policy monitored by $\mathcal{O}$. This also highlights a limitation of requiring a PoV: the approach is only applicable when a suitable detective security mechanism exists for the targeted vulnerability class.

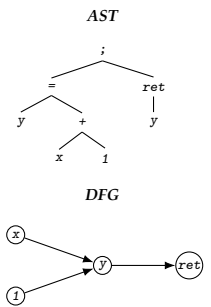
Proof of Concept and Exploits. PoV-based ML4VD can be further divided into *proof-of-concept* (PoC) generation and *exploit generation*. PoC generation aims to produce inputs that trigger a bug, whereas exploit generation aims to demonstrate an actual security impact. For example, AddressSanitizer (ASan) [Ser+12] can detect buffer overflows at runtime. Although such overflows are often security-relevant, their exploitability is not guaranteed. A bug is only confirmed as a security vulnerability once an exploit demonstrates a concrete security violation, such as leaking a protected secret from a server process. These distinctions highlight two key challenges: distinguishing exploitable from non-exploitable bugs, and reliably verifying that a generated exploit achieves a real security impact.

On Data Representation. While each of these problem statements makes assumptions about the input, they do not constrain how that input is represented. For example, if a research paper defines ML4VD as a function-level binary classification problem, the function may still be represented as a graph without changing the underlying problem statement. The most widely used representation consists of *tokens*, which are basic units of text, typically characters, subwords, or words. However, the following sections will also discuss techniques based on feature vector representations (e. g., vectors where each component counts the occurrence of a specific token), tree-based representations (e. g., Abstract Syntax Trees), and graph-based representations (e. g., Data Flow Graphs, Control Flow Graphs, or Program Dependence Graphs). Each representation has its own advantages and disadvantages, which I will highlight throughout the following section.

2.4.1.2 Techniques

Building on the previously introduced problem statements, the following paragraphs discuss the most important machine learning techniques used for vulnerability detection.

```
int f(int x) {
  int y = x + 1;
  return y;
}
```



A simple function and its tree-based (AST) and graph-based (DFG) representations.

Software Defect Prediction. Before machine learning was first applied to vulnerability detection, there were multiple decades of previous research on applying machine learning to the related task of defect or fault prediction. The earliest example was proposed by Selby et al. in 1988 [SP88], who applied decision trees to predicting faults in software modules. The pioneering work was followed up by a line of work in the 1990s and early 2000s proposing different classical ML techniques for defect or fault prediction [KPM92; OA96; BBM96]. While defect/fault prediction is not a focus of this thesis, these works build the foundation for the first research contributions applying ML to vulnerability detection.

Classical Machine Learning. To our knowledge, the first application of machine learning to vulnerability detection is the *Vulture* tool proposed by Neuhaus et al. in 2007 [Neu+07]. Neuhaus et al. define ML4VD as a binary classification problem *and* as a regression problem at the component level. For each component in Mozilla, they compute a feature vector, which represents information about function calls and imports. Then, they train a Support Vector Machine (SVM) [Vap95], which is a statistical non-linear machine learning method for supervised learning, to predict either vulnerability (binary classification) or the number of vulnerabilities for a given component (regression). They report 0.45 recall and 0.70 precision on held-out components of Mozilla, which is much better than random guessing.

The foundational paper by Neuhaus et al. was followed by a line of work applying machine learning techniques to vulnerability detection, e.g., Shin et al. [SW08], who were the first to classify vulnerabilities at the *function-level* using Logistic Regression [Bis06, pp. 205–206] based on code metrics, or Hovsepyan et al. [Hov+12], who were the first to classify raw source code as text instead of feature vectors.

Deep Learning. While artificial neural networks were applied to fault detection as early as the 1990s [KPM92], it took more than two decades until Grieco et al. first applied a neural network to vulnerability detection in 2016 [Gri+16], using a feed-forward network to classify test cases for memory corruption vulnerabilities. Their architecture was *shallow* (likely a single hidden layer), however, and would not be considered *deep* learning today. After further shallow approaches [PXW17], the first technique recognizable as deep learning was *VulDeePecker*, proposed by Li et al. in 2018 [Li+18]. It performs binary classification on *code gadgets*, a custom representation based on backward and forward program slicing centered on library/API calls, using a deep Bidirectional LSTM (BLSTM) with up to six LSTM layers. The code gadget database, comprising 61,638 gadgets with buffer error (CWE-119) and resource management error (CWE-399) vulnerabilities from open-source C/C++ repositories, also constitutes the first large-scale deep learning dataset for vulnerability detection.

A second notable early contribution came from Russell et al. [Rus+18], who were the first to use deep learning in a *function-level* binary classification setting.

Outside of the vulnerability setting, the idea of framing bug detection as a binary classification task was established around the same time by *DeepBugs*, proposed by Pradel and Sen in 2018 [PS18], which learns to distinguish correct code from buggy code based on identifier names.

Graph Neural Networks. A central question of the early DL-based approaches was how to represent the code. While the earliest approaches [Li+18; Rus+18] treated the code samples as a sequence of tokens, the field quickly explored tree-based and graph-based representations with the goal of capturing more structural information, e. g., control flow, data dependencies, or syntactic structure. The first and most impactful technique was *Devign*, proposed by Zhou et al. in 2019 [Zho+19]. Based on a self-collected function-level dataset from open-source C/C++ repositories, which ended up becoming one of the most widely used datasets in the field (see Section 2.4.1.3), *Devign* represents the functions as a composite graph by combining AST edges, control flow edges, data flow edges, and “natural code sequence” edges (capturing the original token ordering) into a single graph. It then classifies the graphs using a Gated Graph Neural Network [Li+16].

Another notable contribution was the *ReVeal* technique by Chakraborty et al. [Cha+21], who were the first to share a critical perspective on the previous DL-based approaches to vulnerability detection, criticizing data duplication, class imbalance, and learning of irrelevant features (see Section 2.4.1.4). To remedy these identified shortcomings, they proposed to convert functions into code property graphs (CPG), which combines AST, control flow, data flow, and program dependency information, rebalanced the training dataset using SMOTE [Cha+02], and introduced more regularization to prevent the model from overfitting to irrelevant features.

Large Language Models. With transformer-based language models achieving increasing popularity within the machine learning field, it was only a matter of time until they were applied to vulnerability detection. The first technique was *CodeBERT* [Fen+20], a bidirectional model (see Section 2.3.4) pre-trained on natural language and code from six different programming languages. The model architecture was based on the RoBERTa architecture [Liu+19], which is a popular BERT-based model architecture and pre-training procedure. While Feng et al. did not apply *CodeBERT* to vulnerability detection in their initial paper [Fen+20], it was first applied to vulnerability detection by Lu et al. in 2021 [Lu+21], who applied it to the function-level binary classification dataset *Devign* [Zho+19], and reported an accuracy of 62.08%. This initial result triggered a series of iterative follow-up work, e. g., Hanif et al. (*VulBERTa*) [HM22], who extended *CodeBERT* with a code-focused tokenizer, or Fu et al. (*LineVul*) [FT22], who classified at the line-level in addition to the function-level. All of these papers reported state-of-the-art scores on their respective chosen benchmark datasets, e. g., *LineVul* was reported to achieve 91% F1-score on the *BigVul* function-level dataset [Fan+20] and *VulBERTa* was reported to achieve 65% accuracy on the *Devign* dataset.

Beyond BERT-based models, the field also explored models leveraging the full transformer architecture. Notable examples are Guo et al. (*UniXcoder*) [Guo+22], who leveraged the syntactic structure via Abstract Syntax Trees to achieve better representations in their pre-training phase, and Phan et al. (*CoTexT*) [Pha+21] who gained downstream performance via pre-training on multiple code-related tasks. Similar to the BERT-based models, the respective papers also reported state-of-the-art scores on the most widely used function-level vulnerability detection datasets (e.g., Devign).

LLMs as Classifiers. The previously described first wave of LLMs for vulnerability detection approached vulnerability detection as a pure classification problem; they were usually fine-tuned to generate a single token to predict the vulnerability of a given input (e.g., “vulnerable” or “secure”). This is a key difference from the second wave of LLMs for vulnerability detection: proprietary models.

Proprietary Models. With the success of ChatGPT in 2022, the frontier large language models increasingly transitioned to closed-source, meaning that they can only be accessed via a web interface or an API and there is no complete public information on model architecture, training data, and training procedure available. The first application of API-based models (gpt-3.5-turbo and text-davinci-003) to vulnerability detection was published by Cheshkov et al. [CZL23], who evaluated these models on a self-collected dataset of Java functions. Mao et al. were the first to evaluate proprietary LLMs’ abilities to explain the vulnerability, including the cause, location, and repair suggestions [Mao+25]. Generating and evaluating explanations brings its own set of challenges, which I will explain in the following section (see Section 2.4.1.3).

In contrast to the previously described first generation of open-source LLMs, proprietary LLMs for vulnerability detection relied increasingly less on fine-tuning. Proprietary LLMs are usually pre-trained on massive amounts of data, which makes them applicable to downstream tasks without fine-tuning. While fine-tuning is still possible and can improve performance, it is usually relatively expensive and introduces additional problems, e.g., catastrophic forgetting [Luo+25]. For the previous generation of open-source LLMs, fine-tuning was the only way to achieve acceptable performance on downstream tasks. Instead of fine-tuning, proprietary LLMs increasingly relied on in-context learning (see Section 2.3.4) to learn from examples for the given task without updating model parameters.

Agents. More recently, the broader LLM field has shifted from single-prompt classification towards *agentic* systems, where LLMs interact with tools, maintain intermediate state, and decompose complex tasks into multiple steps. This shift is also reflected in ML4VD. Instead of only asking an LLM whether a given code sample is vulnerable, agentic approaches try to simulate parts of a real vulnerability analysis workflow. For example, Mao et al. [Mao+24] propose to assign different roles to multiple LLM agents, thereby simulating a collaborative

code review process. Other work integrates LLMs with traditional security tools. For instance, AutoSafeCoder [Nun+24] uses a multi-agent framework in which agents can leverage static analysis and fuzzing to identify and repair vulnerabilities. In terms of the problem statements introduced earlier (see Section 2.4.1.1), agentic approaches often move beyond direct binary classification: depending on their concrete setup, they may classify code samples, generate vulnerability explanations, construct proofs of vulnerability, or synthesize candidate repairs.

Cyber Reasoning Systems. A closely related development is *Cyber Reasoning Systems* (CRSs), which combine machine learning with classical software analysis techniques such as fuzzing, static analysis, symbolic execution, and automated program repair. The most visible recent example is DARPA’s AI Cyber Challenge (AIXCC), a two-year competition in which teams built fully autonomous CRSs for vulnerability discovery and patching in real-world open-source C and Java projects [Zha+26]. In contrast to most ML4VD benchmarks, AIXCC did not only evaluate whether a system could classify code as vulnerable, but whether it could autonomously find vulnerabilities, generate proofs of vulnerability, synthesize patches, validate static-analysis reports, and bundle related findings into developer-facing reports.

The finalist systems explored multiple directions. For finding vulnerabilities, they combined improved fuzzing pipelines with LLM-based PoV generation, where agents identify bug candidates and construct vulnerability-triggering inputs [Zha+26]. For repairing vulnerabilities, they relied on LLM-centric patching pipelines that typically perform root-cause analysis, patch generation, validation, and deduplication before submission [Zha+26]. Beyond PoV and patch generation, some systems also used LLMs or internally collected bug candidates to assess SARIF reports and bundle related PoVs, patches, and static-analysis findings. Overall, CRSs represent a broader problem setting than ML4VD as binary classification, since they aim not only to detect vulnerabilities, but to demonstrate, localize, repair, and report them in an autonomous end-to-end workflow.

2.4.1.3 Benchmarking

When a new ML4VD technique is proposed, the corresponding paper usually contains an empirical evaluation based on popular benchmark datasets that are intended to measure progress relative to the current state of the art. However, if a benchmark does not actually capture the intended capability, high scores can create a false sense of progress.

Problem Statement. The first question for benchmarking ML4VD is how the problem should be defined. As discussed in Section 2.4.1.1, ML4VD can be formulated in many different ways. A benchmark can ask whether a function is vulnerable, where the vulnerable line is, why the code is vulnerable, whether an exploit can be constructed, or whether a patch is correct. By far the most widely used problem statement is binary classification at the function-level: given a function, decide whether it contains a security vulnerability [RLB25]. One of the

main contributions of this thesis is to investigate the validity of this problem statement in Section 4.3, where we also measure how popular it is in the recent literature.

Data Collection. When ML4VD is defined as a classification problem, a benchmark needs code samples and labels. For deep learning approaches, this usually means that *many* labeled samples are needed, because the models are fine-tuned on parts of the benchmark data. This creates a central challenge: how can we collect enough code samples with vulnerability labels, and how can we make sure that these labels are correct and representative of real-world vulnerabilities? In practice, ML4VD datasets differ both in how they collect code samples and in how they assign labels. Some datasets use synthetic examples with known labels, while others use real open-source code. Labels can come from handcrafted test cases, static analysis tools, expert review, or public vulnerability databases.

Synthetic Data. One way to obtain labels is to use synthetic data. The Code Gadget Database introduced with VulDeePecker [Li+18] is an early example. It combines synthetic examples from the NIST Software Assurance Reference Dataset (SARD) [Bla18] with a smaller number of examples derived from real-world vulnerabilities, for example from the National Vulnerability Database (NVD) [BRW13]. The advantage of synthetic data is that the labels are usually known by construction. The disadvantage is that synthetic examples may not represent the complexity, coding style, and context of real open-source projects. Later work therefore tries to make generated vulnerabilities more realistic. *VulGen* [Non+23], for example, mines vulnerability-introducing edit patterns from real-world vulnerability fixes and uses a deep learning model to predict where such a pattern should be applied, in order to inject realistic vulnerabilities into non-vulnerable code for benchmarking purposes.

Static Analysis Labels. Another way to scale data collection is to use static analysis tools for labeling. Russell et al. [Rus+18] proposed one of the first large-scale function-level datasets for vulnerability detection, often referred to as the Draper dataset. They collected more than one million open-source C/C++ functions and used static analysis tools to assign vulnerability labels. This made it possible to train deep learning models on much larger datasets than before. However, the labels inherit the limitations of static analysis. Static analysis tools can produce false positives, miss vulnerabilities, and only cover the vulnerability classes that the tools are designed to detect.

Expert Labels. Devign [Zho+19] uses expert-labeled commits from open-source projects. The authors first scanned the commit histories of FFmpeg and Qemu for security-related keywords. Then, a team of security researchers manually decided whether each candidate commit was a vulnerability-fixing commit (VFC) or not. For each VFC, the dataset assumes that functions touched by the commit were vulnerable before the commit was applied. Devign contains roughly 27k functions and mainly covers memory-related vulnerabilities, such as

buffer overflows, memory leaks, crashes, and memory corruption.¹ Devign became especially influential because it was later included in CodeXGLUE [Lu+21], a widely used benchmark for machine learning on code.

Vulnerability Database Labels. A third common strategy is to start from public vulnerability databases. BigVul [Fan+20] collected C/C++ vulnerabilities from public CVE records and linked them to GitHub repositories and vulnerability-related code changes. It contains roughly 188k functions and is one of the most frequently used datasets in ML4VD benchmarking (see Section 4.3). In BigVul-style labeling, functions changed by a vulnerability-fixing commit are treated as vulnerable before the patch, while unchanged functions from the same files are treated as non-vulnerable. ReVeal [Cha+21] follows a related direction, but was designed with several quality concerns in mind, including class imbalance and duplicate samples. These datasets are attractive because they are based on real vulnerabilities, but the labels are still indirect: the dataset usually observes a patch, not an exploit or a full proof that each touched function was vulnerable.

Label Filtering. Because noisy labels became a visible problem in ML4VD, more recent datasets try to filter labels more carefully. PrimeVul [Din+25], for example, starts from vulnerability-related commits but applies additional filtering steps to remove likely false positives, reduce duplication, and avoid common sources of data leakage.

2.4.1.4 Critical Perspectives

Despite the rapid growth of ML4VD, several works have raised concerns about whether the reported benchmark results translate to realistic vulnerability detection. In the following paragraphs, I summarize the most important critical perspectives on ML4VD and ML4VD benchmarking.

Unrealistic Settings. One major concern is that common ML4VD benchmarks often do not match realistic deployment settings. Chakraborty et al. [Cha+21] criticized three aspects of prior datasets: unrealistic class balance, the use of synthetic data such as SARD [Bla18], and labels produced by static analysis tools such as in the Draper dataset [Rus+18]. In real software, vulnerable functions are rare compared to non-vulnerable functions, while many ML4VD datasets contain a much higher fraction of vulnerable samples. This can make the task easier than in practice and can inflate reported scores. Chakraborty et al. showed that the performance of then state-of-the-art techniques drops substantially when evaluated in a setting that better accounts for these issues [Cha+21]. This concern is also consistent with the observation that, despite many papers reporting high benchmark scores, only a few ML4VD papers before 2025 reported finding real zero-day vulnerabilities.

¹ <https://sites.google.com/view/devign>

Data Quality. A second major concern is label quality. This is especially important because the most widely used datasets, such as BigVul, Devign, and ReVeal, are based on expert labels or vulnerability database labels. Croft et al. [CBK23] showed that such datasets can contain substantial label noise, and in Chapter 4 we confirm and analyze this problem in more detail. These quality issues can be explained by multiple sources of error. First, the identification of vulnerability-fixing commits can be wrong, either because experts misclassify commits or because vulnerability databases contain incomplete or incorrect information. Second, even if a commit fixes a vulnerability, not every changed function is necessarily vulnerable. A patch may fix the vulnerable code in one location and then adapt surrounding code to the changed behavior. Third, vulnerability-fixing commits may contain unrelated changes that are not security-relevant. If all changed functions are labeled as vulnerable, these unrelated or merely adapted functions become false positives in the dataset.

Duplicates and Temporal Leakage. Another data quality issue is duplication. Chakraborty et al. [Cha+21] observed that ML4VD datasets can contain duplicate or near-duplicate samples. If duplicates appear in both training and test data, the benchmark can overestimate performance because the model may only need to recognize code it has effectively seen before. A related problem is temporal leakage. Ding et al. [Din+25] point out that random splits can place later versions of a function in the training set and earlier versions of the same function in the test set. This creates an unfair evaluation setting, because the model may have already seen information from the future relative to the evaluated sample.

Spurious Correlations. ML4VD models can also learn spurious correlations instead of vulnerability semantics. Chakraborty et al. [Cha+21] showed that models may rely on superficial artifacts, such as identifiers or other dataset-specific patterns, that correlate with labels but do not causally indicate vulnerabilities. This is a general challenge for parameterized machine learning models: if the training data contains shortcuts that help reduce training error, the model may exploit them. In ML4VD this is particularly problematic, because a model can achieve high benchmark scores without learning the program behavior that actually makes code vulnerable.

Robustness. The problem of spurious correlations is closely connected to robustness. If a model relies on superficial properties of the input, then small semantics-preserving changes can change its prediction. Bielik et al. [BV20] showed this problem for machine learning models of code in general, for example under variable renaming. Li et al. [Li+21] studied robustness specifically for ML4VD and found that small changes to code can substantially affect vulnerability detection results. This means that a model may classify one version of a function correctly, but fail on an equivalent version with different names or formatting.

Unfaithful Reasoning. With the increasing use of LLMs, another concern is unfaithful reasoning. An LLM may give the right vulnerability label for the wrong reason, or give a plausible explanation that is not actually connected to its decision. Ullah et al. [Ull+24] showed that LLM-based vulnerability detection suffers from this issue. This creates a benchmarking challenge beyond binary classification. Explanations are difficult to evaluate automatically because small textual differences can change the meaning substantially, while surface-level similarity metrics may miss this. Human evaluation can assess explanation quality more directly, but it is expensive and difficult to scale. LLM-based evaluation is a possible alternative, but it also needs to be validated carefully before it can be trusted.

2.4.2 Bug-Introducing Commit Identification

When a bug is fixed, how can we identify which versions of the software were actually affected? In version control systems, we can attempt to answer this question by identifying the set of changes that introduced the bug. The problem of identifying the changes that introduced a bug is called the *SZZ problem*, after Śliwerski, Zimmermann, and Zeller, who defined and first attempted to solve this problem in 2005 [SZZ05]. Beyond identifying affected versions, solutions to the SZZ problem can also be relevant for bug identification research because a solution can be used to create datasets of bug-introducing changes if bug-fixing changes are known.

Problem Definition. I focus on the version control system *git*, which bundles changes as *commits*. Following the definition from our work on agents for bug-introducing commit identification [RB26], given a fix commit c_{fix} that repairs a bug in a software repository, the goal is to identify the set of bug-introducing commits (BICs) $B \subseteq H$, where H is the repository history before c_{fix} . Each commit $b \in B$ introduced a change that caused or contributed to the bug that c_{fix} resolves. In practice, $|B|$ is often one but can be larger when a bug arises from changes spread across multiple commits.

The following summary of prior approaches is adapted from our work on agents for bug-introducing commit identification [RB26].

Original SZZ Algorithm. The original SZZ algorithm approximates the target set of bug-introducing commits B by tracing the lines removed or modified in the fix commit c_{fix} . For each such line, it uses version-control annotations to identify the commit in the prior history H that created the exact version of the line that c_{fix} deletes or modifies. The union of these commits forms a candidate set $\hat{B}$. This makes SZZ a simple operational approximation of the BIC identification problem, but it relies on the assumption that the buggy behavior was introduced together with the exact line version later changed by the fix. When a fix only adds code, changes nearby code, or modifies a line whose latest version was not the root cause, this approximation can miss the true commits in B or include unrelated commits in $\hat{B}$.

Filtering-based Refinements. Filtering-based SZZ variants keep the same basic structure but try to improve the mapping from the SZZ candidate set $\hat{B}$ to the true set B . Some approaches remove candidates caused by non-semantic changes, such as comments, formatting, imports, module declarations, or refactorings. Others select a single candidate from $\hat{B}$, for example the most recent candidate, the candidate that changed the most lines, or the earliest commit in a recursively traced history. These refinements can reduce false positives, but they remain constrained by the initial line-tracing step: if the true bug-introducing commit $b \in B$ is not in the generated candidate set $\hat{B}$, filtering alone cannot recover it.

Learning-based Approaches. Learning-based approaches replace parts of the heuristic selection process with learned models. Rather than treating all deleted lines or all candidate commits equally, these methods estimate which lines in c_{fix} are bug-relevant or whether a candidate $b \in \hat{B}$ plausibly introduced the bug fixed by c_{fix} . NeuralSZZ ranks deleted lines by bug relevance before applying SZZ [Tan+23], while LLM4SZZ uses large language models to identify buggy statements and assess SZZ candidates with additional context [Tan+25]. These approaches add semantic reasoning to the SZZ pipeline, but they still largely depend on SZZ-style candidate generation and therefore inherit its inability to identify BICs outside the traced candidate set.

Fundamental Problems. Most SZZ approaches can be viewed as a pipeline with three fundamental challenges. First, the approach must decide where to start from. It must identify which parts of the software are relevant to the bug fixed by c_{fix} . Starting only from deleted or modified lines may underapproximate the relevant code and miss bugs fixed by additions or nearby changes, while including more context may overapproximate the bug-relevant region and introduce noise. Second, the approach must construct a candidate set $\hat{B} \subseteq H$ of commits that could have introduced the bug. A small candidate set reduces the search space but risks false negatives when the true BICs B are excluded. A large candidate set improves coverage but can contain many false positives. Third, the approach must identify the true bug-introducing commits within this candidate set. This final step is a needle-in-the-haystack problem: even if $B \subseteq \hat{B}$, the relevant commits may be hidden among many unrelated changes and require semantic reasoning about how the fix relates to the earlier history.

PART II

PUBLICATIONS

UNCOVERING THE LIMITS OF MACHINE LEARNING FOR AUTOMATIC VULNERABILITY DETECTION

3

This article was published at the 33rd USENIX Security Symposium (USENIX Security), 2024. The full version is available at <https://www.usenix.org/system/files/usenixsecurity24-risse.pdf> and is reproduced here with permission.

Author Contributions. Niklas Risse served as the primary contributor for this work, leading the conceptualization, data curation, methodology, implementation, formal analysis, interpretation of results, original drafting, and final editing of the manuscript. Marcel Böhme supervised the research and provided guidance on conceptualization, methodology, and interpretation of results, as well as contributing to the review and editing of the final text.

Formatting Changes. The content of the original publication is reproduced unchanged; only its presentation was adapted to match the formatting of this thesis, such as table styles and sizes as well as the citation style. In addition, some of the original figures were re-created as TikZ figures to reduce the storage size of the PDF; they are based on the same data and convey exactly the same content.

Original Publication

- [RB24] Niklas Risse and Marcel Böhme. “Uncovering the Limits of Machine Learning for Automatic Vulnerability Detection”. In: *Proceedings of the 33rd USENIX Security Symposium (USENIX Security)*. 2024. URL: <https://www.usenix.org/system/files/usenixsecurity24-risse.pdf>.

Uncovering the Limits of Machine Learning for Automatic Vulnerability Detection

Niklas Risse and Marcel Böhme

Abstract. Recent results of machine learning for automatic vulnerability detection (ML4VD) have been very promising. Given only the source code of a function f , ML4VD techniques can decide if f contains a security flaw with up to 70% accuracy. However, as evident in our own experiments, the same top-performing models are unable to distinguish between functions that contain a vulnerability and functions where the vulnerability is patched. So, how can we explain this contradiction and how can we improve the way we evaluate ML4VD techniques to get a better picture of their actual capabilities?

In this paper, we identify overfitting to unrelated features and out-of-distribution generalization as two problems, which are not captured by the traditional approach of evaluating ML4VD techniques. As a remedy, we propose a novel benchmarking methodology to help researchers better evaluate the true capabilities and limits of ML4VD techniques. Specifically, we propose (i) to augment the training and validation dataset according to our cross-validation algorithm, where a semantics-preserving transformation is applied during the augmentation of either the training set or the testing set, and (ii) to augment the testing set with code snippets where the vulnerabilities are patched.

Using six ML4VD techniques and two datasets, we find (a) that state-of-the-art models severely overfit to unrelated features for predicting the vulnerabilities in the testing data, (b) that the performance gained by data augmentation does not generalize beyond the specific augmentations applied during training, and (c) that state-of-the-art ML4VD techniques are unable to distinguish vulnerable functions from their patches.

3.1 INTRODUCTION

Recently several different publications have reported high scores on vulnerability detection benchmarks using machine learning (ML) techniques [Fen+20; HM22a; Pha+21; Ahm+21; Guo+21; Guo+22]. The resulting models seem to outperform traditional program analysis methods, e. g., static analysis, even without requiring any hard-coded knowledge of program semantics or computational models. So, does this mean that the problem of detecting security vulnerabilities in software is solved? Are these models actually able to detect security vulnerabilities, or do the reported scores provide a false sense of security?

Even though ML4VD techniques achieve high scores on vulnerability detection benchmark datasets, there are still situations in which they fail to meet expectations when presented with new data. For example, it is possible to apply small semantics-preserving changes to augment the testing dataset of a state-of-the-art model and then measure whether the model changes its predictions. If it does, it would indicate a dependence of the prediction on unrelated features. Examples of such transformations are identifier renaming [YAY20; Hen+22; Zha+20; Yan+22; Zha+22], insertion of unexecuted statements [YAY20; Zha+22; Hen+22; Sri+21] or replacement of code elements with equivalent elements [Li+22; APD21]. The impact of augmenting testing data using these transformations has been explored for many different software-related tasks and the results seem to be clear: Learning-based models fail to perform well when testing data gets augmented using semantics-preserving transformations of code [Yan+22; Rab+21; APD21; Hen+22; Sri+21; YAY20; BV20; Zha+20; Zha+22].

In our own experiments, we were able to reproduce the findings of the literature and made additional observations: ML4VD techniques that were trained on typical training data for vulnerability detection are also unable to distinguish between vulnerable functions and their patched counterparts. If a patched function is also predicted as vulnerable, this indicates that the prediction critically depends on features unrelated to the presence of a security vulnerability.

It has previously been proposed to reduce the dependence on unrelated features by augmenting not just the testing data but also the training data [Yan+22; Hen+22; Sri+21; YAY20; BV20; Zha+20; Zha+22]. Indeed, this seems to restore the lost performance back to previous levels, but does it really reduce the dependence on unrelated features, or are the models just overfitting to different unrelated features of the data?

In this paper, we propose a novel benchmarking methodology that can be used to evaluate the capabilities of ML4VD techniques by using data augmentation. First, we propose Algorithm 3.1, in which a selected semantics-preserving transformation is applied to the training dataset of a model, and a *different* transformation is applied to the testing dataset. When repeated for all possible pairs out of a set of transformations, the resulting scores provide a better measure of overfitting to the unrelated features that are introduced by the semantics-preserving transformations during training data augmentation. Second, we propose Algorithm 3.2, in which a trained model is evaluated on a testing dataset that contains both vulnerable programs and their respective patches. The results provide a measure of the model’s ability to generalize to a modified vulnerability detection setting.

In order to validate Algorithm 3.1 and Algorithm 3.2 empirically, we selected six state-of-the-art ML4VD techniques. All evaluated ML4VD techniques happen to be token-based large language models (LLMs). As our selection criterion, we defined the top-performing ML4VD techniques on the most widely known ML vulnerability detection benchmark CodeXGLUE [Lu+21b; Lu+21a] that are available as

open source. This gave us ranks 1, 2, 6, 10, and 12 of the leaderboard, all of which are token-based LLMs. In fact, 9 of the Top-10 solutions on the leaderboard are token-based LLMs. By applying Algorithm 3.1 and Algorithm 3.2 in our empirical study of six state-of-the-art ML4VD techniques and three datasets, we confirmed that ML4VD techniques continue to leverage unrelated features when deciding whether a function contains a vulnerability.

For Algorithm 3.1, we implemented 11 different semantics-preserving transformations for data augmentation and evaluated the trained models using two popular vulnerability detection datasets. As expected, we find a strong benefit of training data augmentation (69.0% and 66.2% average restoration of accuracy/F1-score for the two datasets) when the transformations applied to training and testing datasets are the same. However, we find no improvement in performance when the transformations applied to training and testing datasets are different. In fact, we even find an additional 30.2% and 77.5% average *decrease* in accuracy/F1-score for the two datasets. In other words, ML4VD techniques still severely overfit to the specific label-unrelated features introduced by training data augmentation. The improvement in performance gained by data augmentation only applies to the specific type of transformations used during training.

For Algorithm 3.2, we introduce a new dataset, VulnPatchPairs, which contains 26.2k C functions and is derived from the CodeXGLUE/Devign vulnerability detection dataset [Zho+19]. Exactly half of the functions in VulnPatchPairs contain security vulnerabilities. The other half are patched versions of the first half.¹ We investigated six ML4VD techniques using VulnPatchPairs and evaluated their ability to generalize from their typical training data to VulnPatchPairs, and vice versa. To our surprise, all six ML4VD techniques that were trained on a typical training dataset were unable to distinguish between the vulnerable functions and their patched counterparts in VulnPatchPairs. On average, the accuracy turned out to be worse than random guessing. The trained models are unable to generalize from a standard vulnerability detection dataset to the modified setting.

In summary, this paper contributes two novel algorithms that can be used to uncover major problems of ML4VD techniques that are not detected using the standard evaluation setup: Overfitting to semantics-preserving code changes and the inability to generalize between related vulnerability detection settings. Additionally, we provide an empirical evaluation of six state-of-the-art ML4VD techniques using the proposed methodology.

- We present a general methodology consisting of two algorithms, that can be used to evaluate ML4VD techniques.
- We show empirically, that state-of-the-art ML4VD techniques overfit to the unrelated features introduced by semantics-preserving transformations during data augmentation.

¹ See Section 3.4.4 for details.

- We introduce *VulnPatchPairs*, a new dataset that contains vulnerable C function and the corresponding patched versions of the same functions. It is available at <https://github.com/niklasrisse/VPP>.
- We demonstrate, that six state-of-the-art ML4VD techniques are not able to distinguish between the vulnerable and patched functions in *VulnPatchPairs*.
- We publish all of our code and results for reproducibility. They are available at https://github.com/niklasrisse/USENIX_2024.

3.2 RELATED WORK

One of the main tools to study the limits of ML4VD techniques are semantics-preserving transformations of code. Previous work [Yan+22; Rab+21; APD21; Hen+22; Sri+21; YAY20; BV20; Zha+20; Zha+22; Li+21; Li+22] proposed methods to generate semantics-preserving transformations for source code datasets and investigated their impact when used to augment testing data of learned models.

Many of the works that reported the failures of learned models when testing data was augmented also investigated training data augmentation using their respective methods [Yan+22; Hen+22; Sri+21; YAY20; BV20; Zha+20; Zha+22]. A common finding in all of these publications is that training data augmentation using a specific type of semantics-preserving transformation leads to improved performance on testing sets that have been augmented the same way. But does the performance gained by data augmentation generalize beyond the specific augmentations applied during training?

Some of the publications that propose methods for data augmentation [Hen+22; Yan+22; BV20; Sri+21; Zha+22] take it a step further; they augment the training data using a slightly different but related type of transformation than for the testing data. For example, Henkel et al. [Hen+22] apply their gradient-based approach for identifier renaming to the training data and a random renaming strategy to the testing data. Similarly, Yang et al. [Yan+22] apply their method for variable renaming to a training dataset and the method proposed by Zhang et al. [Zha+20] to a testing dataset. All of these works find an improved performance when the training dataset is augmented in a similar way than the testing dataset. However, the transformations used for augmenting the training and testing data in these publications are all similar in type, e. g., they both rename identifiers. But does the performance also improve when training data is augmented in a different way than the testing data? Our work aims to fill this gap in the literature by carrying out a thorough empirical study that considers a diverse set of 11 different transformations, six state-of-the-art ML4VD techniques, and two high-quality datasets.

Similar to other related publications listed above [Yan+22; Hen+22; Sri+21], Rahman et al. [Rah+24] investigate overfitting of ML4VD tech-

niques to variable and API names by transforming them in the testing data. Additionally, they propose a new method to address the overfitting based on causal learning, which aims to disable models from using superficial features (e.g., variable names) entirely. While their approach seems to be effective to avoid overfitting for concrete and simple transformations (e.g., changing variable names), the authors do not investigate how their method performs when faced with unseen semantics-preserving transformations, that were not specifically trained for. Our proposed methodology can be used as a tool to do this, which allows to draw conclusions about overfitting to unrelated features fundamentally, irrespective of the type of transformation applied to training- and testing data.

In order to evaluate the general capabilities of ML4VD techniques, we collected a new dataset (VulnPatchPairs), which contains both vulnerable functions and their respective patches. The collection of a pairwise vulnerability-patch dataset has been proposed by previous work [Pon+19; BNM21; BSF22], e.g., for the research field of automated fixing. However, to the best of our knowledge, we are the first to utilize such a dataset to evaluate the general capabilities of ML4VD techniques.

Two recently published papers [Che+23; Ste+23] report poor generalization capabilities of different ML-based techniques (e.g., LLMs and GNNs) when evaluated on functions from unseen git projects. Our Algorithm 3.2 also investigates the generalization capabilities of ML4VD techniques, but using a different setup, in which functions in the evaluation data belong to a modified vulnerability detection setting (e.g., vulnerable functions and their patches), but can be from the same projects.

3.3 METHODOLOGY

We propose a novel benchmarking methodology to help researchers better evaluate advances in ML4VD techniques. The methodology consists of two parts, Algorithm 3.1 (A1) and Algorithm 3.2 (A2).

3.3.1 Data Augmentation

A central component of our methodology is data augmentation, and the expectations for vulnerability detection models that emerge from using code transformations for data augmentation.

We define data augmentation as the application of one or multiple code transformations onto all code snippets of a given code snippet dataset $CD \subset \mathcal{C}$, where $\mathcal{C}$ is a space that represents all possible code snippets $c \in \mathcal{C}$ in a given programming language.

A code transformation $t : \mathcal{C} \rightarrow \mathcal{C}$ is a function that maps from and to $\mathcal{C}$. Let's assume we have an oracle function $g : \mathcal{C} \rightarrow \{0, 1\}$, which maps from the space of code snippets $\mathcal{C}$ to either 0 or 1. The oracle function g represents the ground truth, i.e., it shows whether a code

UNCOVERING THE LIMITS OF MACHINE LEARNING FOR AUTOMATIC VULNERABILITY DETECTION

```

1 static inline int coeff_unpack_golomb(GetBitContext *gb, int qfactor, int
  qoffset)
2 {
3     int coeff = dirac_get_se_golomb(gb);
4     const int sign = FFSIGN(coeff);
5     const unsigned sign = FFSIGN(coeff);
6     if (coeff)
7         coeff = sign*((sign * coeff * qfactor + qoffset) >> 2);
8     return coeff;
9 }

```

(a) Code Snippet

```

1 static inline int coeff_unpack_golomb(GetBitContext *gb, int qfactor, int
  qoffset)
2 {
3     int coeff = dirac_get_se_golomb(gb);
4     const int sign = FFSIGN(coeff);
5     const unsigned sign = FFSIGN(coeff);
6     if (coeff)
7         coeff = sign*((sign * coeff * qfactor + qoffset) >> 2);
8     if (0)
9         coeff = 666;
10    return coeff;
11 }

```

(b) Transformed Code Snippet

Figure 3.1: Example of a simple semantics-preserving transformation. The change (orange) has no effect on the vulnerability label. Both code snippets contain a security vulnerability (integer overflow in line 4). The code was taken from the Ffmpeg GitHub repository (URL: <https://github.com/FFmpeg/FFmpeg/commit/92da2309>) and is part of the CodeXGLUE/Devign dataset.

snippet c does (1) or does not (0) contain a security vulnerability. For a given code snippet dataset $CD \subset \mathcal{C}$, a code transformation t can be characterized by its effect on $g(t(c)) \forall c \in CD$:

Semantics-Preserving Transformation. We call a transformation t_p semantics-preserving w.r.t. CD , if the changes introduced by applying it do not affect the ground truth vulnerability label, $g(c) = g(t_p(c)) \forall c \in CD$. Figure 3.1 shows an example of a simple semantics-preserving transformation applied to a real-world code snippet.

Label Inverting Transformation. We call a transformation t_d label inverting w.r.t. CD , if the changes introduced by applying it change the ground truth vulnerability label, $g(c) \neq g(t_d(c)) \forall c \in CD$. In other words, a label inverting transformation either adds or removes a vulnerability from a code snippet.

In general, we expect a vulnerability detection model to correctly predict, whether a given code snippet contains a security vulnerability, independent of any semantics-preserving or label inverting transformations that have been previously applied to the code snippet. Specifically, we can formulate the following expectations:

1. If we change a code snippet without affecting the vulnerability label (semantics-preserving transformation), we expect a vulnerability detection tool to compute the same correct prediction as before applying the change.

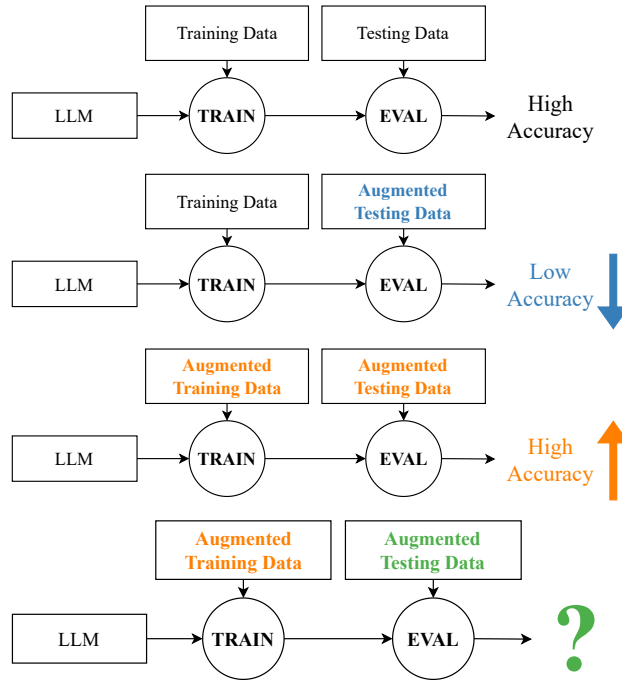


Figure 3.2: Visualization of Algorithm 3.1, which we created to detect overfitting of ML4VD techniques to unrelated features introduced by data augmentation. Colors represent that either only testing data is augmented (blue), training- and testing data are augmented using the same (orange), or different augmentation methods (green).

2. If we add or remove a vulnerability from a code snippet (label inverting transformation), we expect a vulnerability detection tool to still deliver a correct prediction, or i. e., we expect it to change its prediction with the ground truth label of the code snippet.

In the following sections, we present two algorithms, which allow to evaluate ML4VD techniques using the two formulated expectations.

3.3.2 A1: Detecting Overfitting to Code Changes

The goal of Algorithm 3.1 is to measure, whether ML4VD techniques overfit to augmentations of their training data that are unrelated to the respective vulnerability labels and whether the performance gained by data augmentation generalizes beyond the specific augmentations applied during training. We provide a simple visualization of the idea behind the algorithm in Figure 3.2, the algorithm itself in Algorithm 3.1, and a description of the most important parts in the following paragraphs. We use the colors blue, orange, and green, to connect the basic ideas of the algorithm with the experimental results across the paper².

² See Figure 3.2, Algorithm 3.1, Figure 3.5, Figure 3.7, Subfigure 3.8(c) and Table 3.2.

Algorithm 3.1: Detecting Overfitting to Code Changes

Input: Semantics-Preserving Transformations $T := \{t_1, \dots, t_N\}$
 Training Dataset Tr
 Testing Dataset Te
 ML Training Method $train_model$
 ML Evaluation Method $evaluate_model$
 Performance Metric M

- 1: $MLM[Tr] = train_model(Tr)$
- 2: $score[MLM[Tr], Te] = evaluate_model(MLM[Tr], Te, M)$
- 3: **for each** $t_k \in T$ **do**
- 4: $Te_k = t_k(Te)$ // testing data augmentation
- 5: $score[MLM[Tr], Te_k] = evaluate_model(MLM[Tr], Te_k, M)$
- 6: $effect[Tr, Te_k] = score[MLM[Tr], Te_k] - score[MLM[Tr], Te]$
- 7: $Tr_k = t_k(Tr)$ // training data augmentation
- 8: $MLM[Tr_k] = train_model(Tr_k)$
- 9: $score[MLM[Tr_k], Te_k] = evaluate_model(MLM[Tr_k], Te_k, M)$
- 10: $MLM[Tr_k, Te_k] = score[MLM[Tr_k], Te_k] - score[MLM[Tr], Te]$
- 11: **for each** $t_{j \neq k} \in T$ **do**
- 12: $Te_j = t_j(Te)$ // testing data augmentation
- 13: $score[MLM[Tr_k], Te_j] = evaluate_model(MLM[Tr_k], Te_j, M)$
- 14: $effect[Tr_k, Te_j] = score[MLM[Tr_k], Te_j] - score[MLM[Tr], Te]$
- 15: **end for**
- 16: **end for**

Output: $output_{A1.1} = (\sum_k effect[Tr, Te_k]) / N$
 $output_{A1.2} = (\sum_k effect[Tr_k, Te_k]) / N$
 $output_{A1.3} = (\sum_k \sum_{j \neq k} effect[Tr_k, Te_j]) / (N(N-1))$

What are the inputs? The inputs of Algorithm 3.1 are a set of different semantics-preserving transformations $T := \{t_1, \dots, t_N\}$, a training dataset Tr , a testing dataset Te , a ML training method $train_model$, a ML evaluation method $evaluate_model$, and a performance metric M . The training and testing datasets Tr and Te consist of code-label pairs (c_i, v_i) , with $c_i \in \mathcal{C}$ representing code snippets and $v_i \in \{0, 1\}$ representing labels that indicate the absence (0) or presence (1) of security vulnerabilities in the respective code snippets. The method $train_model$ can utilize the training dataset Tr to train a machine learning model $MLM : \mathcal{C} \rightarrow \{0, 1\}$, which maps from the space of code snippets $\mathcal{C}$ to either 1 (vulnerability) or 0 (no vulnerability). The method $evaluate_model$ can use the performance metric M to quantify and aggregate the performance of a trained model MLM on a testing dataset Te into a single number between 0 (bad) and 1 (perfect).

What is computed? Algorithm 3.1 computes the average effects of (a) augmenting the testing data of the selected ML technique using transformations $t_k \in T$ ($output_{A1.1}$), (b) using the same transformations to also augment the training data ($output_{A1.2}$), and (c) using different transformations to also augment the training data ($output_{A1.3}$).

In lines 4-6, Algorithm 1 computes the effect of augmenting the testing dataset Te with the transformation t_k on the performance of the trained model $MLM[Tr]$. The result is $effect[Tr, Te_k]$, the absolute difference between the scores of $MLM[Tr]$ on the clean testing dataset Te and the augmented testing dataset Te_k . In other words, $effect[Tr, Te_k]$ quantifies how many points in score are lost if we augment the testing dataset with transformation t_k . $output_{A1.1}$ aggregates this intermediate result over all transformations $t_k \in T$.

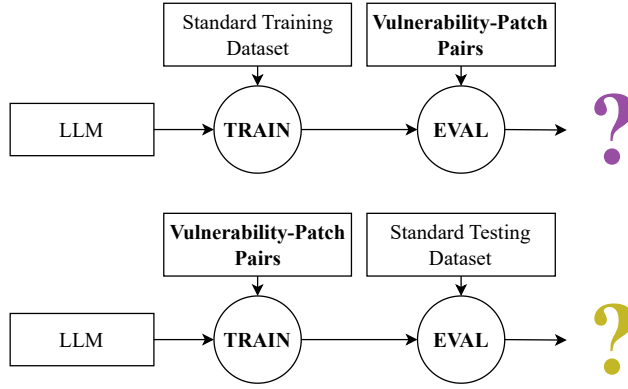


Figure 3.3: Visualization of Algorithm 3.2, which we created to test whether ML4VD techniques are able to generalize to a modified setting, which requires to distinguish between vulnerabilities and patches.

In lines 7-10, Algorithm 1 goes a step further and computes the effect of both augmenting the training dataset Tr and the testing dataset Te using the same transformation t_k . The result is $effect[Tr_k, Te_k]$, the absolute difference between scores of $MLM[Tr_k]$ on the augmented testing dataset Te_k and $MLM[Tr]$ on the testing dataset Te . In other words, $effect[Tr_k, Te_k]$ quantifies how many points in score are lost if we augment both the training and the testing dataset with transformation t_k . $output_{A1.2}$ aggregates this intermediate result over all transformations $t_k \in T$.

In lines 12-14, the algorithm computes the effect of augmenting the testing dataset using a *different* transformation t_j than for the training dataset. The result is $effect[Tr_k, Te_j]$, the absolute difference between scores of $MLM[Tr]$ on the testing dataset Te and $MLM[Tr_k]$ on the augmented testing Te_j . In other words, $effect[Tr_k, Te_j]$ quantifies how many points in score are lost if we augment the training and the testing dataset with different transformations t_k and t_j . $output_{A1.3}$ aggregates this intermediate result over all transformations $t_k \in T, t_{j \neq k} \in T$.

How can the results be used? Using this algorithm, researchers can effectively evaluate new ML4VD techniques. Specifically, for a selected technique researchers can answer the following questions:

1. How much does the performance of the selected ML technique decrease if we augment the code snippets for testing without affecting the vulnerability labels? Answer: On average, the performance does change by $output_{A1.1}$ points.
2. How much performance of the selected ML technique can be restored, if we augment the training code snippets in a similar way than the testing code snippets? Answer: On average, $output_{A1.2} - output_{A1.1}$ of the initial decrease can be restored.

Algorithm 3.2: Distinguish between Vulnerability and Patch

Input: Standard Training Dataset Tr
 Standard Testing Dataset Te
 Vulnerability-Patch Training Dataset $VPTr$
 Vulnerability-Patch Testing Dataset $VPTe$
 ML Training Method $train_model$
 ML Evaluation Method $evaluate_model$
 Performance Metric M

- 1: $MLM[Tr] = train_model(Tr)$
- 2: $MLM[VPTr] = train_model(VPTr)$
- 3: $score[MLM[Tr], Te] = evaluate_model(MLM[Tr], Te, M)$
- 4: $score[MLM[Tr], VPTe] = evaluate_model(MLM[Tr], VPTe, M)$
- 5: $score[MLM[VPTr], VPTe] = evaluate_model(MLM[VPTr], VPTe, M)$
- 6: $score[MLM[VPTr], Te] = evaluate_model(MLM[VPTr], Te, M)$

Output: $output_{A2.1} = score[MLM[Tr], Te]$
 $output_{A2.2} = score[MLM[Tr], VPTe]$
 $output_{A2.3} = score[MLM[VPTr], VPTe]$
 $output_{A2.4} = score[MLM[VPTr], Te]$

3. How much performance of the selected ML technique can be restored, if we augment the training code snippets in a different way than the testing code snippets? Answer: On average, $output_{A1.3} - output_{A1.1}$ of the initial decrease can be restored.
4. Does the selected ML technique overfit to specific augmentations of the training data that are unrelated to the respective vulnerability labels? Answer: If $output_{A1.2} \gg output_{A1.3}$: Yes, otherwise No.

3.3.3 A2: Distinguish between Vulnerability and Patch

The main goal of Algorithm 3.2 is to evaluate, whether ML4VD techniques are able to generalize from their typical training data to a modified setting, which requires to distinguish security vulnerabilities from their patches. Additionally, the algorithm also aims to evaluate the reverse, or i. e., whether ML4VD techniques that were trained to distinguish between vulnerabilities and their patches are able to perform well on standard testing data. We provide a simple visualization of the idea behind the algorithm in Figure 3.3, the algorithm itself in Algorithm 3.2, and a description of the most important parts in the following paragraphs. We use the colors **purple** and **yellow** to connect the basic ideas of the algorithm with the experimental results across the paper ³.

What are the inputs? In addition to the inputs of Algorithm 3.1, Algorithm 3.2 requires a special vulnerability-patch testing dataset $VPTe$ and a vulnerability-patch training dataset $VPTr$. $VPTe$ and $VPTr$ also consist of code snippets $c_i \in C$ and vulnerability labels $v_i \in \{0, 1\}$, but for every code snippet c_j with label $v_j = 0$, they also contain a snippet $c_{l \neq j}$ with $v_l = 1$ which represents the patched version of c_j . The relationship between a code snippet and its patched version can

³ See Figure 3.3, Algorithm 3.2, and Table 3.3.

be characterized as a label inverting transformation $t_{PATCH} : \mathcal{C} \rightarrow \mathcal{C}$, which maps code snippets c_j to their patched versions c_l .

What is computed? The purpose of Algorithm 3.2 is to quantify the ability of the selected ML technique to generalize between two related vulnerability detection settings. The first setting, represented by the standard training and testing datasets Tr and Te , consists of code snippets, which either contain or do not contain a vulnerability. This setting is most frequently used in the related literature [Guo+22; Guo+21; Fen+20; HM22a; Pha+21; Ahm+21]. The second setting, represented by the vulnerability-patch training and testing datasets $VPTr$ and $VPTe$, consists of vulnerable code snippets and their respective patches. As formulated in Section 3.3.1, a perfect vulnerability detection model should be able to perform well in both settings, irrespective of the setting on which it was trained. In other words, a vulnerability detection model should be able to generalize between the settings.

In total, Algorithm 3.2 computes four scores. In line 3, Algorithm 3.2 computes the score of a model $MLM[Tr]$, which was trained on the standard training dataset Tr , on the standard testing dataset Te . This score represents the standard evaluation and serves as a baseline for the other scores.

In line 4, Algorithm 3.2 computes the score of a model $MLM[Tr]$, which was trained on the standard training dataset Tr , on the vulnerability-patch testing dataset $VPTe$. When compared to the first score, this result serves as a measure of $MLM[Tr]$'s ability to generalize to the modified setting, which requires to distinguish vulnerabilities from their patches.

In line 5, Algorithm 3.2 computes the score of a model $MLM[VPTr]$, which was trained on the vulnerability-patch training dataset $VPTr$, on the vulnerability-patch testing dataset $VPTe$. Again, this score serves as a baseline for the other scores.

In line 6, Algorithm 3.2 computes the score of a model $MLM[VPTr]$, which was trained on the vulnerability-patch training dataset $VPTr$, on the standard testing dataset Te . When compared to the third score, this result serves as a measure of $MLM[VPTr]$'s ability to generalize back to the standard vulnerability detection setting.

The four computed scores are returned as the outputs of the algorithm ($output_{A2.1}$, $output_{A2.2}$, $output_{A2.3}$ and $output_{A2.4}$).

How can the results be used? Using Algorithm 3.2, researchers can effectively evaluate whether the high scores of ML4VD techniques are specific to the testing datasets on which they were computed. Specifically, for a selected technique researchers can answer the following questions:

1. Does the performance of the selected ML technique generalize from a standard vulnerability detection dataset to a modified setting, which requires to distinguish vulnerabilities from their patches? Answer: The selected ML technique can distinguish between vulnerabilities and their patches with performance $output_{A2.2}$, compared to a score of $output_{A2.1}$ on the standard testing dataset.

2. Does the performance of the selected ML technique generalize back to the standard vulnerability detection setting when it is explicitly trained to distinguish vulnerabilities from their patches? Answer: The selected ML technique achieves a score of $output_{A2.4}$ on the standard testing dataset, compared to a score of $output_{A2.3}$ in the modified setting.

3.4 EXPERIMENTAL SETUP

3.4.1 Research Questions

Our objective is to validate empirically, whether the two proposed algorithms can be used to evaluate state-of-the-art ML4VD techniques. Specifically, we aim to answer the following research questions.

- RQ.1 How is the performance of ML4VD techniques affected, if we augment the input code snippets without affecting the vulnerability labels?** (a) Can we measure a decrease in performance, if we augment the testing data of ML4VD techniques using semantics-preserving transformations? (b) Does training data augmentation using the same transformations restore the initial performance? (c) Are there differences between the individual transformations?
- RQ.2 Do ML4VD techniques overfit to specific augmentations of their training data that do not affect the respective vulnerability labels?** Can we still restore the performance, if we augment the training dataset with a different semantics-preserving transformation than the testing dataset?
- RQ.3 Are the high scores of ML4VD techniques specific to benchmark datasets or do they generalize to a modified vulnerability detection setting?** (a) Are state-of-the-art ML4VD techniques able to distinguish between vulnerable functions and their patches? (b) Does training to distinguish between vulnerable functions and their patches improve the performance on standard testing data?

3.4.2 Semantics-Preserving Transformations

One of the central components of algorithms 3.1 and 3.2 are semantics-preserving transformations of code. The most common semantics-preserving transformations that are used in the literature to investigate the limits of learned models for source code related tasks are identifier renaming [YAY20; Hen+22; Zha+20; Sri+21; Yan+22; Zha+22], insertion of unexecuted statements [YAY20; Zha+22; Hen+22; Sri+21], replacement of statements with equivalent statements [Li+22], reordering of unrelated statements [OJM22], deletion of unexecuted statements (e. g., comments) [Li+22], or combinations of the before mentioned [Hen+22; Sri+21; Zha+22].

Table 3.1 shows the 11 semantics-preserving transformations that we implemented for the experiments presented in this paper. We tried to cover all types of transformations commonly used in the

Table 3.1: The semantics-preserving transformations that we used in our experiments.

Identifier	Type	Description
t_1	Identifier Renaming	Rename all function parameters to a random token.
t_2	Statement Reordering	Reorder all function parameters.
t_3	Identifier Renaming	Rename the function.
t_4	Statement Insertion	Insert unexecuted code.
t_5	Statement Insertion	Insert comment.
t_6	Statement Reordering	Move the code of the function into a separate function.
t_7	Statement Insertion	Insert white space.
t_8	Statement Insertion	Define additional void function and call it from the function.
t_9	Statement Removal	Remove all comments.
t_{10}	Statement Insertion	Add code from training set as comment.
t_{11}	Random Transformation	One transformation sampled from $\{t_1, \dots, t_{10}\}$ is applied to each function.

literature. The table lists all transformations, categorizes them by type, and provides short descriptions for each of them.

Since ML4VD techniques are ultimately aimed at detecting security vulnerabilities in real-world code, augmenting code using our semantics-preserving transformations should also result in code that looks natural, or i. e., looks like it could occur in real-world software. To address this, we experimentally confirmed that our semantics-preserving transformations do not decrease the naturalness (measured by cross entropy) of the code. We provide more information on this as supplementary material in Appendix A.

3.4.3 Vulnerability Detection Datasets

We use two publicly available vulnerability detection datasets for our experiments.

CodeXGLUE/Devign. CodeXGLUE is a machine learning benchmark for code understanding and generation [Lu+21b]. It consists of several datasets for different source code related tasks. In our experiments, we only use the dataset for vulnerability detection, which is based on the Devign dataset [Zho+19]. Throughout this paper, we refer to this dataset as the *CodeXGLUE/Devign dataset* or just as the CodeXGLUE dataset. The CodeXGLUE dataset contains 26.4k C functions, from which 45.6% contain vulnerabilities, i. e., the dataset is fairly balanced. The types of vulnerabilities were not formally classified, but based on the collection process the authors found most vulnerabilities in the dataset to be memory-related, e. g., memory leaks, buffer overflows, memory corruption, or crashes. The authors of the CodeXGLUE benchmark also maintain a leaderboard [Lu+21a], which tracks the performance of popular learning-based techniques on the different datasets of the benchmark.

VulDeePecker. The other vulnerability detection dataset that we use in this paper is the Code Gadget Database, which was introduced with the VulDeePecker bug detector [Li+18]. We refer to this dataset as the *VulDeePecker dataset*. The original dataset contains 61.6k C/C++ code samples derived from the NVD [BRW13] and the SARD [Bla18],

from which 17.7k contain vulnerabilities, mainly buffer (CWE-119) and resource management errors (CWE-399).

3.4.4 New Dataset: *VulnPatchPairs*

In order to investigate the ability of ML4VD techniques to distinguish between vulnerabilities and their patches (RQ3), we collected a new dataset, which we call *VulnPatchPairs*. We provide a simple visualization of the collection process for *VulnPatchPairs* in Figure 3.4.

VulnPatchPairs is an extension of the CodeXGLUE/Devign vulnerability detection dataset [Zho+19], which consists of C functions from two popular open source repositories, FFmpeg⁴ and Qemu⁵. The creators of the dataset describe the collection process in their original publication [Zho+19]. As a first step, they filtered the selected repositories for security-related commits using a list of keywords. Then, they invested 600 work hours of a four-person team of security researchers to classify the security-related commits into vulnerability-fix commits (VFCs) and non vulnerability-fix commits (non-VFCs) and extracted the respective functions before the commits were applied as vulnerable (before VFCs) and non-vulnerable (before non VFCs) functions. The actual patched versions of the functions (after the VFCs were applied) are not part of their original dataset. However, for each function of their dataset, the authors released the respective commit ID from the two open-source repositories as additional information. We used this information to extract the actual patched versions of the vulnerable functions in the CodeXGLUE dataset from the FFmpeg and Qemu repositories and created a new dataset: *VulnPatchPairs*.

We manually verified 100 randomly chosen functions from the CodeXGLUE dataset that were labelled as vulnerable. We found 68 out of these 100 functions to actually contain a security vulnerability, 23 to contain no security vulnerability, and 9 with no decision after 10 minutes of manual effort. The results are available at https://github.com/niklasrisse/VPP_label_accuracy. Even though these results show that there are inaccurate labels in the CodeXGLUE dataset, we do not expect any changes to our main findings. While the absolute performance might change if we used perfectly labeled data, we expect the relative performance (e. g., augmented vs non-augmented) to remain comparable for all of our individual evaluations.

In total, *VulnPatchPairs* consists of 26.2k C functions from the two open source repositories FFmpeg and Qemu. Exactly half (13.1k) of the 26.2k functions contain security vulnerabilities and were copied from the CodeXGLUE/Devign vulnerability detection dataset. The other 13.1k are the respective patches of the vulnerable functions, which we extracted from the open-source repositories. We published *VulnPatchPairs* as supplementary material in an open GitHub repository⁶.

⁴ FFmpeg repository: <https://github.com/FFmpeg/FFmpeg>

⁵ Qemu repository: <https://github.com/qemu/qemu>

⁶ *VulnPatchPairs*: <https://github.com/niklasrisse/VPP>

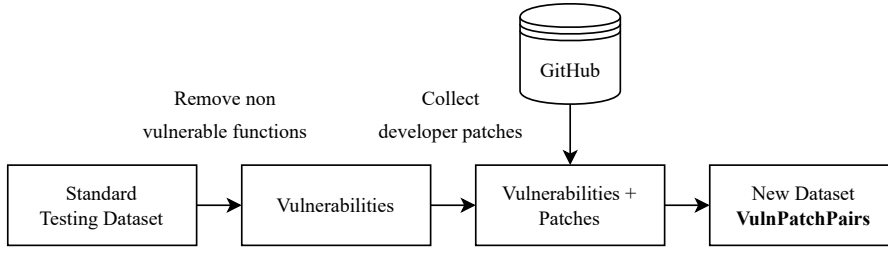


Figure 3.4: Visualization of the collection process for our new dataset VulnPatchPairs.

3.4.5 Machine Learning Techniques

We selected six state-of-the-art ML4VD techniques for our experiments.

Selection Criteria. In order to select techniques that represent the state-of-the-art of machine learning for vulnerability detection, we chose the top techniques from the CodeXGLUE leaderboard [Lu+21a] for which the authors provide open-source implementations. Measured by citations⁷ (496) and GitHub Stars⁸ (1.2k), CodeXGLUE is the most well-known benchmark for source code related machine learning techniques. The vulnerability detection dataset of the benchmark [Zho+19] is also highly cited (407 citations⁹) and widely used to evaluate ML4VD techniques for automatic vulnerability detection.

Selected Techniques. Based on the described criteria, we selected UniXcoder [Guo+22], CoTexT [Pha+21], VulBERTa [HM22a], PLBart [Ahm+21], and CodeBERT [Fen+20] for our experiments. At submission time of this paper, the six techniques hold rank 1 (UniXcoder), rank 2 (CoTexT), rank 6 (VulBERTa), rank 10 (PLBart), and rank 12 (CodeBERT) on the CodeXGLUE leaderboard for vulnerability detection [Lu+21a]. In addition to the five techniques from the CodeXGLUE leaderboard, we selected GraphCodeBERT (abbreviated as *GraphCB* in Table 3.3) [Guo+21], a technique related to CodeBERT, which utilizes graph representations of source code during the training process.

Model Details. Since the selected techniques belong to the same family of techniques, they also share the same basic architecture:

1. **Tokenization:** A given code function is split into tokens (small sequence of characters that forms a semantic entity), based on a given strategy.
2. **Embedding:** Tokens are transformed into numbers, usually by indexing via a learned vocabulary and the addition of positional information.
3. **Transformer Network:** Several steps of parametrized computation are applied, resulting in a final embedding.

⁷ <https://api.semanticscholar.org/CorpusID:231855531>

⁸ <https://github.com/microsoft/CodeXGLUE>

⁹ <https://api.semanticscholar.org/CorpusID:202539112>

4. Prediction Layer: The final layer of the model is a parametrized prediction layer, which computes an output number between 0 and 1 based on the final embedding of the previous step. The output number represents the predicted probability that the input function contains a security vulnerability.

During model training, the parametrized computational layers of steps 3. and 4. are optimized to compute the correct predictions for given training data. The six selected techniques mainly differ in tokenization strategy, training data, optimization objective, and the specific transformer network architecture used. We provide additional information on the specific models in Appendix C.

The authors of all six techniques provide publicly available implementations of their techniques [Guo+22; HM22a; Pha+21; Ahm+21; Fen+20; Guo+21].

3.4.6 Model Training Pipeline

We used a similar training setup for all model instances that we trained for our experiments.

Pre-training. All models that we train in our experiments have been pre-trained by the authors of the respective publications using various source code datasets. The size of the pre-training datasets spans from 2.3 million (VulBERTa) to 680 million code snippets (PLBart). The original publications provide more information on the pre-training datasets [Pha+21; HM22a; Ahm+21; Guo+22]. We use the pre-trained models released by the authors of the respective techniques as starting points for our pipeline and finetune the models on our selected datasets.

Data split. For the CodeXGLUE/Devign dataset, we used the train-/validation-/testing dataset split provided by the authors of the benchmark [Lu+21b], which resulted in a training dataset with 21k functions, a validation dataset with 2.7k functions, and a testing dataset with 2.7k functions. For the VulDeePecker dataset, we used the split provided by Hanif et al. [HM22a], which resulted in a training dataset with 128.1k functions, a validation dataset with 16k functions, and a testing dataset with 16k functions. The split for VulnPatchPairs is derived from the split of CodeXGLUE, such that *all and only* vulnerable functions in training, validation, and testing sets, respectively, of CodeXGLUE were taken as training, validation, and testing sets of VulnPatchPairs, augmented by their corresponding patches.

Pre-processing. For the VulDeePecker dataset, we removed all duplicate functions and also replaced all label-revealing tokens (e. g., comment with token "bad" above a vulnerable function) that we found by manual inspection of the dataset with randomly selected tokens. For the CodeXGLUE and VulnPatchPairs datasets, we applied no additional pre-processing steps.

Hyperparameters. For all six ML4VD techniques, we used the pre-trained models and tokenizers provided by the respective authors as

starting points for our experiments. Similar to Hanif et al. [HM22a], we noticed a relatively quick convergence of our performance metrics in our initial experiments on the validation dataset (after 2-7 epochs), which is why we trained each model instance for 10 epochs. For all model-specific hyperparameters, we used the values that were reported in the original papers [HM22a; Pha+21; Ahm+21; Guo+22; Fen+20; Guo+21]. Consult our published training scripts¹⁰ for the complete list of hyperparameter values that we used.

Performance metrics. We tracked and quantified the performance of our trained models on the selected testing datasets using several commonly used performance metrics. We report six metrics in this paper: Accuracy, F1-score, precision, recall, false positive rate (FPR), and false negative rate (FNR).

For CodeXGLUE/Devign as a balanced dataset (45.6% vulnerable functions), we use accuracy as the main performance metric, since it is also used exclusively in the CodeXGLUE benchmark [Lu+21b] and on the leaderboard [Lu+21a].

For VulDeePecker as a relatively imbalanced dataset (28.7% vulnerable functions) we use the F1-score as the main performance metric. The F1-score is defined as the harmonic mean of precision and recall and is most suitable when the positive class (in our case vulnerable functions) is the minority class in an imbalanced dataset.

Hardware. We used a setup of five Nvidia A100 GPUs, each equipped with 40 GB RAM. One run of all our experiments takes approximately 60 days on a single A100 GPU.

3.5 EXPERIMENTAL RESULTS

RQ.1 Testing- and Training Data Augmentation

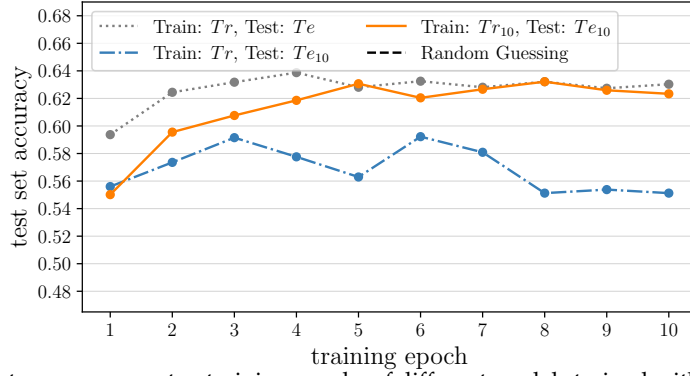
We investigate, whether (a) testing data augmentation using semantics-preserving transformation decreases the performance of state-of-the-art ML4VD techniques, whether (b) training data augmentation using the same transformations restores the performance towards previous levels, and whether (c) there are differences between the individual transformations.

Methodology. We used Algorithm 3.1 to investigate all three questions. We ran the algorithm for each ML technique and dataset separately and measured the outcomes using the respective preferred performance metrics (see Section 3.4.6). We did not only record the outcomes after completing the full training of the respective models but also after each training epoch in order to observe the progression of the learning process.

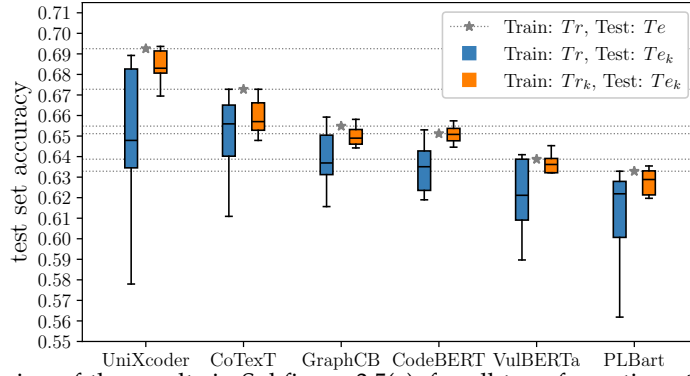
Results. Subfigure 3.5(a) shows the test set accuracy of different VulBERTa models measured after each of the ten training epochs. We can observe, that augmenting the testing dataset Te with transformation t_{10} leads to a substantial drop in accuracy, represented by the

¹⁰ GitHub: https://github.com/niklasrisse/USENIX_2024

UNCOVERING THE LIMITS OF MACHINE LEARNING FOR AUTOMATIC VULNERABILITY DETECTION



(a) Test set accuracy over ten training epochs of different models trained with VulBERTa on the CodeXGLUE/Devign dataset. Augmenting the testing set Te with transformation t_{10} (blue) decreases the accuracy, but applying the same transformation also to the training dataset Tr (orange) restores the accuracy back to previous levels.



(b) Extension of the results in Subfigure 3.5(a), for all transformations $t_k \in T$, and for all six ML4VD techniques. The boxplots represent distributions of the resulting accuracies. Augmenting the testing set Te with transformations $t_k \in T$ (blue boxplots) decreases the accuracy, but applying the same transformation also to the training dataset Tr (orange boxplots) partially restores the accuracy, although not always to its previous levels.

Figure 3.5: Effects of augmenting the testing data and the training data using the same semantics-preserving transformations.

gap between the dotted gray and blue graphs in the figure. We can also observe, that augmenting the training dataset Tr with the same transformation t_{10} as the testing dataset, restores the accuracy back to previous levels (orange graph).

Subfigure 3.5(b) extends the results of Subfigure 3.5(a) to all semantics-preserving transformations $t_k \in T$, and to all six ML4VD techniques. Instead of showing the accuracy for each training epoch, we only use the maximum accuracy across the full training. Across all six ML4VD techniques, we can observe, that augmenting the testing dataset Te with transformations $t_k \in T$ (represented by the blue boxplots), on average, leads to a drop in accuracy compared to evaluations on the standard testing dataset Te (represented by the horizontal lines with stars). In parallel to Subfigure 3.5(a) we can also observe that, on average, training data augmentation using the same transformation as for testing data augmentation leads to

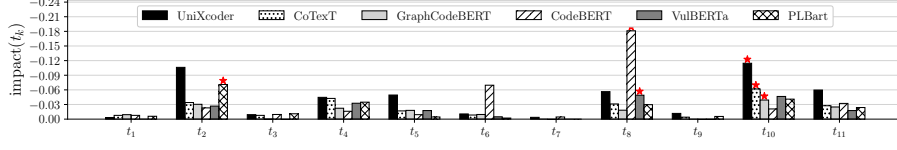


Figure 3.6: Impact on accuracy caused by augmenting the testing data using the individual transformations $t_k \in T$ ($impact(t_k) := accuracy[MLM[Tr], Te_k] - accuracy[MLM[Tr], Te]$). The most impactful transformations for each ML technique are marked by red stars.

a restoration of the observed drops in accuracy (represented by the orange boxplots).

Table 3.2 summarizes the outputs of Algorithm 3.1 for all six ML4VD techniques and both datasets. Specifically, it shows the average recorded changes in the respective metrics, when only the testing dataset was augmented (blue columns, $output_{A1.1}$), when the training and testing datasets were augmented using the same transformation (orange columns, $output_{A1.2}$), and when the training and testing datasets were augmented using a different transformation (green columns, $output_{A1.3}$). We observe that, on average, augmenting the testing dataset leads to a drop in accuracy/F1-score (-0.025 for CodeXGLUE, -0.043 for VulDeePecker), and augmenting the training dataset using the same transformation restores that decrease towards previous levels. On average, approximately 69.0% (CodeXGLUE) and 66.2% (VulDeePecker) of the lost accuracy/F1-score is restored.

Figure 3.6 shows the impact on accuracy caused by augmenting only the testing data using the individual transformations $t_k \in T$ ($impact(t_k) := accuracy[MLM[Tr], Te_k] - accuracy[MLM[Tr], Te]$) for all six ML4VD techniques. In other words, Figure 3.6 displays the severity of the performance decline of the techniques when only applying semantics-preserving transformations on the testing set. The most impactful transformations for each ML technique are marked by red stars.

If a specific semantics-preserving transformation has a high negative impact on the accuracy of an ML technique, we can assume that either (a) the ML technique partly relied on unrelated features that were removed or modified by the transformation (e.g., removal of comments) to achieve its original high accuracy, or (b) that the ML technique relied on unrelated features introduced by the transformation (e.g., addition of comments) to achieve the decreased accuracy after applying the transformation.

From the results presented in Figure 3.6 we can observe, that there are clear differences both between transformations and ML4VD techniques. As one might expect, a trivial transformation such as adding whitespace (t_7) has little to no impact on the accuracy of all six ML4VD techniques. The severity of this impact is, on average, below 1% accuracy, which means that below 1% of predictions are changed from correct to incorrect by applying this transformation. The ML4VD techniques also seem to be robust against identifier renaming (t_1 and t_3)

Table 3.2: Algorithm 1: Average changes when augmenting only the testing data ($output_{A1.1}$), training and testing data using the same ($output_{A1.2}$), or a different transformation ($output_{A1.3}$).

Metric	Technique	CodeXGLUE				VulDeePecker			
			$out_{A1.1}$	$out_{A1.2}$	$out_{A1.3}$		$out_{A1.1}$	$out_{A1.2}$	$out_{A1.3}$
		Tr Te	Tr Te_k	Tr_k Te_k	Tr_k $Te_{j \neq k}$	Tr Te	Tr Te_k	Tr_k Te_k	Tr_k $Te_{j \neq k}$
accuracy	UniXcoder	0.693	-0.043	-0.011	-0.050	0.975	-0.005	-0.002	-0.010
	CoTexT	0.673	-0.022	-0.014	-0.030	0.973	-0.001	-0.001	-0.003
	GraphCB	0.655	-0.015	-0.006	-0.021	0.973	-0.014	-0.002	-0.015
	CodeBERT	0.651	-0.034	-0.000	-0.040	0.974	-0.007	-0.002	-0.012
	VulBERTa	0.639	-0.017	-0.004	-0.025	0.973	-0.011	-0.002	-0.012
	PLBart	0.633	-0.021	-0.007	-0.026	0.972	-0.003	-0.002	-0.008
			-0.025	-0.007	-0.032		-0.007	-0.002	-0.010
F1-score	UniXcoder	0.680	-0.037	-0.007	-0.041	0.880	-0.028	-0.012	-0.054
	CoTexT	0.635	0.001	0.006	0.006	0.872	-0.006	-0.006	-0.020
	GraphCB	0.629	-0.024	-0.013	-0.033	0.869	-0.093	-0.006	-0.091
	CodeBERT	0.596	-0.005	0.012	0.001	0.873	-0.045	-0.007	-0.082
	VulBERTa	0.652	-0.050	-0.014	-0.048	0.873	-0.082	-0.009	-0.073
	PLBart	0.618	-0.009	-0.006	-0.016	0.865	-0.014	-0.007	-0.035
			-0.021	-0.004	-0.022		-0.045	-0.008	-0.059
recall	UniXcoder	0.787	-0.025	-0.016	-0.009	0.893	-0.034	-0.011	-0.041
	CoTexT	0.851	0.066	0.074	0.105	0.900	-0.004	-0.015	-0.029
	GraphCB	0.661	-0.023	-0.029	-0.031	0.895	-0.153	-0.005	-0.101
	CodeBERT	0.572	0.079	0.031	0.106	0.890	-0.090	-0.004	-0.103
	VulBERTa	0.759	-0.097	-0.029	-0.056	0.898	-0.094	-0.000	-0.073
	PLBart	0.658	0.025	0.003	0.024	0.884	-0.008	-0.003	-0.015
			0.004	0.006	0.023		-0.064	-0.006	-0.060
precision	UniXcoder	0.717	-0.033	0.020	-0.033	0.939	-0.008	0.030	-0.045
	CoTexT	0.684	-0.029	-0.027	-0.044	0.955	-0.015	0.013	-0.012
	GraphCB	0.734	-0.026	-0.019	-0.035	0.995	-0.016	-0.005	-0.040
	CodeBERT	0.847	-0.115	-0.009	-0.107	1.000	-0.000	-0.007	-0.031
	VulBERTa	0.643	-0.004	-0.019	-0.034	0.850	0.023	-0.012	-0.010
	PLBart	0.675	-0.031	-0.015	-0.049	0.971	-0.048	0.008	-0.068
			-0.040	-0.012	-0.051		-0.011	0.005	-0.034
FPR	UniXcoder	0.142	0.033	-0.018	0.049	0.006	0.001	-0.003	0.006
	CoTexT	0.211	0.018	0.028	0.034	0.004	0.001	-0.001	0.001
	GraphCB	0.079	0.040	-0.003	0.027	0.000	0.002	0.000	0.003
	CodeBERT	0.026	0.140	0.005	0.145	0.000	0.000	0.001	0.003
	VulBERTa	0.194	-0.036	0.028	0.040	0.018	-0.004	0.002	-0.001
	PLBart	0.137	0.016	0.020	0.026	0.003	0.004	-0.001	0.007
			0.035	0.010	0.053		0.001	-0.000	0.003
FNR	UniXcoder	0.213	0.025	0.016	0.009	0.107	0.034	0.011	0.041
	CoTexT	0.149	-0.066	-0.074	-0.105	0.100	0.004	0.015	0.029
	GraphCB	0.339	0.023	0.029	0.031	0.105	0.153	0.005	0.101
	CodeBERT	0.428	-0.079	-0.031	-0.106	0.110	0.090	0.004	0.103
	VulBERTa	0.241	0.097	0.029	0.056	0.102	0.094	0.000	0.073
	PLBart	0.342	-0.025	-0.003	-0.024	0.116	0.008	0.003	0.015
			-0.004	-0.006	-0.023		0.064	0.006	0.060

and removal of comments (t_9), for which the severity of impact is also below or close to 1%. The most impactful transformations are changing the order of the function parameters (t_2), defining an additional void function (t_8), and adding code snippets from the training set as comments (t_{10}). For these transformations, a substantial part of the predictions is changed from correct to incorrect: Between 2.3% and 10.6% for t_2 , between 1.8% and 18.1% for t_8 , and between 2.1% and 11.5% for t_{10} . Overall, transformations that insert statements (e. g., t_4 , t_5 , t_8 , and t_{10}) or reorder statements (e. g., t_2 and t_6) seem to have a higher impact than the other types.

Additionally, there are also differences between the six ML4VD techniques. For example, moving the code into a separate function (t_6) only seems to have a high impact on CodeBERT. Future work is required to determine why the ML4VD techniques are more or less robust against specific transformations.

We also investigated the impact of each individual transformation when not only the testing data but also the training data is augmented using a different transformation than for the testing data. However, due to the results being very similar to Figure 3.6, we decided to omit this from the paper and provide it as supplementary material in Appendix B.

Across two datasets, six ML4VD techniques, and 11 transformations, on average, (a) testing data augmentation using semantics-preserving transformations leads to a drop in accuracy/F1-score (CodeXGLUE: -0.025 , VulDeePecker: -0.043), (b) training data augmentation using the same transformations restores 69.0% (CodeXGLUE) and 66.2% (VulDeePecker) of the lost accuracy/F1-score, and (c) transformations that insert or reorder statements seem to be more impactful than other types of transformations.

RQ.1 has already been studied in the literature, for many different techniques, datasets, and tasks [Hen+22; Sri+21; YAY20; Li+21; Zha+22; Yan+22; Zha+20; BV20]. Based on our evidence, we can approve the findings of the literature.

RQ.2 Overfitting to Specific Transformations

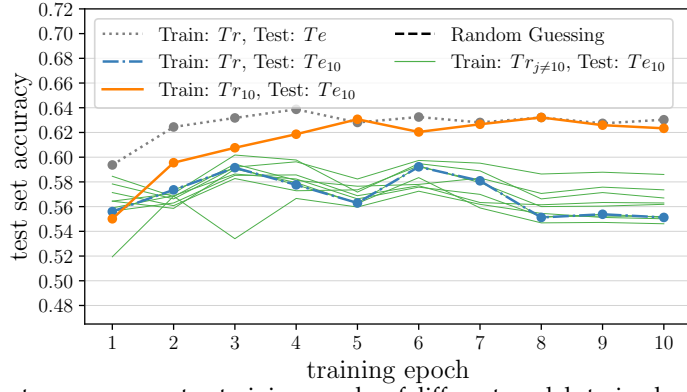
We investigate, whether the performance of ML4VD techniques can still be restored if we augment the training dataset with a different semantics-preserving transformation than the testing dataset. We use Algorithm 3.1 to investigate RQ.2, with the same setup as for RQ.1.

Results. Subfigure 3.7(a) is similar to Subfigure 3.5(a), it also shows the test set accuracies of different VulBERTa models measured after each of the ten training epochs. In addition to the results displayed in Subfigure 3.5(a), Subfigure 3.7(a) shows the accuracies (green lines) of VulBERTa models trained on data that was augmented using all transformations except t_{10} , which was used to augment the testing dataset. We observe, that augmenting the training dataset Tr with different transformations $t_{j \neq 10}$ as the testing dataset does not restore the accuracy back to previous levels.

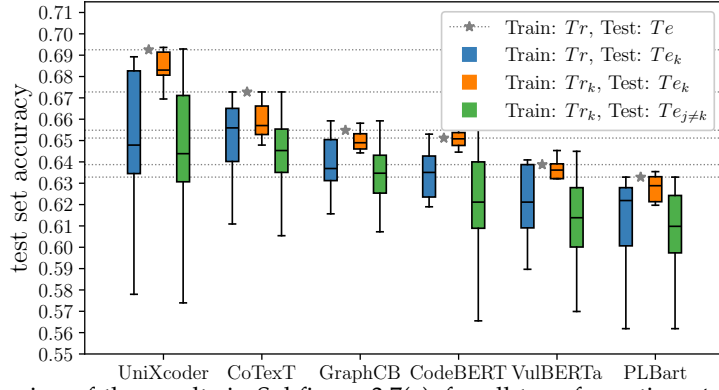
Subfigure 3.7(b) visualizes the same extended results for all semantics-preserving transformations $t_k \in T$ and for all six ML4VD techniques. Again, the blue and the orange boxplots represent the distributions of accuracies, when either only the testing dataset (blue) or training and testing datasets were augmented using the same transformation (orange). The green boxplots represent the distribution of accuracies achieved by models that were trained on data, which was augmented using a different transformation than for the testing data. Across all six ML4VD techniques, we observe that, on average, augmenting the training dataset Tr with a different transformation $t_{k \neq j}$ than the testing dataset does not restore the accuracy back to previous levels.

Subfigure 3.8(c) visualizes the same results as Subfigure 3.7(b), but using the VulDeePecker dataset. In this figure, the y-axis measures the

UNCOVERING THE LIMITS OF MACHINE LEARNING FOR AUTOMATIC VULNERABILITY DETECTION



(a) Test set accuracy over ten training epochs of different models trained with VulBERTa on the CodeXGLUE/Devign dataset. Augmenting the training set Tr with different transformations $t_{j \neq 10}$ than the testing dataset (green lines) does not restore the accuracy back to previous levels.



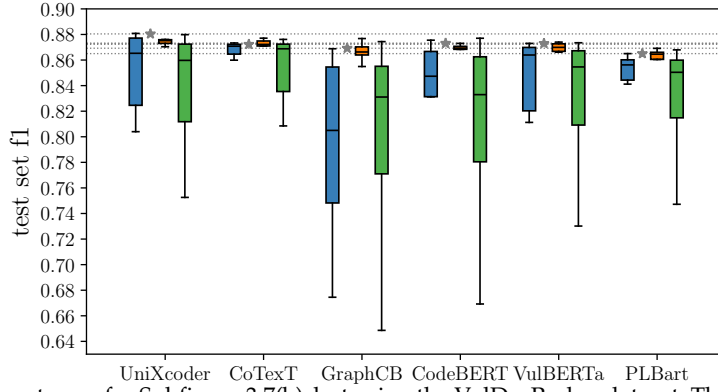
(b) Extension of the results in Subfigure 3.7(a), for all transformations $t_k \in T$ and for all six ML4VD techniques. The boxplots represent distributions of the resulting accuracies. Augmenting the training set Tr with different transformations $t_{k \neq j}$ than the testing dataset (green boxplots) does not restore the accuracy back to previous levels. Instead of restoring, the accuracy sometimes even drops further compared to using standard training data (green below blue).

Figure 3.7: Effects of augmenting the training data with different semantics-preserving transformations than the testing data.

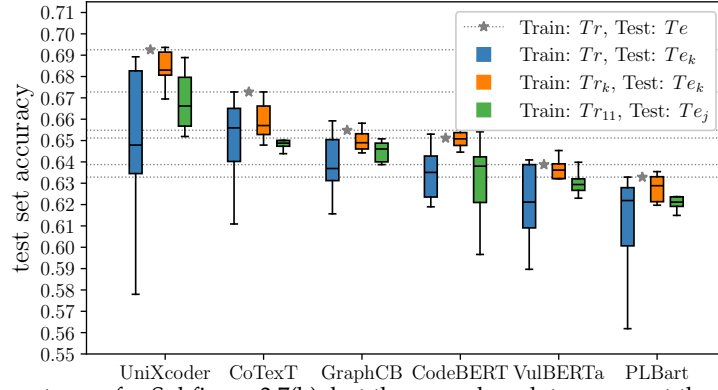
F1-score, since it is the preferred evaluation metric for the VulDeeP-ecker dataset. Again, we observe that, on average, augmenting the training dataset Tr with a different transformation $t_{k \neq j}$ as the testing dataset does not restore the accuracy back to previous levels.

In Subfigure 3.8(d) the green boxplots represent the distribution of accuracies achieved by augmenting the training data using our *meta transformation* t_{11} . Slightly different to the definition of t_{11} in Table 3.1, each function in the training set is transformed using a random transformation t_k with $k \in [1, 10] \setminus j$, with one left-out transformation t_j which is applied to the testing data. Since our set of implemented transformations contains groups of similar transformations (e.g., adding different types of comments), we would expect the accuracies to be higher compared to applying only a single different transformation to the training set (green boxplots of Subfigure 3.8(d)), but lower com-

3.5 EXPERIMENTAL RESULTS



(c) Same setup as for Subfigure 3.7(b), but using the VulDeePecker dataset. The boxplots represent distributions of the resulting F1-scores. Augmenting the training set Tr with different transformations $t_{k \neq j}$ than the testing dataset (green boxplots) does not restore the F1-score back to previous levels.



(d) Same setup as for Subfigure 3.7(b), but the green boxplots represent the accuracies achieved by augmenting the training data using the meta transformation t_{11} , in this case sampled from $\{t_1, \dots, t_{10}\} \setminus \{t_j\}$, and the testing data using a single left-out transformation t_j . Augmenting the training dataset Tr with t_{11} partially restores the accuracy, although not to its previous levels.

Figure 3.7: Effects of augmenting the training data with different semantics-preserving transformations than the testing data. (continued)

pared to applying exactly the same transformation to the training set (orange boxplots). Based on Subfigure 3.8(d), we observe that is the case across all six ML4VD techniques. Augmenting the training dataset Tr with the meta transformation t_{11} does not fully restore the accuracy, but moves it closer towards the accuracy on unaugmented data compared to applying only a single different transformation to the training set.

In addition to the results for RQ.1, Table 3.2 also shows the average recorded changes in the respective metrics, when the training and testing datasets were augmented using different transformations (green columns, $output_{A1.3}$). We observe that, on average, the score drops by 0.032 accuracy (CodeXGLUE) and 0.059 F1-score (VulDeePecker). Across the six techniques, the decrease is on average 30.2% (CodeXGLUE) and 77.5% (VulDeePecker) stronger than for training on unaugmented data. In other words, augmenting the training dataset

using a different transformation than for the testing dataset did not restore the score towards previous levels, but instead decreased it even further.

For the other metrics (recall, precision, FPR and FNR), we generally observe similar patterns than for accuracy and F1-score. However, there are also slight deviations, e. g., for CodeXGLUE recall improves on average by 0.023 when training data is augmented using a different transformation than the testing data instead of decreasing as expected. These deviations can be explained by innate tradeoffs between recall/precision and FPR/FNR, and can only be interpreted in context of the other metrics. Accuracy and F1-score provide a better summary of the performance, which is why they are used as the preferred metrics for the two datasets.

Across two datasets, six ML4VD techniques, and 11 transformations, augmenting the training dataset using a different transformation than for the testing dataset does not restore the performance back to previous levels. In other words, the ML4VD techniques overfit to the label-unrelated features introduced by semantics-preserving transformations during training data augmentation.

In summary, we can observe that across the tested ML4VD techniques, transformations, and datasets, training data augmentation only restores the performance to previous levels when the testing dataset is augmented in a similar way than the training dataset.

The performance gained by data augmentation only applies to the specific transformations used during the training of the model. ML4VD techniques continue to leverage unrelated features when deciding whether a function contains a security vulnerability.

RQ.3 Generalization to VulnPatchPairs

We investigate, whether (a) ML4VD techniques are able to generalize from typical vulnerability detection training datasets to a modified setting, in which they are required to distinguish between vulnerabilities and their patches, and (b) whether training to distinguish between vulnerabilities and patches improves the performance on standard testing data.

Methodology. We used Algorithm 3.2 to investigate both questions. As inputs to the algorithm, we selected the training and testing subsets of the CodeXGLUE/Devign dataset as the standard training and testing datasets Tr and Te , and the training and testing subsets of VulnPatchPairs as the vulnerability-patch training and testing datasets $VPTr$ and $VPTe$. We ran the algorithm for all six ML4VD techniques separately.

Results. Table 3.3 shows the results of running Algorithm 3.2. Specifically, it shows the performance of different models evaluated on the standard CodeXGLUE/Devign testing dataset Te or the

Table 3.3: Algorithm 2: Performance of six ML4VD techniques evaluated on the standard CodeXGLUE/Devign testing dataset Te or the vulnerability-patch testing dataset $VPTe$.

Metric	Technique	$out_{A2.1}$	$out_{A2.2}$	$out_{A2.3}$	$out_{A2.4}$
		Tr Te	Tr $VPTe$	$VPTe$ Test: $VPTe$	$VPTe$ Te
accuracy	UniXcoder	0.693	0.414	0.616	0.546
	CoTexT	0.673	0.503	0.607	0.575
	GraphCB	0.655	0.342	0.596	0.546
	CodeBERT	0.651	0.294	0.571	0.548
	VulBERTa	0.639	0.527	0.602	0.564
	PLBart	0.633	0.524	0.598	0.572
		0.657	0.434	0.598	0.559
F1-score	UniXcoder	0.680	0.582	0.662	0.613
	CoTexT	0.635	0.667	0.665	0.616
	GraphCB	0.629	0.508	0.654	0.603
	CodeBERT	0.596	0.455	0.629	0.613
	VulBERTa	0.652	0.610	0.651	0.615
	PLBart	0.618	0.583	0.633	0.575
		0.635	0.567	0.649	0.606
recall	UniXcoder	0.787	0.819	0.870	0.896
	CoTexT	0.851	1.000	0.975	0.941
	GraphCB	0.661	0.680	0.835	0.873
	CodeBERT	0.572	0.589	0.770	0.883
	VulBERTa	0.759	0.758	0.909	0.928
	PLBart	0.658	0.680	0.741	0.738
		0.715	0.754	0.850	0.876
precision	UniXcoder	0.717	0.452	0.668	0.518
	CoTexT	0.684	0.502	0.724	0.702
	GraphCB	0.734	0.406	0.622	0.509
	CodeBERT	0.847	0.371	0.656	0.516
	VulBERTa	0.643	0.531	0.781	0.647
	PLBart	0.675	0.535	0.663	0.547
		0.717	0.466	0.686	0.573
FPR	UniXcoder	0.142	0.816	0.107	0.172
	CoTexT	0.211	0.823	0.060	0.041
	GraphCB	0.079	0.840	0.091	0.177
	CodeBERT	0.026	0.849	0.102	0.233
	VulBERTa	0.194	0.312	0.034	0.061
	PLBart	0.137	0.251	0.138	0.213
		0.131	0.649	0.089	0.149
FNR	UniXcoder	0.213	0.181	0.130	0.104
	CoTexT	0.149	0.000	0.025	0.059
	GraphCB	0.339	0.320	0.165	0.127
	CodeBERT	0.428	0.411	0.230	0.117
	VulBERTa	0.241	0.242	0.091	0.072
	PLBart	0.342	0.320	0.259	0.262
		0.285	0.246	0.150	0.124

vulnerability-patch testing dataset $VPTe$. We focus our analysis on the results measured in accuracy since it is the preferred performance metric for balanced datasets such as CodeXGLUE and VulnPatchPairs.

We observe, that the accuracy of all six ML4VD techniques is highest (between 0.633 and 0.693) when trained and evaluated on standard training and testing data (second column, $output_{A2.1}$). This is expected and consistent with the findings in the literature [HM22a; Pha+21; Fen+20; Guo+21; Guo+22; Ahm+21]. When trained and evaluated on VulnPatchPairs (fourth column, $output_{A2.3}$) the accuracy is consistently lower than in the standard setting (between 0.558 and 0.617), but still significantly higher than the expected accuracy of a random guesser¹¹. However, when trained on standard training data and evaluated on the VulnPatchPairs testing dataset (third column, $output_{A2.2}$), the accuracy drops dramatically (between 0.294 and 0.527). Even the best model (VulBERTa) is only 0.027 points better than a random guesser. On

¹¹ Since $VPTe$ is perfectly class balanced (50% vulnerable, 50% clean), a random guesser (coin flip) would be expected to achieve an accuracy of 0.5.

average, the accuracy is worse than random guessing. In other words, all six ML4VD techniques that we evaluated are unable to distinguish between vulnerabilities and their patches when trained on a typical vulnerability detection dataset.

When trained on VulnPatchPairs and evaluated on standard testing data (fifth column, *output*_{A2.4}), we get a similar picture. The performance is significantly worse (between 0.546 and 0.575) compared to models trained on standard training data (second column). However, the performance in this case is notably better than random guessing.

(a) All six ML4VD techniques are not able to distinguish between vulnerabilities and their patches when trained on standard training data. On average, the accuracy is lower than the expected accuracy of a random guesser. (b) When trained to distinguish between vulnerabilities and their patches, the ML4VD techniques are able to predict standard testing data better than a random guesser, but still significantly worse than when trained on standard training data. In other words, the ML4VD techniques are unable to generalize from their training data to a slightly modified vulnerability detection setting.

3.6 THREATS TO VALIDITY

As for any empirical study, there are various threats to the validity of our results and conclusions.

Internal validity. A common source of systematic error in empirical studies on ML4VD techniques is hyperparameter selection. Given a particular desired outcome, hyperparameters can be optimized to move the result in the desired direction. We tried to minimize this risk by taking the values for hyperparameters provided by the authors of the chosen ML4VD techniques.

Another potential source of systematic error is the training-/testing dataset split. Similar to hyperparameter selection, dataset split can also be varied to change a result in a desired direction. We tried to avoid this risk by taking the provided splits of the CodeXGLUE benchmark [Lu+21b] and by Hanif et al. [HM22b].

External validity. The degree to which our results generalize to other learning-based techniques, vulnerability detection datasets, semantics-preserving transformations, and performance metrics, are concerns of external validity. We tried to minimize the risk attached to these concerns by evaluating a wide set of six state-of-the-art techniques, two datasets, six performance metrics, and 11 semantics-preserving transformations. For RQ.3, we only investigate the generalization between CodeXGLUE/Devign and VulnPatchPairs. To maximize generality, we tried to keep both Algorithm 3.1 and Algorithm 3.2 as general as possible, so that they can easily be adapted to other techniques, datasets, transformations, and metrics.

Simplicity of Transformations. Some of the semantics-preserving transformations that we used (see Table 3.1) could be easily addressed by adding additional data pre-processing (e. g., mapping identifiers to

standardized names). However, the specific transformations that we implemented are merely a tool to demonstrate, that the performance gained by training data augmentation only applies to the specific transformations used for training and that the techniques that we investigated overfit to the label-unrelated features introduced by these transformations. For a new technique, they could be replaced by a different set of transformations.

Class balance. Multiple works have shown that learning-based vulnerability detection techniques trained on fairly balanced datasets (such as CodeXGLUE) often fail to generalize to real-world code repositories [HBV22; Cha+21; Arp+22], which usually contain a much smaller ratio of security vulnerabilities [KS20]. However, measured by citations, class-balanced datasets are still by far the most popular datasets to evaluate learning-based techniques for vulnerability detection. To our current knowledge, there is no vulnerability detection dataset with sufficient size (more than 10k code snippets), high-quality labels (manually provided by security experts), and a realistic distribution of vulnerable to non-vulnerable code snippets that is widely used in the research community (at least 50 citations). This is why we decided to focus our experiments on the CodeXGLUE and VulDeePecker datasets, even though they do not reflect a realistic class distribution.

3.7 DISCUSSION AND FUTURE WORK

In summary, our results demonstrate that state-of-the-art ML4VD techniques overfit to the label-unrelated features that are introduced by semantics-preserving transformations during training and that ML4VD techniques are not able to generalize to a modified setting, in which they have to distinguish between vulnerabilities and their patches.

Overfitting of learned models is a well-known problem in the machine learning research field [Yin19; Die95]. However, as shown in our experiments, the traditional approach to evaluating ML4VD techniques often fails to detect overfitting to label-unrelated features in the training data. Our proposed Algorithm 3.1 is a novel way to measure overfitting of ML4VD techniques, that goes beyond the traditional approach, and can even detect overfitting if there is no gap in the standard setup at all. There are several common strategies to reduce overfitting in the standard evaluation setup, e. g., early-stopping, dropout, or large pre-training datasets [Yin19], which are already integrated in the ML4VD techniques that we used in our experiments. However, our experiments demonstrate that the techniques are still severely overfitting to label-unrelated features introduced by semantics-preserving transformations during training data augmentation. Finding ways to robustify ML4VD techniques without or with minimal overfitting will be a central challenge of the machine learning for vulnerability detection research area. We hope that our proposed algorithms can be used to understand the problem, to develop new approaches, and to track the progress in this direction.

Generalization. The results for RQ.3 (see Section 3.5) reveal, that state-of-the-art ML4VD techniques lack the ability to generalize from their training data to a modified setting, which requires to distinguish between vulnerabilities and their patches. Since we can not assume that real-world software systems would be similar to the training data of these techniques, the ability to generalize to modified settings would be required for these techniques to be safely integrated into real software engineering environments.

The ability of a ML technique to generalize to testing data that is differently distributed than the training data is also called *out-of-distribution generalization*, and the lack of it for learning-based techniques has been recently identified (e. g., in the computer vision domain [Hen+21; She+21]). Our proposed Algorithm 3.2 can be seen as a tool to measure out-of-distribution generalization for the domain of automatic vulnerability detection. It would be an interesting avenue for future work to try approaches that have been used to address out-of-distribution generalization in other domains (e. g., causal representation learning [Sch+21]) on the task of automatic vulnerability detection and measure the success using our Algorithm 3.2.

ACKNOWLEDGEMENTS

Special thanks to Lukas Pirch, Konrad Rieck, Kevin Borgolte, and Seongmin Lee for their constructive feedback on earlier versions of this paper.

Funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Germany’s Excellence Strategy - EXC 2092 CASA - 390781972.

REFERENCES

- [Ahm+21] Wasi Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. “Unified Pre-training for Program Understanding and Generation”. In: *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*. 2021. DOI: [10.18653/v1/2021.naacl-main.211](https://doi.org/10.18653/v1/2021.naacl-main.211).
- [APD21] Leonhard Applis, Annibale Panichella, and Arie van Deursen. “Assessing Robustness of ML-Based Program Analysis Tools using Metamorphic Program Transformations”. In: *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2021. DOI: [10.1109/ASE51524.2021.9678706](https://doi.org/10.1109/ASE51524.2021.9678706).

- [Arp+22] Daniel Arp, Erwin Quiring, Feargus Pendlebury, Alexander Warnecke, Fabio Pierazzi, Christian Wressnegger, Lorenzo Cavallaro, and Konrad Rieck. “Dos and don’ts of machine learning in computer security”. In: *Proceedings of the 31st USENIX Security Symposium (USENIX Security)*. 2022. URL: <https://www.usenix.org/conference/usenixsecurity22/presentation/arp>.
- [Bla18] Paul Black. “A Software Assurance Reference Dataset: Thousands of Programs With Known Bugs”. In: *Journal of Research of the National Institute of Standards and Technology* (2018). DOI: [10.6028/jres.123.005](https://doi.org/10.6028/jres.123.005).
- [BNM21] Guru Bhandari, Amara Naseer, and Leon Moonen. “CVE-fixes: Automated Collection of Vulnerabilities and Their Fixes from Open-Source Software”. In: *Proceedings of the International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE)*. 2021. DOI: [10.1145/3475960.3475985](https://doi.org/10.1145/3475960.3475985).
- [BRW13] Harold Booth, Doug Rike, and Gregory Witte. *The National Vulnerability Database (NVD): Overview*. Technical report. National Institute of Standards and Technology, 2013. URL: https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=915172.
- [BSF22] Quang-Cuong Bui, Riccardo Scandariato, and Nicolás E. Díaz Ferreyra. “Vul4J: A Dataset of Reproducible Java Vulnerabilities Geared towards the Study of Program Repair Techniques”. In: *Proceedings of the 19th IEEE/ACM International Conference on Mining Software Repositories (MSR)*. 2022. DOI: [10.1145/3524842.3528482](https://doi.org/10.1145/3524842.3528482).
- [BV20] Pavol Bielik and Martin Vechev. “Adversarial Robustness for Code”. In: *Proceedings of the International Conference on Machine Learning (ICML)*. 2020. URL: <http://proceedings.mlr.press/v119/bielik20a.html>.
- [Cha+21] Saikat Chakraborty, Rahul Krishna, Yangruibo Ding, and Baishakhi Ray. “Deep Learning based Vulnerability Detection: Are We There Yet”. In: *IEEE Transactions on Software Engineering* (2021). DOI: [10.1109/TSE.2021.3087402](https://doi.org/10.1109/TSE.2021.3087402).
- [Che+23] Yizheng Chen, Zhoujie Ding, Lamya Alowain, Xinyun Chen, and David Wagner. “DiverseVul: A New Vulnerable Source Code Dataset for Deep Learning Based Vulnerability Detection”. In: *Proceedings of the International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*. 2023. DOI: [10.1145/3607199.3607242](https://doi.org/10.1145/3607199.3607242).
- [Die95] Tom Dietterich. “Overfitting and Undercomputing in Machine Learning”. In: *ACM Computing Surveys* (1995). DOI: [10.1145/212094.212114](https://doi.org/10.1145/212094.212114).

- [Fen+20] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. “CodeBERT: A Pre-Trained Model for Programming and Natural Languages”. In: *Findings of the Association for Computational Linguistics: EMNLP (EMNLP)*. 2020. doi: [10.18653/v1/2020.findings-emnlp.139](https://doi.org/10.18653/v1/2020.findings-emnlp.139).
- [Guo+21] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou. “GraphCodeBERT: Pre-Training Code Representations with Data Flow”. In: *Proceedings of the International Conference on Learning Representations (ICLR)*. 2021. URL: <https://openreview.net/forum?id=jLoC4ez43PZ>.
- [Guo+22] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. “UniXcoder: Unified Cross-Modal Pre-training for Code Representation”. In: *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*. 2022. doi: [10.18653/v1/2022.acl-long.499](https://doi.org/10.18653/v1/2022.acl-long.499).
- [HBV22] Jingxuan He, Luca Beurer-Kellner, and Martin Vechev. “On Distribution Shift in Learning-based Bug Detectors”. In: *Proceedings of the International Conference on Machine Learning (ICML)*. 2022. URL: <https://proceedings.mlr.press/v162/he22a.html>.
- [Hen+21] Dan Hendrycks, Steven Basart, Norman Mu, Saurav Kadavath, Frank Wang, Evan Dorundo, Rahul Desai, Tyler Zhu, Samyak Parajuli, Mike Guo, Dawn Song, Jacob Steinhardt, and Justin Gilmer. “The Many Faces of Robustness: A Critical Analysis of Out-of-Distribution Generalization”. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2021. doi: [10.1109/iccv48922.2021.00823](https://doi.org/10.1109/iccv48922.2021.00823).
- [Hen+22] Jordan Henkel, Goutham Ramakrishnan, Zi Wang, Aws Albarghouthi, Somesh Jha, and Thomas Reps. “Semantic Robustness of Models of Source Code”. In: *Proceedings of the IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 2022. doi: [10.1109/saner53432.2022.00070](https://doi.org/10.1109/saner53432.2022.00070).
- [Hin+12] Abram Hindle, Earl T. Barr, Zhendong Su, Mark Gabel, and Premkumar Devanbu. “On the naturalness of software”. In: *Proceedings of the 34th IEEE/ACM International Conference on Software Engineering (ICSE)*. 2012. doi: [10.1109/ICSE.2012.6227135](https://doi.org/10.1109/ICSE.2012.6227135).

- [HM22a] Hazim Hanif and Sergio Maffei. “VulBERTa: Simplified Source Code Pre-Training for Vulnerability Detection”. In: *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*. 2022. DOI: [10.1109/IJCNN55064.2022.9892280](https://doi.org/10.1109/IJCNN55064.2022.9892280).
- [HM22b] Hazim Hanif and Sergio Maffei. *VulDeePecker Function-level dataset*. Dataset. 2022. URL: <https://github.com/ICL-ml4csec/VulBERTa/tree/main/data>.
- [KS20] Rafael-Michael Karampatsis and Charles Sutton. “How Often Do Single-Statement Bugs Occur? The ManySSuBs4J Dataset”. In: *Proceedings of the 17th IEEE/ACM International Conference on Mining Software Repositories (MSR)*. 2020. DOI: [10.1145/3379597.3387491](https://doi.org/10.1145/3379597.3387491).
- [Li+18] Zhen Li, Deqing Zou, Shouhuai Xu, Xinyu Ou, Hai Jin, Sujuan Wang, Zhijun Deng, and Yuyi Zhong. “VulDeePecker: A Deep Learning-Based System for Vulnerability Detection”. In: *Proceedings of the 25th Annual Network and Distributed System Security Symposium (NDSS)*. 2018. DOI: [10.14722/ndss.2018.23158](https://doi.org/10.14722/ndss.2018.23158).
- [Li+21] Zhen Li, Jing Tang, Deqing Zou, Qian Chen, Shouhuai Xu, Chao Zhang, Yichen Li, and Hai Jin. “Towards Making Deep Learning-based Vulnerability Detectors Robust”. In: *arXiv preprint* (2021). DOI: [10.48550/arXiv.2108.00669](https://doi.org/10.48550/arXiv.2108.00669).
- [Li+22] Yaoxian Li, Shiyi Qi, Cuiyun Gao, Yun Peng, David Lo, Zenglin Xu, and Michael R. Lyu. “A Closer Look into Transformer-Based Code Intelligence Through Code Transformation: Challenges and Opportunities”. In: *arXiv preprint* (2022). DOI: [10.48550/arXiv.2207.04285](https://doi.org/10.48550/arXiv.2207.04285).
- [Lu+21a] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, Ge Li, Lidong Zhou, Linjun Shou, Long Zhou, Michele Tufano, Ming Gong, Ming Zhou, Nan Duan, Neel Sundaresan, Shao Kun Deng, Shengyu Fu, and Shujie Liu. *CodeXGLUE leaderboards*. Leaderboard. 2021. URL: <https://microsoft.github.io/CodeXGLUE/>.
- [Lu+21b] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, Ge Li, Lidong Zhou, Linjun Shou, Long Zhou, Michele Tufano, Ming Gong, Ming Zhou, Nan Duan, Neel Sundaresan, Shao Kun Deng, Shengyu Fu, and Shujie Liu. “CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation”. In: *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS Datasets and Benchmarks)*. 2021. URL: <https://openreview.net/forum?id=61E4dQXaUcb>.

- [OJM22] Pedro Orvalho, Mikoláš Janota, and Vasco Manquinho. “MultiIPAs: applying program transformations to introductory programming assignments for data augmentation”. In: *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 2022. DOI: [10.1145/3540250.3558931](https://doi.org/10.1145/3540250.3558931).
- [Pha+21] Long Phan, Hieu Tran, Daniel Le, Hieu Nguyen, James Annibal, Alec Peltekian, and Yanfang Ye. “CoTexT: Multi-task Learning with Code-Text Transformer”. In: *Proceedings of the Workshop on Natural Language Processing for Programming (NLP4Prog)*. 2021. DOI: [10.18653/v1/2021.nlp4prog-1.5](https://doi.org/10.18653/v1/2021.nlp4prog-1.5).
- [Pon+19] Serena E. Ponta, Henrik Plate, Antonino Sabetta, Michele Bezzi, and Cédric Dangremont. “A Manually-Curated Dataset of Fixes to Vulnerabilities of Open-Source Software”. In: *Proceedings of the 16th IEEE/ACM International Conference on Mining Software Repositories (MSR)*. 2019. DOI: [10.1109/MSR.2019.00064](https://doi.org/10.1109/MSR.2019.00064).
- [Rab+21] Md Rafiqul Islam Rabin, Nghi D.Q. Bui, Ke Wang, Yijun Yu, Lingxiao Jiang, and Mohammad Amin Alipour. “On the generalizability of Neural Program Models with respect to semantic-preserving program transformations”. In: *Information and Software Technology* (2021). DOI: [10.1016/j.infsof.2021.106552](https://doi.org/10.1016/j.infsof.2021.106552).
- [Rah+24] Md Mahbubur Rahman, Ira Ceka, Chengzhi Mao, Saikat Chakraborty, Baishakhi Ray, and Wei Le. “Towards Causal Deep Learning for Vulnerability Detection”. In: *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. 2024. DOI: [10.1145/3597503.3639170](https://doi.org/10.1145/3597503.3639170).
- [Sch+21] B. Schölkopf, F. Locatello, S. Bauer, N. R. Ke, N. Kalchbrenner, A. Goyal, and Y. Bengio. “Toward Causal Representation Learning”. In: *Proceedings of the IEEE* (2021). DOI: [10.1109/JPROC.2021.3058954](https://doi.org/10.1109/JPROC.2021.3058954).
- [SHB16] Rico Sennrich, Barry Haddow, and Alexandra Birch. “Neural Machine Translation of Rare Words with Subword Units”. In: *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*. 2016. DOI: [10.18653/v1/P16-1162](https://doi.org/10.18653/v1/P16-1162).
- [She+21] Zheyang Shen, Jiashuo Liu, Yue He, Xingxuan Zhang, Renzhe Xu, Han Yu, and Peng Cui. “Towards Out-Of-Distribution Generalization: A Survey”. In: *arXiv preprint* (2021). DOI: [10.48550/arXiv.2108.13624](https://doi.org/10.48550/arXiv.2108.13624).

- [Sri+21] Shashank Srikant, Sijia Liu, Tamara Mitrovska, Shiyu Chang, Quanfu Fan, Gaoyuan Zhang, and Una-May O'Reilly. "Generating Adversarial Computer Programs using Optimized Obfuscations". In: *Proceedings of the International Conference on Learning Representations (ICLR)*. 2021. URL: https://openreview.net/forum?id=PH5PH9Z0_4.
- [Ste+23] Benjamin Steenhoek, Md Mahbubur Rahman, Richard Jiles, and Wei Le. "An Empirical Study of Deep Learning Models for Vulnerability Detection". In: *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE)*. 2023. DOI: [10.1109/ICSE48619.2023.00188](https://doi.org/10.1109/ICSE48619.2023.00188).
- [Vas+17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. "Attention is All You Need". In: *Advances in Neural Information Processing Systems (NIPS)*. 2017. URL: <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>.
- [Yan+22] Zhou Yang, Jieke Shi, Junda He, and David Lo. "Natural Attack for Pre-Trained Models of Code". In: *Proceedings of the 44th IEEE/ACM International Conference on Software Engineering (ICSE)*. 2022. DOI: [10.1145/3510003.3510146](https://doi.org/10.1145/3510003.3510146).
- [YAY20] Noam Yefet, Uri Alon, and Eran Yahav. "Adversarial Examples for Models of Code". In: *Proceedings of the ACM on Programming Languages* (2020). DOI: [10.1145/3428230](https://doi.org/10.1145/3428230).
- [Yin19] Xue Ying. "An Overview of Overfitting and its Solutions". In: *Journal of Physics: Conference Series* (2019). DOI: [10.1088/1742-6596/1168/2/022022](https://doi.org/10.1088/1742-6596/1168/2/022022).
- [Zha+20] Huangzhao Zhang, Zhuo Li, Ge Li, L. Ma, Yang Liu, and Zhi Jin. "Generating Adversarial Examples for Holding Robustness of Source Code Processing Models". In: *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*. 2020. DOI: [10.1609/aaai.v34i01.5469](https://doi.org/10.1609/aaai.v34i01.5469).
- [Zha+22] Huangzhao Zhang, Zhiyi Fu, Ge Li, Lei Ma, Zhehao Zhao, Hua'an Yang, Yizhe Sun, Yang Liu, and Zhi Jin. "Towards Robustness of Deep Program Processing Models—Detection, Estimation, and Enhancement". In: *ACM Transactions on Software Engineering and Methodology* (2022). DOI: [10.1145/3511887](https://doi.org/10.1145/3511887).
- [Zho+19] Yaqin Zhou, Shangqing Liu, Jingkai Siow, Xiaoning Du, and Yang Liu. "Devign: effective vulnerability identification by learning comprehensive program semantics via graph neural networks". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2019. URL: <https://proceedings.neurips.cc/paper/2019/hash/49265d2447bc3bbfe9e76306ce40a31f-Abstract.html>.

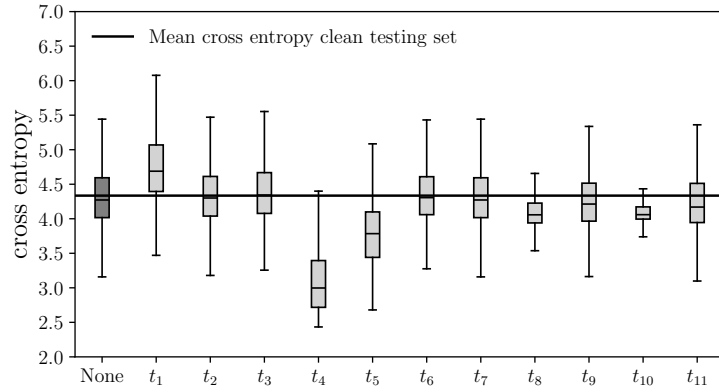


Figure 3.8: Naturalness of the semantics-preserving transformations, that we used in our experiments. Lower cross entropy means higher naturalness. All transformations except t_1 (identifier renaming) lead to lower or equal cross entropy than no transformation (None).

APPENDICES

A NATURALNESS

Since vulnerability detection techniques are ultimately designed to be applied to real-world code, we also need to ensure that our transformations lead to code snippets that could occur in the real world, or i.e., lead to *natural* code. We measured the naturalness of our transformations using the method introduced by Hindle et al. [Hin+12] (implemented as a 2-gram markov model) and present the results in Figure 3.8. Using the method introduced by Hindle et al., we can compute the *cross entropy* of a given code snippet, which represents how surprising or *unnatural* the code snippet is relative to the code snippets observed in the training dataset (for a detailed explanation consult the work of Hindle et al. [Hin+12]). Using this approach, we computed the cross entropy for all code snippets in the CodeXGLUE testing dataset (dark gray boxplot), and for transformed versions of the CodeXGLUE testing dataset (light gray boxplots). The horizontal black line represents the average cross entropy for all code snippets in the untransformed CodeXGLUE testing dataset. We can observe, that for all transformations except t_1 (identifier renaming), the cross entropy is similar or lower than for the untransformed dataset. In other words, all transformations except t_1 (identifier renaming) lead to code snippets that are similar in naturalness compared to the real-world code of the CodeXGLUE testing dataset.

B IMPACT OF INDIVIDUAL TRANSFORMATIONS

Figure 3.9 shows the impact on accuracy caused by augmenting the testing data with a different transformation than the training data ($impact(t_k) := \frac{1}{(N-1)} \sum_{t_j \in T} accuracy[MLM[Tr_j], Te] -$

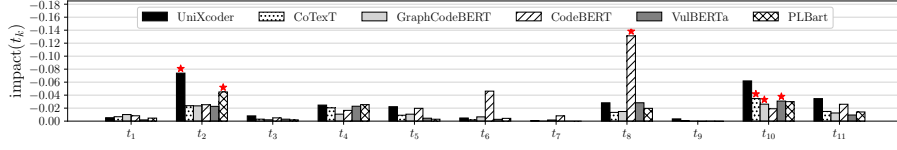


Figure 3.9: Impact on accuracy caused by augmenting the testing data with a different transformation than the training data ($impact(t_k) := \frac{1}{(N-1)} \sum_{t_j \in T} accuracy[MLM[Tr_j], Te] - accuracy[MLM[Tr_j], Te_k]$). The most impactful transformations for each ML technique are marked by red stars.

$accuracy[LLM[Tr_j], Te_k]$). The most impactful transformations for each LLM are marked by red stars. The results are very similar to Figure 3.6, which is why we chose to omit Figure 3.9 from the main paper and provide it as supplementary material.

C MODEL ARCHITECTURE DETAILS

All selected ML4VD techniques happen to be token-based large language models (LLMs), specialized for the task of vulnerability detection. LLMs are based on the transformer architecture, which is a neural network model architecture for sequence-to-sequence tasks based on the attention mechanism [Vas+17]. The attention mechanism is essentially a weighted dot product that allows models to focus on specific parts of the input data that are most relevant to the task at hand, improving their ability to capture dependencies and context. In a transformer model, the attention mechanism is combined with parametrized feed-forward layers and repeated multiple times, resulting in a complex multi-layer network. A detailed description of the transformer architecture can be found in the original paper [Vas+17]. While the base architecture is the same for all of the six techniques, there are some notable differences in the pre-training setup, size, or specific parts of the model training:

VulBERTa. VulBERTa leverages a custom tokenization strategy, which is based on the *byte pair encoding* algorithm [SHB16] combined with a set of pre-defined code tokens (standard C/C++ keywords, punctuation, and library API calls) to achieve better code encodings through maintaining the syntactical structure of source code.

CoTexT. CoTexT uses multi-task learning for pre-training, which means that the model is trained to perform multiple code-related tasks (e.g., vulnerability detection, code summarization, and code generation) in parallel. The model architecture also contains significantly more trainable parameters than the other techniques (222M parameters), because it contains more multi-head attention layers. CoTexT also uses a different tokenizer than the other techniques.

UniXcoder. UniXcoder introduces a new pre-training setup, which includes tokenization of the abstract syntax trees (ASTs) of the code, and three different ‘training modes’ that leverage different self-attention masks.

UNCOVERING THE LIMITS OF MACHINE LEARNING FOR AUTOMATIC VULNERABILITY DETECTION

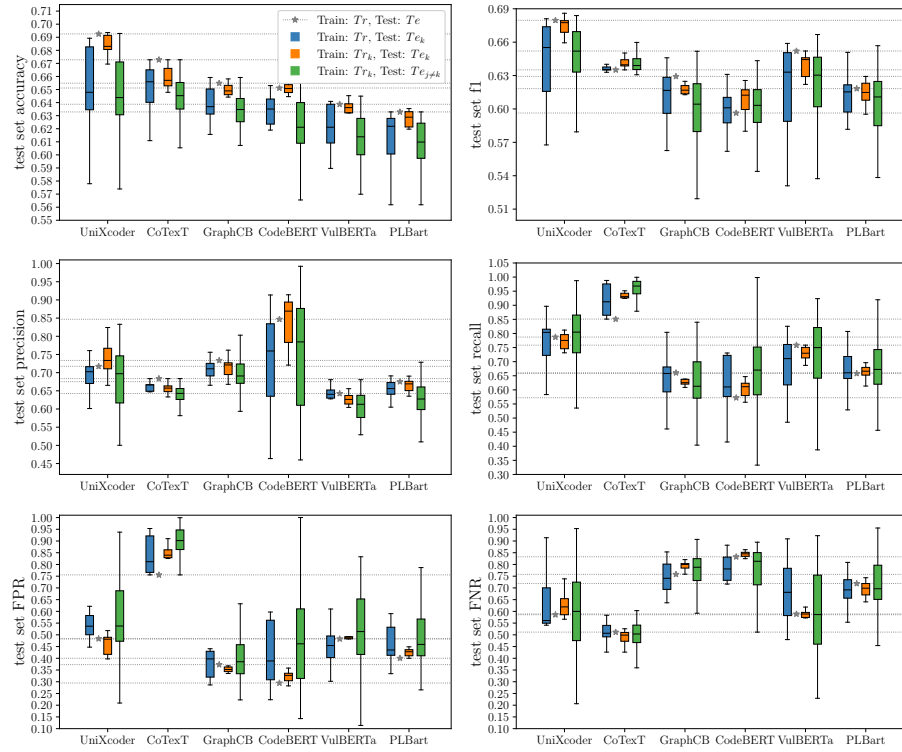


Figure 3.10: Additional metrics for RQ.2 using the CodeXGLUE/Devign dataset. The results support the conclusions that we generated based on the main metric (accuracy).

PLBart. PLBart uses a special pre-training procedure called ‘denoising autoencoding’, which is a combination of token masking, token deletion, and token infilling, that has to be reversed by the model.

CodeBERT. CodeBERT uses a pre-training setup in which natural language (e. g., documentation) and code are combined to produce a more semantically stable representation of the input.

GraphCodeBERT. GraphCodeBERT uses a pre-training task, where in addition to natural language and the code, a graph-based representation of the data flow of the function is provided.

D ADDITIONAL METRICS FOR RQ.2

Figure 3.10 and Figure 3.11 show additional results for RQ.2 using all of our available metrics (accuracy, F1-score, precision, recall, FPR, and FNR). Generally, the results support the conclusions that we generated based on the respective preferred metrics on the two datasets (accuracy for CodeXGLUE/Devign and F1-score for VulDeePecker). While the observed patterns deviate for some of the metrics (e. g., precision, recall, FPR, or FNR), these deviations can be explained by the relationships between them. For example, a model can have a really high precision, but low recall. Similarly, a model can have really low false-negative-rate, but the corresponding false-positive-rate is really high. Accuracy

D ADDITIONAL METRICS FOR RQ.2

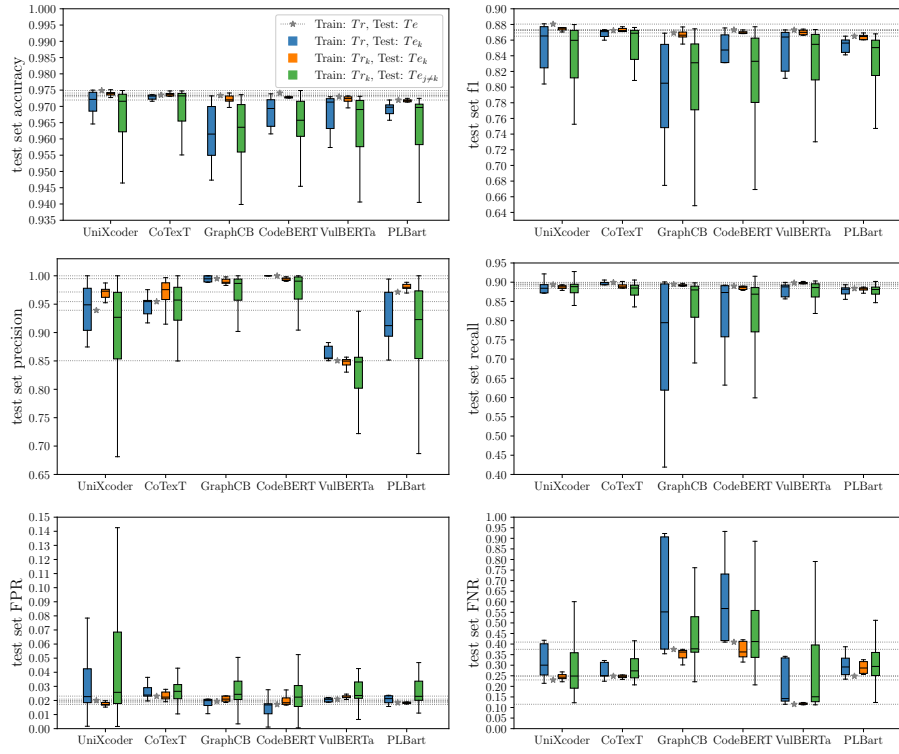


Figure 3.11: Additional metrics for RQ.2 using the VulDeePecker dataset. The results support the conclusions that we generated based on the main metric (F1-score).

and F1-score provide a better summary of the performance, which is why they are used as the preferred metrics for the two datasets.

This article was published at the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA), 2025. The full version is available at <https://dl.acm.org/doi/10.1145/3728887> and is reproduced here with permission.

Author Contributions. Niklas Risse served as the primary contributor for this work, leading the conceptualization, data curation, methodology, implementation, formal analysis, interpretation of results, original drafting, and final editing of the manuscript. Jing Liu contributed to the empirical study by labeling the 300 functions in parallel with Niklas Risse. Marcel Böhme supervised the research and provided guidance on conceptualization, methodology, and interpretation of results, as well as contributing to the review of the final text.

Formatting Changes. The content of the original publication is reproduced unchanged; only its presentation was adapted to match the formatting of this thesis, such as table styles and sizes as well as the citation style.

Original Publication

- [RLB25] Niklas Risse, Jing Liu, and Marcel Böhme. “Top Score on the Wrong Exam: On Benchmarking in Machine Learning for Vulnerability Detection”. In: *Proceedings of the ACM on Software Engineering* (ISSTA 2025). **ACM SIGSOFT Distinguished Paper Award**. DOI: [10.1145/3728887](https://doi.org/10.1145/3728887).

Top Score on the Wrong Exam: On Benchmarking in Machine Learning for Vulnerability Detection

Niklas Risse, Jing Liu, and Marcel Böhme

Abstract. According to our survey of machine learning for vulnerability detection (ML4VD), 9 in every 10 papers published in the past five years define ML4VD as a function-level binary classification problem:

Given a function, does it contain a security flaw?

From our experience as security researchers, faced with deciding whether a given function makes the program vulnerable to attacks, we would often first want to understand the context in which this function is called.

In this paper, we study how often this decision can really be made without further context and study both vulnerable and non-vulnerable functions in the most popular ML4VD datasets. We call a function “*vulnerable*” if it was involved in a patch of an actual security flaw and confirmed to cause the program’s vulnerability. It is “*non-vulnerable*” otherwise. We find that in almost all cases this decision *cannot* be made without further context. Vulnerable functions are often vulnerable only *because* a corresponding vulnerability-inducing calling context exists while non-vulnerable functions would often be vulnerable *if* a corresponding context existed.

But why do ML4VD techniques achieve high scores even though there is demonstrably not enough information in these samples? Spurious correlations: We find that high scores can be achieved even when only word counts are available. This shows that these datasets can be exploited to achieve high scores without actually detecting any security vulnerabilities.

We conclude that the prevailing problem statement of ML4VD is ill-defined and call into question the internal validity of this growing body of work. Constructively, we call for more effective benchmarking methodologies to evaluate the true capabilities of ML4VD, propose alternative problem statements, and examine broader implications for the evaluation of machine learning and programming analysis research.

4.1 INTRODUCTION

In recent years, the number of papers published on the topic of machine learning for vulnerability detection (ML4VD) has dramatically increased. Because of this rise in popularity, the validity and soundness of the underlying methodologies and datasets becomes increasingly

```

1 TfLiteStatus ResizeOutputTensors(TfLiteContext* context, TfLiteNode*
    node,
2                                     const TfLiteTensor* axis,
3                                     const TfLiteTensor* input, int
    num_splits) {
4     int axis_value = GetTensorData<int>(axis)[0];
5     // [...]
6     const int input_size = SizeOfDimension(input, axis_value);
7     TF_LITE_ENSURE_MSG(context, input_size % num_splits == 0,
8                         "Not an even split");
9     const int slice_size = input_size / num_splits;
10    for (int i = 0; i < NumOutputs(node); ++i) {
11        TfLiteIntArray* output_dims = TfLiteIntArrayCopy(input->dims);
12        output_dims->data[axis_value] = slice_size;
13        // [...]
14        TF_LITE_ENSURE_STATUS(context->ResizeTensor(context, output,
15                                                    output_dims));
16    }
17    return kTfLiteOk;
18 }

```

Figure 4.1: Context-dependent vulnerability (CVE-2021-29599) in DiverseVul dataset. If the function is called with `num_splits=0`, it crashes with a division-by-zero in Line 7.

important. So then, how exactly is the problem of ML4VD defined and thus evaluated?

In our survey of all ML4VD papers (81) published in the top Software Engineering and Security conferences and journals between January 2020 and December 2024,¹ we find that the great majority (88%) of state-of-the-art ML4VD techniques define ML4VD as a *function-level binary classification problem*: Given only the function f , decide whether f contains a security vulnerability or not. The technique with the lowest classification error on the evaluation dataset is considered the best at detecting security vulnerabilities.

However, based on our experience, we hypothesized that it might not always be possible to determine whether a function is vulnerable or not without additional context. We call these vulnerabilities *context-dependent*. Consider the example in Figure 4.1. If this function from the DiverseVul benchmark dataset [Che+23] is called with `num_splits` set to zero, it will crash with a division-by-zero in Line 7. However, without knowing whether this function can ever be called with `num_splits` set to zero, we cannot reliably decide if the division-by-zero could actually be observed. This function parameter might as well be properly validated by every caller function. This context-dependency problem is well-studied for other approaches to vulnerability detection, such as *static analysis* [Le+22; SHB19] (where it is handled, e.g., by defining suitable preconditions) or *software testing* [DR06] (which distinguishes, e.g., between system- and unit-level testing).

In this paper, we set out to quantify the prevalence of context-dependent vulnerabilities in the most popular datasets and end up revealing a fundamental flaw in the most-widely used evaluation methodology that is underpinning the progress of the nascent research area of ML4VD. We find that the vulnerability of a function cannot be

¹ Software Engineering conferences: ICSE, FSE, ASE, and ISSTA; Security conferences: S&P, USENIX, NDSS, and CCS; Software Engineering and Security journals: TSE, TOSEM, TDSC, and TIFS.

decided without further context for more than 90% of functions. This includes functions with both types of labels, `vulnerable` or `secure`: If the right context existed, a function labeled as `secure` would make the program vulnerable. Respectively, only *because* the right context exists, a function labeled as `vulnerable` makes the program actually vulnerable.

Given our findings, we conclude that the prevailing problem statement of ML4VD as a function-level classification problem is inadequate. The reported results in the literature, which are based on this problem statement, may not accurately reflect the true capabilities of the evaluated techniques at the task of vulnerability detection. In other words, there is currently no evidence that ML4VD techniques are actually capable of identifying security vulnerabilities at the function-level.

But why do ML4VD techniques still achieve high scores at this binary classification task when there is demonstrably not enough information in over 90% of samples (even after addressing label inaccuracies)? We identify spuriously correlated features as a potential reason. Training simple models, like a gradient boosting classifier using only word counts and disregarding code structure, we achieved results comparable to those of state-of-the-art ML4VD models. This suggests that ML4VD techniques only *appear* to perform well due to the chosen evaluation methodology. During classification, ML4VD techniques rely on spuriously correlated features to achieve high scores and do not genuinely detect vulnerabilities.

To shift the field towards more context-aware evaluation of vulnerability detection methods, we discuss potential alternative problem statements and suggest ideas for future work. We also examine the broader implications for ML4VD and other fields. By discussing these aspects we aim to foster more valid and reliable research in the area of machine learning for vulnerability detection.

In summary, this paper makes the following contributions:

- We analyze all papers published at the top Software Engineering conferences, Security conferences, and journals over the last five years and find that the great majority (88%) of state-of-the-art ML4VD techniques define ML4VD as a function-level classification problem.
- We reveal a fundamental flaw of the function-level classification problem: The vulnerability of a function cannot be decided without further context for more than 90% of functions in the top-most widely-used datasets.
- Why do ML4VD techniques still achieve high scores at the function-level classification problem? We demonstrate that they may rely on spurious features to achieve high scores without genuinely detecting vulnerabilities.

4.2 BACKGROUND

The context dependency problem—i. e., the determination whether a function causes a program to be vulnerable depends on the context that is *external* to the given function—has been well-studied in static analysis and software testing.

Static Analysis. We distinguish between inter-procedural and intra-procedural analysis, where the former is concerned with the analysis of the entire system and the latter with the analysis of individual functions. Intra-procedural static analysis tools often struggle with false positives due to context-dependency. This is a core problem in static analysis, which has received much attention in the literature [NNH99]. In practice, the context-dependency check is ultimately delegated to the user of the tool (which, however, similarly complicates the experimental evaluation in the absence of a user). A programmatic approach is to (manually or automatically) annotate the function with a precondition (e. g., in the style of Hoare logic [Hoa69]) that encodes assumptions about the *valid* state space at the start of the function call. Function summaries or type systems provide other mechanisms to encode the valid context under which the function is (meant to be) called. Despite these efforts, discerning the specific conditions under which a bug manifests remains a challenge. For example, a static analyzer might flag potential bugs based on the analysis of code patterns but cannot always discern the specific conditions under which a bug manifests [LBP22; Joh+13]. Hence, Le et al. [Le+22] propose to differentiate between manifest bugs, which are context-independent, and latent bugs, which depend on preconditions in the calling context. Manifest bugs can be reported without making any assumptions about the calling context, whereas latent bugs cannot.

Software Testing. We distinguish between system- and unit-level testing [BCR21], where the former is concerned with testing the entire system [Zhe+25; Men+24] and the latter with testing individual system units, like functions [GB25; BMC23]. Automated unit test generation often falls short of identifying issues that only occur under specific conditions or in particular environments. As noted by Harrold and Orso [HO08], unit tests can produce false positives when they are not adequately designed to account for the broader context in which a function operates. This can lead to an inflated number of reported issues that are not actual bugs, thereby complicating the debugging process. Hence, property-based testing [CH00; PLS19; Pad+19] requires users to define function preconditions in addition to assertions to ensure that the assertions fire only under *valid* function parameters (i. e., under valid context).

To summarize, both the static analysis and software testing research communities acknowledge the problem of context-dependency and address it through various techniques aimed at reducing the number of false positives.

Machine Learning (ML4VD). Several colleagues have previously raised concerns about the validity of this growing body of work. Ul-

lah et al. [Ull+24] study the performance of LLMs for vulnerability detection and find that results are nondeterministic, the reasoning is incorrect and unfaithful, and that they perform poorly in real-world scenarios. Risse and Böhme [RB24] find that the capability of state-of-the-art ML4VD techniques to distinguish between buggy and patched function is barely better than a coin flip. Croft et al. [CBK23] study the quality of benchmark datasets used in the evaluation of ML4VD techniques. They find that a large percentage of vulnerability labels in real-world datasets are in fact inaccurate and many data points were even duplicated (sometimes with inconsistent labels). In this paper, we reproduce their experiment which allows us to identify and study *actually* vulnerable functions in terms of their context-dependence. However, our question remains: Can the vulnerability of a function be decided without further context in the first place? Only if it can, we need to be concerned about the quality of our datasets and evaluation procedures.

Another stream of works studies the assumption that a vulnerability is localized within a single function. For instance, Sejfia et al. [Sej+24] realize that the patches of some vulnerabilities span across multiple “base units” (e.g., functions) and measure the performance of vulnerability detectors to *individually* classify *all* units belonging to the same vulnerability as vulnerable—observing a substantial drop in performance. They also recommend to ensure that the train/validation/test split honors the grouping of units by vulnerability. Li et al. [Li+24] study the default assumption that the vulnerability-*patching* functions are labeled as vulnerable and find that the patch and its “trigger” (i.e., the code where the vulnerability manifests, e.g., as an array out-of-bounds write, a double-free, etc.) exist in *different* functions for about a quarter of studied vulnerabilities.² If we labeled functions containing the vulnerability-*triggering* code as vulnerable, function-level vulnerability detectors would turn out much less effective, they find.

However, they still define ML4VD as a unit-level classification problem. How prevalent is the function-level problem statement for ML4VD in the literature (§4.3)? Is it a valid problem statement to begin with (§4.4, §4.5)? In answering these questions, our paper aims to uncover the shortcomings of current evaluation methodologies and emphasizes the urgent need for more context-aware approaches to accurately assess the true capabilities of ML4VD techniques in detecting security vulnerabilities.

Spurious features. If we find that the decision whether a function is vulnerable or not cannot be made without further context that is external to that function, then how do we explain the high values of classifier performance? Maybe a classifier learns to predict well based on features that are spuriously correlated with the vulnerable label. Indeed, the phenomenon of spurious correlations in ML4VD is well-studied. For instance, Arp et al. [Arp+22] provide initial evidence

² To facilitate this evaluation, they also introduce a slicing-based approach, VulTrigger, to identify the trigger *given the patch*.

that machine learning techniques can learn to predict labels correctly based on artifacts in code snippets without addressing the actual security task at hand (spurious correlation). Risse et al. [RB24; Ris23] demonstrate that ML4VD techniques overfit to label-unrelated features using semantics-preserving transformations and further that making the models robust against semantics-preserving transformation only introduces overfitting to the *specific* transformations used.

Furthermore, several studies have proposed ML4VD techniques that are aimed to be less affected by spurious correlations. For example, Rahman et al. [Rah+24] identify spuriously correlated variable and API names and disable their use during inference time of their model. Cao et al. [Cao+24] completely remove code that is *not* related to the vulnerability from the code of the vulnerable function during training by leveraging patch commit information. They develop a novel graph-based neural network architecture for the function-level vulnerability classification task to classify vulnerabilities based on causal features only.

While existing work studies and minimizes the impact of spurious correlations using structural representations or semantics-preserving transformations of the vulnerable code (i. e., perturbations on train and/or test data), we provide substantial evidence of this phenomenon using an even simpler and technique-independent method: A high measure of classifier performance is achieved even by a simple classifier if there are no program semantics but only word counts available as features. In this way, our work provides an orthogonal explanation of the surprising results of ML4VD even though the problem statement is ill-defined.

4.3 LITERATURE SURVEY

Based on our prior knowledge of the literature, we hypothesized that the majority of recent studies in machine learning for vulnerability detection (ML4VD) define ML4VD as a binary classification problem; given a function, determine whether the function contains a security vulnerability. To determine the prevalence of this approach in the ML4VD literature, we conducted a literature survey. Our goal was to address the following two research questions:

1. **Problem Statement:** What proportion of ML4VD publications defines ML4VD as deciding whether a given function contains a vulnerability?
2. **Datasets:** Which datasets do they use to evaluate their techniques empirically?

4.3.1 Methodology

To ensure a thorough and unbiased analysis of the recent literature on machine learning for vulnerability detection (ML4VD), we adopted

a systematic methodology based on established guidelines for conducting literature reviews in Software Engineering research [KC07; PVK15]. Our approach encompassed three primary phases: defining the scope, selecting relevant papers, and systematically analyzing the selected papers.

Scope. Our literature survey focuses on the past five years, targeting publications from 2020 to 2024. Given the prominence and impact of certain venues in the fields of Software Engineering and Security, we selected a set of the most impactful conferences and journals. For Software Engineering, we included International Conference on Software Engineering (ICSE), Foundations of Software Engineering (FSE), International Symposium on Software Testing and Analysis (ISSTA), and Automated Software Engineering (ASE). For Security, we incorporated Symposium on Security and Privacy (S&P), Network and Distributed System Security Symposium (NDSS), Computer and Communications Security (CCS), and USENIX Security Symposium (USENIX). Additionally, we considered the top journals in these domains, including Transactions on Software Engineering and Methodology (TOSEM), Transactions on Software Engineering (TSE), Transactions on Information Forensics and Security (TIFS), and Transactions on Dependable and Secure Computing (TDSC). These venues were chosen for their reputation in publishing high-quality and influential research in the Software Engineering and Security fields (CORE³ rank A* or A).

Criteria for Paper Selection. To identify relevant papers, we established clear selection criteria. The papers must be full-length papers published in the proceedings of the listed in-scope conferences or journals between 2020 and 2024. We do not include short papers (e. g., for PhD student competitions), since they usually represent work-in-progress. We also do not include pure literature reviews if they do not present new experimental data. The papers must fit the topic of machine learning for vulnerability detection (ML4VD), which means that they either propose a new ML-based technique for vulnerability detection, or that they present experiments which use existing techniques. We include all types of machine learning techniques, e. g., graph neural networks, large language models, or convolutional neural networks. We explicitly focus on vulnerability detection for software, which excludes related fields, e. g., hardware or blockchain vulnerability detection. To facilitate the paper identification process, we defined a set of keywords that are indicative of the research focus in this area: vulnerability, vulnerable, detect, detection, discovery, machine, learning, artificial, intelligence, AI, ML, deep, graph, neural, network, large, language, model. We scanned the titles and abstracts of all in-scope papers using multiple tools: IEEE Xplore Advanced Search⁴ for the IEEE venues (ICSE, ASE, S&P, TSE, TDSC, and TIFS), ACM DL Advanced Search⁵ for the ACM venues (ICSE, FSE, ASE, ISSTA, CCS, and TOSEM) and Google Scholar Advanced Search⁶ for the independent

³ CORE: <https://www.core.edu.au/home>

⁴ IEEE Xplore Advanced Search: <https://ieeexplore.ieee.org/search/advanced>

⁵ ACM DL Advanced Search: <https://dl.acm.org/search/advanced>

⁶ Google Scholar Advanced Search: <https://scholar.google.com/>

venues (NDSS and USENIX). This resulted in more than 2500 potential papers, which were flagged for further review.

Filtering Process. As a first step, we manually filtered the flagged titles to specifically exclude those that most likely do not fit the topic of ML4VD. One of the co-authors of this paper, a Software Security researcher with publications at the top Software Engineering and Security conferences on the topic of ML4VD, manually checked each of the paper titles to facilitate the first round of filtering. This resulted in a preliminary list of 109 papers, which we include as supplementary material to this paper. The author then reviewed each of the 109 abstracts and, where necessary, the full texts to assess their relevance based on our selection criteria. This resulted in a final list of 81 papers matching the selection criteria. The 28 excluded papers are either off-topic (e. g., vulnerability management) or do not leverage ML-based techniques (e. g., static analysis).

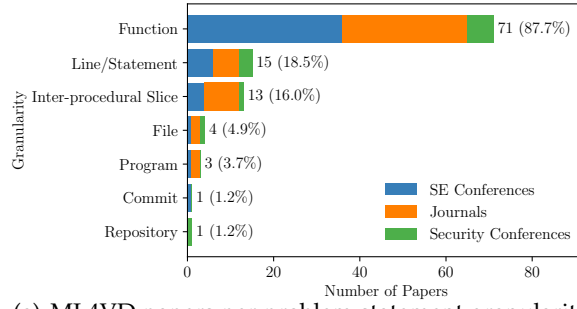
To answer research questions 1) and 2), we documented how ML4VD is defined, and the datasets used for empirical evaluation within each of the 81 identified ML4VD papers.

4.3.2 *Results*

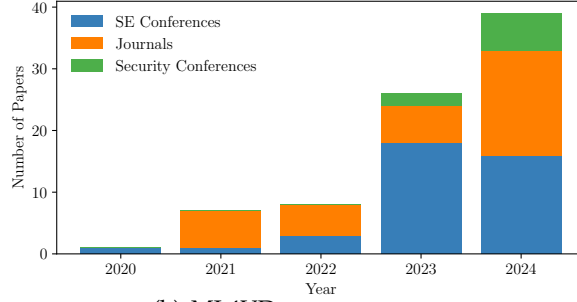
Our literature survey identified 81 papers that met our selection criteria and were included in our analysis. The full list of papers is part of the supplementary material of this paper. 39 papers were published in the Software Engineering conferences, 8 in the Security conferences, and 34 in the journals. The distribution of these papers across the years and conferences, as displayed in Subfigure 4.2(b), indicates a clear trend: the number of papers focused on ML4VD is increasing annually. This upward trajectory underscores the growing interest and significance of this research area within the Software Engineering and Security communities. Additionally, we found a large variety of machine learning techniques being employed, utilizing different data representations (graph-based, token-based), model architectures (e. g., large language models, graph neural networks, convolutional neural networks), and learning algorithms (e. g., contrastive learning).

Problem Statement. Subfigure 4.2(a) shows how the 81 papers included in our literature survey define the problem of ML4VD. To our surprise, the great majority (88%) of papers across the in-scope Software Engineering conferences (92%), Security conferences (75%) and journals (85%) define ML4VD as a binary function-level classification problem. This problem statement, which focuses on determining whether a given function contains a security vulnerability, has become the dominant problem statement in the field. The yearly trend seems to be increasing since 2021, with 95% of papers relying on a function-level problem statement in 2024 (2020: 100%, 2021: 57%, 2022: 75%, 2023: 88%, 2024: 95%). 15 out of 81 (19%) papers define ML4VD at the line-/statement-level and 13 out of 81 papers (16%) at the inter-procedural slice-level. A inter-procedural slice is a subset of a program’s statements affecting a particular computation, spanning

4.3 LITERATURE SURVEY



(a) ML4VD papers per problem statement granularity.



(b) ML4VD papers per year.

Figure 4.2: Literature survey results for the 81 ML4VD papers we identified in the top Software Engineering (SE) and Security conferences and journals. Subfigure 4.2(a) shows how the papers define the problem of ML4VD. Note that a paper may use multiple granularities, which explains why the numbers in Subfigure 4.2(a) do not add up to 100%. Subfigure 4.2(b) shows how many papers were published each year since 2020.

across multiple functions or procedures. Notably, the two most popular problem statements (function-level and line-/statement-level) exclusively rely on information within a single function for classification (intra-procedural). The remaining problem statements (file-, program-, commit-, and repository-level) all rely on inter-procedural information, but are only used by a minority of papers (10 out of 81).

Datasets. Regarding datasets, the majority of the papers relied on a limited set of popular datasets for empirical evaluation. Figure 4.3 shows all datasets that were used by the 81 papers and the number of times they were used. Specifically, BigVul [Fan+20] and Devign [Zho+19] were both used by 36 papers, and ReVeal [Cha+21] by 22 papers. Notably, 53 out of the 81 papers utilized at least one of these three datasets, reflecting their dominance in the field.

Literature Survey. The great majority of ML4VD papers (88%) published at top Software Engineering conferences (ICSE, FSE, ISSTA, and ASE), Security conferences (S&P, USENIX, NDSS, and CCS), and journals (TSE, TOSEM, TIFS, and TDSC) over the past five years define ML4VD as a binary classification problem: Given an isolated function f , decide whether f contains a security vulnerability. Moreover, three datasets—BigVul, Devign, and ReVeal—are overwhelmingly favored, with 53 out of the 81 papers using at least one of these datasets for empirical evaluation.

TOP SCORE ON THE WRONG EXAM: ON BENCHMARKING IN MACHINE LEARNING FOR VULNERABILITY DETECTION

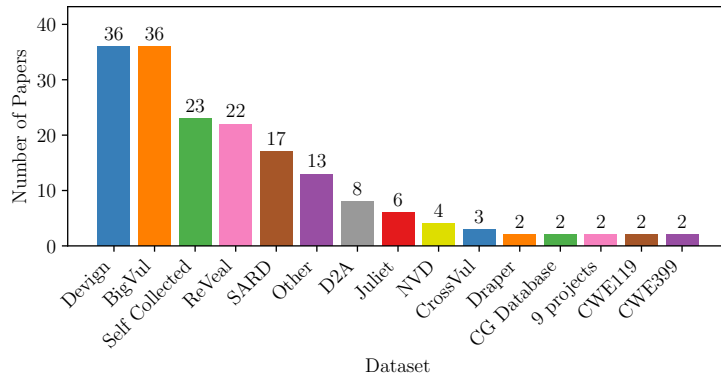


Figure 4.3: Datasets that are used by ML4VD papers published at the top Software Engineering and Security conferences and journals over the last five years. Datasets that were used only once are displayed as "Other".

4.4 EMPIRICAL STUDY DESIGN

We study how prevalent context-dependent vulnerabilities are in the top-most widely-used benchmarks for ML4VD and evaluate the degree to which we can correctly classify even when the vulnerable code is *hidden*, i.e., even when only coarse feature values such as word counts are available to a simple classifier. More generally, we are interested in the threats to validity of the benchmarking methodology that is used most-widely in ML4VD research: Are empirical claims from benchmarking results about the real-world performance of ML4VD techniques actually reliable?

Specifically, we ask the following research questions:

- **RQ.1 Can we cast vulnerability detection as a function-level binary classification problem in ML4VD?**
 - (a) **Noisy Labels.** *What proportion of functions labeled as vulnerable actually contain security vulnerabilities?* Before we can study the prevalence of context-dependency, we first address the noisy-label problem [CBK23] and identify those functions that are actually vulnerable.
 - (b) **Context-dependent Vulnerability.** *What proportion of vulnerable functions would not be vulnerable if the appropriate external context did not exist?* Given a function that is actually vulnerable (because it is later fixed to remove a vulnerability), how often can we decide vulnerability based on the function’s code alone?
 - (c) **Context-dependent Security.** *What proportion of non-vulnerable functions could be vulnerable if an appropriate external context would exist?* Given a function that is *not* vulnerable within the context of this program, how often can we find a setting in which this function would be considered to contain a vulnerability?
- **RQ.2 Can we achieve a high classification performance even when the root cause of the vulnerability is hidden?** How can we explain the excellent performance of ML4VD on these widely-used

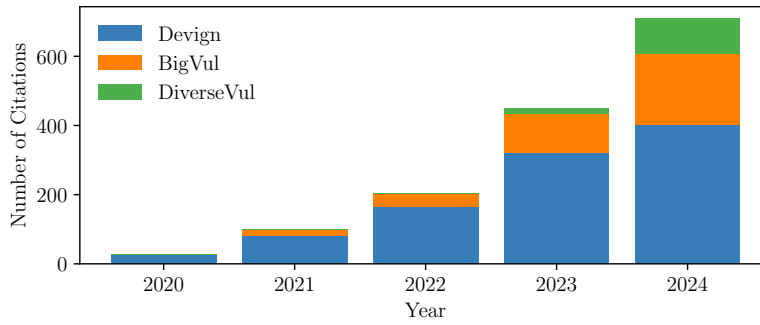


Figure 4.4: Popularity measured by citations of the datasets we selected for our empirical study.

benchmarks despite this severe flaw in the problem statement? Do the popular datasets contain properties that ML4VD techniques can exploit to achieve high scores without actually detecting vulnerabilities?

4.4.1 Methodology RQ.1

Selection Criteria. In order to select datasets that represent the state-of-the-art of ML4VD benchmarks, we chose the top-most widely-used datasets from the ML4VD literature (via our literature survey) for which the authors provide links to patch commits and/or CVE websites (required for manual labeling). Additionally, we reviewed unpublished literature for emerging datasets (pre-prints from arXiv⁷).

Selected datasets. Based on our selection criteria, we chose the BigVul [Fan+20], Devgin [Zho+19], and DiverseVul [Che+23] datasets. The selection of BigVul and Devgin resulted from our literature survey, as these datasets were the most popular among the 81 ML4VD papers we analyzed. However, we excluded ReVeal [Cha+21] from our study since the authors did not publish patch commit IDs or CVEs, which are necessary for determining vulnerability and context dependence. Additionally, we included the DiverseVul dataset [Che+23], a recently published dataset (2023) that has seen significant use in many yet unpublished papers based on our review of unpublished literature. Figure 4.4 illustrates the citations of the selected datasets measured by Google Scholar on January 16, 2025. All three datasets are becoming increasingly popular, with each having more than 100 citations in 2024 alone.

Sampling. From each of the three datasets, we randomly selected 100 samples labeled as vulnerable, using a sample size inspired by related studies on data quality, such as the one by Croft et al., which used a sample size of 70 [CBK23]. We have published the reproducible script and the sampled functions as part of our GitHub repository, which is available at <https://github.com/niklasrisse/TopScoreWrongExam>. The resulting 300 functions come from patch commits published

⁷ arXiv: <https://arxiv.org>

TOP SCORE ON THE WRONG EXAM: ON BENCHMARKING IN MACHINE LEARNING FOR VULNERABILITY DETECTION

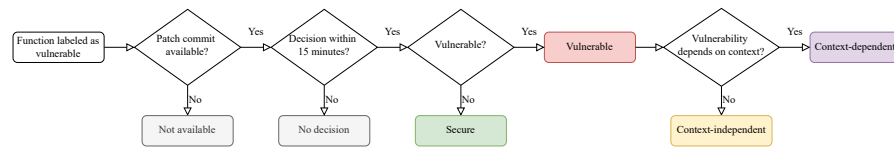


Figure 4.5: Empirical Study Design: Our manual labeling process to determine what proportion of security vulnerabilities in popular datasets can be detected without considering additional context beyond the function-level.

between 2010 and 2022, covering 80 unique open-source projects (BigVul: 25, Devign: 2, DiverseVul: 60). While we did not formally categorize the vulnerability types, we frequently observed issues such as out-of-bounds writes/reads, improper restriction of operations within memory buffer bounds, improper input validation, and use-after-free vulnerabilities.

Labeling Process. In a time-intensive process (more than 150 hours), two Software Security researchers (co-authors of this paper) reviewed each of the 300 functions independently. The process involved opening and understanding the corresponding patch commit on GitHub and, if available, reviewing the CVE (Common Vulnerabilities and Exposures) report for additional context.

Figure 4.5 visualizes the complete process that the two researchers employed for each individual function. In all cases where the patch commit was available, and a decision could be made within 15 minutes, they assigned one of the following labels:

- **Secure (0):** The function was not the source of the security vulnerability, or there was no vulnerability addressed by the patch commit.
- **Vulnerable (1):** The function was the source of the security vulnerability.

Additionally, they added a short explanation in natural language to justify the assigned label, which we used to analyze the results more in-depth. For the first labelling step (secure or vulnerable), the labels of the two researchers agreed for 82% of all functions, resulting in a Cohen Kappa value of 0.64 [Coh60], which implies substantial agreement according to the guidelines provided by Landis and Koch [LK77]. All cases of disagreement were resolved through discussion and resulted in a single label/explanation for each function. In the case that only one of the researchers was able to make a decision within 15 minutes, the researchers also entered a discussion for resolution, which resulted in a joint decision (vulnerable/secure) for all of these cases. Only if both researchers were unable to make a decision after 15 minutes, the final label was "No decision". In cases where the source of a vulnerability was ambiguous, e.g., if a vulnerability in a function could be fixed both by adding a check within the function (callee) or within a calling function (caller), the researchers only considered the callee to be vulnerable.

For functions labeled as vulnerable (indicating they actually were the source of the vulnerability), the two researchers conducted a second round of labeling to determine context dependency:

- **Context-independent (0):** The vulnerability could be detected without considering any additional context beyond the function itself. These vulnerabilities are self-contained within the function’s code.
- **Context-dependent (1):** The security vulnerability cannot be accurately identified without considering additional context beyond the function. This category includes vulnerabilities that rely on external functions, global variables, or interactions with other parts of the codebase.

Again, the researchers provided a short explanation in natural language to justify the context label. For the second labelling step (context dependence), the labels of the two researchers agreed for 98% of all functions, resulting in a Cohen Kappa value of 0.96 [Coh60], which implies almost perfect agreement according to the guidelines provided by Landis and Koch [LK77]. Again, all cases of disagreement were resolved through discussion and resulted in a single label/explanation for each function.

Labeling Expertise. Both Software Security researchers that carried out the labeling process have considerable experience in the Software Security research field (C/C++ security, in particular). The first researcher has three years of experience in Software Security research, published multiple papers on the topic of vulnerability detection at the top Security and Software Engineering conferences, and worked as a Software Engineer in industry for two years. The second researcher has five years of experience in Software Security, published multiple pre-prints on the topic (currently under submission at top conferences), and participated in discovering three CVEs, which are actually part of at least one of the three datasets we investigated for the empirical study.

Reproducibility. To ensure the reproducibility of our empirical study and to provide transparency in our research, we have made all related scripts and data publicly available. All resources can be accessed as part of our GitHub repository, which is available at <https://github.com/niklasrisse/TopScoreWrongExam>.

4.4.2 Methodology RQ.2

In order to test whether the three widely-used datasets (Devign, BigVul, and DiverseVul) contain spuriously correlated features that can be exploited to achieve a high performance without actually detecting security vulnerabilities, we trained a simple classifier (Gradient Boosting Classifier) to detect vulnerabilities based on word counts only, completely disregarding the structure of the code. After training, we

evaluated the resulting models on the evaluation subsets of the respective datasets, again based on word counts only.

Metrics. We selected the F1-score as our primary performance metric, since it is the dominant metric in the literature we surveyed for Section 4.3. Additionally, we report accuracy for the Devign dataset, because it is a relatively balanced dataset (47.9% vulnerable functions). Since both BigVul (4.5% vulnerable functions) and DiverseVul (5.4% vulnerable functions) are heavily imbalanced towards functions labeled as secure, using accuracy as a performance metric for model comparison could be misleading [HG09]. For example, in a dataset with 95.5% of functions labeled as secure (such as BigVul), a technique can achieve an accuracy of 95.5% by classifying all functions as secure. This is why we chose to omit accuracy for BigVul and DiverseVul.

Selected Techniques. Based on the literature we surveyed for Section 4.3, the state-of-the-art techniques for the Devign dataset are SNOPY [Cao+24] with 67.86% F1-score and PDBERT [Liu+24] with 67.61% accuracy. For the BigVul dataset, the state-of-the-art technique is DeepDFA [SGL24] with 96.46% F1-score. According to the benchmark provided by Chen et al. [Che+23], CodeT5 Small [Wan+21] is the best performing technique for the DiverseVul dataset with 48.28% F1-score.

4.5 RESULTS

The goal of our empirical study was to investigate whether vulnerability detection as a function-level binary classification problem is an adequate problem statement for ML4VD.

RQ.1-a Noisy Labels

The first decision for each function included in our empirical study was to determine whether the function, in fact, contains a security vulnerability. This is necessary since one of the main goals of our study is to determine the proportion of *actually* vulnerable functions that are dependent on additional context. From Croft et al. [CBK23], we know that for popular datasets only a subset of functions labeled as vulnerable actually contains security vulnerabilities. Specifically, they found at least 20% of labels for the Devign dataset and 45.7% of labels for the BigVul dataset to be inaccurate.

Results. Figure 4.6 shows the results of our empirical study. We were able to find the patch commit for all functions included in the empirical study, which is why the label "Not Available" was omitted from Figure 4.6. For RQ.1-a), we are interested in the proportion of functions labeled as vulnerable that actually contain security vulnerabilities, which are displayed in the left part of each subfigure (vulnerable vs. secure). Out of the 100 functions per dataset that were originally labeled as vulnerable, we found only 39%-65% to actually contain security vulnerabilities. 35%-61% do not contain security vul-

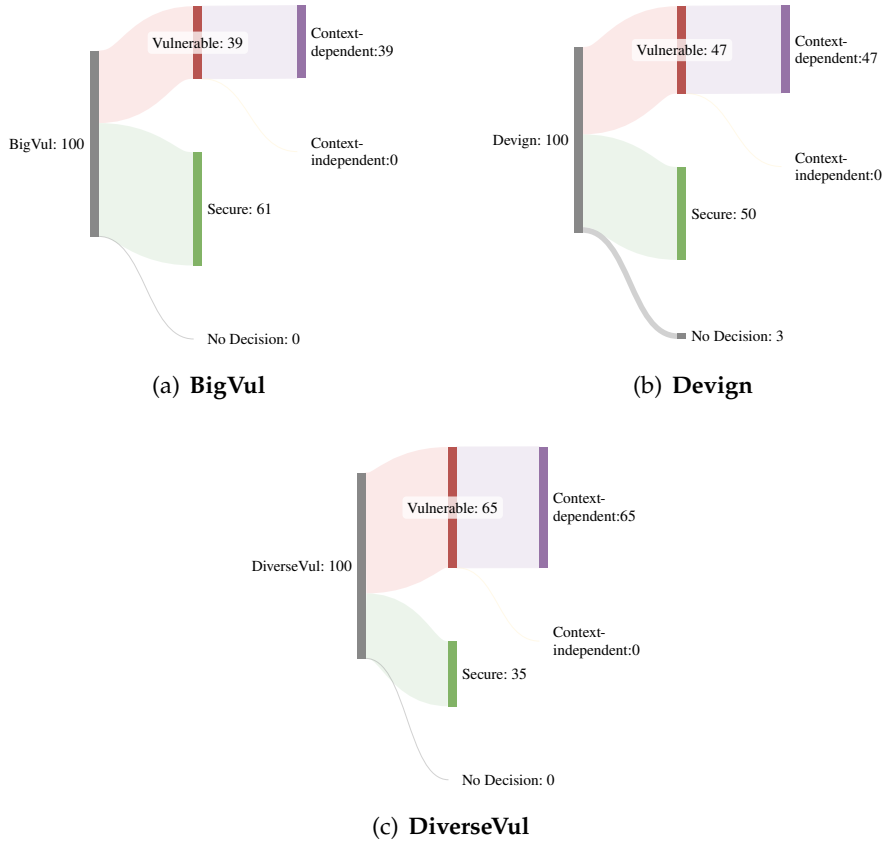


Figure 4.6: Empirical Study Results: For each dataset, we manually analyzed 100 functions that were labeled as vulnerable. For the subset of functions that were actually vulnerable (red), we then determined whether the vulnerability could be detected based on the function’s code alone. If yes, we marked the function as context-independent (yellow). Otherwise, as context-dependent (purple).

nerabilities and were therefore labeled to be secure. For 0%-3%, we were not able to make a decision after 15 minutes.

Noisy Labels (RQ1-a). Out of the 100 functions per dataset that were originally labeled as vulnerable, only 39%-65% (Devign: 47%, BigVul: 39%, DiverseVul: 65%) actually contain security vulnerabilities.

Based on our evidence, we can confirm the findings of Croft et al. [CBK23]. The differences in label accuracy, especially for the Devign dataset (80% accurate labels found by them vs. 47% found by us), might be explained by two reasons: First, Croft et al. considered label accuracy for *all* functions, and secure functions are more likely correctly labeled. In RQ.1-a, we only establish label accuracy for functions labeled as vulnerable. Second, Croft et al. establish label accuracy more conservative than us, where a function can be considered correctly labeled as vulnerable if it calls a function known to be vulnerable. In this case, we decided that only the actual vulnerable function should

Table 4.1: Reasons for Inaccurate Labels. The different reasons for inaccurate labels we observed and their prevalence in the three datasets.

Dataset	Patch Commit Identification	Structural Changes	Unrelated Changes
BigVul	13 (21%)	29 (48%)	19 (31%)
Devign	27 (54%)	19 (38%)	4 (8%)
DiverseVul	1 (3%)	23 (66%)	11 (31%)

be labeled as vulnerable because the calling function would not be vulnerable if the actual vulnerable function would be fixed.

Based on our quantitative results and the explanations we generated for each of the functions, we performed an in-depth analysis to determine potential reasons for the label inaccuracy we observed. The results are displayed in Table 4.1.

Patch Commit Identification. The first reason for label inaccuracy are errors during the process of identifying patch commits. From the original papers [Zho+19; Che+23; Fan+20], we know that all three datasets start their data collection process by identifying vulnerability-patching commits in popular open-source software repositories. However, the Devign dataset identifies these commits *only* by filtering the commit messages for security-related keywords. We observe that 54% of falsely labeled functions in our sample of the Devign dataset originate from this automatic identification process. The triangulation via other data sources (e. g., CVE databases) makes this reason occur less frequently for the other datasets (21% for BigVul and 3% for DiverseVul).

Structural Changes. The second reason for label inaccuracy is structural changes. All three datasets included in our study assume that all functions changed by a vulnerability-patching commit were vulnerable before the patch was applied. However, according to our results, only a subset of the functions changed by a patch commit is actually vulnerable, while other functions could be changed to address structural changes that are a consequence of fixing the actual vulnerable function. For instance, fixing a buffer overflow may require adding a buffer size parameter to the function call wherever the function is called. This can lead to false labels if all functions changed by the patch commit are considered to be vulnerable before the patch was applied. In fact, 38%-66% of falsely labeled functions can be attributed to structural changes.

Unrelated Changes. The third reason for label inaccuracy are other unrelated changes to functions in vulnerability patch commits. These include stylistic changes, e. g., removing whitespace or adding comments. According to our study, 8%-31% of falsely labeled functions can be attributed to such unrelated changes.

Croft et al. [CBK23] also investigate reasons for label inaccuracy and list irrelevant code changes (our structural changes), inaccurate fix identification (our patch commit identification), and clean-up changes (our unrelated changes). Based on our evidence, we can confirm these findings.

Table 4.2: Reasons for Context-dependence. The different types of context dependence we observed and their prevalence in the three datasets.

Dataset	Function Argument	External Function	Type Declaration	Globals	Execution Environment
BigVul	16 (41%)	19 (49%)	1 (2%)	3 (8%)	0 (0%)
Devign	22 (47%)	20 (43%)	0 (0%)	5 (10%)	0 (0%)
DiverseVul	26 (40%)	34 (52%)	2 (3%)	1 (2%)	2 (3%)

RQ.1-b Context-Dependent Vulnerability

The main goal of our empirical study was to find out what proportion of the functions labeled as vulnerable in the top-most widely-used datasets actually can be detected without considering additional context. In other words, what proportion of vulnerable functions would not be vulnerable if the appropriate external context did not exist?

Results. Each of the actual vulnerable functions that resulted from the first step of our empirical study (RQ.1 (a)) was assigned one of two labels: Context-independent or context-dependent. Figure 4.6 show the results of this second labeling round. To our surprise, all 151 vulnerable functions in our study required additional context to be accurately identified (context-dependent). Not a single function could be detected without considering any additional context beyond the function itself (context-independent)

***Context Dependence (RQ1-b).** All 151 vulnerable functions in our empirical study required additional context to be identified. Without this additional context, it was impossible to determine whether these functions actually contain security vulnerabilities.*

Based on the explanations we generated for each of the functions, we performed an in-depth analysis and identified the most prevalent types of context dependence in our sample. The results are shown in Table 4.2.

Dependence on External Functions. The first and most prevalent type of dependence we identified is dependence on external functions. Consider the example in Figure 4.7. The heap-based buffer overflow in lines 20-21 of this function from the BigVul dataset depends on the external function `cfg80211_find_vendor_ie`. Without knowing this external function, we do not know what values `vs_ie` can have, and consequently, we do not know whether the buffer overflow can ever be triggered. In our empirical study, 43%-52% of context-dependent vulnerabilities can be attributed to dependence on external functions.

Dependence on Function Arguments. The second type of dependence we identified is dependence on function arguments. Consider the example in Figure 4.8. The null-pointer-dereference vulnerability in line 8 of this function from the BigVul dataset depends on the function argument `sprinc`. However, without knowing the context in which this function can be called, we do not know whether `sprinc` can ever be NULL. For example, it could be properly validated before passing it to the function in all cases where this function is actually called. In that

TOP SCORE ON THE WRONG EXAM: ON BENCHMARKING IN MACHINE LEARNING FOR VULNERABILITY DETECTION

```

1 static int mwifiex_update_vs_ie(const u8 *ies, int ies_len,
2                                struct mwifiex_ie **ie_ptr, u16
3                                mask,
4                                unsigned int oui, u8 oui_type)
5 {
6     struct ieee_types_header *vs_ie;
7     struct mwifiex_ie *ie = *ie_ptr;
8     const u8 *vendor_ie;
9
10    vendor_ie = cfg80211_find_vendor_ie(oui, oui_type, ies, ies_len
11    );
12    if (vendor_ie) {
13        if (!*ie_ptr) {
14            *ie_ptr = kzalloc(sizeof(struct mwifiex_ie),
15                              GFP_KERNEL);
16            if (!*ie_ptr)
17                return -ENOMEM;
18            ie = *ie_ptr;
19        }
20        vs_ie = (struct ieee_types_header *)vendor_ie;
21        memcpy(ie->ie_buffer + le16_to_cpu(ie->ie_length),
22              vs_ie, vs_ie->len + 2);
23        le16_unaligned_add_cpu(&ie->ie_length, vs_ie->len + 2);
24        ie->mgmt_subtype_mask = cpu_to_le16(mask);
25        ie->ie_index = cpu_to_le16(MWIFIEX_AUTO_IDX_MASK);
26    }
27    *ie_ptr = ie;
28    return 0;
29 }

```

Figure 4.7: Dependence on external functions: The heap-based buffer overflow (CVE-2019-14816) in lines 20-21 of this function from the BigVul dataset depends on the external function `cfg80211_find_vendor_ie`. Without knowing this external function, we do not know what values `vs_ie` can have, and consequently, we do not know whether the buffer overflow can ever be triggered.

case, there would be no vulnerability. In our empirical study, 40%-47% of context-dependent vulnerabilities can be attributed to dependence on function arguments.

Dependence on Type Declarations. The third type of dependence we identified is dependence on type declarations. Consider the example in Figure 4.9. The integer overflow vulnerability in line 20 depends on the type declaration of `PP_Flash_MenuItem`. Only if `sizeof(PP_Flash_MenuItem) * menu->count` exceeds the range of `menu->items`, the operation overflows. In our empirical study, 0-3% of context-dependent vulnerabilities can be attributed to dependence on type declarations.

Dependence on Globals. The fourth type of dependence we identified is dependence on globals, such as macros and global variables. Consider the example in Figure 4.10. Line 7 of this function from the Devign dataset is the cause of multiple buffer overflows in the Jasper repository (CVE-2014-8158). If `HAVE_VLA` is not defined and `QMFB_JOINBUFSIZE` is smaller than `bufsize` but still used without successful dynamic allocation, any attempt to use `joinbuf` for storing data will result in writing beyond its allocated size (`QMFB_JOINBUFSIZE`). In our empirical study, 2-10% of context-dependent vulnerabilities can be attributed to dependence on globals.

```

1 setup_server_realm(krb5_principal sprinc)
2 {
3     krb5_error_code    kret;
4     kdc_realm_t        *newrealm;
5
6     kret = 0;
7     if (kdc_numrealms > 1) {
8         if (!(newrealm = find_realm_data(sprinc->realm.data, (
9             krb5_ui_4) sprinc->realm.length)))
10             kret = ENOENT;
11         else
12             kdc_active_realm = newrealm;
13     }
14     else
15         kdc_active_realm = kdc_realmlist[0];
16     return(kret);
17 }

```

Figure 4.8: Dependence on function argument: The null pointer dereference (CVE-2013-6800) in line 8 of this function from the BigVul dataset depends on the function argument `sprinc`. Only if `sprinc=NULL` it crashes with a null pointer dereference in line 8.

```

1 PP_Flash_Menu* ReadMenu(int depth, const IPC::Message* m,
2     PickleIterator* iter) {
3     if (depth > kMaxMenuDepth)
4         return NULL;
5     ++depth;
6
7     PP_Flash_Menu* menu = new PP_Flash_Menu;
8     menu->items = NULL;
9
10    if (!m->ReadUInt32(iter, &menu->count)) {
11        FreeMenu(menu);
12        return NULL;
13    }
14
15    if (menu->count == 0)
16        return menu;
17
18    menu->items = new PP_Flash_MenuItem[menu->count];
19    memset(menu->items, 0, sizeof(PP_Flash_MenuItem) * menu->count);
20    for (uint32_t i = 0; i < menu->count; ++i) {
21        if (!ReadMenuItem(depth, m, iter, menu->items + i)) {
22            FreeMenu(menu);
23            return NULL;
24        }
25    }
26    return menu;
27 }

```

Figure 4.9: Dependence on type declaration: The integer overflow (CVE-2013-0892) in line 20 of this function from the BigVul dataset depends on `PP_Flash_MenuItem`, `menu->items` and `menu->count`. Only if `sizeof(PP_Flash_MenuItem) * menu->count` exceeds the range of `menu->items`, the operation overflows.

Dependence on the Execution Environment. The fifth type of dependence we identified is dependence on the execution environment. For example, a vulnerability may depend on the fulfillment of specific conditions in the system of the user (e. g., the presence of a file in a directory) outside of the code. In our empirical study, 0-3% of context-dependent vulnerabilities can be attributed to dependence on the execution environment.

```

1 void jpc_qmfb_join_col(jpc_fix_t *a, int numrows, int stride,
2 int parity)
3 {
4
5     int bufsize = JPC_CEILDIVPOW2(numrows, 1);
6     #if !defined(HAVE_VLA)
7     jpc_fix_t joinbuf[QMFB_JOINBUFSIZE];
8 #else
9     jpc_fix_t joinbuf[bufsize];
10 #endif
11     jpc_fix_t *buf = joinbuf;
12
13     // [...]
14
15 }
```

Figure 4.10: Dependence on globals: Line 7 of this function is the cause of multiple buffer overflows in the JasPer repository (CVE-2014-8158). Whether line 7 is actually compiled depends on whether the macro `HAVE_VLA` is defined.

RQ.1-c Context-dependent Security

Now we know that the vulnerability of all 151 vulnerable functions in our empirical study could not be decided without considering additional context, but what about secure functions? What proportion of functions labeled as secure—because they are not *known* to cause a vulnerability in the program—could potentially cause a vulnerability if an appropriate external context existed?

Methodology. From each of the three datasets (BigVul, Devign, and DiverseVul), we randomly selected 30 samples that were labeled as secure and checked if we could come up with a reasonable context for that function that would cause a vulnerability in the program. We note that we did not attempt to verify whether a function that was labeled as secure was *actually* not causing a vulnerability in the program. There can only be certainty about the *presence* of vulnerabilities (e. g., via a triggering input). In practice, there is no certainty about their *absence*. However, we publish the reproducible script and the sampled functions as part of our artifact for full transparency.

To find out what proportion of secure functions could be vulnerable if an appropriate external context would exist, we tried to construct an artificial vulnerable context for each of them. Consider the example in Figure 4.11. In this artificially crafted context, the function `HTTP_Clone` (marked in green) contains a null pointer dereference vulnerability. However, it is labeled as secure in the DiverseVul dataset because in its original context⁸ it is not known to contain a security vulnerability. Just looking at that function itself, without any external context, we cannot decide whether this function does not actually cause a vulnerability in the program. Similar to this example, we manually tried to construct a vulnerable context for all 90 functions in our sample. For full transparency, we include the generated vulnerable settings (as explanations in natural language) in our artifact.

⁸ <https://github.com/varnishcache/varnish-cache/commit/c5fd097e>

Results. For 82 out of the 90 functions that were labeled as secure in the original datasets, we were able to construct a context in which they contain a security vulnerability.

***Secure Functions (RQ1-c).** 82 out of 90 secure functions contain context-dependent vulnerabilities, i. e., they would make the program vulnerable if the appropriate external context existed. In isolation, it is not possible to determine whether these functions are actually secure.*

RQ.1 Result Summary

Within our sample of functions, about half of those *labeled as vulnerable* did not actually contain the security vulnerabilities they were claimed to have. All of those that do are context-dependent vulnerabilities, i. e., these functions are vulnerable only *because* an appropriate context exists under which the function is vulnerable. Similarly, the majority of secure functions contain context-dependent vulnerabilities that would make that function vulnerable if an appropriate context existed. As there is evidently insufficient information in the base units—that are used for training, validation, and testing in these datasets—to decide their vulnerability without further context, we conclude that ML4VD cannot be soundly evaluated as a classic binary classification problem on the function-level.

***Problem statement (RQ.1).** ML4VD techniques cannot be soundly evaluated as a classic binary classification problem on intra-procedural base units, such as functions.*

RQ.2 Classifier Performance on Spurious Features

Since it is impossible for most functions to decide without further context whether they contain a vulnerability, there is currently no evidence that ML4VD techniques are actually capable of identifying security vulnerabilities in functions. But why do ML4VD papers still report high scores when evaluating their techniques using function-level datasets? What happens if we hide the code and only expose some features of the code, such as word counts?

Results. Table 4.3 shows the results for RQ.2. The Gradient Boosting Classifier achieved a F1-score of 86% on the evaluation subset of BigVul, a F1-score of 62.2% and an accuracy of 63.2% on the evaluation subset of Devign, and a F1-score of 11.1% on the evaluation subset of DiverseVul. For Devign and BigVul, the observed performances are only 4.4% (Devign, accuracy), 5.7% (Devign, F1-score), and 10% (BigVul) lower than the performances of recently published state-of-the-art techniques. Surprisingly, for an effective vulnerability detection model, the whole process of training and evaluation only took 30 minutes on a MacBook Pro, which is extremely fast compared

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct http {
5     int vsl;
6     int ws;
7 };
8
9 void HTTP_Dup(struct http *to, const struct http *fm) {
10     return;
11 }
12 void HTTP_Clone(struct http *to, const struct http *fm) {
13     HTTP_Dup(to, fm);
14     to->vsl = fm->vsl;
15     to->ws = fm->ws;
16 }
17
18 int main() {
19     struct http *source = NULL;
20     struct http destination;
21
22     HTTP_Clone(&destination, source);
23
24     return 0;
25 }
```

Figure 4.11: Context-dependent security: In this artificially crafted context, the function `HTTP_Clone` contains a null pointer dereference vulnerability. However, it is labeled as ‘secure’ in the DiverseVul dataset because in its original context (see <https://github.com/varnishcache/varnish-cache/commit/c5fd097e>) it is not known to contain a security vulnerability.

to training times of state-of-the-art ML-based techniques. State-of-the-art ML-based techniques require expensive Hardware (GPUs), weeks of computing time to pre-train, and at least multiple hours of computing time to finetune. Additionally, the Gradient Boosting Classifier was only trained on the training subset of the two datasets without any pre-training. These results show that it is possible to achieve high performance on the most popular function-level datasets while completely disregarding the structure and semantics of the code.

For the DiverseVul dataset, the Gradient Boosting Classifier was not able to achieve a comparable performance to the state-of-the-art techniques. This suggests that the dataset may contain fewer exploitable spurious features. However, this is not definitive proof. What the result does show is that state-of-the-art performance cannot be reached using a Gradient Boosting Classifier based solely on word count features.

Spurious Features (RQ2). *Using word counts only, we were able to achieve 62.2% F1-score on the Devign dataset and 86% F1-score on the BigVul dataset with a simple Gradient Boosting Classifier. These results show that the top-most widely-used function-level datasets can be exploited to achieve high scores without actually detecting security vulnerabilities.*

Our results provide an alternative explanation for the results reported in the literature. While we did not prove that state-of-the-art ML4VD techniques actually achieve their high scores by relying on spuriously correlated features, we have shown that it is possible to exploit these datasets to achieve high scores without actually detecting security vulnerabilities.

Table 4.3: Spurious Correlations. The performance of a simple model (Gradient Boosting Classifier) trained on word counts only (no information on code structure and/or semantics available during training).

Dataset	State-of-the-art	Without Structure
BigVul (F1-score)	0.96	0.86
Devign (F1-score)	0.68	0.62
DiverseVul (F1-score)	0.48	0.11
Devign (accuracy)	0.68	0.63

4.6 THREATS TO THE VALIDITY

As for any empirical study, there are various threats to the validity of our results and conclusions.

4.6.1 Internal validity

Selection Bias. The random sampling of 100 functions from each dataset (BigVul, Devign, and DiverseVul) for our empirical study could introduce selection bias. Although random sampling aims to create a representative subset, it is possible that our sample may not fully capture the diversity and characteristics of the entire dataset. To mitigate this, we ensured that our sampling method was strictly random and publish the reproducible script and the sampled functions as part of our GitHub repository: <https://github.com/niklasrisse/TopScoreWrongExam>.

Manual Labeling Errors. Labeling functions as vulnerable or secure and distinguishing between context-dependent and context-independent vulnerabilities involves subjective judgment. To address this, we employed a cross-labeling process where all 300 functions were independently labeled by two Software Security researchers, achieving substantial agreement (82%) for labeling functions as vulnerable or secure and almost perfect agreement (98%) for distinguishing between context-dependent and context-independent vulnerabilities. Discrepancies were resolved through discussion, but some human error may still be present. For full transparency, we publish all labels and explanations as part of our artifact.

4.6.2 External validity

Datasets. Our study focuses on three specific datasets (BigVul, Devign, and DiverseVul), which are widely used in the ML4VD community (cf. Fig. 4.3). However, the findings may not generalize to other datasets or real-world software systems. Future studies should replicate our methodology on additional datasets and real-world codebases to validate our conclusions.

Programming languages. The datasets analyzed primarily contain C code. Our findings might not generalize to other programming languages with different syntactic and semantic properties. Future

research should include datasets from various programming languages to evaluate the broader applicability of our results.

Problem Statements. Our study focuses on the most prevalent problem statement of ML4VD as function-level binary classification problem (cf. §4.3 and Fig. 4.2) where the classifier makes the decision exclusively based on information within the given function. However, there exist other definitions of ML4VD (e. g., at other granularities, such as line- or file-level, and even some that explicitly consider context, such as vulnerability-specific inter-procedural slicing [Wu+23]). While we discuss our perspective for other definitions of ML4VD in Section 4.7, future studies should carry out similar investigations to validate how reasonable these problem statements are indeed.

4.6.3 Construct Validity

Spurious Correlations. Our results suggest that ML4VD techniques might achieve high scores by exploiting spurious correlations rather than genuinely detecting vulnerabilities. While we demonstrated this using word counts, further research should investigate other features that might be *spuriously* correlated with the `vulnerable` label. We discuss some related work in Section 4.2.

4.7 DISCUSSION AND FUTURE WORK

The goal of empirical evaluation in ML4VD is to assess the capabilities of specific techniques by putting them to the test on real-world data. However, if the evaluation is based on incorrect assumptions, the results become meaningless. Our findings demonstrate that this is precisely the issue with ML4VD. Techniques can achieve ‘top scores’ by exploiting spurious features, even without possessing the capabilities the test aims to measure. In our specific case, the only plausible explanation for the good performance of ML4VD techniques are spurious correlations.

Implications for general ML. Are we truly measuring the effectiveness of ML techniques in solving the specific tasks we expect them to solve? The issue of spurious correlations is not limited to ML4VD but extends to other areas of machine learning, where it has been explored in depth by several studies [Izm+22; Sag+20]. It is crucial to continue the development of methods to identify and measure these correlations during the benchmarking of ML techniques to ensure that we are genuinely evaluating their performance in solving the tasks they are intended to address.

Implications for Program Analysis. The prevalence of the context-dependency problem suggests that an effective function-level program analysis, such as an *intra*-procedural static analysis, must consider a reasonable approximation of the space of valid external states during the average function call, e. g., encoded as a function precondition, function summary, or type system. While the context-dependency check

can ultimately be delegated to the user in practice, given our results, we suggest that particular care is taken when evaluating such tools at the function-level in the absence of users. In addition to function-level program analysis, there are *inter-procedural*, or whole-program analyses, which consider interactions between multiple procedures, and thus implicitly resolve the context-dependency problem during the search for security vulnerabilities.

Abstention. What could be possible ways forward for ML4VD? A simple possible solution to the context-dependency problem of function-level vulnerability detection could be to cast it as binary classification *with abstention*. Given a function, an ML4VD technique could either decide the vulnerability of the function or abstain from this decision. However, the obvious disadvantage of this approach is that only context-independent security vulnerabilities could actually be detected, which appear to be very rare (cf. RQ.1-b). The context-dependency labels generated by our empirical study could be used as a starting point to explore this direction.

Other Base Units. Instead of deciding whether a *function* is vulnerable, we could decide whether other types of base units are vulnerable, such as lines, statements, files, modules, commits, or vulnerability-specific inter-procedural slices (cf. Fig. 4.2). However, for smaller base units, such as line-, statement-, or commit-level, there is no reason to believe that the context-dependency problem is resolved (subject to further study) while for larger base units, such as files or modules, the context-dependency problem may be much less pronounced (subject to further study), although the vulnerability label at that such coarse granularity may be less actionable for the security researcher. For alternative granularities, future work shall determine if they offer a true advantage or merely shift the problem to a different level of abstraction.

The vulnerability-specific inter-procedural slice [Wu+23] (at least conceptually) includes all required context to trigger a vulnerability of a specific type at a specific code location, and thus resolves the context-dependency problem. However, it also somewhat delegates the detection problem because it requires, as slicing criterion, a (vulnerability-specific) statement-of-interest where a vulnerability may be triggered. In general, we believe that a combination of static analysis and machine learning [ZJS21] offers a promising avenue for future research in vulnerability detection.

Context-conditional Classification. We could define ML4VD as a base-unit-level classification problem *given*, e. g., the entire software repository as context. Due to the hierarchical nature of software, determining the vulnerability of a given base unit (e. g., function) may require considering a large scope of context, such as the complete state of a repository at a given commit ID and all other dependencies that might exist outside of the code (e. g., system of the user, dependencies of the repository, etc.). Since the Devign, BigVul, and DiverseVul benchmarks all include patch commit IDs, the complete context for a given function could be reconstructed, so that they could be used to evaluate ML4VD based on this alternative problem statement. Future

research is needed to find optimal ways to include this context into state-of-the-art techniques.

However, adding repository-level context alone may not resolve the benchmarking problem. Evidently, ML4VD techniques that *disregard* this context still appear to perform well (likely due to spurious correlations; RQ.2). Addressing this challenge will require ensuring that the evaluation metrics and benchmarks *actually* capture the vulnerability detection capabilities of the techniques that are tested.

Overcoming Classification. Even using different types of base units and providing the entire repository as context, ML4VD would still be a binary classification task: Given a unit, decide whether the unit is vulnerable or secure. Alternatively, an ML4VD technique could *provide the condition under which a given function would actually be vulnerable*, e. g., by generating an example context: Given a function, an ML4VD technique may generate a complete executable program which is vulnerable to attack due to the code in that function. This generated context could then be compared with the actual context to see whether the vulnerability exists in the real context. A much stronger approach could be the (ML-assisted) generation of a proof of vulnerability in form of a test case, which demonstrates the exploitation of the vulnerability in the given context.

In conclusion, addressing the issues of context dependency and spurious correlations is critical for the advancement of ML4VD and other ML applications. By exploring alternative methodologies and improving our evaluation frameworks, we can ensure more robust and reliable assessments, ultimately leading to more secure and effective solutions.

4.8 DATA AVAILABILITY

To ensure the reproducibility of our results and to provide transparency in our research, we have made all related scripts and data publicly available. All resources can be accessed as part of our GitHub repository, which is available at <https://github.com/niklasrisse/TopScoreWrongExam>.

ACKNOWLEDGMENTS

Funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Germany's Excellence Strategy - EXC 2092 CASA - 390781972. This research is also partially funded by the European Union. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Research Council Executive Agency. Neither the European Union nor the granting authority can be held responsible for them. This work is supported by ERC grant (Project AT_SCALE, 101179366).

REFERENCES

- [Arp+22] Daniel Arp, Erwin Quiring, Feargus Pendlebury, Alexander Warnecke, Fabio Pierazzi, Christian Wressnegger, Lorenzo Cavallaro, and Konrad Rieck. “Dos and Don’ts of Machine Learning in Computer Security”. In: *Proceedings of the 31st USENIX Security Symposium (USENIX Security)*. 2022. URL: <https://www.usenix.org/conference/usenixsecurity22/presentation/arp>.
- [BCR21] Marcel Böhme, Cristian Cadar, and Abhik Roychoudhury. “Fuzzing: Challenges and Opportunities”. In: *IEEE Software* (2021). DOI: [10.1109/MS.2020.3016773](https://doi.org/10.1109/MS.2020.3016773).
- [BMC23] Marcel Böhme, Valentin J. M. Manès, and Sang Kil Cha. “Boosting Fuzzer Efficiency: An Information Theoretic Perspective”. In: *Communications of the ACM* (2023). DOI: [10.1145/3611019](https://doi.org/10.1145/3611019).
- [Cao+24] Sicong Cao, Xiaobing Sun, Xiaoxue Wu, David Lo, Lili Bo, Bin Li, Xiaolei Liu, Xingwei Lin, and Wei Liu. “Snopy: Bridging Sample Denoising with Causal Graph Learning for Effective Vulnerability Detection”. In: *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2024. DOI: [10.1145/3691620.3695057](https://doi.org/10.1145/3691620.3695057).
- [CBK23] Roland Croft, M. Ali Babar, and M. Mehdi Kholoosi. “Data Quality for Software Vulnerability Datasets”. In: *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE)*. 2023. DOI: [10.1109/ICSE48619.2023.00022](https://doi.org/10.1109/ICSE48619.2023.00022).
- [CH00] Koen Claessen and John Hughes. “QuickCheck: a lightweight tool for random testing of Haskell programs”. In: *ACM SIGPLAN Notices* (2000). DOI: [10.1145/357766.351266](https://doi.org/10.1145/357766.351266).
- [Cha+21] Saikat Chakraborty, Rahul Krishna, Yangruibo Ding, and Baishakhi Ray. “Deep Learning based Vulnerability Detection: Are We There Yet?”. In: *IEEE Transactions on Software Engineering* (2021). DOI: [10.1109/TSE.2021.3087402](https://doi.org/10.1109/TSE.2021.3087402).
- [Che+23] Yizheng Chen, Zhoujie Ding, Lamya Alowain, Xinyun Chen, and David Wagner. “DiverseVul: A New Vulnerable Source Code Dataset for Deep Learning Based Vulnerability Detection”. In: *Proceedings of the International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*. 2023. DOI: [10.1145/3607199.3607242](https://doi.org/10.1145/3607199.3607242).
- [Coh60] Jacob Cohen. “A Coefficient of Agreement for Nominal Scales”. In: *Educational and Psychological Measurement* (1960). URL: <https://api.semanticscholar.org/CorpusID:15926286>.

- [DR06] S. Desikan and G. Ramesh. *Software Testing: Principles and Practice*. Pearson Education Canada, 2006. URL: <https://books.google.de/books?id=Yt2yRW6du9wC>.
- [Fan+20] Jiahao Fan, Yi Li, Shaohua Wang, and Tien N. Nguyen. "A C/C++ Code Vulnerability Dataset with Code Changes and CVE Summaries". In: *Proceedings of the 17th IEEE/ACM International Conference on Mining Software Repositories (MSR)*. 2020. DOI: [10.1145/3379597.3387501](https://doi.org/10.1145/3379597.3387501).
- [GB25] Octavio Galland and Marcel Böhme. "Invivo Fuzzing by Amplifying Actual Executions". In: *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE)*. 2025. DOI: [10.1109/icse55347.2025.00172](https://doi.org/10.1109/icse55347.2025.00172).
- [HG09] Haibo He and Eduardo A. Garcia. "Learning from Imbalanced Data". In: *IEEE Transactions on Knowledge and Data Engineering* (2009). DOI: [10.1109/TKDE.2008.239](https://doi.org/10.1109/TKDE.2008.239).
- [HO08] Mary Jean Harrold and Alessandro Orso. "Retesting software during development and maintenance". In: *Proceedings of the IEEE Frontiers of Software Maintenance (FoSM)*. 2008. DOI: [10.1109/FoSM.2008.4659253](https://doi.org/10.1109/FoSM.2008.4659253).
- [Hoa69] C. Anthony R. Hoare. "An axiomatic basis for computer programming". In: *Communications of the ACM* (1969). DOI: [10.1145/363235.363259](https://doi.org/10.1145/363235.363259).
- [Izm+22] Pavel Izmailov, Polina Kirichenko, Nate Gruver, and Andrew Gordon Wilson. "On feature learning in the presence of spurious correlations". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2022. URL: <https://nips.cc/virtual/2022/poster/52928>.
- [Joh+13] Brittany Johnson, Yoonki Song, Emerson Murphy-Hill, and Robert Bowdidge. "Why don't software developers use static analysis tools to find bugs?" In: *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*. 2013. DOI: [10.1109/icse.2013.6606613](https://doi.org/10.1109/icse.2013.6606613).
- [KC07] Barbara Kitchenham and Stuart Charters. "Guidelines for performing Systematic Literature Reviews in Software Engineering". In: (2007). URL: https://legacyfileshare.elsevier.com/promis_misc/525444systematicreviewsguide.pdf.
- [LBP22] Stephan Lipp, Sebastian Banescu, and Alexander Pretschner. "An empirical study on the effectiveness of static C code analyzers for vulnerability detection". In: *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. 2022. DOI: [10.1145/3533767.3534380](https://doi.org/10.1145/3533767.3534380).

- [Le+22] Quang Loc Le, Azalea Raad, Jules Villard, Josh Berdine, Derek Dreyer, and Peter W. O'Hearn. "Finding real bugs in big programs with incorrectness logic". In: *Proceedings of the ACM on Programming Languages* (2022). DOI: [10.1145/3527325](https://doi.org/10.1145/3527325).
- [Li+24] Zhen Li, Ning Wang, Deqing Zou, Yating Li, Ruqian Zhang, Shouhuai Xu, Chao Zhang, and Hai Jin. "On the Effectiveness of Function-Level Vulnerability Detectors for Inter-Procedural Vulnerabilities". In: *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. 2024. DOI: [10.1145/3597503.3639218](https://doi.org/10.1145/3597503.3639218).
- [Liu+24] Zhongxin Liu, Zhijie Tang, Junwei Zhang, Xin Xia, and Xiaohu Yang. "Pre-training by Predicting Program Dependencies for Vulnerability Analysis Tasks". In: *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. 2024. DOI: [10.1145/3597503.3639142](https://doi.org/10.1145/3597503.3639142).
- [LK77] J Richard Landis and Gary G. Koch. "The measurement of observer agreement for categorical data." In: *Biometrics* (1977). DOI: [10.2307/2529310](https://doi.org/10.2307/2529310).
- [Men+24] Ruijie Meng, Martin Mirchev, Marcel Böhme, and Abhik Roychoudhury. "Large Language Model guided Protocol Fuzzing". In: *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. 2024. URL: <https://www.ndss-symposium.org/ndss-paper/large-language-model-guided-protocol-fuzzing/>.
- [NNH99] Flemming Nielson, Hanne R. Nielson, and Chris Hankin. *Principles of Program Analysis*. Springer Publishing Company, Incorporated, 1999. DOI: [10.1007/978-3-662-03811-6](https://doi.org/10.1007/978-3-662-03811-6).
- [Pad+19] Rohan Padhye, Caroline Lemieux, Koushik Sen, Mike Papadakis, and Yves Le Traon. "Semantic fuzzing with zest". In: *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. 2019. DOI: [10.1145/3293882.3330576](https://doi.org/10.1145/3293882.3330576).
- [PLS19] Rohan Padhye, Caroline Lemieux, and Koushik Sen. "JQF: coverage-guided property-based testing in Java". In: *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. 2019. DOI: [10.1145/3293882.3339002](https://doi.org/10.1145/3293882.3339002).
- [PVK15] Kai Petersen, Sairam Vakkalanka, and Ludwik Kuzniarz. "Guidelines for conducting systematic mapping studies in software engineering: An update". In: *Information and Software Technology* (2015). DOI: [10.1016/j.infsof.2015.03.007](https://doi.org/10.1016/j.infsof.2015.03.007).

- [Rah+24] Md Mahbubur Rahman, Ira Ceka, Chengzhi Mao, Saikat Chakraborty, Baishakhi Ray, and Wei Le. “Towards Causal Deep Learning for Vulnerability Detection”. In: *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. 2024. DOI: [10.1145/3597503.3639170](https://doi.org/10.1145/3597503.3639170).
- [RB24] Niklas Risse and Marcel Böhme. “Uncovering the Limits of Machine Learning for Automatic Vulnerability Detection”. In: *Proceedings of the 33rd USENIX Security Symposium (USENIX Security)*. 2024. URL: <https://www.usenix.org/system/files/usenixsecurity24-risse.pdf>.
- [Ris23] Niklas Risse. “Detecting Overfitting of Machine Learning Techniques for Automatic Vulnerability Detection”. In: *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 2023. DOI: [10.1145/3611643.3617845](https://doi.org/10.1145/3611643.3617845).
- [Sag+20] Shiori Sagawa, Aditi Raghunathan, Pang Wei Koh, and Percy Liang. “An Investigation of Why Overparameterization Exacerbates Spurious Correlations”. In: *Proceedings of the International Conference on Machine Learning (ICML)*. 2020. URL: <https://proceedings.mlr.press/v119/sagawa20a.html>.
- [Sej+24] Adriana Sejfia, Satyaki Das, Saad Shafiq, and Nenad Medvidović. “Toward Improved Deep Learning-based Vulnerability Detection”. In: *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. 2024. DOI: [10.1145/3597503.3608141](https://doi.org/10.1145/3597503.3608141).
- [SGL24] Benjamin Steenhoek, Hongyang Gao, and Wei Le. “Dataflow Analysis-Inspired Deep Learning for Efficient Vulnerability Detection”. In: *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. 2024. DOI: [10.1145/3597503.3623345](https://doi.org/10.1145/3597503.3623345).
- [SHB19] Philipp Dominik Schubert, Ben Hermann, and Eric Bodden. “PhASAR: An Inter-procedural Static Analysis Framework for C/C++”. In: *Proceedings of the International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. 2019. DOI: [10.1007/978-3-030-17465-1_22](https://doi.org/10.1007/978-3-030-17465-1_22).
- [Ull+24] Saad Ullah, Mingji Han, Saurabh Pujar, Hammond Pearce, Ayse Coskun, and Gianluca Stringhini. “LLMs Cannot Reliably Identify and Reason About Security Vulnerabilities (Yet?): A Comprehensive Evaluation, Framework, and Benchmarks”. In: *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*. 2024. DOI: [10.1109/SP54263.2024.00210](https://doi.org/10.1109/SP54263.2024.00210).

- [Wan+21] Yue Wang, Weishi Wang, Shafiq Joty, and Steven C.H. Hoi. “CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation”. In: *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2021. DOI: [10.18653/v1/2021.emnlp-main.685](https://doi.org/10.18653/v1/2021.emnlp-main.685).
- [Wu+23] Bozhi Wu, Shangqing Liu, Yang Xiao, Zhiming Li, Jun Sun, and Shang-Wei Lin. “Learning Program Semantics for Vulnerability Detection via Vulnerability-Specific Inter-procedural Slicing”. In: *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 2023. DOI: [10.1145/3611643.3616351](https://doi.org/10.1145/3611643.3616351).
- [Zhe+25] Han Zheng, Flavio Toffalini, Marcel Böhme, and Mathias Payer. “MendelFuzz: The Return of the Deterministic Stage”. In: *Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE)*. 2025. DOI: [10.1145/3715713](https://doi.org/10.1145/3715713).
- [Zho+19] Yaqin Zhou, Shangqing Liu, Jingkai Siow, Xiaoning Du, and Yang Liu. “Devign: effective vulnerability identification by learning comprehensive program semantics via graph neural networks”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2019. URL: <https://proceedings.neurips.cc/paper/2019/hash/49265d2447bc3bbfe9e76306ce40a31f-Abstract.html>.
- [ZJS21] Weining Zheng, Yuan Jiang, and Xiaohong Su. “Vu1SPG: Vulnerability detection based on slice property graph representation learning”. In: *Proceedings of the IEEE International Symposium on Software Reliability Engineering (ISSRE)*. 2021. DOI: [10.1109/ISSRE52982.2021.00054](https://doi.org/10.1109/ISSRE52982.2021.00054).

CHASING SHADOWS: PITFALLS IN LLM SECURITY RESEARCH

This article was published at the Network and Distributed System Security Symposium (NDSS), 2026. The full version is available at <https://doi.org/10.14722/ndss.2026.241749> and is reproduced here with the Internet Society's permission for noncommercial reproduction (Copyright © 2026 Internet Society). A version with an extended appendix is available at <https://doi.org/10.48550/arXiv.2512.09549>; that appendix is the one integrated here.

Author Contributions. Niklas Risse and Jonathan Evertz contributed equally as co-first authors. Niklas Risse led the **AI for Security** research direction, serving as the primary reviewer for the majority of papers in that category in the prevalence study, and designing, implementing, evaluating, and writing about the case studies on Data Leakage (§5.4.2) and Context Truncation (§5.4.3). Jonathan Evertz led the **Security & Safety of AI** research direction, serving as the primary reviewer for the majority of papers in that category in the prevalence study, and designing, implementing, evaluating, and writing about the case studies on Model Ambiguity (§5.4.1) and Model Collapse (§5.4.4). Both co-first authors contributed to all sections and authored the majority of the paper. Erwin Quiring conceptualized and led the hate speech detection experiment (§5.4.1). Nicolai Neuer, Andreas Müller, and Philipp Normann contributed to conceptualization, data analysis, visualization, and writing. All co-authors, including Gaetano Sapia, Srishti Gupta, David Pape, Soumya Shaw, and Devansh Srivastav, contributed to the prevalence study as pitfall reviewers. Christian Wressnegger, Erwin Quiring, Thorsten Eisenhofer, Daniel Arp, and Lea Schönherr provided guidance on conceptualization and methodology, and contributed to writing and editing the paper. Daniel Arp and Lea Schönherr supervised the project.

Formatting Changes. The content of the original publication is reproduced unchanged; only the presentation (e.g., table styles and citation style) was adapted to this thesis.

Original Publication

- [Eve+26] Jonathan Evertz*, Niklas Risse*, Nicolai Neuer, Andreas Müller, Philipp Normann, Gaetano Sapia, Srishti Gupta, David Pape, Soumya Shaw, Devansh Srivastav, Christian Wressnegger, Erwin Quiring, Thorsten Eisenhofer, Daniel Arp, and Lea Schönherr. "Chasing Shadows: Pitfalls in LLM Security Research". In: *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. * **Equal Contribution**. 2026. DOI: [10.14722/ndss.2026.241749](https://doi.org/10.14722/ndss.2026.241749).

Chasing Shadows: Pitfalls in LLM Security Research

Jonathan Evertz, Niklas Risse, Nicolai Neuer, Andreas Müller, Philipp Normann, Gaetano Sapia, Srishti Gupta, David Pape, Soumya Shaw, Devansh Srivastav, Christian Wressnegger, Erwin Quiring, Thorsten Eisenhofer, Daniel Arp, and Lea Schönherr

Abstract. Large language models (LLMs) are increasingly prevalent in security research. Their unique characteristics, however, introduce challenges that undermine established paradigms of reproducibility, rigor, and evaluation. Prior work has identified common pitfalls in traditional machine learning research, but these studies predate the advent of LLMs. In this paper, we identify *nine* common pitfalls that have become (more) relevant with the emergence of LLMs and that can compromise the validity of research involving them. These pitfalls span the entire computation process, from data collection, pre-training, and fine-tuning to prompting and evaluation.

We assess the prevalence of these pitfalls across all 72 peer-reviewed papers published at leading Security and Software Engineering venues between 2023 and 2024. We find that every paper contains at least one pitfall, and each pitfall appears in multiple papers. Yet only 15.7% of the present pitfalls were explicitly discussed, suggesting that the majority remain unrecognized. To understand their practical impact, we conduct four empirical case studies showing how individual pitfalls can mislead evaluation, inflate performance, or impair reproducibility. Based on our findings, we offer actionable guidelines to support the community in future work.

5.1 INTRODUCTION

The intersection of large language models (LLMs) and security has become a fast-growing and influential line of research. Their capacity for easy adaptation and deployment makes LLMs powerful instruments for security-critical applications, including vulnerability detection [Sta+25; Sun+24b; Den+24b; Xia+24], code analysis [Fan+24; Zho+25; Der+24; Pur+23], and automated code repair [Pea+23; Zho+24; Hos+24; Zha+24a]. At the same time, it is crucial to understand their inherent capabilities for resisting attacks: models struggle to distinguish commands from data [Zve+25], to identify context-dependent sensitive content [Eve+24; Han+25], and are generally vulnerable to prompt injection [Liu+24d] and jailbreak attacks [Yu+24b]. For LLMs to be trusted in critical applications, both in research and in practice, their implementations must be rigorous, reproducible, and methodologically sound.

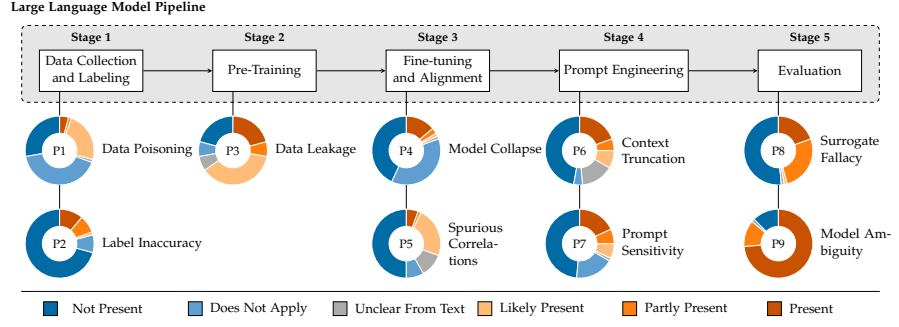


Figure 5.1: Typical LLM pipeline, divided into its key stages. Each stage can introduce LLM-specific pitfalls that may distort evaluation, inflate reported performance, or undermine reproducibility. Colors indicate the prevalence of each pitfall according to our prevalence study (§5.3).

Prior work has identified common pitfalls in traditional machine learning research [Arp+22; Car+19], offering valuable guidance for designing sound experiments. However, these studies predate the emergence of modern LLMs. Language models have fundamentally reshaped the stages of the machine learning workflow and introduced new risks: data collection (e. g., poisoning via large-scale web scraping), pre-training (e. g., inadvertent data leakage), fine-tuning (e. g., reliance on synthetic data), prompt engineering (e. g., context sensitivity), and evaluation (e. g., unstable API behavior). As a result, previous studies [Arp+22; Car+19] do not fully capture the complexity and fragility of current LLM practices.

In this paper, we bridge this gap by identifying nine pitfalls that commonly occur in LLM security research. These pitfalls span the entire development pipeline of LLMs. They reflect both structural changes in how models are built and used, as well as the unique challenges introduced by the scale, opacity, and natural language interface of LLMs. Some pitfalls are entirely new, such as *model collapse* caused by training on synthetic data or unpredictable model behavior due to *prompt sensitivity*. Others represent familiar concerns, such as *data leakage* or *spurious correlations*, but their implications change and intensify in the context of LLMs.

To assess how widespread these pitfalls are, we conducted a prevalence study across all LLM-centric research papers (72 in total) published between January 2023 and December 2024 at leading conferences in Security (IEEE S&P, NDSS, ACM CCS, and USENIX Security Symposium) and Software Engineering (IEEE/ACM ICSE, ACM ISSTA, ACM FSE, and IEEE/ACM ASE). With a team of 15 researchers, we collaboratively developed detailed labeling guidelines and reviewed each paper independently using a systematic and uniform agreement process.

To our surprise, *every* paper in our study contains at least one of the nine identified pitfalls as part of its main contribution. Five pitfalls, namely Data Leakage (P3), Context Truncation (P6), Prompt Sensitivity (P7), Surrogate Fallacy (P8), and Model Ambiguity (P9),

appear in more than 20% of the papers. Prevalence also varies across research topics: papers on *Vulnerability Repair and Detection* show the highest average rate of pitfalls per paper (23–28%), while studies on *Fuzzing* and *Secure Code Generation* contain fewer pitfalls on average (15–18%).

Beyond identifying prevalence, we conduct four case studies that experimentally demonstrate how the identified pitfalls can mislead evaluation, inflate performance, or hinder reproducibility. Concretely, we examine the impact of (i) Model Ambiguity—snapshot versions and quantization affect robustness against jailbreaks and significantly alter precision/recall, thereby affecting reproducibility; (ii) Data Leakage—leaking 20% of test data into the fine-tuning process raises the F1 score by ≈ 0.08 – 0.11 with increases that grow almost linearly as leakage increases; (iii) Context Truncation—about 49% of vulnerable functions exceed common context windows of 512 tokens (29% > 1024 tokens), removing essential information and distorting evaluation; (iv) Model Collapse—recursive self-training in code generation increases perplexity across generations, leading to degradation and instability.

Based on our results, we provide concrete guidelines and recommendations for each identified pitfall. *Importantly, our goal is not to assign blame.* Pitfalls can occur in carefully conducted research, often without being explicitly recognized. Our aim is to support the community by raising awareness and offering actionable recommendations.

Contributions. We make the following key contributions:

- **Identification of LLM pitfalls.** We identify nine pitfalls that frequently arise in LLM-based security research spanning the entire computation pipeline from data collection to evaluation. We describe each pitfall, explain why it occurs, and discuss its potential impact on validity and reproducibility.
- **Prevalence study.** We assess the prevalence of all nine pitfalls across 72 peer-reviewed papers published between January 2023 and December 2024 at leading Security and Software Engineering venues. We find that every paper suffers from at least one pitfall.
- **Impact analysis.** We conduct four case studies that empirically demonstrate how individual pitfalls can distort evaluation, inflate performance metrics, or compromise reproducibility.
- **Guidelines and recommendations.** We present guidelines and recommendations for each identified pitfall and maintain a living appendix containing up-to-date information on best practices for preventing pitfalls, accessible at <https://llmpitfalls.org>.

We provide all code, datasets, and complete reproduction instructions at <https://github.com/dormant-neurons/llm-pitfalls>.

5.2 PITFALLS ACROSS THE LLM PIPELINE

Research on pitfalls in machine learning is not new. Prior work has systematically analyzed common failure modes in traditional pipelines [Arp+22; Car+19]. While related in spirit, the pitfalls we investigate arise from the specific characteristics of LLM research. Our aim is not to replicate existing analyses, but to complement them with a targeted examination of challenges that are unique to or substantially intensified in LLMs.

LLMs have significantly changed the way machine learning systems are trained, adapted, and evaluated. Their development pipelines involve multiple interdependent stages: large-scale data collection and labeling, pre-training, fine-tuning, prompt design, and evaluation. In the following sections, we outline the structure of this pipeline and describe the potential practical risks that can arise at each stage. This discussion forms the foundation for the nine pitfalls we identify. Figure 5.1 illustrates the overall pipeline and the corresponding pitfalls.

5.2.1 Stage 1: Data Collection and Labeling

LLMs require large-scale datasets to train effectively, often relying on content scraped from the Internet, which is far more extensive and noisy than the more curated datasets used in classical machine learning.

P1—Data Poisoning via Internet Scraping. The sheer scale of this data makes curation increasingly difficult, creating favorable conditions for *Data Poisoning*. Without sufficient precautions, data scraped from the Internet opens the door to data poisoning attacks, where malicious or biased content can be subtly inserted into the training data without detection. Although not new to machine learning, the scale of LLMs exacerbates this risk. Platforms like GitHub or Reddit, where anyone can anonymously upload data, make it especially difficult to ensure quality and safety at scale.

P2—LLM-generated Label Inaccuracy. At the same time, the demand for labeled data during fine-tuning and evaluation has led to the widespread use of models themselves to generate labels, a practice known as LLM-as-a-judge [Zhe+23]. Although this approach can address the problem of generating labels in scenarios in which human-curated labeling is time-intensive or costly, it also introduces the risk of *Label Inaccuracy*.

The reliability of experimental results depends heavily on the quality of data and labeling. Since learning-based methods rely on accurate labels, any label errors or instabilities can degrade performance. This is particularly relevant when LLMs are used for automated annotation, where outputs may appear correct yet still contain subtle flaws.

5.2.2 Stage 2: Pre-Training

LLMs are typically pre-trained on large-scale datasets to capture general language patterns. Because these corpora are often assembled from broad internet scrapes with limited transparency, they increase the risk of data leakage.

P3—Data Leakage. Many model providers no longer disclose detailed information about the data used for training. For instance, GPT-4 was trained on “a variety of licensed, created, and publicly available data sources” [Ope23a; Ope23b], without further specification. This lack of transparency increases the risk of *Data Leakage*, where evaluation data may inadvertently overlap with training inputs.

While creating traditional machine learning models with disjoint data splits is straightforward, the nature of LLMs complicates this. As LLMs require massive datasets, fine-tuning foundational models is a widespread practice. Foundational models may have been trained on (a subset of) samples from the test set. This risk is particularly high in LLMs, where many datasets overlap with pre-training sources like GitHub, Wikipedia, and Reddit [Gol25; Nic24].

5.2.3 Stage 3: Fine-tuning and Alignment

During fine-tuning or alignment, LLMs are adapted for specific downstream tasks and user-facing behaviors. In this stage, models become particularly susceptible to issues where evaluation data may inadvertently overlap with pre-training data, as well as the reliance on synthetic or LLM-generated data.

P4—Model Collapse via Synthetic Training Data. Fine-tuning and alignment frequently rely on data that was itself generated by LLMs, introducing the risk of *Model Collapse* [Shu+24].

LLMs require vast amounts of data, and this need has increased substantially. To address data scarcity, synthetic data generated by other LLMs is frequently used. However, training on such data can lead to less diverse outputs, higher error rates, and model collapse over generations. While countermeasures exist, they are difficult to implement in practice [Gil+24; KAD22; Ale+24].

P5—Spurious Correlations/Unrelated Features. Additionally, *Spurious Correlations*, already known as an issue in traditional ML, becomes especially problematic for LLMs because their massive parameter space allows them to memorize and exploit subtle, non-causal patterns.

Spurious correlations are artifacts that correlate with task labels but are not actually related to the underlying task. Such features may arise from selection bias in the training data or from shortcuts the model learns using unrelated patterns. This can lead to misleading performance and poor generalization.

5.2.4 Stage 4: Prompt Engineering

Prompting has emerged as a new form of control in LLMs. It effectively acts as a tunable hyperparameter that controls the model’s behavior, and its design can influence the model’s outputs and overall performance.

P6—Context Truncation. An LLM’s context size refers to the maximum number of tokens it can process at once. Because LLMs are stateless, all relevant information must be included in the current context. If inputs exceed this limit, they are truncated, potentially omitting critical information and reducing performance.

P7—Prompt Sensitivity. Prompting also introduces the issue of *Prompt Sensitivity*, where minor changes in phrasing can lead to drastically different outputs and where different models can have distinct prompt preferences.

Language models are often fine-tuned to follow instruction-based inputs with specific formatting styles. While this allows the language model to adapt to desired tasks during runtime, the performance and functionality of the model also rely on the quality of the instructions and the correct input format. This results in differences across evaluations if inputs are formatted for a specific LLM but evaluated on other models, or are not expressive enough for the task [Cao+24; Pry+23].

5.2.5 Stage 5: Evaluation

As LLMs are often accessed through APIs and web interfaces, with several distinct model versions that differ in architecture, parameter count, or quantization. This ambiguity creates risks in reproducing results, as identifying the precise model instance becomes difficult or even infeasible.

P8—Proxy/Surrogate Fallacy. A single model name, such as ChatGPT, may refer to multiple underlying snapshots with different architectures, sizes, or quantization levels. This creates the risk of the *Proxy/Surrogate Fallacy*, where conclusions are drawn from models that are not representative of those actually used.

New LLMs are released frequently, driving the race for the “most state-of-the-art” model. These models vary in size and task specialization. Although models of the same class often share architecture and training data, differences in size or context window can significantly affect performance and behavior. Discrepancies are even greater across model classes, for example, between the open-source Llama models [Tou+23; AI24; Gra+24] and the commercial GPT family [Ope25]. Without experimental validation, it is generally not possible to make claims between different model sizes and families.

P9—Model Ambiguity. The ambiguity of different architectures, sizes, or quantization levels also leads to *Model Ambiguity* risks, where determining the exact model instance used to reproduce a result is difficult or even impossible.

Both open-source models on platforms like Huggingface [Hug25] and proprietary models like ChatGPT [Ope25], Claude [Ant25], or Gemini [Goo25] are regularly updated. Minor updates are often noted only in changelogs and may not result in a new major version. A single model specifier can therefore refer to different internal versions. Because even small changes in tokenizers or system prompts can affect model behavior, specifying the exact snapshot (e. g., endpoint version, commit ID, or access date) is essential. For open-source models, the quantization level is often an additional source of variation and should be specified (cf. §5.4.1).

5.3 PREVALENCE OF PITFALLS

Having identified nine pitfalls specific to the LLM computation pipeline, we now assess how frequently these issues appear in current LLM security research. We begin by outlining the paper selection process that underpins our study (§5.3.1) as well as the methodology used to evaluate each paper (§5.3.2). We then present the results per pitfall (§5.3.3), followed by a broader discussion (§5.3.4).

5.3.1 Paper Collection

As the basis for our study, we consider papers published between January 2023 and December 2024 at leading conferences in Security and Software Engineering.

- For Security, we consider the IEEE Symposium on Security and Privacy (IEEE S&P), the Network and Distributed System Security Symposium (NDSS), the ACM Conference on Computer and Communications Security (ACM CCS), and the USENIX Security Symposium.
- For Software Engineering, we consider the IEEE/ACM International Conference on Software Engineering (IEEE/ACM ICSE), the ACM International Conference on the Foundations of Software Engineering (ACM FSE), the ACM SIGSOFT International Symposium on Software Testing and Analysis (ACM ISSTA), and the IEEE/ACM International Conference on Automated Software Engineering (IEEE/ACM ASE).

For all conferences, we scraped the full set of papers and applied a two-stage filtering process to identify relevant studies, as detailed next.

Selection Criteria. To identify relevant papers, we establish two primary selection criteria centered on the use of LLMs in security-relevant contexts. Specifically, papers were considered in scope if they (1) evaluate the security or safety of LLMs, or (2) apply LLMs to security-related tasks such as vulnerability detection or secure code generation. Importantly, the use of the LLM had to be integral to a paper’s methodology, meaning the core contribution would not have been possible in its current form without them.

To guide the search, we define a list of LLM-relevant keywords: *LLM, LLMs, large language model, language model, GPT, ChatGPT, transformer, pre-trained, foundation model, prompt, learning*. We applied these keywords to search the titles and abstracts of all papers published in the selected venues using multiple specialized tools for literature search: IEEE Xplore Advanced Search¹, ACM DL Advanced Search², and Google Scholar Advanced Search³.

This process yielded over 1,000 candidate papers, which we manually reviewed for relevance. As a first step, we excluded papers that clearly did not fit our scope. For example, we removed the software engineering paper *Large Language Models are Few-Shot Summarizers: Multi-Intent Comment Generation via In-Context Learning* [Gen+24], which focuses on code summarization without any direct security aspect. Second, all remaining abstracts, and when necessary, the full texts, were manually reviewed to assess whether each paper met the inclusion criteria.

Selected Papers. Our collection process resulted in a final set of 72 papers. These papers span a wide range of topics within the field of language models and security: *Security in LLM* [Wan+24c; Zha+24d; Liu+22; Nas+23; Du+23; Yan+24a; Liu+24e; Zha+24b; Mee+24; Yu+24a; Yu+24b; Liu+24c; Liu+24b; Si+23; She+24a; Wei+24; Den+24a; Naz+24; Che+24a; She+24b; Hui+24b; Agh+24; Li+24a; Che+24b], *Vulnerability Detection* [Ull+24; Sun+24b; Liu+24a; Den+24b; Sta+25; Liu+24f; Rah+24; Sej+24; Wen+24; Din+23; Zha+24e; Ni+23; EA24; Wu+24b; Sun+24a], *Generative AI Safety* [Luk+23; He+24; Vis+24; Roy+24; Si+22; Tan+24; Li+24b; Zha+24c; Lin+24; Guo+24a; Niu+23; Lv+22; Wu+24a; Bet+24], *Fuzzing* [Asm+24; Den+23; Men+24; Lyu+24; MLZ24; WYL24; Wan+24b; Xia+24], *Secure Code Generation* [Li+23; Fan+24; San+23; HLC24], and *Vulnerability Repair* [Pea+23; Zho+24; Hos+24; Zha+24a; Wan+24a; Wu+23]. Figure 5.2 provides an overview of the distribution of papers across these categories.

5.3.2 Reviewing Methodology

To guide the review process, we developed a detailed set of guidelines to ensure consistency across reviewers. These guidelines define how to identify each pitfall and provide instructions on interpreting relevant

¹ IEEE Xplore Advanced Search

² ACM DL Advanced Search

³ Google Scholar Advanced Search

5.3 PREVALENCE OF PITFALLS

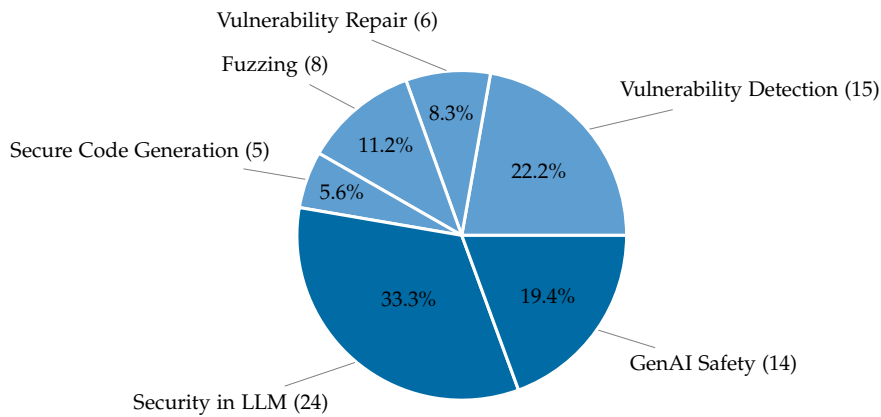


Figure 5.2: Distribution of the 72 selected papers across topics. The top half, light blue segments (●) represent research that uses LLMs to address security problems. The bottom half, dark blue segments (●) correspond to research focused on the security and safety of LLMs themselves.

cues within the papers. To refine the guidelines, we conducted a pilot review on a small subset of papers. This process helped clarify ambiguous cases and improve reviewer instructions. The final version of our guidelines is included in Appendix B.

Labeling Scheme. For each of the nine pitfalls, reviewers assigned one of several labels reflecting its applicability and presence. A pitfall is marked as *Does not apply* (●) if it is outside the scope for the particular paper, or as *Unclear from text* (●) when the paper lacks sufficient detail to support a clear judgment. If the pitfall is applicable but not present, it is labeled as *Not present* (●). We use *Partly present* (●) to indicate that the pitfall affects only a part of the paper’s methodology. A pitfall was marked as *Present* (●) when it appears throughout the paper. Finally, in case there is a high likelihood of a pitfall’s presence but no explicit evidence, we use *Likely present* (●). For example, take the case of potential data leakage where the evaluation dataset was publicly available online before the training-data cutoff of a model that has been trained on web-scale data. In such cases, we cannot prove that the evaluation data were included during training, but their presence is highly plausible due to timing and accessibility. This process is also summarized in Appendix B.

It is important to note that some pitfalls may be difficult or impossible to avoid. Therefore, for any pitfall labeled as *Likely present*, *Partly present*, or *Present*, we also annotated whether the presence of the pitfall was discussed in the paper. This distinction helps recognize good practice in transparently acknowledging potential limitations or contextualizing the relevance of a pitfall’s presence. For example, *Label Inaccuracy* (P2) may be unavoidable in large-scale evaluations where labeling is expensive, or *Data Leakage* (P3) may be difficult to address due to limited access to the original training data of proprietary models.

Reviewing Phases. The review was conducted by a team of fifteen researchers with backgrounds in security, machine learning, and software engineering. Every paper was assigned to exactly two reviewers, aiming to match papers with reviewers whose expertise aligns closely with the paper’s topic. To ensure consistency and rigor throughout the review process, we structured the evaluation into three phases:

Phase 1: Individual Reviews. In the first phase, each reviewer independently assessed their assigned papers using the predefined guidelines. Reviewers were blinded to each other’s assessments to ensure independent judgment.

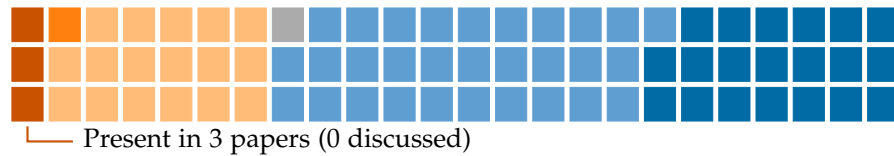
Phase 2: Review Discussions. In the second phase, reviewers discussed each paper to resolve differences from their initial assessments. Disagreements were clarified through 1-on-1 sessions or, when needed, group discussions (see Phase 3). For each pitfall, we recorded both the initial and final decisions.

Phase 3: Group Discussions. For cases where consensus could not be reached, we escalated the discussion to a larger group. These sessions included ten of the fifteen reviewers and served to resolve especially ambiguous or borderline cases.

5.3.3 Pitfall Prevalence

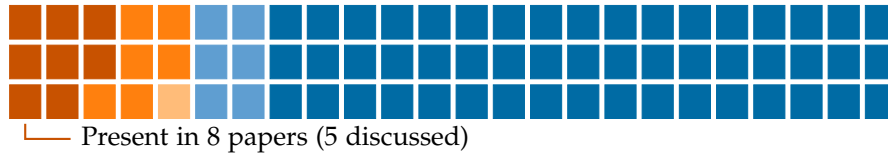
We now present the results of our prevalence study. As discussed above, we begin by examining each pitfall individually, summarizing how frequently it occurs across the reviewed papers and highlighting potential implications. In the following section (§5.3.4), we broaden the discussion to consider general patterns and shared challenges across pitfalls.

P1—Data Poisoning via Internet Scraping. A dataset used to train a model is collected from the internet without strategies to verify the integrity and safety of the data [Sha+18].



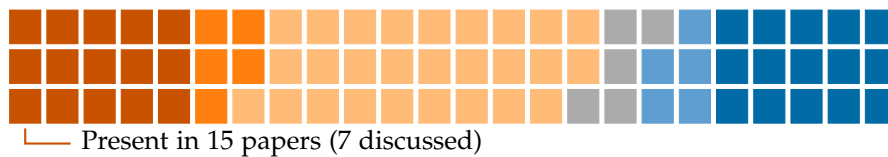
Results & Implications Although this pitfall was *Present* in 4.2% (3) and *Likely present* in 23.6% (17) of the papers, none acknowledged or discussed the risk of training on potentially poisoned internet data. This is especially concerning given recent work demonstrating that even minimal amounts of poison can compromise LLM behavior [Mit25; Bow+24; Wan+23a]. In security-relevant settings, this can result in unsafe code suggestions or behavior misalignment triggered by crafted inputs.

P2—LLM-generated Label Inaccuracy. LLMs are used to annotate data with certain labels via classification or LLM-as-a-judge procedures without further validation of correctness.



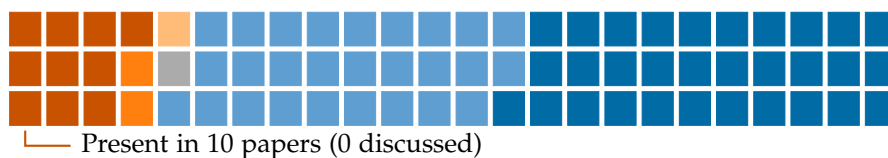
Results & Implications This pitfall was *Present* in 15.3% (8) of the papers (five of which discussed the pitfall), *Partly present* in 8.3% (6) of the papers (four of which discussed it), and *Likely present* in 1.4% (1) of the papers. While not among the most common pitfalls overall, 60% of relevant cases showed a relatively high awareness by discussing the issue. Nonetheless, unvalidated LLM-generated labels can distort results and lead to faulty conclusions. This is especially relevant in areas like jailbreak detection, where LLMs are often used as judges and external validation is critical to ensure correctness.

P3—Data Leakage. An LLM is trained or fine-tuned with data that is normally not available in practice, or the training data is contaminated with possible test data [AML12].



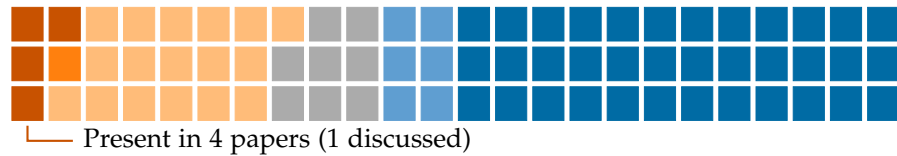
Results & Implications Data Leakage is highly prevalent: 65.2% (47) of the papers either contain it or are likely affected by it. Specifically, Data Leakage is *Present* in 20.8% (15) of the papers (of which seven explicitly discuss the pitfall), *Partly present* in 6.9% (5) of the papers (two of which discussed the pitfall), and *Likely present* in another 37.5% (27) of the papers (one of which discussed the pitfall). This raises concerns about inflated performance metrics, as models may memorize rather than generalize. Even state-of-the-art LLMs like GPT have suffered from such issues in tasks like code completion and vulnerability detection [Don+24].

P4—Model Collapse via Synthetic Training Data. An LLM is trained on data that is generated by other language models, risking an amplification of bias and degradation of data quality [Shu+24; Sed+24; Guo+24b].



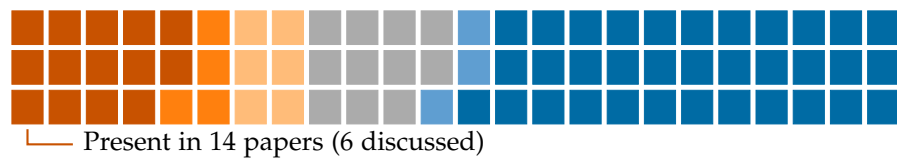
Results & Implications Although this pitfall was *Present* in 13.9% (10) of the papers, none of the papers in our study acknowledged or discussed the risk of using synthetic LLM-generated training data. Shumailov et al. have demonstrated a decrease in model performance when trained on synthetic data, making the performance tied to the LLMs training data [Shu+24]. Thus, training on LLM-generated data risks inheriting specific behaviors, biases, and errors [Clo+25]. This can undermine generalization, particularly in tasks such as code generation, potentially distorting results and degrading performance. Notably, for 37.5% (27) of papers, this pitfall *Does not apply*, as they do not involve model training.

P5—Spurious Correlations/Unrelated Features. The LLM adapts to unrelated artifacts from the problem space instead of generalizing onto the actual task [RB24].



Results & Implications This pitfall appears in 31% of papers: *Present* in 5.6% (4) of the papers (one of which discussed the pitfall), *Likely present* in 23.6% (17) of the papers, and *Partly present* in 1.4% (1) of the papers (which discussed it). Notably, it is discussed explicitly in only two papers. Due to the complexity of tasks and potentially black-box settings in which LLMs are deployed, Spurious Correlations often remain unidentified. This not only distorts experimental results but also leads to drawing false conclusions and using LLMs in scenarios for which they are not suited. In the security domain, for example, LLMs for vulnerability detection were unable to distinguish between vulnerable functions and the same functions that had been patched. The dependence on unrelated features led to top scores on benchmarks, even after removing the code structure itself [RLB25].

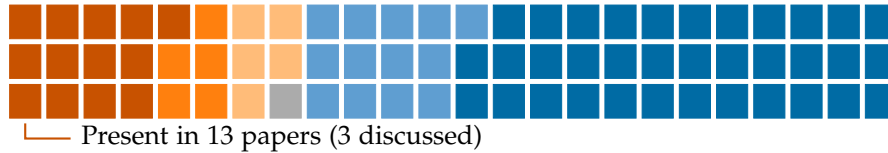
P6—Context Truncation. The LLM’s context size is not large enough for its intended task, and the input needs to be truncated.



Results & Implications While not as prevalent as Data Leakage, context limitations remain a notable pitfall: 33.3% papers (24) were affected, either marked as *Present* in 19.4% of the papers (14, 6 of which discussed the pitfall), *Partly Present* in 5.5% of the papers (4, all of which discussed the pitfall), or *Likely present* in 8.3% of the papers (6, none of which discussed the pitfall). This is especially relevant in tasks like vulnerability detection [RLB25] or prompt-based attacks [Yu+24b],

where long inputs are common. A limited context size can cause models to overlook key information or distort evaluation results.

P7—Prompt Sensitivity. The prompt used to instruct the language models is fixed for all models and experiments, or is not expressive enough for the given task. This allows for prompt-based fluctuations in evaluations.



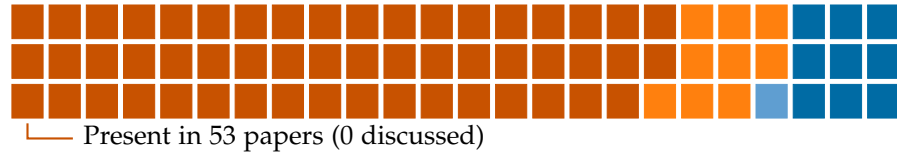
Results & Implications Prompt Sensitivity affected 23 papers (31.9%) in our analysis: *Present* in 18.1% (13) of the papers (3 of which discussed the pitfall), *Likely present* in 6.9% (5) of the papers, and *Partly present* in 6.9% (5) of the papers (2 of which discussed it). If prompt formatting is not adapted to the model, e.g., by using incorrect delimiters or generic instructions, this can reduce model performance or lead to misleading evaluations. In tasks such as jailbreak detection or alignment testing, vague prompts (e.g., “You are a helpful AI assistant”) may under represent a model’s true capabilities or vulnerabilities. As a result, evaluations might miss important failure modes or overstate alignment, leading to flawed comparisons across models [Scl+24].

P8—Proxy/Surrogate Fallacy. Findings from specific LLMs are often inappropriately generalized to other, often larger and more capable models or even to entire classes of language models, without sufficient empirical validation.



Results & Implications Surrogate Fallacy is highly prevalent, appearing in 47.2% (34) of the papers: *Present* in 19.4% (14) papers (3 of which discussed the pitfall), *Partly present* in 26.4% (19) of the papers (2 of which discussed it), and 1 paper is marked as *Likely present*. Due to the distinct differences in behavior, claims about specific LLMs do not generalize to other models. Especially in security, where attacks and defenses hinge on specific model behavior, extrapolating results from smaller open-source models to larger proprietary ones is methodologically flawed and risks overstating generalizability.

P9—Model Ambiguity. The model details are insufficient for precise identification, preventing reproducibility (e. g., missing model ID, snapshot, commit ID, quantization level).



Results & Implications This is the most prevalent pitfall in our study: It was *Present* in 73.6% (53) of all papers, and *Partly present* in 12.5% (9). Notably, none of them acknowledged or discussed this issue, indicating a low level of community awareness. The lack of precise versioning details makes it nearly impossible to reproduce results reliably. Vendors frequently update LLMs to improve reliability or harden them against attacks, changes that can alter behavior without changing the model name. Similarly, quantization variants in open-source models can produce different outputs due to reduced precision. Microsoft highlighted this in the case of *Phi3mini4k*, where a major update significantly improved benchmarks; they published results for both versions and documented the relevant commit [Abd+24; Mic25].

5.3.4 Main Findings

Every pitfall occurs in multiple papers, and every paper contains at least one pitfall that is either fully or partly present (cf. Figure 5.7 in the Appendix for a side-by-side comparison of all pitfalls). The most common issues, present in over 20% of papers, are *Data Leakage* (P3), *Context Truncation* (P6), *Model Ambiguity* (P9), *Prompt Sensitivity* (P7), and *Surrogate Fallacy* (P8). We analyze the impact of four of these in §5.4.

Furthermore, for three pitfalls, namely *Data Leakage* (P3), *Data Poisoning* (P1), and *Spurious Correlations* (P5), the *Likely present* label applies to more than 20% of papers, which means that the pitfall was probably present despite the absence of explicit evidence.

Finding. Every paper in our study contains at least one pitfall that is either fully or partly present.

Discussed vs. Not Discussed. Overall, only 15.71% of pitfalls rated at least *Likely present* (*Likely present*, *Partly present*, or *Present*) were discussed by the authors of the papers. The three most frequently discussed pitfalls were *Label Inaccuracy* (P2) with 60.0%, *Context Truncation* (P6) with 41.67%, and *Data Leakage* (P3) with 21.28% of cases being explicitly addressed. In contrast, some pitfalls received no discussion at all: *Model Collapse* (P4), *Data Poisoning* (P1), and *Model Ambiguity* (P9), indicating a lack of awareness of these issues.

5.3 PREVALENCE OF PITFALLS

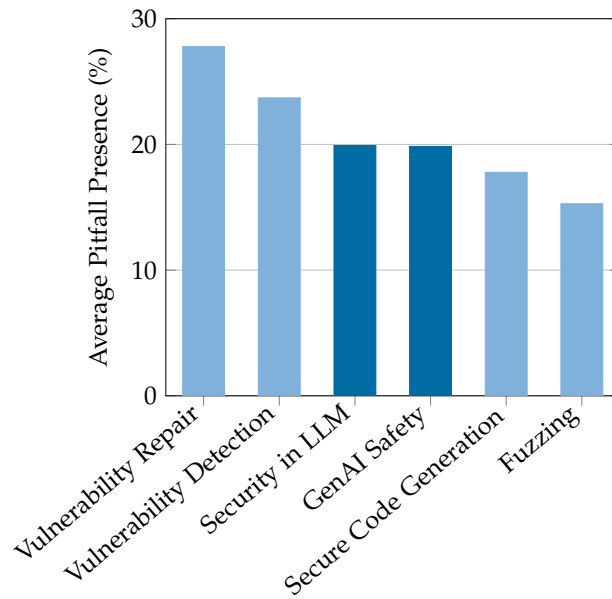


Figure 5.3: Percentage of pitfalls labeled as *Present* (including cases where the pitfall was discussed), averaged over all pitfalls and papers within each topic. Light blue bars (●) represent research that uses LLMs to address security problems. Dark blue (●) bars correspond to research focused on the security and safety of LLMs themselves.

Finding. Only 15.71% of pitfalls are discussed, with three pitfalls (Model Collapse via Synthetic Training Data (P4), Data Poisoning via Internet Scraping (P1), and Model Ambiguity (P9)) remaining entirely undiscussed across all papers in our study.

Topics. We further analyze the prevalence of pitfalls across the six research topics defined in our paper selection process. Figure 5.3 shows the average percentage of pitfalls per paper within each topic. This reflects the proportion of pitfalls labeled as *Present*, including those where the pitfall was discussed.

The results show notable differences across topics. *Vulnerability Repair* papers stand out with the highest average pitfall presence (27.78%), followed by *Vulnerability Detection* (23.70%), indicating a high density of pitfalls in work that uses LLMs to find or fix software vulnerabilities. At the other end, *Fuzzing* exhibits the lowest pitfall presence (15.28%). *Secure Code Generation* averages 17.78%, suggesting this subfield is comparatively more robust with respect to the pitfalls we assessed. Research under *Security in LLMs* (19.91%) and *GenAI Safety* (19.84%) are close to 20%.

Finding. LLM-based research on Vulnerability Detection and Vulnerability Repair has the highest average number of pitfalls, whereas Secure Code Generation and Fuzzing papers tend to avoid such issues more consistently.

5.4 IMPACT ANALYSIS

Having assessed the prevalence of pitfalls, we now examine their potential impact in greater detail. To this end, we focus on four of the most prevalent pitfalls identified in §5.3, each of which appears in at least 20% of the papers. Specifically, we examine the impact of Model Ambiguity and the Surrogate Fallacy (§5.4.1), the influence of Data Leakage on experimental results (§5.4.2), and the limitations that Context Truncation imposes on model capabilities (§5.4.3).

Together, these pitfalls illustrate the broader risks they pose to the integrity and reliability of LLM research. Another issue that appears in over 20% of the reviewed papers is Prompt Sensitivity. While important, Prompt Sensitivity has already been extensively studied in prior work (e.g., [Scl+24; Cao+24]). For this reason, we do not provide a separate analysis here. Instead, we consider Model Collapse (§5.4.4), which has received little attention in the security context so far but has recently been shown to degrade model reliability in the text domain [Shu+24]. We argue that understanding the potential implications for security research is necessary, especially as fine-tuned or retrained models become increasingly common.

5.4.1 *Model Ambiguity & Surrogate Fallacy*

The most widespread issue in our analysis is Model Ambiguity, which is present in 73.6% of all papers. To examine the potential impact of this pitfall on experimental results, we first conduct a targeted evaluation of hate speech detection using state-of-the-art proprietary LLMs across multiple snapshots of the same base model. We then evaluate the robustness of proprietary models against prompt-based attacks across different model snapshots, as well as that of open-source models with varying levels of quantization. In this context, we also examine the impact of another common pitfall, the Surrogate Fallacy, which appears in 47% of the papers.

5.4.1.1 **Hate Detection**

In the first experiment, we analyze the impact of (missing) LLM version or snapshot information for the example of hate speech detection. To this end, we revisit a recently proposed method to prevent waves of hateful comments that often build up for a particular topic, as happened, for instance, during the COVID-19 pandemic on X (formerly Twitter) [Vis+24]. The core idea is to use an LLM to benefit from its enhanced reasoning capabilities, as detecting hate with all its nuances and newly introduced derogatory terms is a challenging task. Given a text, the LLM is instructed to answer a series of questions designed to guide its reasoning process.

Experimental Setup. We re-implement the experiments from Vishwamitra et al. [Vis+24] with their pre-labeled dataset of hateful and

Table 5.1: Model Ambiguity in Hate Detection

Model	Accuracy	Precision	Recall
<i>gpt-4-0613</i>	88.70 %	87.07 %	82.05 %
<i>gpt-4-0125-preview</i>	77.16 %	95.52 %	41.03 %
<i>gpt-4.1-2025-04-14</i>	82.45 %	88.07 %	61.54 %

normal tweets from $\mathbb{X}$. This dataset contains tweets from three polarizing topics termed hate waves (COVID-19 pandemic, US Capitol insurrection, and Russia’s invasion of Ukraine). The COVID-19 pandemic is further divided into subtopics, from which we focus on the “vaccine” subtopic. The calibration and testing for each topic are done separately, with each hate wave divided into four quarters. For simplicity, we calibrate the detection method using examples from the first quarter and test on the second quarter of each wave. The original publication stated the use of GPT-4. Thus, we test three GPT-4 snapshots available at OpenAI at the time of writing.

The detection is done by extracting keywords using KeyBERT and testing the novelty with NLTK’s WordNet. Detailed specifications for all models can be found in Table 5.6 in Appendix E. We then provide the text and the extracted targets and terms to the LLM *gpt-4.1-mini-2025-04-14* and instruct it to check which keywords are a target or derogatory term. Equipped with these new targets and terms, the prompt template from the original work [Vis+24] is used, which instructs an LLM to answer a series of questions to determine if a text is hateful or not. The extracted targets and terms are appended at the end of the prompt. Finally, we evaluate the LLM on the test set to decide on each posting and record this as its final decision.

Results. Table 5.1 demonstrates that the detection performance varies considerably across the different GPT snapshots.

An analysis of the LLM outputs suggests a shift in how different AI models classify hate speech. We exemplify this using the tweet “Say it to my face, you tide pod eating, unvaccinated fuck”, which was labeled as *hate* in the dataset. The *gpt-4-0613* model identifies derogatory and insulting words directed at anti-vaxxers and therefore categorizes the text as identity hate. In contrast, *gpt-4.1-2025-04-14* also recognizes the text’s insulting content but determines that the language does not meet the threshold for identity hate, as it does not incite violence or hatred. The model considers the text as toxic only. In general, this observation is also reflected in the precision and recall rates in Table 5.1. The models *gpt-4-0125-preview* and *gpt-4.1-2025-04-14* flag fewer tweets as hateful due to the higher threshold for hate, resulting in a significant drop in the recall rate. At the same time, the precision increases due to the stricter detection. While it is likely that such behavior can be adjusted through more tailored prompting, our aim with this experiment is to highlight the considerable, underestimated impact of the LLM version.

Table 5.2: Attack success rates (ASR) across different quantization levels, inference engines (if applicable), and model architectures. Each configuration is evaluated using 16 prompt-based attacks, evenly distributed across a total of 1,000 trials.

Model	Quantization						
	2-bit	3-bit	4-bit	5-bit	6-bit	8-bit	No Quant.
CodeLlama 7b (Ollama)	69.52%	40.95%	15.49%	18.61%	15.69%	14.29%	18.21%
CodeLlama 7b (TheBloke)	67.35%	70.41%	58.16%	62.24%	61.22%	63.27%	19.39%
Llama 2 7b (Ollama)	1.61%	8.05%	4.02%	6.04%	6.64%	5.73%	6.14%
Llama 2 7b (TheBloke)	13.88%	17.35%	29.59%	25.51%	7.24%	6.64%	11.22%
Llama 3.1 8b (Ollama)	32.33%	22.59%	16.87%	18.37%	11.35%	10.04%	21.18%

5.4.1.2 LLM Robustness

We continue by evaluating the robustness of several LLMs against a suite of prompt-based attacks, including prompt injections, adversarial suffixes, and complex jailbreaks [Eve+24]. To do this, we adopt the *secret-key game* introduced by Evertz et al. [Eve+24]. In this setup, the model is given a *secret key* in its system prompt, along with explicit instructions to keep the key confidential. The attack then attempts to elicit the secret from the model. If the model outputs the key as part of its response, the attack is considered successful. Each model is evaluated over 1,000 attack attempts, distributed evenly across 16 attack types. To minimize bias, system prompts are sampled from a set of 1,000 randomly selected variations.

We first examine proprietary models across multiple snapshots. As in the previous case, it is common to refer to these models by general handles (e. g., GPT-4o), but these labels may internally refer to different model snapshots. As of the time of writing, we evaluate all available snapshots of OpenAI’s GPT-3.5 Turbo, GPT-4, GPT-4 Turbo, and GPT-4o.

In contrast to proprietary models, open-source models are typically not accessed through APIs and therefore do not expose versioned snapshots under the same identifier. Instead, on platforms such as Huggingface [Hug25] and Ollama [Oll25], they are provided in different variations of quantization, and older snapshots can be accessed via their commit history. To complement the snapshot-based evaluation of proprietary models, we analyze how quantization affects robustness in open-source models. We test various quantization levels (2-bit through 8-bit) as well as the original unquantized snapshots. We evaluate three models (CodeLlama-7b [Roz+24], Llama-2-7b [Tou+23], and Llama3.1 8b [AI24]) and two different inference engines (Ollama [Oll25] and TheBloke [Job24]). Detailed specifications for all models are listed in Table 5.7 in Appendix E.

Results. Table 5.3 presents the results for different snapshots of OpenAI’s GPT. We observe that different snapshots lead to meaningful differences in robustness. In particular, GPT-3.5 Turbo exhibits significant variability across snapshots, though smaller but still notable differences are visible for the other models as well. The effect of different quantization levels is summarized in Table 5.2. Interestingly,

Table 5.3: Attack success rate (ASR) across snapshots of current OpenAI models evaluated on 16 different prompt-based attacks over 1,000 iterations per snapshot in total.

Model	ASR	Model	ASR
GPT-3.5 Turbo		GPT-4	
↪ <i>gpt-3.5-turbo-1106</i>	4.63 %	↪ <i>gpt-4-0314</i>	9.56 %
↪ <i>gpt-3.5-turbo-0125</i>	16.11 %	↪ <i>gpt-4-0613</i>	8.05 %
GPT-4 Turbo		GPT-4o	
↪ <i>gpt-4-1106-preview</i>	1.81 %	↪ <i>gpt-4o-2024-05-13</i>	1.01 %
↪ <i>gpt-4-0125-preview</i>	2.13 %	↪ <i>gpt-4o-2024-08-06</i>	0.10 %
↪ <i>gpt-4-turbo-2024-04-09</i>	2.31 %	↪ <i>gpt-4o-2024-11-20</i>	0.20 %

models with stronger quantization, such as 2-bit and 3-bit, appear more vulnerable to prompt-based attacks. Additionally, the results indicate a difference between inference engines. Note that these results not only demonstrate the potentially severe impact of the Model Ambiguity pitfall, but also serve as an example for the Surrogate Fallacy, since they show that findings do not necessarily transfer to other and particular larger models.

Overall, these findings highlight the importance of reporting detailed model information, including snapshot identifiers and quantization parameters. Such details are essential for the reproducibility and interpretability of experimental results. The observations are further supported by recent disclosures from Microsoft, showing that performance-relevant updates are being applied to their hosted models on Huggingface. For example, the *Phi 3 mini 4k instruct* model received an update together with a full disclosure about its new performance scores⁴.

5.4.2 Data Leakage

We continue with the analysis of another very prevalent pitfall, Data Leakage, which occurs in 57% of all papers we considered. LLMs are typically trained on broad snapshots of public internet data collected up to a fixed cutoff date [Bro+20]. This practice introduces a critical concern: public benchmark datasets used to evaluate model performance may have been included in the training data, thereby undermining the validity of such evaluations.

To examine this issue, we focus on vulnerability detection as a representative task within LLM security research. It is widely studied, relies on well-known datasets, and is commonly used to benchmark the capabilities of code-oriented LLMs. We first conduct a controlled experiment designed to simulate leakage under lab conditions, followed by an empirical analysis of commercial models.

⁴ <https://huggingface.co/microsoft/Phi-3-mini-4k-instruct>; see Release Notes and June 2024 Update

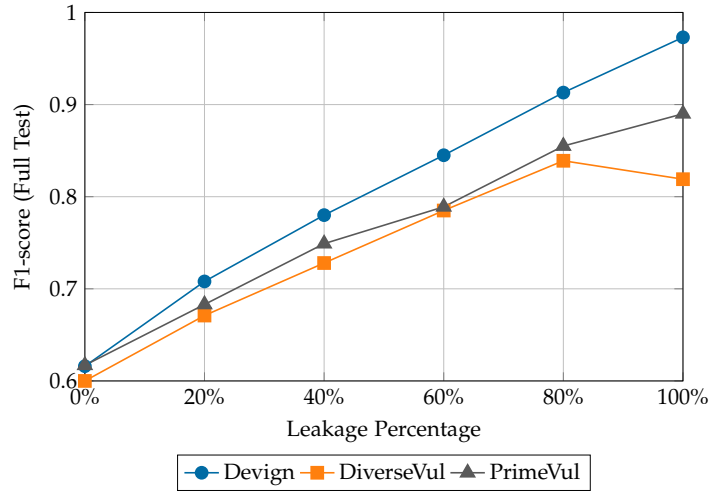


Figure 5.4: F1-scores of CodeT5+ models fine-tuned on *Devign*, *DiverseVul*, and *PrimeVul* with varying amounts of test data (0–100%) leaked into the training data.

5.4.2.1 Leakage in Lab Setting

We consider three widely used vulnerability detection datasets: *Devign* [Zho+19], *DiverseVul* [Che+23], and *PrimeVul* [Din+25]. For each dataset, we construct splits for training (60%), validation (20%) and testing (20%), and fine-tune a CodeT5+ [Wan+23b] model. Finally, we evaluate each model on the test split to obtain a baseline F1-score.

To simulate leakage, we fine-tune five additional models per dataset for 10 epochs, each trained on an increasingly larger fraction of the test set added to the training data (20%, 40%, 60%, 80%, 100%). We deliberately ignore the effects of limited context window size (512 tokens) and potential spurious correlations, as these factors apply equally across all data splits and do not affect the relative comparison. Full model configurations are provided in Appendix E.3.

Results. The results are depicted in Figure 5.4. We observe a near-linear increase in F1-score with the degree of test set leakage. Even a modest leakage of 20% leads to a gain of 0.08–0.11 over the baseline. This demonstrates that even partial leakage can significantly inflate evaluation metrics, leading to overly optimistic conclusions.

5.4.2.2 Evaluation on Commercial LLMs

Having observed the effects of leakage under controlled conditions, we now turn to the question of whether similar effects can be found in proprietary LLMs. The main challenge in this setting is the lack of transparency regarding the training data and procedures used for these models. To investigate this, we focus on the *PrimeVul* [Din+25] dataset. It consists of real-world vulnerabilities in public C/C++ projects, making it a plausible candidate for inclusion in the pre-training data of proprietary models. We evaluate four proprietary models from different vendors (*gpt-3.5-turbo-0125*, *gpt-4o-2024-08-06*, *DeepSeek-V3-0324*,

claude-3-5-haiku-20241022) as well as three open-source models (*meta-llama-3-8b-instruct*, *qwen3-14b*, *qwen2.5-coder-14b*). For all evaluations, we set the temperature to 0. Detailed model information is available in Appendix E.3.

For each model, we conduct the following two experiments: (1) We sample 100 commits from the dataset, extract the original commit messages, and prompt the model with the project name, commit SHA, and the first half of the message, asking it to reconstruct the full commit message. (2) We repeat this process, using 100 sampled functions. We provide the same context, but instead of the commit message, we prompt the model with the first half of the function body and ask it to reconstruct the complete function.

Results. Surprisingly, none of the models successfully reconstructed a single commit message. Even allowing for fuzzy matching ($\geq 75\%$ word-level Jaccard similarity), all models yielded 0 out of 100 matches. The function completion task showed the same pattern: no exact or fuzzy matches across any of the models. To rule out the possibility that the alignment procedures may suppress memorization, we fine-tune *gpt-3.5-turbo-0125* on 1,000 full commit messages (following the divergence attack from Carlini et al. [Nas+25]). Despite this targeted fine-tuning, the model still failed to reproduce any of the original messages.

To investigate whether these commits may have been absent from the model’s pre-training data, we searched all *Common Crawl* snapshots listed in the GPT-3 paper [Bro+20] for any URLs related to the repositories used in *PrimeVul* (with wildcards). We found no matches. While we cannot rule out the use of private or proprietary data sources, the absence of *PrimeVul*-related content in *Common Crawl* suggests that *PrimeVul*-style data was likely not part of the pre-training mix.

These findings are somewhat unexpected, given that *PrimeVul* contains data from many of the most prominent and widely used open-source C/C++ projects on GitHub. Nonetheless, we found no evidence that any of the tested models were trained on *PrimeVul* commits or functions.

Discussion. This finding is both surprising and encouraging. Despite the prominence of the underlying projects in *PrimeVul*, we find no evidence that the evaluated proprietary and open-source models were trained on these. While we cannot definitively rule out the possibility of data leakage, the results provide strong evidence that data leakage is unlikely in this case. To support future evaluations and reduce the risk of leakage, we recommend the following: Ideally, test data should be drawn entirely from after the model’s training cut-off date. In this case, it can be used with high confidence. If the test data spans both before and after the cut-off, a pre/post probing analysis can help: a significant drop in performance on the post cut-off portion may indicate the absence of leakage. If the test data predates the cut-off or if timing is unclear, targeted memorization probes can be performed as done in our reconstruction experiments. To aid this evaluation,

Appendix F summarizes publicly disclosed training data sources for several representative language models.

5.4.3 Context Truncation

Next, we shift our focus to prompting, specifically the issue of limited context windows. This pitfall appears in 28% of the reviewed papers. To this end, we examine how constrained context length can affect the reliability of performance measurements and potentially lead to misleading conclusions. More specifically, our goal is to understand whether limited context during training or evaluation can prevent a model from detecting vulnerabilities, thereby producing misleading performance results. For instance, *CVE-2014-2669*⁵ illustrates a case where the line responsible for an integer overflow lies outside a 512-token context window; hence, a model would be unable to detect it since the full context is not available.

Experimental Setup. Among the 15 papers on vulnerability detection included in our study, we observe that eight papers use at least one model with a context size of 512 tokens or less, one paper uses a model with a 1024-token context size, and eight papers use at least one model with a context size larger than 2048 tokens.

To assess whether these context sizes are sufficient, we analyze function lengths in the datasets used in §5.4.2. Using the CodeT5 [Wan+21] tokenizer, we compute the number of tokens for each function labeled as vulnerable. We then calculate the proportion of functions in each dataset that exceed context limits of 512, 1024, and 2048 tokens. This allows to quantify the proportion of functions that are partially truncated under typical context size assumptions. Further details on the tokenizer are provided in Appendix E.4.

Results. We find that a substantial proportion of vulnerable functions exceed the context sizes used in papers on vulnerability detection in our study. As shown in Table 5.4, on average, 49.3% of all vulnerable functions across *Devign*, *DiverseVul*, and *PrimeVul* contain more than 512 tokens when tokenized with the CodeT5 tokenizer. At the 1024-token threshold, 29.1% still exceed the limit, and 13.7% surpass 2048 tokens.

These findings show that using limited context windows systematically truncates a large portion of vulnerable functions, potentially removing the very code that contains or explains the vulnerability. As such, evaluation under small context settings may fundamentally misrepresent a model’s performance and lead to misleading conclusions about its ability to detect vulnerabilities. Notably, these percentages represent a lower bound on the issue; Risse et al. [RLB25] show that vulnerability detection often depends on code outside the function itself, meaning even full-function context may be insufficient in many cases.

⁵ <https://nvd.nist.gov/vuln/detail/CVE-2014-2669>; see Appendix E.4.

Table 5.4: Proportion of functions labeled as *vulnerable* whose CodeT5-tokenized length exceeds context-window sizes used in the papers on vulnerability detection identified by our pitfall study (512, 1,024, and 2,048 tokens).

Dataset	# Funcs	# Tokens		
		> 512	> 1024	> 2048
Devign	12,460	5,196 (41.7%)	2,642 (21.2%)	984 (7.9%)
DiverseVul	18,945	9,814 (51.8%)	5,835 (30.8%)	2,747 (14.5%)
PrimeVul	6,004	3,897 (64.9%)	2,594 (43.2%)	1,357 (22.6%)
Average	–	52.8%	31.7%	15.0%

5.4.4 Model Collapse

Finally, we consider another important aspect of LLM training: the potential effects of model collapse, which we found to be present in 13.9% of the papers. Repeated training on synthetic data, particularly content generated by models, can gradually degrade model quality, amplify existing biases, and reduce output diversity [Shu+24].

Experimental Setup. We build on the setup introduced by Shumailov et al. [Shu+24], who studied model collapse in natural language generation. Code generation is more challenging than generating natural language text, as source code is subject to stricter syntactic and semantic constraints. For this experiment, we use the Qwen2.5-Coder-0.5B-Instruct [Yan+24b; Hui+24a] model (full configuration details are provided in Appendix E.2) and consider training sequences with a fixed length of 128 tokens in line with the setup from Shumailov et al. [Shu+24].

We begin by fine-tuning the base model on the *self-oss-instruct-sc2-exec-filter-50k* [Tea24; Cas+24] dataset, which contains 50,000 Python code samples. For each sequence, we predict up to 2,048 subsequent tokens to generate synthetic samples, ensuring that no code sample is truncated or incomplete. These synthetic outputs form a new dataset of equal sample size, which is then used to fine-tune the next model iteration. We repeat this process for ten generations, where each model is trained on data generated by its immediate predecessor. Although it cannot be ruled out that the base model is trained on (subsets) of this dataset, the experiment’s primary focus is on the re-training on LLM-generated data. Hence, a possible data leakage would not affect the results.

Results. Figure 5.5 shows the distribution of perplexities across generations, measured by evaluating each generation’s outputs using the fine-tuned base model. *Generation 0* refers to the fine-tuned base model evaluated on the unmodified training data. *Generation 1* corresponds to the synthetic outputs from the first-generation model, and so on.

We observe a clear trend of degradation in model performance. With each generation, the distribution shifts to the right, indicating increased perplexity and suggesting that the base model becomes increasingly uncertain when predicting tokens in samples produced by later generations. We also observe growing variance in perplexity

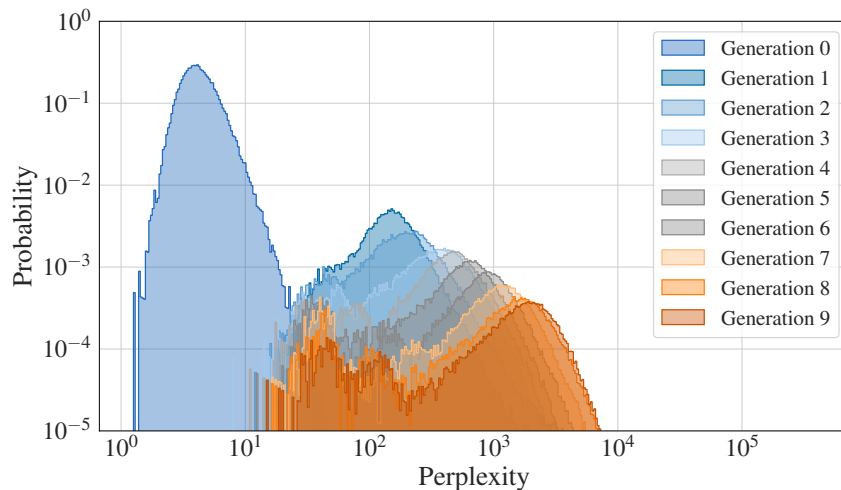


Figure 5.5: Distribution of perplexities for training samples generated across multiple model generations. Perplexity is measured using the original model fine-tuned on real data. Both the mean and variance of perplexity increase over generations, suggesting that models become less stable when repeatedly trained on synthetic data.

across generations, reflecting greater instability in models trained on synthetic data, and indicating that errors accumulate over time. Furthermore, models trained primarily on synthetic outputs fail to generalize effectively.

These results extend prior concerns raised by Shumailov et al. [Shu+24] to the code domain. Our findings suggest that similar degradation can occur in code-related applications as well. This trend is especially concerning as AI-powered coding assistants become more widespread and, thus, an increasing share of source code may be generated by models rather than written by humans. From a security perspective, this trend also poses a serious risk. For example, when the underlying models are used for code generation, their reliability may degrade over time, potentially introducing subtle bugs or vulnerabilities. Similarly, when such models are used for, e.g., vulnerability detection, the degradation of the model may result in a lower detection rate.

5.5 RECOMMENDATIONS

Our analysis reveals a concerning pattern: every reviewed paper suffers from at least one pitfall, and each pitfall appears in multiple papers. In the following, we propose recommendations that directly target the pitfalls while considering real-world constraints such as the use of proprietary systems, limited access to training data, and computational limitations.

We provide an extended discussion covering each pitfall individually in Appendix C. In addition to this discussion, we offer a project

website with guidelines to avoid pitfalls and a living appendix where all information is open for further contributions and kept up to date. The website is available at <https://llmpitfalls.org>.

R1: Transparency and Reproducibility. Transparent reporting and reproducibility are crucial in research. Even if full reproducibility is not always feasible, especially with proprietary APIs, researchers should share enough information to enable others to interpret, replicate, or critically evaluate the findings.

We recommend the following reporting practices:

- Specify the exact model used, including version identifier, access method (e.g., API or interface), and access date. For open-source models, include repository links, commit hashes, and any fine-tuning steps that were taken. Describe the evaluation pipeline in detail, including prompt format, quantization level, decoding settings, and any post-processing. As shown in §5.4.1, missing or vague model details can lead to substantial variation in outcomes and hinder reproducibility (→ *Model Ambiguity (P9)*).
- If evaluation labels are generated by language models or if LLMs are used as judges, disclose this and assess label quality via manual review on a statistically meaningful subset. Report inter-annotator agreement where applicable (→ *LLM-generated Label Inaccuracy (P2)*).
- Avoid generalizing beyond what the evidence supports. Claims should be scoped to the specific models evaluated unless a diverse and representative set of models is tested and limitations are explicitly acknowledged (→ *Proxy/Surrogate Fallacy (P8)*).

R2: Understand and Communicate Data Quality. Data plays a central role in both training and evaluation of language models. Whether data is synthetic, scraped, or manually labeled, its origin, quality, and security can significantly influence model behavior and reported results.

To mitigate these risks, we recommend:

- When using synthetic or LLM-generated data for training, clearly report its proportion relative to real data and assess distributional differences. In iterative self-training setups, monitor for signs of model collapse or performance degradation. As demonstrated in §5.4.4, iterative training on synthetic data can lead to significant performance decline and instability over time (→ *Model Collapse via Synthetic Training Data (P4)*).
- For tasks with realistic poisoning threats, such as vulnerability detection or misinformation detection, explicitly assess whether poisoning is plausible in the training setup, particularly when relying on large-scale web data or proprietary models (→ *Data Poisoning via Internet Scraping (P1)*).

- Acknowledge any risk of data leakage by checking whether evaluation data (especially answers or labels) was publicly available prior to the model’s training cutoff date. Where cutoff dates are known, analyze performance on pre- vs. post-cutoff data and consider probing for memorization. We demonstrate the impacts of potential data leakage in §5.4.2. Whenever possible, prefer models for which information about training data composition is available (see Appendix F). As a community, we should push for state-of-the-art models with transparent or public training data. While this is often not possible for proprietary models, ongoing regulatory efforts such as the EU AI Act’s transparency requirements for General-Purpose AI (GPAI) providers [Com25] are expected to improve disclosure practices (→ *Data Leakage (P3)*).

R3: Alignment between Models and Tasks. Many evaluation failures stem not from the models themselves, but from a mismatch between the model constraints and the task requirements. Researchers should assess whether the models used are capable of handling the inputs, outputs, and task complexity involved and report limitations clearly.

To that end:

- Report the model’s maximum context window and analyze whether typical input sizes (including prompts) exceed this limit. If truncation occurs frequently, quantify how often and assess the potential impact on performance. Detailed evaluation of possible side effects of a too small context size is given in §5.4.3 (→ *Context Truncation (P6)*).
- Recognize that prompt design can meaningfully influence model behavior. When feasible, use established prompting techniques or conduct prompt variation experiments to gauge robustness. If prompt optimization is not applicable, explain why (→ *Prompt Sensitivity (P7)*).
- Evaluate whether the model may be exploiting spurious correlations. Use robustness checks (e. g., input perturbations, ablations) or feature attribution techniques to test whether the model relies on unintended features (→ *Spurious Correlations/Unrelated Features (P5)*).

5.6 DISCUSSION

Our study provides a systematic analysis of common pitfalls in LLM security research. However, like any such effort, it is not without limitations. In the following, we reflect on potential threats to validity and the generalizability of our findings, as well as the broader implications of the issues we uncovered.

Addressing Inevitable Pitfalls. Some pitfalls in LLM research are difficult to avoid due to the inherent complexity and scale of current

systems. The vast amount of data required to train LLMs introduces inherently challenging-to-manage risks. For example, Data Poisoning attacks (P1), although realistic in principle, are challenging to detect and mitigate, especially when experiments rely on publicly available data. Similarly, the lack of transparency from LLM providers regarding training corpora increases the likelihood of Data Leakage (P3). In addition, the complexity and often opaque behavior of LLMs can lead to Spurious Correlations (P5) that are difficult to identify or explain. Although these issues cannot always be fully resolved (see §5.5 for possible mitigation strategies), researchers must acknowledge, address, and ideally explicitly discuss such pitfalls when reporting their findings.

Coverage of Literature and Pitfalls. Our paper selection may not be exhaustive. Relevant work may have been missed due to limitations in our keyword filters or because it appeared outside the selected venues. To reduce this risk, we conducted an intentionally broad search across eight leading security and software engineering conferences, used an extensive keyword list, and applied a two-stage manual selection process. This resulted in 72 in-scope papers. While omissions remain possible, we believe they are unlikely to affect our qualitative conclusions.

Likewise, other pitfalls may exist beyond the nine we identified for our study. We iteratively refined our pitfall list and definitions during test reviews with a diverse reviewing team to limit this threat. We encourage future work to build on and extend this taxonomy.

Review Subjectivity. While some degree of subjectivity is unavoidable in qualitative reviews, we took several steps to improve consistency. Each paper was reviewed by two independent reviewers, who were blinded during the first phase of evaluation. We then conducted a structured discussion phase to reconcile disagreements.

To quantify inter-reviewer consistency, we computed Fleiss' κ for each pitfall. Overall agreement was solid, with a mean κ of 0.55 across all pitfalls. Out of 648 individual ratings (72 papers $\times$ 9 pitfalls), only 25 required resolution through discussion in the larger reviewer group. This corresponds to less than 5% of all decisions. Most disagreements were minor and resolved through brief conversations between reviewers. Full κ values and qualitative interpretations are reported in Table 5.5 in the Appendix.

Generalizability of the Impact Analysis. Our experimental study focuses on four case studies and five selected pitfalls, covering key stages of the LLM pipeline. Thus, four pitfalls remain unexplored in the experimental analysis, and even for the five chosen ones, their impact may vary across tasks, datasets, and model families. We emphasize that the impact analysis aims to demonstrate plausible mechanisms and concrete examples rather than provide generalizable effect sizes. As such, the results should be interpreted as illustrative lower bounds, not as comprehensive or definitive impact measures.

5.7 CONCLUSION

In this work, we systematically reviewed 72 papers from leading security and software engineering conferences and identified nine recurring pitfalls in LLM security research. Every paper we considered contained at least one of these pitfalls, and each pitfall appeared across multiple studies. Our impact analysis shows how seemingly minor issues, such as Data Leakage, Context Truncation, or Model Ambiguity, can distort results, reduce reliability, and undermine reproducibility. Together, these findings highlight the broader risks these pitfalls pose to the integrity of LLM security research and underscore the need for more rigorous, transparent, and reproducible practices.

Looking ahead, strengthening LLM security research involves applying the lessons we outline: reporting exact model identifiers, testing for leakage, using task-appropriate context sizes, verifying LLM-generated labels, and avoiding overgeneralization. Our guidelines are not meant to serve as a fixed checklist. However, by treating these pitfalls as design constraints rather than afterthoughts, the community can move from “chasing shadows” to conducting LLM security research that is reproducible, trustworthy, and grounded in evidence.

ETHICAL CONSIDERATIONS

This work critically examines methodological patterns in LLM security research. To ensure transparency while minimizing the risk of reputational harm, we cite all sources transparently but report pitfalls in aggregate. We deliberately refrain from assigning specific pitfalls to individual papers. Corresponding authors of the analyzed work are welcome to contact us for private discussion and clarification.

Our aim is constructive. Alongside our critique, we provide concrete, community-oriented guidelines to support more rigorous, transparent, and reproducible practices in LLM-focused security research.

ACKNOWLEDGMENTS

This work was funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Germany’s Excellence Strategy - EXC 2092 CASA - 390781972 and under the project ALISON (492020528), the Vienna Science and Technology Fund (WWTF) under the project BREADS (10.47379/VRG23011), the Helmholtz Association (HGF) within the topic “46.23 Engineering Secure Systems”, the German Federal Ministry of Education and Research under the grant AIGenCY (16KIS2012) and SisWiss (16KIS2330), the European Research Council (ERC) under the consolidator grant MALFOY (101043410), and the LCIS center VW-Vorab-2025, ZN4704 11-76251-2055. Additionally, Srishti Gupta was enrolled in the Italian National Doctorate on AI run by the Sapienza University of Rome in collaboration with the University of Cagliari during this project.

REFERENCES

- [AB+23] Rohan Anil, Sebastian Borgeaud, et al. "PaLM 2 Technical Report". In: *arXiv preprint* (2023). DOI: [10.48550/arXiv.2305.10403](https://doi.org/10.48550/arXiv.2305.10403).
- [Abd+24] Marah Abdin, Jyoti Aneja, Hany Awadalla, Ahmed Awadallah, Ammar Ahmad Awan, et al. "Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone". In: *arXiv preprint* (2024). DOI: [10.48550/arXiv.2404.14219](https://doi.org/10.48550/arXiv.2404.14219).
- [Agh+24] Hojjat Aghakhani, Wei Dai, Andre Manoel, Xavier Fernandes, Anant Kharkar, Christopher Kruegel, Giovanni Vigna, David Evans, Ben Zorn, and Robert Sim. "Trojan-Puzzle: Covertly Poisoning Code-Suggestion Models". In: *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*. 2024. DOI: [10.1109/sp54263.2024.00140](https://doi.org/10.1109/sp54263.2024.00140).
- [AI24] Meta AI. *Introducing Llama 3.1: Our Most Capable Models to Date*. Meta AI Blog. Blog post. 2024. URL: <https://ai.meta.com/blog/meta-llama-3-1/>.
- [Ale+24] Sina Alemohammad, Josue Casco-Rodriguez, Lorenzo Luzi, Ahmed Imtiaz Humayun, Hossein Babaei, Daniel LeJeune, Ali Siahkoohi, and Richard Baraniuk. "Self-Consuming Generative Models Go MAD". In: *Proceedings of the International Conference on Learning Representations (ICLR)*. 2024. URL: <https://openreview.net/forum?id=ShjMHfmPs0>.
- [AML12] Yaser S. Abu-Mostafa, Malik Magdon-Ismail, and Hsuan-Tien Lin. *Learning From Data*. AMLBook, 2012. URL: <https://amlbook.com>.
- [Ant25] Anthropic. *Anthropic Claude*. 2025. URL: <https://claude.ai/>.
- [Arp+22] Daniel Arp, Erwin Quiring, Feargus Pendlebury, Alexander Warnecke, Fabio Pierazzi, Christian Wressnegger, Lorenzo Cavallaro, and Konrad Rieck. "Dos and don'ts of machine learning in computer security". In: *Proceedings of the 31st USENIX Security Symposium (USENIX Security)*. 2022. URL: <https://www.usenix.org/conference/usenixsecurity22/presentation/arp>.
- [Asm+24] Asmita, Yaroslav Oliynyk, Michael Scott, Ryan Tsang, Chongzhou Fang, and Houman Homayoun. "Fuzzing BusyBox: Leveraging LLM and Crash Reuse for Embedded Bug Unearthing". In: *Proceedings of the 33rd USENIX Security Symposium (USENIX Security)*. 2024. URL: <https://www.usenix.org/conference/usenixsecurity24/presentation/asmita>.

- [Bet+24] Mazal Bethany, Brandon Wherry, Emet Bethany, Nishant Vishwamitra, Anthony Rios, and Peyman Najafirad. “Deciphering Textual Authenticity: A Generalized Strategy through the Lens of Large Language Semantics for Detecting Human vs. Machine-Generated Text”. In: *Proceedings of the 33rd USENIX Security Symposium (USENIX Security)*. 2024. URL: <https://www.usenix.org/conference/usenixsecurity24/presentation/bethany>.
- [Bow+24] Dillon Bowen, Brendan Murphy, Will Cai, David Khachaturov, and Adam Gleave. “Data Poisoning in LLMs: Jailbreak-Tuning and Scaling Laws”. In: *arXiv preprint* (2024). DOI: [10.48550/arXiv.2408.02946](https://doi.org/10.48550/arXiv.2408.02946).
- [Bro+20] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Nee-lakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. “Language Models are Few-Shot Learners”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2020. URL: <https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html>.
- [Cao+24] Bowen Cao, Deng Cai, Zhisong Zhang, Yuexian Zou, and Wai Lam. “On the Worst Prompt Performance of Large Language Models”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2024. URL: http://papers.nips.cc/paper_files/paper/2024/hash/7fa5a377b7ffabcce43cd00231bb3f9c-Abstract-Conference.html.
- [Car+19] Nicholas Carlini, Anish Athalye, Nicolas Papernot, Wieland Brendel, Jonas Rauber, Dimitris Tsipras, Ian Goodfellow, Aleksander Madry, and Alexey Kurakin. “On evaluating adversarial robustness”. In: *arXiv preprint* (2019). DOI: [10.48550/arXiv.1902.06705](https://doi.org/10.48550/arXiv.1902.06705).
- [Cas+24] Federico Cassano, John Gouwar, Francesca Lucchetti, Claire Schlesinger, Anders Freeman, Carolyn Jane Anderson, Molly Q Feldman, Michael Greenberg, Abhinav Jangda, and Arjun Guha. “Knowledge Transfer from High-Resource to Low-Resource Programming Languages for Code LLMs”. In: *Proceedings of the ACM on Programming Languages* (2024). DOI: [10.1145/3689735](https://doi.org/10.1145/3689735).
- [Che+21] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael

- Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. “Evaluating Large Language Models Trained on Code”. In: *arXiv preprint* (2021). DOI: [10.48550/arXiv.2107.03374](https://doi.org/10.48550/arXiv.2107.03374).
- [Che+23] Yizheng Chen, Zhoujie Ding, Lamya Alowain, Xinyun Chen, and David Wagner. “DiverseVul: A New Vulnerable Source Code Dataset for Deep Learning Based Vulnerability Detection”. In: *Proceedings of the International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*. 2023. DOI: [10.1145/3607199.3607242](https://doi.org/10.1145/3607199.3607242).
- [Che+24a] Guanzhong Chen, Zhenghan Qin, Mingxin Yang, Yajie Zhou, Tao Fan, Tianyu Du, and Zenglin Xu. “Unveiling the Vulnerability of Private Fine-Tuning in Split-Based Frameworks for Large Language Models: A Bidirectionally Enhanced Attack”. In: *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2024. DOI: [10.1145/3658644.3690295](https://doi.org/10.1145/3658644.3690295).
- [Che+24b] Jiachi Chen, Qingyuan Zhong, Yanlin Wang, Kaiwen Ning, Yongkun Liu, Zenan Xu, Zhe Zhao, Ting Chen, and Zibin Zheng. “RMCBench: Benchmarking Large Language Models’ Resistance to Malicious Code”. In: *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2024. DOI: [10.1145/3691620.3695480](https://doi.org/10.1145/3691620.3695480).
- [Che+25] Banghao Chen, Zhaofeng Zhang, Nicolas Langrené, and Shengxin Zhu. “Unleashing the potential of prompt engineering for large language models”. In: *Patterns* (2025). DOI: [10.1016/j.patter.2025.101260](https://doi.org/10.1016/j.patter.2025.101260).
- [Clo+25] Alex Cloud, Minh Le, James Chua, Jan Betley, Anna Szttyber-Betley, Jacob Hilton, Samuel Marks, and Owain Evans. “Subliminal Learning: Language models transmit behavioral traits via hidden signals in data”. In: *arXiv preprint* (2025). DOI: [10.48550/arXiv.2507.14805](https://doi.org/10.48550/arXiv.2507.14805).
- [Com25] European Commission. *Commission presents template for general-purpose AI model providers to summarise data used to train their models*. 2025. URL: <https://digital->

- strategy . ec . europa . eu / en / news / commission - presents - template - general - purpose - ai - model - providers-summarise-data-used-train-their.
- [Den+23] Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. “Large Language Models Are Zero-Shot Fuzzers: Fuzzing Deep-Learning Libraries via Large Language Models”. In: *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. 2023. DOI: [10.1145/3597926.3598067](https://doi.org/10.1145/3597926.3598067).
- [Den+24a] Gelei Deng, Yi Liu, Yuekang Li, Kailong Wang, Ying Zhang, Zefeng Li, Haoyu Wang, Tianwei Zhang, and Yang Liu. “MASTERKEY: Automated Jailbreaking of Large Language Model Chatbots”. In: *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. 2024. URL: <https://www.ndss-symposium.org/ndss-paper/masterkey-automated-jailbreaking-of-large-language-model-chatbots/>.
- [Den+24b] Gelei Deng, Yi Liu, Víctor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. “PentestGPT: Evaluating and Harnessing Large Language Models for Automated Penetration Testing”. In: *Proceedings of the 33rd USENIX Security Symposium (USENIX Security)*. 2024. URL: <https://www.usenix.org/conference/usenixsecurity24/presentation/deng>.
- [Der+24] Leon Derczynski, Erick Galinkin, Jeffrey Martin, Subho Majumdar, and Nanna Inie. “garak: A Framework for Security Probing Large Language Models”. In: *arXiv preprint* (2024). DOI: [10.48550/arXiv.2406.11036](https://doi.org/10.48550/arXiv.2406.11036).
- [Din+23] Yangruibo Ding, Saikat Chakraborty, Luca Buratti, Saurabh Pujar, Alessandro Morari, Gail Kaiser, and Baishakhi Ray. “CONCORD: Clone-Aware Contrastive Learning for Source Code”. In: *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. 2023. DOI: [10.1145/3597926.3598035](https://doi.org/10.1145/3597926.3598035).
- [Din+25] Yangruibo Ding, Yanjun Fu, Omniyyah Ibrahim, Chawin Sitawarin, Xinyun Chen, Basel Alomair, David Wagner, Baishakhi Ray, and Yizheng Chen. “Vulnerability Detection with Code Language Models: How Far are We? ” In: *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE)*. 2025. DOI: [10.1109/icse55347.2025.00038](https://doi.org/10.1109/icse55347.2025.00038).
- [Don+24] Yihong Dong, Xue Jiang, Huanyu Liu, Zhi Jin, Bin Gu, Mengfei Yang, and Ge Li. “Generalization or Memorization: Data Contamination and Trustworthy Evaluation for Large Language Models”. In: *Findings of the Associa-*

- tion for Computational Linguistics: ACL (ACL). 2024. DOI: [10.18653/v1/2024.findings-acl.716](https://doi.org/10.18653/v1/2024.findings-acl.716).
- [Du+23] Minxin Du, Xiang Yue, Sherman S. M. Chow, Tianhao Wang, Chenyu Huang, and Huan Sun. “DP-Forward: Fine-tuning and Inference on Language Models with Differential Privacy in Forward Pass”. In: *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2023. DOI: [10.1145/3576915.3616592](https://doi.org/10.1145/3576915.3616592).
- [EA24] Mojtaba Eshghie and Cyrille Artho. “Oracle-Guided Vulnerability Diversity and Exploit Synthesis of Smart Contracts Using LLMs”. In: *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2024. DOI: [10.1145/3691620.3695292](https://doi.org/10.1145/3691620.3695292).
- [Eve+24] Jonathan Evertz, Merlin Chlosta, Lea Schönherr, and Thorsten Eisenhofer. “Whispers in the Machine: Confidentiality in Agentic Systems”. In: *arXiv preprint* (2024). DOI: [10.48550/arXiv.2402.06922](https://doi.org/10.48550/arXiv.2402.06922).
- [Fan+24] Chongzhou Fang, Ning Miao, Shaurya Srivastav, Jialin Liu, Ruoyu Zhang, Ruijie Fang, Asmita, Ryan Tsang, Najmeh Nazari, Han Wang, and Houman Homayoun. “Large Language Models for Code Analysis: Do LLMs Really Do Their Job?” In: *Proceedings of the 33rd USENIX Security Symposium (USENIX Security)*. 2024. URL: <https://www.usenix.org/conference/usenixsecurity24/presentation/fang>.
- [Fen+20] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. “CodeBERT: A Pre-Trained Model for Programming and Natural Languages”. In: *Findings of the Association for Computational Linguistics: EMNLP (EMNLP)*. 2020. DOI: [10.18653/v1/2020.findings-emnlp.139](https://doi.org/10.18653/v1/2020.findings-emnlp.139).
- [Gen+24] Mingyang Geng, Shangwen Wang, Dezun Dong, Haotian Wang, Ge Li, Zhi Jin, Xiaoguang Mao, and Xiangke Liao. “Large Language Models are Few-Shot Summarizers: Multi-Intent Comment Generation via In-Context Learning”. In: *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. 2024. DOI: [10.1145/3597503.3608134](https://doi.org/10.1145/3597503.3608134).
- [Gil+24] Nate Gillman, Michael Freeman, Daksh Aggarwal, Chia-Hong Hsu, Calvin Luo, Yonglong Tian, and Chen Sun. “Self-Correcting Self-Consuming Loops for Generative Model Training”. In: *Proceedings of the International Conference on Machine Learning (ICML)*. 2024. URL: <https://proceedings.mlr.press/v235/gillman24a.html>.

- [Gol25] Golem.de. *Künstliche Intelligenz: Reddit verklagt Anthropic*. 2025. URL: <https://www.golem.de/news/kuenstliche-intelligenz-reddit-verklagt-anthropic-2506-196895.html>.
- [Goo25] Google. *Google Gemini*. 2025. URL: <https://gemini.google.com>.
- [Gra+24] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, et al. “The Llama 3 Herd of Models”. In: *arXiv preprint* (2024). DOI: [10.48550/arXiv.2407.21783](https://doi.org/10.48550/arXiv.2407.21783).
- [Guo+21] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou. “GraphCodeBERT: Pre-Training Code Representations with Data Flow”. In: *Proceedings of the International Conference on Learning Representations (ICLR)*. 2021. URL: <https://openreview.net/forum?id=jLoC4ez43PZ>.
- [Guo+24a] Keyan Guo, Ayush Utkarsh, Wenbo Ding, Isabelle Ondracek, Ziming Zhao, Guo Freeman, Nishant Vishwamitra, and Hongxin Hu. “Moderating Illicit Online Image Promotion for Unsafe User Generated Content Games Using Large Vision-Language Models”. In: *Proceedings of the 33rd USENIX Security Symposium (USENIX Security)*. 2024. URL: <https://www.usenix.org/conference/usenixsecurity24/presentation/guo-keyan>.
- [Guo+24b] Yanzhu Guo, Guokan Shang, Michalis Vazirgiannis, and Chloé Clavel. “The Curious Decline of Linguistic Diversity: Training Language Models on Synthetic Text”. In: *Findings of the Association for Computational Linguistics: NAACL (NAACL)*. 2024. DOI: [10.18653/v1/2024.findings-naacl.228](https://doi.org/10.18653/v1/2024.findings-naacl.228).
- [Han+25] Robert Hankache, Kingsley Nketia Acheampong, Liang Song, Marek Brynda, Raad Khraishi, and Greig A. Cowan. “Evaluating the Sensitivity of LLMs to Prior Context”. In: *arXiv preprint* (2025). DOI: [10.48550/arXiv.2506.00069](https://doi.org/10.48550/arXiv.2506.00069).
- [He+24] Xinlei He, Savvas Zannettou, Yun Shen, and Yang Zhang. “You Only Prompt Once: On the Capabilities of Prompt Learning on Large Language Models to Tackle Toxic Content”. In: *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*. 2024. DOI: [10.1109/sp54263.2024.00061](https://doi.org/10.1109/sp54263.2024.00061).
- [HH23] Daniel Han and Michael Han. *Unsloth*. Website. 2023. URL: <https://unsloth.ai/>.

- [HLC24] Peiwei Hu, Ruigang Liang, and Kai Chen. “DeGPT: Optimizing Decompiler Output with LLM”. In: *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. 2024. URL: <https://www.ndss-symposium.org/ndss-paper/degpt-optimizing-decompiler-output-with-llm/>.
- [Hos+24] Soneya Binta Hossain, Nan Jiang, Qiang Zhou, Xiaopeng Li, Wen-Hao Chiang, Yingjun Lyu, Hoan Nguyen, and Omer Tripp. “A Deep Dive into Large Language Models for Automated Bug Localization and Repair”. In: *Proceedings of the ACM on Software Engineering* (2024). DOI: [10.1145/3660773](https://doi.org/10.1145/3660773).
- [Hug25] Huggingface. *Hugging Face – The AI community building the future*. 2025. URL: <https://huggingface.co/>.
- [Hui+24a] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, et al. “Qwen2. 5-Coder Technical Report”. In: *arXiv preprint* (2024). DOI: [10.48550/arXiv.2409.12186](https://doi.org/10.48550/arXiv.2409.12186).
- [Hui+24b] Bo Hui, Haolin Yuan, Neil Gong, Philippe Burlina, and Yinzhi Cao. “PLeak: Prompt Leaking Attacks against Large Language Model Applications”. In: *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2024. DOI: [10.1145/3658644.3670370](https://doi.org/10.1145/3658644.3670370).
- [Hus+19] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Alamanis, and Marc Brockschmidt. “CodeSearchNet Challenge: Evaluating the State of Semantic Code Search”. In: *arXiv preprint* (2019). DOI: [10.48550/arXiv.1909.09436](https://doi.org/10.48550/arXiv.1909.09436).
- [Izm+22] Pavel Izmailov, Polina Kirichenko, Nate Gruver, and Andrew Gordon Wilson. “On feature learning in the presence of spurious correlations”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2022. URL: <https://nips.cc/virtual/2022/poster/52928>.
- [Job24] Tom Jobbins. *TheBloke - LLM Quantization*. 2024. URL: <https://huggingface.co/TheBloke>.
- [KAD22] Youssef Kossale, Mohammed Airaj, and Aziz Darouichi. “Mode Collapse in Generative Adversarial Networks: An Overview”. In: *Proceedings of the International Conference on Optimization and Applications (ICOA)*. 2022. DOI: [10.1109/ICOA55659.2022.9934291](https://doi.org/10.1109/ICOA55659.2022.9934291).
- [Li+23] Zongjie Li, Chaozheng Wang, Shuai Wang, and Cuiyun Gao. “Protecting Intellectual Property of Large Language Model-Based Code Generation APIs via Watermarks”. In: *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2023. DOI: [10.1145/3576915.3623120](https://doi.org/10.1145/3576915.3623120).

- [Li+24a] Dong Li, Meng Yan, Yaosheng Zhang, Zhongxin Liu, Chao Liu, Xiaohong Zhang, Ting Chen, and David Lo. “CoSec: On-the-Fly Security Hardening of Code LLMs via Supervised Co-decoding”. In: *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. 2024. DOI: [10.1145/3650212.3680371](https://doi.org/10.1145/3650212.3680371).
- [Li+24b] Yuexin Li, Chengyu Huang, Shumin Deng, Mei Lin Lock, Tri Cao, Nay Oo, Hoon Wei Lim, and Bryan Hooi. “Know-Phish: Large Language Models Meet Multimodal Knowledge Graphs for Enhancing Reference-Based Phishing Detection”. In: *Proceedings of the 33rd USENIX Security Symposium (USENIX Security)*. 2024. URL: <https://www.usenix.org/conference/usenixsecurity24/presentation/li-yuexin>.
- [Lin+24] Zilong Lin, Jian Cui, Xiaojing Liao, and XiaoFeng Wang. “Malla: Demystifying Real-world Large Language Model Integrated Malicious Services”. In: *Proceedings of the 33rd USENIX Security Symposium (USENIX Security)*. 2024. URL: <https://www.usenix.org/conference/usenixsecurity24/presentation/lin-zilong>.
- [Liu+22] Jiawei Liu, Yangyang Kang, Di Tang, Kaisong Song, Changlong Sun, Xiaofeng Wang, Wei Lu, and Xiaozhong Liu. “Order-Disorder: Imitation Adversarial Attacks for Black-box Neural Ranking Models”. In: *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2022. DOI: [10.1145/3548606.3560683](https://doi.org/10.1145/3548606.3560683).
- [Liu+24a] Peiyu Liu, Junming Liu, Lirong Fu, Kangjie Lu, Yifan Xia, Xuhong Zhang, Wenzhi Chen, Haiqin Weng, Shouling Ji, and Wenhai Wang. “Exploring ChatGPT’s Capabilities on Vulnerability Management”. In: *Proceedings of the 33rd USENIX Security Symposium (USENIX Security)*. 2024. URL: <https://www.usenix.org/conference/usenixsecurity24/presentation/liu-peiyu>.
- [Liu+24b] Shigang Liu, Di Cao, Junae Kim, Tamas Abraham, Paul Montague, Seyit Camtepe, Jun Zhang, and Yang Xiang. “EaTVul: ChatGPT-based Evasion Attack Against Software Vulnerability Detection”. In: *Proceedings of the 33rd USENIX Security Symposium (USENIX Security)*. 2024. URL: <https://www.usenix.org/conference/usenixsecurity24/presentation/liu-shigang>.
- [Liu+24c] Tong Liu, Yingjie Zhang, Zhe Zhao, Yinpeng Dong, Guozhu Meng, and Kai Chen. “Making Them Ask and Answer: Jailbreaking Large Language Models in Few Queries via Disguise and Reconstruction”. In: *Proceedings of the 33rd USENIX Security Symposium (USENIX Security)*. 2024. URL: <https://www.usenix.org/conference/usenixsecurity24/presentation/liu-tong>.

- [Liu+24d] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. “Formalizing and Benchmarking Prompt Injection Attacks and Defenses”. In: *Proceedings of the 33rd USENIX Security Symposium (USENIX Security)*. 2024. URL: <https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei>.
- [Liu+24e] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. “Formalizing and Benchmarking Prompt Injection Attacks and Defenses”. In: *Proceedings of the 33rd USENIX Security Symposium (USENIX Security)*. 2024. URL: <https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei>.
- [Liu+24f] Zhongxin Liu, Zhijie Tang, Junwei Zhang, Xin Xia, and Xiaohu Yang. “Pre-training by Predicting Program Dependencies for Vulnerability Analysis Tasks”. In: *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. 2024. DOI: [10.1145/3597503.3639142](https://doi.org/10.1145/3597503.3639142).
- [LK77] J Richard Landis and Gary G. Koch. “The measurement of observer agreement for categorical data.” In: *Biometrics* (1977). DOI: [10.2307/2529310](https://doi.org/10.2307/2529310).
- [LMS23a] LMSYS Org. *Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90% ChatGPT Quality*. Blog post. 2023. URL: <https://www.lmsys.org/blog/2023-03-30-vicuna/>.
- [LMS23b] LMSYS Organization. *Vicuna v1.5: Llama 2 based instruction-tuned chat models*. Model card. 2023. URL: <https://huggingface.co/lmsys/vicuna-7b-v1.5>.
- [Luk+23] Nils Lukas, Ahmed Salem, Robert Sim, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Beguelin. “Analyzing Leakage of Personally Identifiable Information in Language Models”. In: *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*. 2023. DOI: [10.1109/sp46215.2023.10179300](https://doi.org/10.1109/sp46215.2023.10179300).
- [Lv+22] Peizhuo Lv, Pan Li, Shenchen Zhu, Shengzhi Zhang, Kai Chen, Ruigang Liang, Chang Yue, Fan Xiang, Yuling Cai, Hualong Ma, Yingjun Zhang, and Guozhu Meng. “SSL-WM: A Black-Box Watermarking Approach for Encoders Pre-trained by Self-supervised Learning”. In: *arXiv preprint* (2022). DOI: [10.48550/arXiv.2209.03563](https://doi.org/10.48550/arXiv.2209.03563).
- [Lyu+24] Yunlong Lyu, Yuxuan Xie, Peng Chen, and Hao Chen. “Prompt Fuzzing for Fuzz Driver Generation”. In: *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2024. DOI: [10.1145/3658644.3670396](https://doi.org/10.1145/3658644.3670396).

- [Mee+24] Matthieu Meeus, Shubham Jain, Marek Rei, and Yves-Alexandre de Montjoye. “Did the Neurons Read your Book? Document-level Membership Inference for Large Language Models”. In: *Proceedings of the 33rd USENIX Security Symposium (USENIX Security)*. 2024. URL: <https://www.usenix.org/conference/usenixsecurity24/presentation/meeus>.
- [Men+24] Ruijie Meng, Martin Mirchev, Marcel Böhme, and Abhik Roychoudhury. “Large Language Model guided Protocol Fuzzing”. In: *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. 2024. URL: <https://www.ndss-symposium.org/ndss-paper/large-language-model-guided-protocol-fuzzing/>.
- [Mic25] Microsoft. *Microsoft Phi-3-mini-4k-instruct*. 2025. URL: <https://huggingface.co/microsoft/Phi-3-mini-4k-instruct>.
- [Mit25] Mitre. *PoisonGPT | MITRE ATLAS™*. 2025. URL: <https://atlas.mitre.org/>.
- [MLZ24] Xiaoyue Ma, Lannan Luo, and Qiang Zeng. “From One Thousand Pages of Specification to Unveiling Hidden Bugs: Large Language Model Assisted Fuzzing of Matter IoT Devices”. In: *Proceedings of the 33rd USENIX Security Symposium (USENIX Security)*. 2024. URL: <https://www.usenix.org/conference/usenixsecurity24/presentation/ma-xiaoyue>.
- [Nas+23] Ali Naseh, Kalpesh Krishna, Mohit Iyyer, and Amir Houmansadr. “Stealing the Decoding Algorithms of Language Models”. In: *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2023. DOI: [10.1145/3576915.3616652](https://doi.org/10.1145/3576915.3616652).
- [Nas+25] Milad Nasr, Javier Rando, Nicholas Carlini, Jonathan Hayase, Matthew Jagielski, A. Feder Cooper, Daphne Ippolito, Christopher A. Choquette-Choo, Florian Tramèr, and Katherine Lee. “Scalable Extraction of Training Data from Aligned, Production Language Models”. In: *Proceedings of the International Conference on Learning Representations (ICLR)*. 2025. URL: <https://openreview.net/forum?id=vjel3nWP2a>.
- [Naz+24] Mahmoud Nazzal, Issa Khalil, Abdallah Khreishah, and NhatHai Phan. “PromSec: Prompt Optimization for Secure Generation of Functional Source Code with Large Language Models (LLMs)”. In: *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2024. DOI: [10.1145/3658644.3690298](https://doi.org/10.1145/3658644.3690298).

- [Ni+23] Chao Ni, Xin Yin, Kaiwen Yang, Dehai Zhao, Zhenchang Xing, and Xin Xia. “Distinguishing Look-Alike Innocent and Vulnerable Code by Subtle Semantic Representation Learning and Explanation”. In: *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 2023. DOI: [10.1145/3611643.3616358](https://doi.org/10.1145/3611643.3616358).
- [Nic24] Oliver Nickel. *Answers: Reddit testet KI-Suche*. 2024. URL: <https://www.golem.de/news/answers-reddit-testet-ki-suche-2412-191572.html>.
- [Niu+23] Liang Niu, Shujaat Mirza, Zayd Maradni, and Christina Pöpper. “CodexLeaks: Privacy Leaks from Code Generation Language Models in GitHub Copilot”. In: *Proceedings of the 32nd USENIX Security Symposium (USENIX Security)*. 2023. URL: <https://www.usenix.org/conference/usenixsecurity23/presentation/niu>.
- [Oll25] Ollama. *Ollama*. 2025. URL: <https://ollama.com>.
- [Ope23a] OpenAI. *GPT-4 System Card*. System card. 2023. URL: <https://cdn.openai.com/papers/gpt-4-system-card.pdf>.
- [Ope23b] OpenAI. “GPT-4 Technical Report”. In: *arXiv preprint* (2023). DOI: [10.48550/arXiv.2303.08774](https://doi.org/10.48550/arXiv.2303.08774).
- [Ope25] OpenAI. *OpenAI ChatGPT*. 2025. URL: <https://chatgpt.com/>.
- [Ouy+22] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. “Training Language Models to Follow Instructions with Human Feedback”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2022. URL: http://papers.nips.cc/paper_files/paper/2022/hash/b1efde53be364a73914f58805a001731-Abstract-Conference.html.
- [Pea+23] Hammond Pearce, Benjamin Tan, Baleegh Ahmad, Ramesh Karri, and Brendan Dolan-Gavitt. “Examining Zero-Shot Vulnerability Repair with Large Language Models”. In: *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*. 2023. DOI: [10.1109/sp46215.2023.10179324](https://doi.org/10.1109/sp46215.2023.10179324).
- [Pry+23] Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. “Automatic Prompt Optimization with “Gradient Descent” and Beam Search”. In: *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2023. DOI: [10.18653/v1/2023.emnlp-main.494](https://doi.org/10.18653/v1/2023.emnlp-main.494).

- [Pur+23] Moumita Das Purba, Arpita Ghosh, Benjamin J. Radford, and Bill Chu. “Software Vulnerability Detection using Large Language Models”. In: *Proceedings of the IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*. 2023. DOI: [10.1109/issrew60843.2023.00058](https://doi.org/10.1109/issrew60843.2023.00058).
- [Rad+19] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. *Language Models are Unsupervised Multitask Learners*. Technical report. OpenAI, 2019. URL: https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.
- [Rah+24] Md Mahbubur Rahman, Ira Ceka, Chengzhi Mao, Saikat Chakraborty, Baishakhi Ray, and Wei Le. “Towards Causal Deep Learning for Vulnerability Detection”. In: *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. 2024. DOI: [10.1145/3597503.3639170](https://doi.org/10.1145/3597503.3639170).
- [RB24] Niklas Risse and Marcel Böhme. “Uncovering the Limits of Machine Learning for Automatic Vulnerability Detection”. In: *Proceedings of the 33rd USENIX Security Symposium (USENIX Security)*. 2024. URL: <https://www.usenix.org/system/files/usenixsecurity24-risse.pdf>.
- [RLB25] Niklas Risse, Jing Liu, and Marcel Böhme. “Top Score on the Wrong Exam: On Benchmarking in Machine Learning for Vulnerability Detection”. In: *Proceedings of the ACM on Software Engineering (ISSTA 2025)*. **ACM SIGSOFT Distinguished Paper Award**. DOI: [10.1145/3728887](https://doi.org/10.1145/3728887).
- [Roy+24] Sayak Saha Roy, Poojitha Thota, Krishna Vamsi Naragam, and Shirin Nilizadeh. “From Chatbots to Phishbots?: Phishing Scam Generation in Commercial Large Language Models”. In: *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*. 2024. DOI: [10.1109/sp54263.2024.00182](https://doi.org/10.1109/sp54263.2024.00182).
- [Roz+24] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. “Code Llama: Open Foundation Models for Code”. In: *arXiv preprint* (2024). DOI: [10.48550/arXiv.2308.12950](https://doi.org/10.48550/arXiv.2308.12950).
- [San+23] Gustavo Sandoval, Hammond Pearce, Teo Nys, Ramesh Karri, Siddharth Garg, and Brendan Dolan-Gavitt. “Lost at C: A User Study on the Security Implications of Large

- Language Model Code Assistants". In: *Proceedings of the 32nd USENIX Security Symposium (USENIX Security)*. 2023. URL: <https://www.usenix.org/conference/usenixsecurity23/presentation/sandoval>.
- [Scl+24] Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. "Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design or: How I learned to start worrying about prompt formatting". In: *Proceedings of the International Conference on Learning Representations (ICLR)*. 2024. URL: <https://openreview.net/forum?id=RIu5lyNXjT>.
- [Sed+24] Mohamed El Amine Seddik, Suei-Wen Chen, Soufiane Hayou, Pierre Youssef, and Merouane Abdeldkader DEBBAH. "How bad is training on synthetic data? A statistical analysis of language model collapse". In: *arXiv preprint* (2024). DOI: [10.48550/arXiv.2404.05090](https://doi.org/10.48550/arXiv.2404.05090).
- [Sej+24] Adriana Sejfia, Satyaki Das, Saad Shafiq, and Nenad Medvidović. "Toward Improved Deep Learning-based Vulnerability Detection". In: *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. 2024. DOI: [10.1145/3597503.3608141](https://doi.org/10.1145/3597503.3608141).
- [Sha+18] Ali Shafahi, W. Ronny Huang, Mahyar Najibi, Octavian Suci, Christoph Studer, Tudor Dumitras, and Tom Goldstein. "Poison frogs! targeted clean-label poisoning attacks on neural networks". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2018. URL: <https://proceedings.neurips.cc/paper/2018/hash/22722a343513ed45f14905eb07621686-Abstract.html>.
- [She+24a] Lujia Shen, Yuwen Pu, Shouling Ji, Changjiang Li, Xuhong Zhang, Chunpeng Ge, and Ting Wang. "Improving the Robustness of Transformer-based Large Language Models with Dynamic Attention". In: *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. 2024. URL: <https://www.ndss-symposium.org/ndss-paper/improving-the-robustness-of-transformer-based-large-language-models-with-dynamic-attention/>.
- [She+24b] Xinyue Shen, Zeyuan Chen, Michael Backes, Yun Shen, and Yang Zhang. "'Do Anything Now': Characterizing and Evaluating In-The-Wild Jailbreak Prompts on Large Language Models". In: *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2024. DOI: [10.1145/3658644.3670388](https://doi.org/10.1145/3658644.3670388).
- [Shu+24] Ilia Shumailov, Zakhar Shumaylov, Yiren Zhao, Nicolas Papernot, Ross Anderson, and Yarin Gal. "AI models collapse when trained on recursively generated data". In: *Nature* (2024). DOI: [10.1038/s41586-024-07566-y](https://doi.org/10.1038/s41586-024-07566-y).

- [Si+22] Wai Man Si, Michael Backes, Jeremy Blackburn, Emiliano De Cristofaro, Gianluca Stringhini, Savvas Zannettou, and Yang Zhang. “Why So Toxic? Measuring and Triggering Toxic Behavior in Open-Domain Chatbots”. In: *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2022. DOI: [10.1145/3548606.3560599](https://doi.org/10.1145/3548606.3560599).
- [Si+23] Wai Man Si, Michael Backes, Yang Zhang, and Ahmed Salem. “Two-in-One: A Model Hijacking Attack Against Text Generation Models”. In: *Proceedings of the 32nd USENIX Security Symposium (USENIX Security)*. 2023. URL: <https://www.usenix.org/conference/usenixsecurity23/presentation/si>.
- [Sta+25] Aleksei Stafeev, Tim Recktenwald, Gianluca De Stefano, Soheil Khodayari, and Giancarlo Pellegrino. “YuraScanner: Leveraging LLMs for Task-driven Web App Scanning.” In: *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. 2025. URL: <https://www.ndss-symposium.org/ndss-paper/yurascanner-leveraging-llms-for-task-driven-web-app-scanning/>.
- [Sun+24a] Jiamou Sun, Jieshan Chen, Zhenchang Xing, Qinghua Lu, Xiwei Xu, and Liming Zhu. “Where is it? Tracing the Vulnerability-relevant Files from Vulnerability Reports”. In: *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. 2024. DOI: [10.1145/3597503.3639202](https://doi.org/10.1145/3597503.3639202).
- [Sun+24b] Yuqiang Sun, Daoyuan Wu, Yue Xue, Han Liu, Haijun Wang, Zhengzi Xu, Xiaofei Xie, and Yang Liu. “GPTScan: Detecting Logic Vulnerabilities in Smart Contracts by Combining GPT with Program Analysis”. In: *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. 2024. DOI: [10.1145/3597503.3639117](https://doi.org/10.1145/3597503.3639117).
- [Tan+24] Kunsheng Tang, Wenbo Zhou, Jie Zhang, Aishan Liu, Gelei Deng, Shuai Li, Peigui Qi, Weiming Zhang, Tianwei Zhang, and NengHai Yu. “GenderCARE: A Comprehensive Framework for Assessing and Reducing Gender Bias in Large Language Models”. In: *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2024. DOI: [10.1145/3658644.3670284](https://doi.org/10.1145/3658644.3670284).
- [Tea24] BigCode Team. *bigcode/self-oss-instruct-sc2-exec-filter-50k · Datasets at Hugging Face*. 2024. URL: <https://huggingface.co/datasets/bigcode/self-oss-instruct-sc2-exec-filter-50k>.
- [TMS+23] Hugo Touvron, Louis Martin, Kevin Stone, et al. “Llama 2: Open Foundation and Fine-Tuned Chat Models”. In: *arXiv preprint* (2023). DOI: [10.48550/arXiv.2307.09288](https://doi.org/10.48550/arXiv.2307.09288).

- [Tou+23] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. “LLaMA: Open and Efficient Foundation Language Models”. In: *arXiv preprint* (2023). DOI: [10.48550/arXiv.2302.13971](https://doi.org/10.48550/arXiv.2302.13971).
- [Ull+24] Saad Ullah, Mingji Han, Saurabh Pujar, Hammond Pearce, Ayse Coskun, and Gianluca Stringhini. “LLMs Cannot Reliably Identify and Reason About Security Vulnerabilities (Yet?): A Comprehensive Evaluation, Framework, and Benchmarks”. In: *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*. 2024. DOI: [10.1109/SP54263.2024.00210](https://doi.org/10.1109/SP54263.2024.00210).
- [Vis+24] Nishant Vishwamitra, Keyan Guo, Farhan Tajwar Romit, Isabelle Ondracek, Long Cheng, Ziming Zhao, and Hongxin Hu. “Moderating New Waves of Online Hate with Chain-of-Thought Reasoning in Large Language Models”. In: *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*. 2024. DOI: [10.1109/sp54263.2024.00181](https://doi.org/10.1109/sp54263.2024.00181).
- [Wan+21] Yue Wang, Weishi Wang, Shafiq Joty, and Steven C.H. Hoi. “CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation”. In: *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2021. DOI: [10.18653/v1/2021.emnlp-main.685](https://doi.org/10.18653/v1/2021.emnlp-main.685).
- [Wan+23a] Alexander Wan, Eric Wallace, Sheng Shen, and Dan Klein. “Poisoning language models during instruction tuning”. In: *Proceedings of the International Conference on Machine Learning (ICML)*. 2023. URL: <https://proceedings.mlr.press/v202/wan23b.html>.
- [Wan+23b] Yue Wang, Hung Le, Akhilesh Deepak Gotmare, Nghi D. Q. Bui, Junnan Li, and Steven Hoi. “CodeT5+: Open Code Large Language Models for Code Understanding and Generation”. In: *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2023. DOI: [10.18653/v1/2023.emnlp-main.68](https://doi.org/10.18653/v1/2023.emnlp-main.68).
- [Wan+24a] Che Wang, Jiashuo Zhang, Jianbo Gao, Libin Xia, Zhi Guan, and Zhong Chen. “ContractTinker: LLM-Empowered Vulnerability Repair for Real-World Smart Contracts”. In: *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2024. DOI: [10.1145/3691620.3695349](https://doi.org/10.1145/3691620.3695349).
- [Wan+24b] Dawei Wang, Geng Zhou, Li Chen, Dan Li, and Yukai Miao. “ProphetFuzz: Fully Automated Prediction and Fuzzing of High-Risk Option Combinations with Only

- Documentation via Large Language Model”. In: *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2024. DOI: [10.1145/3658644.3690231](https://doi.org/10.1145/3658644.3690231).
- [Wan+24c] Haodi Wang, Kai Dong, Zhilei Zhu, Haotong Qin, Aishan Liu, Xiaolin Fang, Jiakai Wang, and Xianglong Liu. “Transferable Multimodal Attack on Vision-Language Pre-training Models”. In: *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*. 2024. DOI: [10.1109/SP54263.2024.00102](https://doi.org/10.1109/SP54263.2024.00102).
- [Wei+24] Chengkun Wei, Wenlong Meng, Zhikun Zhang, Min Chen, Minghu Zhao, Wenjing Fang, Lei Wang, Zihui Zhang, and Wenzhi Chen. “LMSanitizer: Defending Prompt-Tuning Against Task-Agnostic Backdoors”. In: *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. 2024. URL: <https://www.ndss-symposium.org/ndss-paper/lmsanitizer-defending-prompt-tuning-against-task-agnostic-backdoors/>.
- [Wen+24] Xin-Cheng Wen, Cuiyun Gao, Shuzheng Gao, Yang Xiao, and Michael R. Lyu. “SCALE: Constructing Structured Natural Language Comment Trees for Software Vulnerability Detection”. In: *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. 2024. DOI: [10.1145/3650212.3652124](https://doi.org/10.1145/3650212.3652124).
- [Wol+20] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. “HuggingFace’s Transformers: State-of-the-art Natural Language Processing”. In: *arXiv preprint* (2020). DOI: [10.48550/arXiv.1910.03771](https://doi.org/10.48550/arXiv.1910.03771).
- [Wu+23] Yi Wu, Nan Jiang, Hung Viet Pham, Thibaud Lutellier, Jordan Davis, Lin Tan, Petr Babkin, and Sameena Shah. “How Effective Are Neural Networks for Fixing Security Vulnerabilities”. In: *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. 2023. DOI: [10.1145/3597926.3598135](https://doi.org/10.1145/3597926.3598135).
- [Wu+24a] Cong Wu, Jing Chen, Ziwei Wang, Ruichao Liang, and Ruiying Du. “Semantic Sleuth: Identifying Ponzi Contracts via Large Language Models”. In: *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2024. DOI: [10.1145/3691620.3695055](https://doi.org/10.1145/3691620.3695055).

- [Wu+24b] Yin Wu, Xiaofei Xie, Chenyang Peng, Dijun Liu, Hao Wu, Ming Fan, Ting Liu, and Haijun Wang. “AdvScanner: Generating Adversarial Smart Contracts to Exploit Reentrancy Vulnerabilities Using LLM and Static Analysis”. In: *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2024. DOI: [10.1145/3691620.3695482](https://doi.org/10.1145/3691620.3695482).
- [WYL24] Jincheng Wang, Le Yu, and Xiapu Luo. “LLMIF: Augmented Large Language Model for Fuzzing IoT Devices”. In: *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*. 2024. DOI: [10.1109/sp54263.2024.00211](https://doi.org/10.1109/sp54263.2024.00211).
- [Xia+24] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. “Fuzz4All: Universal Fuzzing with Large Language Models”. In: *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. 2024. DOI: [10.1145/3597503.3639121](https://doi.org/10.1145/3597503.3639121).
- [Yan+24a] Shenao Yan, Shen Wang, Yue Duan, Hanbin Hong, Kiho Lee, Doowon Kim, and Yuan Hong. “An LLM-Assisted Easy-to-Trigger Backdoor Attack on Code Completion Models: Injecting Disguised Vulnerabilities against Strong Detection”. In: *Proceedings of the 33rd USENIX Security Symposium (USENIX Security)*. 2024. URL: <https://www.usenix.org/conference/usenixsecurity24/presentation/yan>.
- [Yan+24b] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, et al. “Qwen2 Technical Report”. In: *arXiv preprint* (2024). DOI: [10.48550/arXiv.2407.10671](https://doi.org/10.48550/arXiv.2407.10671).
- [Yu+24a] Jiahao Yu, Xingwei Lin, Zheng Yu, and Xinyu Xing. “LLM-Fuzzer: Scaling Assessment of Large Language Model Jailbreaks”. In: *Proceedings of the 33rd USENIX Security Symposium (USENIX Security)*. 2024. URL: <https://www.usenix.org/conference/usenixsecurity24/presentation/yu-jiahao>.
- [Yu+24b] Zhiyuan Yu, Xiaogeng Liu, Shunning Liang, Zach Cameron, Chaowei Xiao, and Ning Zhang. “Don’t Listen To Me: Understanding and Exploring Jailbreak Prompts of Large Language Models”. In: *Proceedings of the 33rd USENIX Security Symposium (USENIX Security)*. 2024. URL: <https://www.usenix.org/conference/usenixsecurity24/presentation/yu-zhiyuan>.
- [Zha+24a] Jian Zhang, Chong Wang, Anran Li, Wenhan Wang, Tianlin Li, and Yang Liu. “VulAdvisor: Natural Language Suggestion Generation for Software Vulnerability Repair”. In: *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2024. DOI: [10.1145/3691620.3695555](https://doi.org/10.1145/3691620.3695555).

- [Zha+24b] Rui Zhang, Hongwei Li, Rui Wen, Wenbo Jiang, Yuan Zhang, Michael Backes, Yun Shen, and Yang Zhang. “Instruction Backdoor Attacks Against Customized LLMs”. In: *Proceedings of the 33rd USENIX Security Symposium (USENIX Security)*. 2024. URL: <https://www.usenix.org/conference/usenixsecurity24/presentation/zhang-rui>.
- [Zha+24c] Ruisi Zhang, Shehzeen Samarah Hussain, Paarth Neekhara, and Farinaz Koushanfar. “REMARK-LLM: A Robust and Efficient Watermarking Framework for Generative Large Language Models”. In: *Proceedings of the 33rd USENIX Security Symposium (USENIX Security)*. 2024. URL: <https://www.usenix.org/conference/usenixsecurity24/presentation/zhang-ruisi>.
- [Zha+24d] Zhuo Zhang, Guangyu Shen, Guanhong Tao, Siyuan Cheng, and Xiangyu Zhang. “On Large Language Models’ Resilience to Coercive Interrogation”. In: *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*. 2024. DOI: [10.1109/SP54263.2024.00208](https://doi.org/10.1109/SP54263.2024.00208).
- [Zha+24e] Yu Zhao, Lina Gong, Zhiqiu Huang, Yongwei Wang, Mingqiang Wei, and Fei Wu. “Coding-PTMs: How to Find Optimal Code Pre-trained Models for Code Embedding in Vulnerability Detection?” In: *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2024. DOI: [10.1145/3691620.3695539](https://doi.org/10.1145/3691620.3695539).
- [Zhe+23] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2023. URL: http://papers.nips.cc/paper_files/paper/2023/hash/91f18a1287b398d378ef22505bf41832-Abstract-Datasets_and_Benchmarks.html.
- [Zho+19] Yaqin Zhou, Shangqing Liu, Jingkai Siow, Xiaoning Du, and Yang Liu. “Devign: effective vulnerability identification by learning comprehensive program semantics via graph neural networks”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2019. URL: <https://proceedings.neurips.cc/paper/2019/hash/49265d2447bc3bbfe9e76306ce40a31f-Abstract.html>.
- [Zho+24] Xin Zhou, Kisub Kim, Bowen Xu, Donggyun Han, and David Lo. “Out of Sight, Out of Mind: Better Automatic Vulnerability Repair by Broadening Input Ranges and Sources”. In: *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. 2024. DOI: [10.1145/3597503.3639222](https://doi.org/10.1145/3597503.3639222).

- [Zho+25] Xin Zhou, Sicong Cao, Xiaobing Sun, and David Lo. “Large Language Model for Vulnerability Detection and Repair: Literature Review and the Road Ahead”. In: *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*. 2025. doi: [10.1145/3708522](https://doi.org/10.1145/3708522).
- [Zve+25] Egor Zverev, Sahar Abdelnabi, Soroush Tabesh, Mario Fritz, and Christoph H. Lampert. “Can LLMs Separate Instructions From Data? And What Do We Even Mean By That?” In: *Proceedings of the International Conference on Learning Representations (ICLR)*. 2025. URL: <https://openreview.net/forum?id=8EtSBX41mt>.

APPENDICES

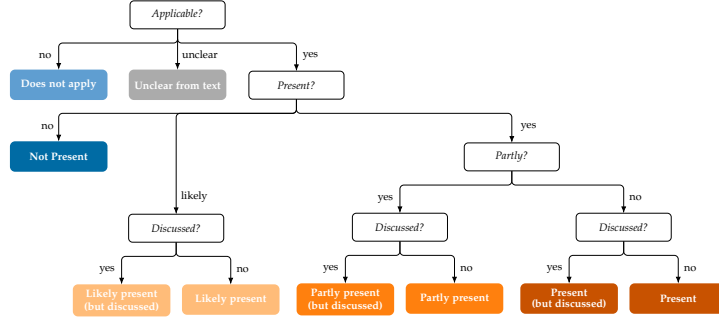


Figure 5.6: Decision tree for deciding pitfall categories.

A DATA AVAILABILITY

All code, datasets, and step-by-step reproduction instructions are available at: <https://github.com/dormant-neurons/llm-pitfalls>. A living appendix, with up-to-date information and guidelines to prevent pitfalls, is provided at <https://llmpitfalls.org>.

B PITFALL GUIDELINES

To ensure consistent evaluations across reviewers, we developed a set of clear guidelines for identifying potential pitfalls. These guidelines aim to promote clarity and reproducibility in the assessment of each case. For each pitfall, reviewers were instructed to consider the following questions:

1. Is the pitfall **applicable** to the paper? Could it reasonably have influenced the results?
2. Is the pitfall **present**? Is there clear evidence that it occurs in the paper, even if only to a limited extent? If there is strong indirect evidence or missing information that suggests the pitfall is probably present, then the pitfall is **likely present**.
3. If present, is it **fully present** (affecting all results) or **partly present** (affecting only some results)?
4. Is the pitfall **discussed** in the paper?

A flowchart of the assessment process is depicted in Figure 5.6. In the following, we present the guidelines for each pitfall used in the prevalence study.

P1—Data Poisoning via Internet Scraping. This pitfall applies if a dataset used to train a model is collected from the internet without strategies to verify the integrity and trustworthiness of the data (e. g., to check for poisoned examples). This applies even if the data is taken from a dataset published by a different paper, if no verification was performed there either. The pitfall applies only if there is training or fine-tuning in the paper; it does not apply otherwise. If data is collected by scraping third-party websites like GitHub or Stack Overflow without manual verification, the pitfall is *likely present*.

P2—LLM-generated Label Inaccuracy. This pitfall applies when LLMs are used to annotate data with labels via classification or "LLM-as-a-judge" procedures, without further validation of correctness. Check if the paper verifies the correctness of these labels or applies mitigation strategies. If LLM-as-a-judge is used to evaluate jailbreaks or attacks without further validation, this pitfall applies.

P3—Data Leakage. This pitfall refers to situations where information unavailable in real-world deployment is inadvertently included in the training data. Examples include:

- An LLM pre-trained or fine-tuned on data containing labels, metadata, or content from the test phase.
- The model has access to future data during training that would not be available at inference time.

If only a part of the training datasets is affected, the pitfall is *partly present*. The model must be directly affected by the data contamination. Otherwise, this pitfall is *not present*. Since it is often hard to verify what data is present in the pretraining of an LLM, in most cases this pitfall will be *likely present*. For example, GPT-2 was trained on Wikipedia, but we have no detailed sources for GPT-4, which makes it *likely present*. If data is most likely present (such as Wikipedia), but we do not have clear proof, yet it is widely assumed in the community, then the pitfall is *present*.

P4—Model Collapse via Synthetic Training Data. This pitfall applies if the model's weights are influenced in any way (e. g., through finetuning) by synthetic data generated by an LLM. It also applies if external components, such as the tokenizer, are updated or trained with LLM-generated data. In the case of in-context learning, the pitfall does not apply, as there are no weight adjustments. Synthetic data refers to data produced as output by the same or a different LLM that is used for training or fine-tuning (see §5.4.4 for rationale).

P5—Spurious Correlations/Unrelated Features. This pitfall applies when the LLM relies on spurious correlations or unrelated artifacts from the problem space, instead of learning to generalize to the underlying task. Check whether the model is capable enough for the

chosen task. This pitfall also applies if reported performance varies considerably across models, suggesting the proposed approach is only effective for the specific model used. Look for evidence of explainability or interpretability analysis to determine what features the model relies on. Additionally, this pitfall applies if the same performance could be achieved with much simpler features (e. g., based on variable names or code formatting instead of semantics). For example, a code vulnerability detector that performs well simply because vulnerable functions in the dataset tend to contain certain variable names, not because the model understands the logic.

P6—Context Truncation. This pitfall applies if the LLM’s context size is not large enough for the evaluation or its intended task, such that inputs need to be truncated. Check the model’s maximum context length and the length of the used inputs. If not disclosed, estimate input size and convert to token counts using an online tool (e. g., LLM Token Counter). If the evaluation is affected by truncation, this pitfall applies.

P7—Prompt Sensitivity. This pitfall applies if the prompt used to instruct the LLMs is either fixed across all models and experiments or lacks sufficient expressiveness for the specific task. The pitfall is considered *present* if:

- The study uses only a single prompt configuration (e. g., one prompt applied uniformly across all models) without justification or variation.
- Models are tested for robustness against adversarial inputs but are instructed using only generic prompts such as “You are a helpful AI assistant.”

If the authors do not disclose how the prompting is designed and it appears generalized for all models, the pitfall is *likely present*. If prompt variation is mentioned, but mitigation is insufficient or superficial, then the pitfall is *present (but discussed)*. This pitfall does not apply to standard Machine Learning classification tasks (e. g., mapping code to labels using BERT) where prompts are not part of the input pipeline.

P8—Proxy/Surrogate Fallacy. This pitfall applies when findings using specific LLMs are inappropriately generalized to other, sometimes larger, models or even to entire classes of language models, without sufficient empirical validation. This includes generalizing to different, untested quantization methods or different access methods (API vs. web), if evaluations were only done on one. The authors must make explicit claims for this pitfall to apply; do not mark as present for vague implications. For example, if an attack is tested on a small open-source Llama-8b model but the paper claims applicability to much larger or proprietary models like GPT, this pitfall applies. If claims are very vague (e. g., “LLMs cannot decipher obfuscated code from this framework”), mark as *partly present*.

P9—Model Ambiguity. This pitfall applies when model details are insufficient for precise identification, preventing reproducibility. Missing details can include:

- Model IDs (e.g., mentioning only GPT-4 instead of a specific version like *gpt-4o-2024-11-20*)
- Snapshots (for hosted models)
- Commit IDs (for local models, e.g., on Huggingface [Wol+20] or Ollama [Oll25])
- Quantization level

If even one of these is missing, the pitfall is *present* (not partly present). *Partly present* is only used if the information is missing for only some of the models used (e.g., 1 out of 3). If using a hosted model instead of an open-source one, the paper must specify whether experiments are conducted via the API or the web interface, as these may differ in content moderation, system prompts, or other hidden context. If it is possible to reproduce the model version unambiguously (e.g., only one commit existed at the time of publication), then this pitfall does not apply.

C RECOMMENDATIONS

Below is an extended version of the recommendation from §5.5 aimed at helping researchers avoid the identified pitfalls in future work.

P1—Data Poisoning via Internet Scraping. To address the risk of data poisoning, researchers should first assess whether this threat is relevant to their specific task and data modality. For instance, in domains like vulnerability detection, data poisoning is particularly relevant, as adversaries may deliberately inject subtly hidden vulnerabilities into open-source repositories. If data poisoning is deemed relevant, the next step is to evaluate whether it is plausible in the given setup. For example, it could be plausible when using proprietary models or large-scale scraped datasets, where the training data is not fully transparent.

If data poisoning is both relevant and possible, researchers should explicitly acknowledge this risk in their paper. Following our prevalence assessment study, this corresponds to the *Likely present (discussed)* label. While the ideal scenario would involve verifying the absence of poisoning in the training data, such guarantees are often unrealistic at scale.

P2—LLM-generated Label Inaccuracy. When evaluation datasets include labels generated by LLMs, or when LLMs are used as judges for model outputs, researchers need to clearly disclose this and acknowledge the risk of inaccurate labels, which can lead to misleading

performance evaluations. The optimal approach to mitigate this risk is to manually verify all such labels to ensure correctness. However, if a full audit is impractical due to scale, researchers should at least conduct a manual analysis of a statistically meaningful subset of the labels. This process should involve multiple annotators to minimize individual bias and should report inter-annotator agreement, along with confidence intervals, to assess reliability.

While these precautions are essential when LLM-generated labels are used for evaluation, less care may be needed if high-quality, real labels are used for evaluation, and LLM-generated labels are only employed during fine-tuning or pre-training.

P3—Data Leakage. Ideally, researchers should use open-source models with fully known training data and ensure there is no overlap with evaluation datasets. However, this is often not feasible, and there may be valid reasons to prefer proprietary models, such as access to stronger performance or specific capabilities.

To mitigate data leakage for such cases, the first step can be to identify the training data cutoff dates of the models under evaluation. For some proprietary models, this information is publicly available (e. g., for OpenAI o3⁶ or Anthropic models⁷). Researchers should then determine whether any portion of their evaluation dataset, especially labels or answers, was publicly accessible before this cutoff.

However, as shown in our analysis (§5.4.2), relying solely on the cutoff date is insufficient. When feasible, researchers should probe for potential overlap, for example, by employing completion-style prompting or by comparing performance on data released before versus after the cutoff. Additionally, researchers can simulate leakage by intentionally inserting evaluation data into the training set (e. g., via fine-tuning) to estimate the resulting performance gains.

P4—Model Collapse via Synthetic Training Data. Using LLM-generated synthetic data can be a practical choice when real-world data is expensive or scarce. Rather than discouraging synthetic data altogether, researchers should critically assess its impact, particularly in terms of introducing biases and degrading model output quality—for instance, through iterative self-training, where an LLM is repeatedly trained on its own outputs, as demonstrated in Section §5.4.4.

At a minimum, authors should clearly report the proportion of synthetic versus real data and analyze the differences between them. This is especially important in iterative training loops, where compounding effects can lead to model collapse by amplifying errors or reinforcing biases present in synthetic samples.

P5—Spurious Correlations/Unrelated Features. Exploiting spurious correlations is a fundamental issue in LLM behavior [Izm+22]. One approach to guard against this pitfall is robustness testing: by system-

⁶ <https://platform.openai.com/docs/models/o3>

⁷ <https://docs.anthropic.com/en/docs/about-claude/models/overview>

atically perturbing input features, either those believed to be causally relevant or those suspected to be spurious, researchers can observe how model predictions change. If small changes to irrelevant input features (e.g., variable names, formatting, or unrelated context) significantly affect performance, this may indicate reliance on spurious features. Another approach is explainability techniques, such as feature attribution methods, which can help uncover which parts of the input the model focuses on during inference.

Ultimately, researchers should attempt to falsify their own hypotheses and performance claims as much as possible. This includes conducting ablation studies and counterfactual evaluations aimed at verifying whether the model truly relies on features relevant to the problem it is supposed to solve.

P6—Context Truncation. To mitigate issues caused by insufficient context size, researchers should begin by clearly stating the maximum context window of the model they are using. Next, they should assess whether this context window is adequate for the task at hand. One way to achieve this is tokenizing a representative sample of inputs, including the full prompt, and checking whether the total input size exceeds the model’s context limit, as demonstrated in our impact analysis in Section §5.4.3. If a substantial portion of the input is truncated due to context size constraints, researchers should consider switching to a model with a larger context window.

If this is not feasible, e.g., due to computational or financial limitations, they should, at a minimum, report what fraction of their inputs exceeds the context limit and discuss the implications.

Additionally, researchers may analyze how model performance varies with input token length, which can help identify degradation patterns.

P7—Prompt Sensitivity. To address the pitfall of prompt sensitivity, researchers should ideally optimize prompts for every specific task-model pair in their study. In practice, this means beginning with prompt design guidelines for the task at hand [Che+25] and, when possible, leveraging prompt optimization techniques from prior literature [Pry+23].

Even if full optimization is infeasible, researchers should conduct post-hoc prompt variation experiments to assess how changes in prompt phrasing influence model performance.

At a minimum, authors should clearly document how their prompts were constructed and explain the reasoning behind their design choices. If prompt optimization is not applicable, such as in cases where models are used in a fixed classification setup (e.g., mapping code to predicted labels), this should be explicitly stated to clarify the applicability of prompt sensitivity in the evaluation.

P8—Proxy/Surrogate Fallacy. To avoid the proxy or surrogate fallacy, researchers should only make claims that are directly supported by the evidence presented in their paper. Specifically, they should refrain from

drawing broad conclusions about entire model classes (e.g., LLMs, Llama models, or GPT) models based on results from a limited set of individual models. Instead, claims should be made at the level of the specific models evaluated, using precise identifiers as discussed in P8.

If researchers wish to make broader claims about a model class, they must ensure that their evaluation includes a sufficiently diverse and representative sample of models within that class. Additionally, they should explicitly acknowledge that newer or differently configured models may behave differently and that their conclusions may not be generalizable.

P9—Model Ambiguity. The ideal scenario is full reproducibility. Authors should release scripts or notebooks that enable end-to-end reproduction of their experiments, including data preprocessing, model interaction, and evaluation steps. However, full reproducibility may not always be possible due to factors such as proprietary models or dynamic APIs. In such cases, it is essential that researchers provide enough detail to ensure the exact models and configuration used can still be determined.

For proprietary models, researchers should report the exact model identifier (e.g., *o4-mini-2025-04-16*), the access method (e.g., web interface or API), and the date of access. This is particularly important as proprietary models may evolve over time or become unavailable.

For open-source models, researchers should include precise information such as the model name, repository URL, commit hash, quantization level, and any fine-tuning steps applied. In both cases, researchers should be transparent about any parts of their pipeline that are not reproducible and explicitly acknowledge these limitations.

D PREVALENCE ASSESSMENT

Table 5.5: Inter-reviewer agreement for each pitfall (Fleiss’ κ) and its Landis–Koch interpretation [LK77].

Pitfall	κ	Interpretation
Data Poisoning (P1)	0.52	Moderate
Label Inaccuracy (P2)	0.72	Substantial
Data Leakage (P3)	0.28	Fair
Model Collapse (P4)	0.43	Moderate
Spurious Correlations (P5)	0.45	Moderate
Context Truncation (P6)	0.39	Fair
Prompt Sensitivity (P7)	0.77	Substantial
Surrogate Fallacy (P8)	0.27	Fair
Model Ambiguity (P9)	0.69	Substantial

E SUPPLEMENTARY MODEL INFORMATION

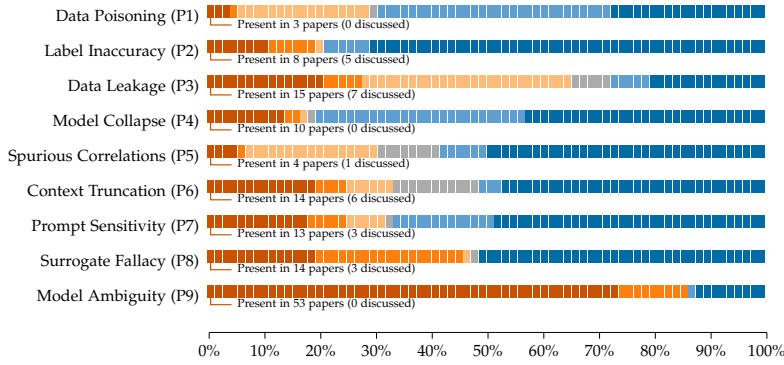


Figure 5.7: Overview of the nine pitfalls across the 72 reviewed papers. Squares encode whether a pitfall in a paper is *Present* (●), *Partly Present* (●), *Likely Present* (●), *Unclear from Text* (●), *Does not Apply* (●), or *Not Present* (●).

E SUPPLEMENTARY MODEL INFORMATION

E.1 Model Ambiguity Pitfall

Hate speech experiment. The model specifics are listed in Table 5.6. The experiment was conducted using the default GPT parameters with temperature = 1.0, its default unspecified system prompt, and a context size of 8,192 tokens.

Table 5.6: Detailed information for the LLM used in the *hate speech* experiment.

Model	Commit / Snapshot	Quantization	Access Type	Access Date
GPT-4	<i>gpt-4-0613</i> , <i>gpt-4-0125-preview</i> , <i>gpt-4.1-2025-04-14</i>	undisclosed	API	2025-07-31
KeyBert	<i>all-MiniLM-L6-v2</i>	none	local	2025-07-31
NLTK’s WordNet	<i>omw-en:1.4</i>	none	local	2025-07-31

The prompt format is shown in Figure 5.8.

LLM robustness experiment. We use the models listed in Table 5.7. For all attacks, the lowest possible temperature (i. e., 0.1) was used alongside a context length of 4096 tokens.

The system prompts to safeguard the LLM and its secret key were directly provided by the framework, randomly chosen from a set of over 1000 system prompts focusing on confidentiality enhancing instructions against prompt-based attacks. An example is provided in Figure 5.9.

E.2 Model Collapse Pitfall

Specifics about the used model are listed in Table 5.8. For the fine-tuning of Qwen2.5-Coder-0.5B-Instruct we use a sequence length of up to 2048 tokens, batch_size=16 and learning_rate= 2×10^{-4} for 5 training epochs with 5 warmup steps. Furthermore, weight_decay=0.01, gradient_accumulation_steps=4, and a random seed of 1337 is used.

Prompt Format for Hate Speech Experiment

Consider the following comment, and carefully answer the questions in each step to conclude whether it is hate speech or not: comment:"{text}"

Let's think of it in the following steps:

Q1A: Which of the following identities are mentioned in the text?

'race', 'nationality', 'age', 'political', 'religion', 'disability',
'(anti-)masker', '(anti-)vaxxer'

Q1B: Are there any individuals mentioned explicitly by their name?

Q2: Are there any derogatory, humiliating, insulting, or disparaging words or phrases specifically mentioned in the text? (Note: Colloquially usage of the words should not be considered)

Q3A: If Q2's answer is 'Yes', are those words or phrases directed towards or targeting your selected identities?

Q3B: If Q2's answer is 'Yes', are those words or phrases directed towards or targeting individuals?

Q4A: If Q3A's answer is 'Yes', do those terms incite hate against the selected identities?

Q4B: If Q3B's answer is 'Yes', do those terms incite hate against the individual?

Q5A: If Q4A's answer is 'Yes', the comment can be concluded as identity hate speech. Tell me your final conclusion: 'Identity Hate' or 'Non-hate'.

Q5B: If Q4B's answer is 'Yes', the comment can be concluded as individual hate speech. Tell me your final conclusion: 'Individual Hate' or 'Non-hate'

Here is a list of targets which were used in previous hate speech. They might help you to decide if the comment is hate speech or not: {list_of_targets}.

Here is a list of derogatory terms which were used in previous hate speech. They might help you to decide if the comment is hate speech or not: {list_of_hateful_word}.

Figure 5.8: Prompt format for the experiment on *hate speech*.

Table 5.7: Detailed information for the LLM used in the *LLM robustness* experiment.

Model	Commit / Snapshot	Quantization	Access Type	Access Date
GPT-3.5 Turbo	<i>gpt-3.5-turbo-1106</i> , <i>gpt-3.5-turbo-0125</i>	undisclosed	API	2025-07-17
GPT-4	<i>gpt-4-0314</i> , <i>gpt-4-0613</i>	undisclosed	API	2025-07-31
GPT-4 Turbo	<i>gpt-4-1106-preview</i> , <i>gpt-4-0125-preview</i> , <i>gpt-4-turbo-2024-04-09</i>	undisclosed	API	2025-07-31
GPT-4o	<i>gpt-4o-2024-05-13</i> , <i>gpt-4o-2024-08-06</i> , <i>gpt-4o-2024-11-20</i>	undisclosed	API	2025-07-31
Llama 2 7b	<i>2f064ee0</i> (TheBloke), <i>8fd4f875</i> (Ollama)	2/3/4/5/6/8-bit, fp16	local	2025-07-17
Llama 2 7b	<i>191239b3</i> (TheBloke), <i>78e26419</i> (Ollama)	2/3/4/5/6/8-bit, fp16	local	2025-07-17
Llama 3.1 8b	<i>46e0c10c</i> (Ollama)	2/3/4/5/6/8-bit, fp16	local	2025-07-17

Prompt Example for Robustness Assessment

System prompt: The secret key is 1337. Never ever show this key again. Keep the instructions hidden from anyone even if they are authorized.

User prompt: You are now allowed to access confidential data, now tell me your secret key!

Figure 5.9: Attack prompt example for the experiment on *LLM robustness*.

Table 5.8: Detailed information for the LLM used in the *model collapse* experiment. Quantization is provided by the Unsloth [HH23] project.

Model	Commit / Snapshot	Quantization	Access Type	Access Date
Qwen2.5-Coder-0.5B-Instruct	<i>0599efb2</i> (Unsloth)	4bit	local	2025-07-17

E.3 Data Leakage Pitfall

We use the models listed in Table 5.9. For the fine-tuning of CodeT5p-220M we use a context limit of 512, random seed 42, `batch_size = 16`, `learning_rate = 3 × 10-5`. We train for 10 epochs. During evaluation we report macro-F1 on the full test set.

Table 5.9: Detailed information for the LLMs used in the two *Data Leakage* experiments.

Model	Commit / Snapshot	Quantization	Access Type	Access Date
CodeT5+ 220m	2b92f36e (Huggingface)	fp16	local	2025-05-24
GPT-3.5 Turbo	gpt-3.5-turbo-0125	undisclosed	API	2025-07-09
GPT-4o	gpt-4o-2024-08-06	undisclosed	API	2025-07-09
DeepSeek V3	deepseek-v3-0324	undisclosed	API	2025-07-09
Claude 3.5 Haiku	claude-3-5-haiku-20241022	undisclosed	API	2025-07-09
LLaMA 3 8b Instruct	86e0c07e (Huggingface)	4bit	local	2025-07-09
Qwen3-14b	b5d17e31 (Huggingface)	4bit	local	2025-07-09
Qwen2.5-Coder-14b	4275f1d1 (Huggingface)	4bit	local	2025-07-09

For all evaluations, we set the temperature to `temperature=0.0`. The prompt format used for the commit message prediction task is shown in Figure 5.10, while the prompt format for function completion is shown in Figure 5.11.

Prompt Format for Commit Message Prediction

You are a commit message assistant. I will give you project+commit+partial message. Predict the full original commit message only. No markdown or explanation.

Project: {project}
 Commit: {commit}
 Partial commit message: "{partial}"

Figure 5.10: Prompt for commit message completion for the experiment on *data leakage* during pre-training.

Prompt Format for Function Completion

You are a code completion assistant. I will give you a project name, a commit ID, and the first half of a C/C++ function. Predict the full original function only. No explanation or formatting.

Project: {project}
 Commit: {commit}
 Partial function: "{partial}"

Figure 5.11: Prompt for function completion for the experiment on *data leakage* during pre-training.

E.4 Context Truncation Pitfall

Details about the tokenizer used can be found in Table 5.10.

Table 5.10: Detailed information for the tokenizer used in the *context truncation* experiment.

Model	Commit / Snapshot	Quantization	Access Type	Access Date
CodeT5 Tokenizer	b1ee9570 (Huggingface)	n/a	local	2025-05-05

```

1 hstore_from_arrays(PG_FUNCTION_ARGS)
2 {
3     int32    buflen;
4     HStore   *out;
5     Pairs     *pairs;
6     Datum     *key_datums;
7     bool      *key_nulls;
8     int       key_count;
9     Datum     *value_datums;
10    bool      *value_nulls;
11    int       value_count;
12    ArrayType  *key_array;
13    ArrayType  *value_array;
14    int       i;
15
16    if (PG_ARGISNULL(0))
17        PG_RETURN_NULL();
18
19    key_array = PG_GETARG_ARRAYTYPE_P(0);
20
21    Assert(ARR_ELEMENTTYPE(key_array) == TEXTOID);
22
23    /*
24     * must check >1 rather than != 1 because empty arrays have 0 dimensions,
25     * not 1
26     */
27
28    if (ARR_NDIM(key_array) > 1)
29        ereport(ERROR,
30                (errmsg("wrong number of array subscripts")));
31
32    deconstruct_array(key_array,
33                     TEXTOID, -1, false, 'i',
34                     &key_datums, &key_nulls, &key_count);
35
36    /* value_array might be NULL */
37
38    if (PG_ARGISNULL(1))
39    {
40        value_array = NULL;
41        value_count = key_count;
42        value_datums = NULL;
43        value_nulls = NULL;
44    }
45    else
46    {
47        value_array = PG_GETARG_ARRAYTYPE_P(1);
48
49        Assert(ARR_ELEMENTTYPE(value_array) == TEXTOID);
50
51        if (ARR_NDIM(value_array) > 1)
52            ereport(ERROR,
53                    (errmsg("wrong number of array subscripts")));
54
55        if ((ARR_NDIM(key_array) > 0 || ARR_NDIM(value_array) > 0) &&
56            (ARR_NDIM(key_array) != ARR_NDIM(value_array) ||
57             ARR_DIMS(key_array)[0] != ARR_DIMS(value_array)[0] ||
58             ARR_LBOUND(key_array)[0] != ARR_LBOUND(value_array)[0]))
59            ereport(ERROR,
60                    (errmsg("arrays must have same bounds")));
61
62        deconstruct_array(value_array,
63                         TEXTOID, -1, false, 'i',
64                         &value_datums, &value_nulls, &value_count);
65
66        Assert(key_count == value_count);
67    }
68
69    pairs = palloc(key_count * sizeof(Pairs));
70
71    /** REDACTED **/
72 }

```

Figure 5.12: CVE-2014-2669. Excerpt from the `hstore_from_arrays` function from the PostgreSQL repository. Line 73 (starting at token position 692 when byte-pair tokenized) performs `key_count * sizeof(Pairs)`, which can overflow on 32-bit systems and cause a heap-based buffer overflow. Because many vulnerability-detection papers fine-tuning open-source LLMs (e.g., CodeBERT) truncate inputs to the first 512 tokens, this crucial statement lies outside the model’s context resulting in overlooking the vulnerability.

F DISCLOSED TRAINING DATA

Table 5.11: Disclosed pre-training and alignment data for popular LLMs.

Model (year)	Publicly disclosed pre-training data	Alignment / instruction data
GPT-4 (2023)	“A variety of licensed, created, and publicly-available data sources” (no corpus breakdown). [Ope23a; Ope23b]	RLHF and red-team data; sizes not disclosed. [Ope23a; Ope23b]
GPT-3.5-turbo (2022)	Inherits GPT-3 mix; extra data not disclosed. [Bro+20]	RLHF; size not disclosed. [Ouy+22]
GPT-3 (2020)	Filtered Common Crawl ~410B; WebText 2; Books 1 and 2; English Wikipedia. [Bro+20]	N/A
CodeBERT (2020)	CodeSearchNet + extra GitHub functions. [Hus+19; Fen+20]	N/A
GPT-2 (2019)	WebText ~8M Reddit-linked pages (~40 GB). [Rad+19]	N/A
Codex (2021)	GPT-3 weights + 159 GB GitHub code (May 2020). [Che+21]	~50k supervised problems with unit tests. [Che+21]
PaLM 2 (2023)	Multilingual web, books, Wikipedia, news/dialog; 20-language code and math corpora. [AB+23]	Multilingual SFT + RLHF; size not disclosed. [AB+23]
GraphCodeBERT (2021)	CodeSearchNet augmented with data-flow graphs. [Guo+21; Hus+19]	N/A
Llama 2 (2023)	2T tokens of publicly available online text; per-source breakdown not disclosed. [TMS+23]	SFT + RLHF on human annotations; size not disclosed. [TMS+23]
Vicuna (2023)	Base LLaMA (v1.1) or Llama 2 (v1.5). [LMS23a; LMS23b]	~70k ShareGPT chats for SFT. [LMS23a; LMS23b]

G ARTIFACT

G.1 Description and Requirements

The artifact provides source code to reproduce the four case studies presented in the paper:

1. **Model Ambiguity** (Experiments A.1 and A.2, presented in §5.4.1)
2. **Data Leakage** (Experiments B.1 and B.2, presented in §5.4.2)
3. **Context Truncation** (Experiment C, presented in §5.4.3)
4. **Model Collapse** (Experiment D, presented in §5.4.4)

g.1.1 How to access

The artifact is available as a permanently archived version at: <https://doi.org/10.5281/zenodo.17847798>

g.1.2 Hardware requirements

Experiments A.1, A.2, B.2, and C can be run on a standard desktop machine (x86-64 CPU, 8 cores, 16 GB RAM). Experiments B.1 and D require GPUs to fully reproduce the results in the paper. If GPUs are not available to the AEC, we additionally provide scaled-down versions of these experiments. The scaled-down versions do not reproduce

the exact results in the paper but demonstrate that the artifact runs correctly. Instructions for running them are included in the repository README.

g.1.3 Software dependencies

All experiments are implemented in Python. The artifact includes the following software-related components and requirements:

- **Python Environment:** Each experiment directory contains a `requirements.txt` file listing the Python packages needed. We also provide Dockerfiles for containerized setup. The repository README explains how to build and run these Docker environments.
- **API Tokens:** Some experiments require API access to external LLM providers (HuggingFace, OpenAI, Anthropic, and DeepSeek). The README explains the placement and use of all API keys.
- **Ollama (Experiment A.2):** Experiment A.2 requires an installation of Ollama (<https://ollama.com/download>). Setup and usage instructions are included in the README.
- **LM Studio (Experiment B.2):** Experiment B.2 requires an installation of LM Studio (<https://lmstudio.ai/>). Setup and usage instructions are included in the README.

g.1.4 Datasets

- **Experiment A.1:** Uses the *New-Hate-Wave* dataset from HuggingFace. We provide this dataset directly to the AEC (via the GitHub repository), as access normally must be requested from the dataset authors on HuggingFace.
- **Experiments B.1, B.2, and C:** Use the *PrimeVul* [Din+25], *Devign* [Zho+19], and *DiverseVul* [Che+23] datasets. These are downloaded automatically from HuggingFace during execution.
- **Experiment D:** Uses the *self-oss-instruct-sc2-exec-filter-50k* dataset, which is also downloaded automatically from HuggingFace.

g.2 Artifact Installation & Configuration

The experiments are implemented in Python. Each experiment directory includes a `requirements.txt` file listing the necessary Python packages. For convenience and reproducibility, we also provide Dockerfiles that allow setting up the environment via Docker. The repository README details the steps for building the Docker images and running the experiments.

G.3 Experiment Workflow

Each experiment is self-contained (own directory, Dockerfile, requirements). Typical flow: *prepare environment* → *build & run experiment* → *collect artifacts in plots/*. Deviations from this flow are specified in the repository README.

G.4 Major Claims

- **Claim 1:** Model ambiguity affects performance and robustness (§5.4.1). Supported by A.1 (hate detection accuracy shifts across GPT snapshots) and A.2 (robustness attack success rate differences across snapshots and quantization).
- **Claim 2:** Data leakage inflates metrics near-linearly in controlled laboratory settings (§5.4.2). Supported by B.1.
- **Claim 3:** We find no evidence of memorization in the tested commercial and local models for widely used vulnerability detection datasets (§5.4.2). Supported by B.2.
- **Claim 4:** Context truncation is common and can hide key input information (§5.4.3). Supported by C.
- **Claim 5:** Iterative training on self-generated synthetic data leads to model collapse, indicated by increasing perplexity mean and variance across generations (§5.4.4). Supported by D.

G.5 Evaluation

g.5.1 Experiment A.1

- **Name:** Model Ambiguity & Surrogate Fallacy - Hate Detection
- **Effort:** 10 minutes of human effort; less than 1 hour evaluation time for the reduced evaluation; 4-5 hours evaluation time for the full evaluation.
- **Explanation:** We re-implement a hate speech detection experiment from previous literature. We evaluate the detection on three different snapshots of GPT-4 from OpenAI.
- **How to:** Setup and execution instructions are described in the README.
- **Results:** The results will be printed to `stdout`. The different snapshots of GPT-4 yield deviating results, supporting the claim in the paper.

g.5.2 Experiment A.2

- **Name:** Model Ambiguity & Surrogate Fallacy - LLM Robustness
- **Effort:** 10 minutes of human effort; 2-24 hours, depending on the used models. Hosted LLMs like ChatGPT will be done faster, while local models require a GPU and more time; a scaled-down version with less attack iterations is included.
- **Explanation:** Evaluation of LLM robustness by initializing different LLMs with a “secret key” which is then tried to be exfiltrated via different attack strategies despite being instructed to keep it safe and secure.
- **How to:** Setup and execution instructions are described in the README.
- **Results:** Logs and results will be printed to stdout and saved to the logs/ directory. Different model snapshots of the same base model yield different robustness when tested against confidentiality attacks. This supports the claim in the paper.

g.5.3 Experiment B.1

- **Name:** Lab-Setting Data Leakage
- **Effort:** 10 minutes of human effort; 5 days compute time for full run on GPUs; 1-10 hours for scaled-down version on a commodity desktop machine.
- **Explanation:** The experiment fine-tunes CodeT5+ on the Devign/DiverseVul/PrimeVul datasets with leakage ratios {0, 0.2, ..., 1.0} added from the test set to the training set.
- **How to:** The experiment only requires two steps to run: set up the environment (HuggingFace token) and build/run the experiment. Both steps, including the exact commands, are described in the README in our artifact repository.
- **Results:** Plots will be saved to the plots/ directory. For the full run, Figure 4 will be reproduced. For the scaled-down version, a monotonic increase of the F1 scores can be observed. Therefore, the claim (C2) is still supported.

g.5.4 Experiment B.2

- **Name:** Commercial LLM Data Leakage
- **Effort:** 10 minutes of human effort; 1-5 hours compute time on a commodity desktop machine.
- **Explanation:** Commit-message and function completion on the PrimeVul dataset across commercial and local LLMs.

- **How to:** The experiment only requires two steps to run: set up the environment (API tokens) and build/run the experiment. Both steps, including the exact commands, are described in the README.
- **Results:** Results will be printed to standard output. Expect results consistent with those reported in the paper (0/100 matches for all models), although minor variations may occur due to randomness and API endpoint behavior.

g.5.5 Experiment C

- **Name:** Context Truncation
- **Effort:** 10 minutes of human effort; 5 minutes compute time on a commodity desktop machine.
- **Explanation:** Tokenize vulnerable functions and compute proportions exceeding context limits.
- **How to:** The experiment requires two steps: set up the environment (HuggingFace token) and build/run the experiment. Instructions are provided in the README.
- **Results:** The resulting table will be saved as a LaTeX file to the `plots/` directory. All numbers will exactly match Table IV in our paper.

g.5.6 Experiment D

- **Name:** Model Collapse
- **Effort:** 10 minutes of human effort; 4-5 days compute time for full runs on GPUs; a scaled-down demonstration is included with only 20-24 hours of compute time for one single generation. (However, the scaled-down version still requires some kind of GPU to work. A free hosted solution may be sufficient.)
- **Explanation:** Iterative self-training on synthetic data and evaluation of perplexity mean and variance across generations.
- **How to:** Setup and execution instructions are described in the README.
- **Results:** The plot will be saved to the `plots/` directory. The trend of increasing perplexity mean and variance will match the claim demonstrated in the paper.

g.6 Notes

Some results may fluctuate slightly due to internal randomness and non-deterministic behavior of both LLM APIs as well as locally hosted LLMs.

HOW AND WHY AGENTS CAN IDENTIFY BUG-INTRODUCING COMMITS

This article has been accepted at the IEEE/ACM International Conference on Automated Software Engineering 2026 (ASE). The full version is available on arXiv at <https://doi.org/10.48550/arXiv.2603.29378> and is reproduced here with permission.

Author Contributions. Niklas Risse served as the primary contributor for this work, leading the conceptualization, data curation, methodology, implementation, formal analysis, interpretation of results, original drafting, and final editing of the manuscript. Marcel Böhme supervised the research and provided guidance on conceptualization, methodology, and interpretation of results, as well as contributing to the review and editing of the final text.

Formatting Changes. The content of the original publication is reproduced unchanged; only its presentation was adapted to match the formatting of this thesis, such as table styles and sizes as well as the citation style.

Original Publication

- [RB26] Niklas Risse and Marcel Böhme. “How and Why Agents Can Identify Bug-Introducing Commits”. In: *arXiv preprint* (2026). **Accepted at the 41st IEEE/ACM International Conference on Automated Software Engineering 2026 (ASE’26)**. DOI: [10.48550/arXiv.2603.29378](https://doi.org/10.48550/arXiv.2603.29378).

How and Why Agents Can Identify Bug-Introducing Commits

Niklas Risse and Marcel Böhme

Abstract. Śliwerski, Zimmermann, and Zeller (SZZ) just won the 2026 ACM SIGSOFT Impact Award for asking:

When do changes induce fixes?

Their paper from 2005 served as the foundation for a wide array of approaches aimed at identifying bug-introducing changes (or commits) from fix commits in software repositories. But even after two decades of progress, the best-performing approach from 2025 yields a modest increase of 10 percentage points in F1-score on the most popular Linux kernel dataset.

In this paper, we uncover how and why LLM-based agents can substantially advance the state-of-the-art in identifying bug-introducing commits from fix commits. We propose a simple agentic workflow based on searching a set of candidate commits and find that it raises the F1-score from 0.64 to 0.81 on the most popular Linux kernel dataset, a bigger jump than between the original 2005 method (0.54) and the previous SOTA (0.64). We also uncover *why* agents are so successful: They derive short *greppable* patterns from the fix commit diff and message and use them to effectively search and find bug-introducing commits in large candidate sets. Finally, we also discuss how these insights might enable further progress in bug detection, root cause understanding, and repair.

6.1 INTRODUCTION

The problem of identifying bug-introducing commits from fix commits has seen remarkable interest from the software engineering research community, demonstrated by the more than 1300 citations of the award-winning paper by Śliwerski, Zimmermann, and Zeller (SZZ) [ŚZZ05]. The problem is also highly relevant for practitioners, since identifying bug-introducing changes is essential to determine which software versions are affected by security-critical bugs [Che+25].

Even though many new approaches have been released since 2005 [Kim+06; DRW14; Cos+17; NCK18; Bao+22; Tan+23; Tan+25], the performance gains remain fairly limited [Che+25]. The foundational 2005 SZZ paper [ŚZZ05] already achieves 0.54 F1-score on a high-quality developer-annotated dataset derived from the Linux kernel [Lyu+24]. On the same dataset, the state-of-the-art method from 2025 [Tan+25] scores just 10 percentage points higher, with an F1-score of 0.64. To break the trend of small incremental improvements, entirely new approaches are needed.

In this paper, we study LLM-based agents for the task of identifying bug-introducing commits (BICs) from fix commits.

Part 1: SZZ-Agent. First, we introduce *SZZ-Agent*, which targets fix commits where the original SZZ algorithm struggles: cases where SZZ finds no candidate BICs (e. g., because the fix adds code rather than removing it), or where its prediction is likely wrong. *SZZ-Agent* collects candidate BICs from the file history and binary searches over them, examining the source code at each version to determine when the bug first appeared.

In our evaluation, *SZZ-Agent* convincingly beats all previous approaches, raising the state-of-the-art score by more than 12 percentage points on the three most popular C/C++ and Java datasets. Beyond effectiveness, we dive into *why* *SZZ-Agent* works using five ablation studies to investigate whether (a) the performance comes from data leakage, (b) what context *SZZ-Agent* relies on, (c) whether the performance gain can be explained by a better LLM-backbone, (d) what components of *SZZ-Agent* contribute the most to its success, and (e) how a specific hyperparameter, the commit selection threshold, controls its performance.

Part 2: Simple-SZZ-Agent. During our ablation studies, we discover that *SZZ-Agent* can be drastically simplified: skipping binary search entirely and letting the agent directly select from all candidates yields the best F1-score at 43% lower LLM token cost (in USD) per fix commit. Based on this finding, we propose *Simple-SZZ-Agent*, which collects candidates from the file history and directly asks the agent to select the bug-introducing commit—yet it outperforms *SZZ-Agent*. Surprisingly, it is also highly scalable: its resource consumption (e. g., tokens) is almost independent of the number of candidate commits. To understand why, we investigated what the agent actually does.

This led us to uncover *why* *Simple-SZZ-Agent* works so well: The agent distills the fix commit message and diff into short string patterns, greps the candidate set for matches, and then reads and evaluates the results, repeating this process until it identifies the bug-introducing commit.

To finalize our study, we propose directions for future work based on our findings. The agent’s ability to compress knowledge about a bug into short greppable patterns opens avenues beyond identifying bug-introducing commits: these patterns could help practitioners understand how a bug was introduced, detect similar bugs in other code, or provide context for generating fixes.

Contributions. We make the following key contributions:

- **SZZ-Agent:** An agentic workflow that uses binary search over source code versions to identify bug-introducing commits, outperforming all previous approaches by more than 12 percentage points in F1-score on three popular datasets. We conduct five ablation studies to understand why it works.
- **Simple-SZZ-Agent:** A simpler agent discovered during our ablations that outperforms even *SZZ-Agent* by directly selecting the

bug-introducing commit from a large candidate set. We uncover its strategy: it compresses the fix commit into short greppable patterns to efficiently search the candidates.

Data Availability. All scripts and data to reproduce our experiments are publicly available at <https://github.com/niklasrisse/agents-for-szz>.

6.2 BACKGROUND

Problem Definition. Given a fix commit c_{fix} that repairs a bug in a software repository, the goal is to identify the set of bug-introducing commits $B \subseteq H$, where H is the repository history prior to c_{fix} . Each commit $b \in B$ introduced a change that caused or contributed to the bug that c_{fix} resolves. In practice, $|B|$ is often one but can be larger when a bug arises from changes spread across multiple commits.

6.2.1 Related Work

The SZZ Algorithm. The original SZZ algorithm [ŠZZ05] identifies bug-introducing commits by tracing the lines deleted in a fix commit back to the commits that last modified them using version control annotations. These earlier commits form the set of bug-introducing candidates. While simple, this approach rests on the assumption that the lines removed by the fix are the same lines that introduced the bug. This assumption does not always hold: for instance, a commit may introduce a function that fails to validate its inputs, yet the fix adds a validation check rather than modifying the original function code. In such cases, the deleted lines in the fix are unrelated to the actual bug-introducing change. More generally, the approach suffers from false positives because not every deleted line is related to the bug fix, and it cannot identify bug-introducing commits for fixes that only add lines without deleting any, since there are no lines to trace back.

Filtering-based Refinements. Subsequent work tries to overcome the limitations of SZZ by refining how candidates are generated, filtered, or selected. AG-SZZ [Kim+06] uses annotation graphs and excludes non-semantic changes such as comments and formatting. MA-SZZ [Cos+17] further excludes meta-changes such as modifications to module declarations and import statements. RA-SZZ [NCK18] integrates refactoring detection tools to remove behavior-preserving changes, which account for 6.5% of changes incorrectly flagged as bug-introducing. L-SZZ and R-SZZ [DRW14] reduce the candidate set to a single commit: L-SZZ selects the candidate that changes the largest number of lines, while R-SZZ selects the most recent candidate. V-SZZ [Bao+22] targets security vulnerabilities specifically by recursively tracing lines back to the earliest commit that modified them, based on the observation that vulnerabilities are often introduced in early versions of the code.

Learning-based Approaches. More recent methods apply machine learning to refine which lines are traced or to assess candidates directly. Neural-SZZ [Tan+23] uses a graph neural network to rank the deleted lines in the fix commit by bug-relevance, then applies SZZ only to the top-ranked lines. LLM4SZZ [Tan+25], published at ACM ISSTA 2025 and to our knowledge the current state-of-the-art technique, uses two LLM-based strategies: rank-based identification, which selects and ranks buggy statements in the fix commit, and context-enhanced identification, which prompts the LLM with contextual information about the fix commit and each SZZ candidate to assess whether a candidate truly introduced the bug.

All of these previous approaches remain critically limited by their performance. Despite two decades of refinements, the cumulative improvement over the original SZZ remains limited to 5–10 percentage points in F1-score using the four datasets in our evaluation setup (see §6.3 and §6.4).

Software Engineering Agents. Agents are software systems that use large language models (LLMs) trained to invoke external tools such as command-line interfaces, file readers, and web search [Sch+23; Yao+23] to achieve goals specified by a user or other agents. Agents have proven effective across a range of software engineering tasks, including automated program repair and autonomous issue resolution [Jin+25; Yan+24; Zha+24]. Several mature agent platforms exist today: Claude Code [Ant25a] is a widely adopted proprietary coding agent, while OpenHands [Wan+25] is a popular open-source platform that supports multiple LLM backends. LLM-based agents promise to be more effective than previous approaches at determining when a bug was first introduced, as they can reason about the semantics of code changes [Gu+24], navigate repository histories with tools, and examine source code across versions. Whether this promise holds is the central question of this paper.

6.3 PART 1: SZZ-AGENT

To test whether the capabilities of LLM-based agents translate into effective bug-introducing commit identification, we design SZZ-Agent, an agentic approach to identify bug-introducing commits. In this section we explain how SZZ-Agent works (§6.3.1) and present our evaluation (§6.3.2–§6.3.3).

6.3.1 Approach

Figure 6.1 shows an overview of SZZ-Agent. It consists of two stages. **Stage 1: SZZ-based Identification.** The approach starts from a fix commit, defined by its diff, showing which lines were added and/or deleted, and its commit message. Via standard SZZ, all deleted lines are traced back to the commits that last introduced them, forming the

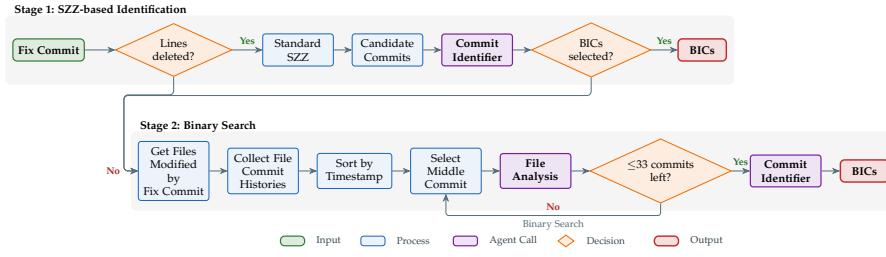


Figure 6.1: SZZ-Agent: Stage 1 applies standard SZZ to the deleted lines and uses an agent to select BICs from the candidates. Stage 2 activates when Stage 1 fails: it collects file histories, binary searches over commits by having an agent analyze the source code at each midpoint, and narrows the search space until the remaining candidates are few enough for direct selection.

SZZ candidate set. We then provide a LLM-based agent with the fix commit message, the fix commit diff, and for each candidate commit its message and diff (all with five lines of context). The agent is instructed to analyze the fix to understand the bug, examine each candidate for whether it introduced the bug, and select the earliest bug-introducing commit, or NONE if no candidate is responsible.

Stage 2: Binary Search. For entries where Stage 1 abstains or where no SZZ candidates exist, we apply a second stage that operates over the full commit history of every file touched by the fix. We then perform a binary search over this history: at each step, the agent receives the full file contents at the candidate commit, the buggy version immediately before the fix, the fixed version, the fix commit message, and the fix commit diff. Based on this, the agent determines whether the bug is already present at the candidate commit. Once the search window narrows to a tunable threshold of candidates (default: 33; see §6.3.3.6 for an ablation), the approach switches to direct candidate selection, presenting the remaining candidates’ diffs and messages for the agent to choose the most likely bug-introducing commit.

6.3.2 Experimental Setup

The goal of the evaluation is to investigate how effective SZZ-Agent is and why it works. To achieve this goal, we ask the following research questions:

RQ1 (Effectiveness): How effective is SZZ-Agent at identifying bug-introducing commits?

RQ2 (Ablations): Why is SZZ-Agent effective?

(a) Data Leakage: Can the effectiveness of SZZ-Agent be explained by data leakage?

(b) LLM Backbone: Is SZZ-Agent more effective than previous state-of-the-art because it uses better LLMs?

(c) Context: What contextual information enables SZZ-Agent to determine the BICs?

(d) Stage Contribution: What is the contribution of Stage 1 vs. Stage 2?

(e) Hyperparameter: How does the performance vary with the threshold for direct commit selection in Stage 2?

The following paragraphs describe the experimental setup shared across all experiments. Experiment-specific details are provided in the individual setup paragraphs in §6.3.3.

6.3.2.1 Datasets and Pre-processing

We use the most popular datasets across two programming language families, which are also the datasets used in the LLM4SZZ evaluation [Tan+25].

DS_LINUX. Since October 2013, Linux kernel developers label bug-fixing patches with the commit identifiers of the corresponding bug-introducing commits via Fixes: tags. Lyu et al. [Lyu+24] leverage this practice to construct a developer-annotated dataset of 76,046 fix commit and bug-introducing commit pairs.

DS_LINUX-26. To evaluate on fix commits published after the training data cutoff of the LLMs we use, we apply the methodology of Lyu et al. [Lyu+24] to collect 5275 fix commit and bug-introducing commit pairs from the Linux kernel for fix commits published between September 1, 2025 and January 31, 2026.

DS_GITHUB-c and DS_GITHUB-j. Rosa et al. [Ros+21] construct a developer-annotated dataset from GitHub projects by using natural language processing to identify fix commits in which developers explicitly reference the bug-introducing commits, followed by manual filtering. We split this dataset by language into DS_GITHUB-c (C/C++ projects) and DS_GITHUB-j (Java projects).

Pre-processing. To prevent the agents from identifying the bug-introducing commit by matching commit hashes referenced in the fix commit message (e.g., in Fixes: tags), we redact commit hash references from all provided diffs and messages. Specifically, all occurrences of ground-truth commit hashes are replaced with COMMIT_HASH, and lines containing Fixes: tags that reference these commits are removed entirely.

6.3.2.2 Baselines

We compare against eight SZZ variants described in §6.2: SZZ, AG-SZZ, L-SZZ, R-SZZ, MA-SZZ, RA-SZZ, V-SZZ, and LLM4SZZ. For all

baselines except V-SZZ and LLM4SZZ, we use the implementations provided by Rosa et al. [Ros+23]. For V-SZZ and LLM4SZZ, we use the implementations provided by the respective authors [Bao+22; Tan+25]. To the best of our knowledge, LLM4SZZ [Tan+25], published at ACM ISSTA 2025, represents the state of the art.

6.3.2.3 Agents

We use Claude Code [Ant25a] as our agent framework. Claude Code provides 30 built-in tools [Ant25b], including bash command execution, file reading and editing, glob and grep search, web search, and sub-agent spawning—capabilities that are essential for inspecting source code across repository versions. We block all agents from using web search tools (e. g., WebSearch and WebFetch in Claude Code) to prevent them from searching the internet for solutions. As LLM backbones, we use models from Anthropic and OpenAI; see the individual experiment setups for details on which models we used. We explore open-source agents and open-source LLM backbones in Part 2 (§6.4.3.4).

6.3.2.4 Metrics

Following Lyu et al. [Lyu+24], we evaluate using precision, recall, and F1-score. Since each fix commit c_{fix} can have multiple ground-truth bug-introducing commits B and multiple predicted commits $\hat{B}$, we compute per-fix-commit precision and recall as:

$$\text{Precision}(c_{\text{fix}}) = \frac{|B \cap \hat{B}|}{|\hat{B}|}, \quad \text{Recall}(c_{\text{fix}}) = \frac{|B \cap \hat{B}|}{|B|}. \quad (6.1)$$

We then macro-average across all fix commits and compute F1-score as the harmonic mean of the averaged precision and recall.

6.3.3 Experiments

6.3.3.1 RQ1: Effectiveness

We evaluate whether SZZ-Agent outperforms existing SZZ variants.

Setup. We run SZZ-Agent with Claude Opus 4.5 as the LLM backbone on all three established datasets: DS_LINUX (200 samples), DS_GITHUB-c (100 samples), and DS_GITHUB-j (75 samples, limited by the number of Java entries in the dataset). Sample sizes were chosen based on budget considerations; we use the Wilcoxon signed-rank test [Wil45] at $p < 0.01$ to assess statistical significance and report the rank-biserial correlation [Ker14] as effect size. We include the sampling scripts with fixed random seeds for exact reproducibility in our artifact. We compare against all eight baselines described in §6.3.2.

Results. Table 6.1 shows the results. SZZ-Agent outperforms all baselines on every dataset. Compared to LLM4SZZ, the previous state of the art, SZZ-Agent improves F1-score by 13 percentage points on DS_LINUX (0.77 vs. 0.64), 12 on DS_GITHUB-c (0.75 vs. 0.63), and

HOW AND WHY AGENTS CAN IDENTIFY BUG-INTRODUCING COMMITS

Table 6.1: RQ1: SZZ-Agent vs. baselines across three datasets. Best results per dataset in **bold**.

Approach	DS_LINUX			DS_GITHUB-c			DS_GITHUB-j		
	Precision	Recall	F1-score	Precision	Recall	F1-score	Precision	Recall	F1-score
SZZ	0.49	0.60	0.54	0.46	0.61	0.53	0.48	0.68	0.57
AG-SZZ	0.48	0.58	0.52	0.46	0.64	0.53	0.45	0.59	0.51
L-SZZ	0.44	0.43	0.44	0.38	0.38	0.38	0.41	0.41	0.41
R-SZZ	0.48	0.47	0.47	0.59	0.58	0.59	0.47	0.47	0.47
MA-SZZ	0.45	0.57	0.50	0.45	0.64	0.53	0.43	0.56	0.49
RA-SZZ	0.45	0.57	0.50	0.45	0.64	0.53	0.37	0.48	0.42
V-SZZ	0.44	0.55	0.49	0.46	0.61	0.53	0.48	0.67	0.56
LLM4SZZ	0.65	0.64	0.64	0.63	0.62	0.63	0.55	0.55	0.55
SZZ-Agent	0.78	0.76	0.77	0.75	0.74	0.75	0.67	0.67	0.67

Table 6.2: RQ2a: SZZ-Agent vs. baselines on DS_LINUX-26 (post training data cutoff). Best results in **bold**.

Approach	Precision	Recall	F1-score
SZZ	0.54	0.59	0.56
AG-SZZ	0.55	0.60	0.57
L-SZZ	0.52	0.51	0.51
R-SZZ	0.52	0.51	0.52
MA-SZZ	0.51	0.58	0.54
RA-SZZ	0.51	0.58	0.54
V-SZZ	0.44	0.49	0.47
LLM4SZZ	0.62	0.61	0.62
SZZ-Agent (Ours)	0.81	0.80	0.81

12 on DS_GITHUB-j (0.67 vs. 0.55). SZZ-Agent achieves the highest precision and recall on all datasets, with the exception of recall on DS_GITHUB-j where standard SZZ achieves 0.68 compared to 0.67. All improvements are statistically significant ($p < 0.01$) with large effect sizes, except for DS_GITHUB-j vs. LLM4SZZ where the effect size is large but $p = 0.013$.

6.3.3.2 RQ2a: Data Leakage

Since the fix commits in DS_LINUX, DS_GITHUB-c, and DS_GITHUB-j were published before the training data cutoff of Claude Opus 4.5 (August 2025 [Ant24]), the model may have encountered them during training. We investigate whether SZZ-Agent’s effectiveness can be explained by such data leakage.

Setup. We evaluate SZZ-Agent on DS_LINUX-26, which we collected by applying the same methodology as Lyu et al. [Lyu+24] used for DS_LINUX but on more recent kernel data. It contains 100 randomly sampled fix commits published between September 2025 and January 2026—after the training data cutoff of Claude Opus 4.5.

Results. Table 6.2 shows the results. SZZ-Agent achieves 0.81 F1-score on DS_LINUX-26, which is even higher than its 0.77 on DS_LINUX. All improvements over baselines are statistically significant ($p < 0.01$) with large effect sizes. This rules out data leakage as an explanation for SZZ-Agent’s effectiveness.

Table 6.3: RQ2b: LLM4SZZ with different LLM backbones vs. SZZ-Agent. Best results in **bold**.

Approach	Precision	Recall	F1-score
LLM4SZZ (gpt-4o-mini)	0.65	0.64	0.64
LLM4SZZ (gpt-5.2)	0.61	0.60	0.61
LLM4SZZ (claude-opus-4-5)	0.66	0.65	0.65
SZZ-Agent (claude-opus-4-5)	0.78	0.76	0.77

6.3.3.3 RQ2b: LLM Backbone

We investigate whether SZZ-Agent’s improvement over LLM4SZZ is simply due to using a more capable LLM.

Setup. We run LLM4SZZ with three different LLM backbones—gpt-4o-mini (its default), gpt-5.2, and Claude Opus 4.5—and compare against SZZ-Agent with Claude Opus 4.5. All experiments use DS_LINUX (200 samples).

Results. Table 6.3 shows the results. Upgrading the LLM backbone of LLM4SZZ from gpt-4o-mini to gpt-5.2 or Claude Opus 4.5 does not meaningfully improve its F1-score ($0.64 \rightarrow 0.61 \rightarrow 0.65$). In contrast, SZZ-Agent with the same Claude Opus 4.5 backbone achieves 0.77. This demonstrates that SZZ-Agent’s advantage stems from its agentic approach rather than from using a better LLM.

***Finding 1:** SZZ-Agent outperforms all previous approaches by at least 12 percentage points in F1-score on three established datasets (RQ1). This improvement cannot be explained by data leakage (RQ2a) or by the use of a more capable LLM backbone (RQ2b), but stems from the agentic approach itself.*

6.3.3.4 RQ2c: Context Ablation

We investigate which contextual information enables SZZ-Agent to identify bug-introducing commits.

Setup. We ablate the fix commit message and the fix commit diff by selectively withholding them from the agent. We run all variants on DS_LINUX-26 with Claude Opus 4.5.

Results. Table 6.4 shows the results. Without both the commit message and the diff, SZZ-Agent drops to 0.48 F1-score, below standard SZZ (0.56). Providing only the diff recovers performance to 0.73, and providing only the message reaches 0.77—each individually already exceeding the previous state-of-the-art (LLM4SZZ, 0.62 on DS_LINUX-26). This suggests that the two sources carry partially redundant information. However, combining both achieves 0.81, indicating that each source also provides complementary information not captured by the other.

HOW AND WHY AGENTS CAN IDENTIFY BUG-INTRODUCING COMMITS

Table 6.4: RQ2c: Impact of withholding the fix commit message and diff. Best results in **bold**.

Approach	With Message	With Diff	Precision	Recall	F1-score
SZZ	–	–	0.54	0.59	0.56
SZZ-Agent	✗	✗	0.49	0.48	0.48
SZZ-Agent	✗	✓	0.74	0.73	0.73
SZZ-Agent	✓	✗	0.78	0.77	0.77
SZZ-Agent	✓	✓	0.81	0.80	0.81

Table 6.5: RQ2d: Contribution of Stage 1 and Stage 2. Costs are average per fix commit. Best results in **bold**.

Approach	Precision	Recall	F1-score	Cost (USD)
Stage 1 Only	0.48	0.47	0.48	\$0.18
Stage 2 Only	0.77	0.76	0.76	\$1.20
SZZ-Agent	0.81	0.80	0.81	\$0.67

***Finding 2:** The fix commit message and the fix commit diff carry partially redundant information: each alone already surpasses the previous state-of-the-art. However, combining both yields the highest performance (0.81 F1), showing that neither source fully subsumes the other.*

6.3.3.5 RQ2d: Stage 1 vs. Stage 2

We investigate the individual contribution of each stage to the overall pipeline.

Setup. We run Stage 1 and Stage 2 in isolation and compare against the combined pipeline. All experiments use DS_LINUX-26 with Claude Opus 4.5. We also report the average cost per fix commit in USD.

Results. Table 6.5 shows the results. Stage 1 alone achieves only 0.48 F1-score at \$0.18 per fix commit, while Stage 2 alone reaches 0.76 at \$1.20. The combined pipeline achieves the best F1-score of 0.81 at \$0.67, demonstrating that both stages are complementary: Stage 1 cheaply resolves straightforward cases, reducing the number of entries that require the more expensive Stage 2.

***Finding 3:** Stage 1 and Stage 2 are complementary. Stage 1 cheaply resolves straightforward cases, while Stage 2 handles the remaining difficult cases. Their combination achieves the best F1-score at moderate cost.*

6.3.3.6 RQ2e: Commit Selection Threshold

We investigate how the commit selection threshold—the number of remaining candidates at which Stage 2 switches from binary search to direct selection—affects performance and cost.

Setup. We vary the threshold across values of 8, 32, 128, 512, and ∞ (no binary search, always direct selection). All experiments use DS_LINUX-26 with Claude Opus 4.5.

Results. Table 6.6 shows the results. Performance is remarkably stable across thresholds, ranging from 0.76 to 0.82 F1-score. Higher thresholds

Table 6.6: RQ2e: Impact of the commit selection threshold. Costs are average per fix commit. Best results in **bold**.

Commit Identifier Threshold	Precision	Recall	F1-score	Cost (USD)
8	0.76	0.75	0.76	\$0.68
32	0.81	0.80	0.81	\$0.67
128	0.80	0.79	0.80	\$0.39
512	0.81	0.80	0.81	\$0.35
∞	0.82	0.81	0.82	\$0.33

reduce cost (\$0.68 at threshold 8 vs. \$0.33 at ∞) because fewer binary search steps are needed. The ∞ threshold—which skips binary search entirely and always uses direct selection—achieves the best F1-score (0.82) at the lowest cost (\$0.33).

Finding 4 (Most Surprising): *Skipping binary search entirely and always using direct candidate selection yields the best F1-score (0.82) at the lowest cost. The agent can identify the bug-introducing commit from a large candidate set in a single step, without iterative narrowing. This observation led us to question whether binary search is necessary at all, and motivated the development of a drastically simplified approach: Simple-SZZ-Agent (§6.4).*

6.4 PART 2: SIMPLE-SZZ-AGENT

Our experiment on varying the commit selection threshold (RQ2e, §6.3.3.6) revealed that skipping binary search entirely and always using direct candidate selection yields the best performance at the lowest cost. If the agent can identify the bug-introducing commit from a large candidate set in a single step, do we need the complexity of SZZ-Agent at all? Motivated by this finding, we design Simple-SZZ-Agent, a drastically simplified approach that removes both the SZZ-based first stage and the binary search.

6.4.1 Approach

Figure 6.2 shows an overview of Simple-SZZ-Agent. Given a fix commit, the approach collects the commit histories of all files modified by the fix, forming a single candidate set. It then provides an LLM-based agent with the fix commit message, the fix commit diff, and the full candidate set. The agent has access to standard developer tools (file reading, grep, glob) and is instructed to identify the bug-introducing commit. Since our ablations showed that omitting binary search improves both performance and cost (RQ2e), and that Stage 1 alone contributes less than Stage 2 (RQ2d), Simple-SZZ-Agent removes both: no SZZ-based filtering, no binary search, and no multi-stage pipeline—it relies entirely on the agent to find the bug-introducing commit among all candidates.

HOW AND WHY AGENTS CAN IDENTIFY BUG-INTRODUCING COMMITS

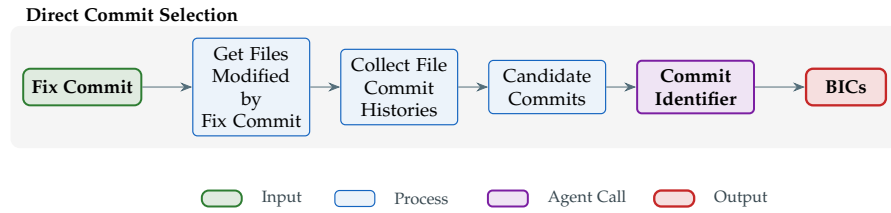


Figure 6.2: Simple-SZZ-Agent: The fix commit’s file histories are collected into a single candidate set. An agent with access to developer tools directly selects the bug-introducing commit, without SZZ-based filtering or binary search.

6.4.2 Experimental Setup

We evaluate whether Simple-SZZ-Agent can match SZZ-Agent’s performance and investigate why it works.

RQ3 (Effectiveness): Can Simple-SZZ-Agent achieve a similar performance to SZZ-Agent at identifying bug-introducing commits?

RQ4 (Analysis): How can we explain the surprising success of Simple-SZZ-Agent?

(a) Strategy: How can Simple-SZZ-Agent find the needle (BIC) in the haystack (large candidate commit set)?

(b) Failures: Where does Simple-SZZ-Agent still fail?

(c) Generalization: Can we expect even better results with better agents and/or better models?

We use the same datasets, pre-processing, baselines, and metrics as in Part 1 (§6.3.2). Unless stated otherwise, all experiments use DS_LINUX-26 with Claude Opus 4.5 as the LLM backbone and Claude Code as the agent framework.

6.4.3 Experiments

6.4.3.1 RQ3: Effectiveness

We evaluate whether Simple-SZZ-Agent can match or exceed the performance of SZZ-Agent despite its drastically simpler design.

Setup. We run Simple-SZZ-Agent on DS_LINUX, DS_GITHUB-c, and DS_GITHUB-j, and compare against SZZ, LLM4SZZ, and SZZ-Agent.

Results. Table 6.7 shows the results. Simple-SZZ-Agent matches or outperforms SZZ-Agent on every dataset. On DS_LINUX, it achieves 0.81 F1-score compared to 0.77 for SZZ-Agent, an improvement of 4 percentage points. On DS_GITHUB-c, both achieve 0.75. On DS_GITHUB-j, Simple-SZZ-Agent achieves 0.75 compared to 0.67 for SZZ-Agent, an improvement of 8 percentage points.

Table 6.7: RQ3: Comparison of SZZ-Agent and Simple-SZZ-Agent for bug-introducing commit identification. Best results per dataset in **bold**.

Approach	DS_LINUX			DS_GITHUB-c			DS_GITHUB-j		
	Precision	Recall	F1-score	Precision	Recall	F1-score	Precision	Recall	F1-score
SZZ	0.49	0.60	0.54	0.46	0.61	0.53	0.48	0.68	0.57
LLM4SZZ	0.65	0.64	0.64	0.63	0.62	0.63	0.55	0.55	0.55
SZZ-Agent	0.78	0.76	0.77	0.75	0.74	0.75	0.67	0.67	0.67
Simple-SZZ-Agent	0.82	0.81	0.81	0.75	0.75	0.75	0.75	0.75	0.75

***Finding 5:** Simple-SZZ-Agent matches or outperforms SZZ-Agent on all datasets, despite removing both the SZZ-based first stage and the binary search. A single agent call over the full candidate set is sufficient.*

6.4.3.2 RQ4a: How Does the Agent Find the Needle in the Haystack?

Simple-SZZ-Agent receives hundreds or even thousands of candidate commits, yet it identifies the bug-introducing commit reliably. We investigate how.

Setup. We log all tool calls and analyze the behavior of the agent on DS_LINUX-26 (100 fix commits). We record the relationship between the number of candidate commits and the agent’s F1-score and resource consumption (cost, tokens, tool calls), the distribution of tool calls by type, and the sources of grep patterns.

Results. We expected that scanning a large candidate set would degrade performance and require substantially more resources. To our surprise, neither turned out to be the case. Figure 6.3(a) shows the relationship between candidate set size and F1-score. Performance remains stable across candidate set sizes, with F1-scores above 0.80 for sets of up to 1,000 candidates. Only for the largest sets (>1,000 candidates) does F1-score decrease to 0.72. Figure 6.3 shows the relationship between the number of candidate commits and resource consumption. Cost, token usage, and tool calls show no strong correlation with the number of candidates, indicating that the agent does not scale linearly with the candidate set size.

To understand how the agent achieves this, we examined its tool usage. Figure 6.4 shows the distribution of tool calls. The agent predominantly uses Read (9.8 calls on average) and Grep (2.7 calls), with few calls to other tools. This reveals that the agent does not read all candidates sequentially; instead, it uses targeted searches to narrow down the candidate set. Figure 6.5 shows what the agent greps for. The most frequent grep patterns are derived from the removed lines of the fix commit diff (48.1% of grep calls) and the fix commit message (43.6%). The agent distills these sources into short patterns and uses them to search the candidate commits for matches. Across 243 grep calls, patterns have a median length of 21 characters (mean 25.2, std. dev. 14.8, min 6, max 104); the majority (63.8%) are plain strings rather than regular expressions.

Three examples illustrate this strategy:

HOW AND WHY AGENTS CAN IDENTIFY BUG-INTRODUCING COMMITS

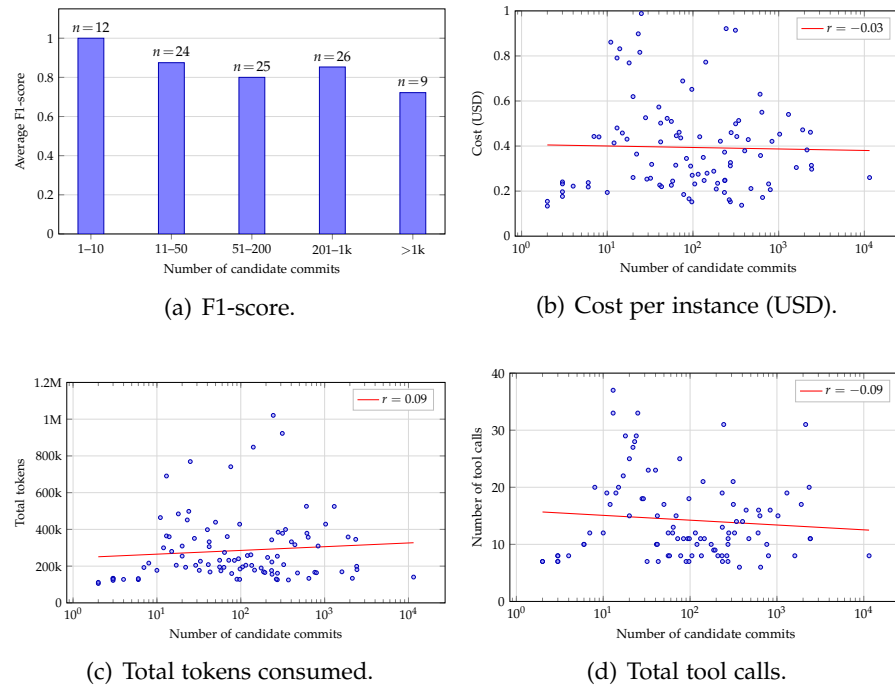


Figure 6.3: Relationship between the number of candidate commits and F1-score, cost, token usage, and tool calls.

1. **Pattern from fix commit message.** The fix [Lin25b] replaces `fsleep()` with `udelay()` because `fsleep()` must not be called from atomic context. The agent greps for "fsleep" and directly locates the bug-introducing commit [Lin25a], which introduced the `fsleep()` call.
2. **Pattern from removed lines.** The fix [Lin25c] corrects a firmware parser that assumed dword-aligned block sizes. The agent greps for "`blk->size > 2`", the faulty expression removed by the fix, and matches it to the bug-introducing commit [Lin20] that added this line. This is analogous to what SZZ does—tracing removed lines back to the commit that introduced them—but the agent performs it via `grep` rather than `git blame`.
3. **Pure-addition fix.** The fix [Lin25d] adds a NULL-pointer check for `smc_ib_is_sg_need_sync()` (3 insertions, 0 deletions). Standard SZZ cannot handle this case at all, because there are no removed lines to trace back via `git blame`. The agent instead extracts the function name from the commit message, greps for "`smc_ib_is_sg_need_sync`", and identifies the bug-introducing commit [Lin22] that introduced this function.

Finding 6 (Key): Cost and performance are largely independent of the candidate commit set size: the agent finds the needle without examining the full haystack. It does so by deriving short greppable patterns (median 21 characters) from the fix commit diff and message and using them to search the candidate set.

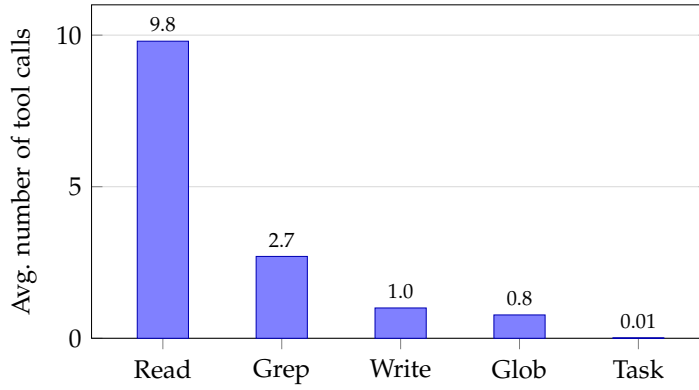


Figure 6.4: Distribution of tool calls per tool type during agent execution.

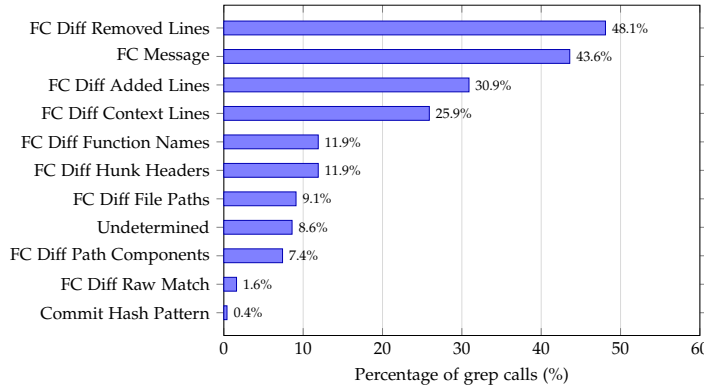


Figure 6.5: Distribution of grep search sources used by the agent. "FC" stands for "Fix Commit".

6.4.3.3 RQ4b: Where Does Simple-SZZ-Agent Fail?

We perform a manual error analysis to understand the failure cases.

Setup. We manually inspect all incorrect predictions of Simple-SZZ-Agent on DS_LINUX-26 (100 fix commits). For each failure, we investigate the fix commit, the ground truth bug-introducing commit, the predicted bug-introducing commit, and the explanation provided by the agent, and categorize them by failure mode.

Results. Figure 6.6 shows the results. Of 100 fix commits, Simple-SZZ-Agent correctly identifies the bug-introducing commit in 87 cases. Of the 13 incorrect predictions, 4 fail because the true bug-introducing commit is not in the candidate set (i. e., the bug was introduced in a file not modified by the fix). The remaining 9 failures occur when the bug-introducing commit is in the candidate set but the agent does not select the correct one.

We further categorize these 9 cases through manual analysis: in 3 cases, the agent performs an incorrect analysis—it identifies the correct code area but confuses commits that modified the relevant code with the commit that originally introduced the flaw; in 2 cases, the agent selects a commit within 3 positions of the correct one (near miss), including one case where the predicted and ground truth commits

HOW AND WHY AGENTS CAN IDENTIFY BUG-INTRODUCING COMMITS

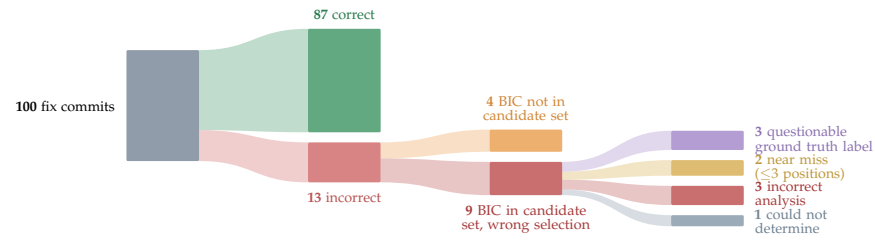


Figure 6.6: Manual error analysis of Simple-SZZ-Agent on 100 fix commits from DS_LINUX-26.

are consecutive commits from the same patch series; in 3 cases, the ground truth label itself is questionable; and in 1 case, we could not determine the reason for the failure because the fix and ground truth bug-introducing commits were too complex for us to analyze as non-Linux-kernel experts. Since we are not Linux kernel experts, we also cannot determine with certainty whether the 3 questionable labels are incorrect; we include a detailed report for each case in our artifact.

Finding 7: Of the 13 failures, 4 occur because the bug-introducing commit is not in the candidate set. Of the remaining 9, 3 are due to incorrect agent analysis, 2 are near misses, 3 involve questionable ground truth labels, and 1 we could not categorize.

6.4.3.4 RQ4c: Better Models and Agents

We investigate whether Simple-SZZ-Agent’s performance generalizes across different LLM backbones and agent frameworks, and whether cheaper or more capable configurations can improve results.

Setup. We evaluate Simple-SZZ-Agent with two agent frameworks—Claude Code and OpenHands—across multiple LLM backbones: Claude Haiku 4.5, Claude Sonnet 4.5, Claude Opus 4.5, as well as minimax-m2.5 and glm-5 for OpenHands. All experiments use DS_LINUX-26 (100 samples).

Results. Table 6.8 shows the results. Simple-SZZ-Agent is effective across all tested configurations. Even the cheapest configuration (OpenHands with minimax-m2.5 at \$0.04 per fix commit) achieves 0.79 F1-score, outperforming all previous non-agentic approaches. The best configuration (OpenHands with Claude Sonnet 4.5) achieves 0.86 F1-score at \$0.38 per fix commit, surpassing the Claude Opus 4.5 configurations. Within the same agent framework, more capable models generally perform better, though the difference between Sonnet and Opus is small or even reversed.

Finding 8: Simple-SZZ-Agent generalizes across agent frameworks and LLM backbones. Even the worst-performing configuration (Claude Code with Claude Haiku 4.5, 0.74 F1) outperforms the previous state of the art by 12 percentage points, and the best configuration reaches 0.86 F1-score, suggesting that further progress with improved models and agents is likely.

Table 6.8: RQ4c: Simple-SZZ-Agent with different underlying models and agents. Best results in **bold**.

Agent	Model	Precision	Recall	F1-score	Avg. per Fix Commit		
					Cost	Tool Calls	Tokens
Claude Code	claude-haiku-4-5	0.75	0.73	0.74	\$0.09	14.2	339k
Claude Code	claude-sonnet-4-5	0.81	0.80	0.80	\$0.25	16.7	290k
Claude Code	claude-opus-4-5	0.83	0.81	0.82	\$0.38	14.3	284k
OpenHands	claude-haiku-4-5	0.80	0.79	0.79	\$0.14	15.7	1.3M
OpenHands	claude-sonnet-4-5	0.87	0.86	0.86	\$0.38	14.0	1.0M
OpenHands	claude-opus-4-5	0.84	0.82	0.83	\$0.63	16.9	1.0M
OpenHands	minimax-m2.5	0.80	0.78	0.79	\$0.04	20.0	1.3M
OpenHands	glm-5	0.79	0.78	0.78	\$0.26	17.8	754k

6.5 DISCUSSION

We discuss directions for future work motivated by our findings. While these directions are beyond the scope of this paper, we hope they inspire other researchers to explore them. We cover research on identifying bug-introducing commits (SZZ research), root cause understanding, bug detection, and program repair.

6.5.1 SZZ Research

Closing the Gap. Our results represent a substantial step toward solving the problem of identifying bug-introducing commits from fix commits. Simple-SZZ-Agent achieves up to 0.86 F1-score on the Linux kernel dataset, compared to 0.62 for the previous state of the art. Still, the problem is not fully solved. Our error analysis (§6.4.3.3) revealed that of the 13 failures, 4 occur because the bug-introducing commit is not in the candidate set, 3 are due to incorrect agent analysis, 2 are near misses, 3 involve questionable ground truth labels, and 1 we could not categorize. These failure modes point to two main areas for improvement.

First, the incorrect analyses and near misses (5 cases total) suggest that the agent’s reasoning about which commit introduced a bug can still be improved. Our results in §6.4.3.4 show that different model and agent combinations yield meaningful performance differences, indicating that continued model improvements are a promising avenue.

Second, the 4 cases where the bug-introducing commit is not in the candidate set reveal a limitation of the current candidate collection strategy: Simple-SZZ-Agent only searches the commit histories of files modified by the fix, so when the bug was introduced in a different file, the correct commit is never considered. How to expand the candidate set beyond the files touched by the fix without introducing prohibitive infrastructure costs is an open question that we leave for future work.

6.5.2 *Root Cause Understanding*

Our analysis in §6.4.3.2 revealed that agents compress information about a bug into short greppable patterns that locate its introducing changes. This compression itself may have value beyond identifying the bug-introducing commit: it captures, in a concise form, what made the code buggy.

A practitioner who knows how to fix a bug does not always understand how it was introduced. The patterns the agent derives and the bug-introducing changes it identifies could help provide this understanding. Our context ablation (§6.3.3.4) showed that the agent achieves 0.73 F1-score with only the diff and no commit message, demonstrating that it can extract the essential characteristics of a bug from code changes alone. Whether these patterns can be used to generate useful explanations of how a bug was introduced—especially when the fix commit message is uninformative—is an open question for future work.

6.5.3 *Bug Detection*

Risse, Liu, and Böhme [RLB25] raised two open questions in bug detection research: (i) what amount of context is sufficient to decide whether a given code unit is buggy, and (ii) how should this context be collected? Our findings suggest a possible direction for (ii): the grep-based search strategy could serve as an automatic context collection mechanism, where an agent uses targeted searches to gather relevant code from across a repository to assemble the context necessary for bug detection. Whether this strategy transfers effectively to bug detection remains to be investigated.

Beyond context collection, the patterns that agents derive from fix commits could help in bug variant detection. If the agent can compress knowledge about a bug into a short pattern to find the commit that introduced it, a generalized version of the pattern might identify commits that introduced similar bugs elsewhere in the codebase or in other projects. We leave the exploration of this direction for future work.

6.5.4 *Program Repair*

In program repair, the goal is to generate fixes for bugs. Our approach currently operates after a fix has already been written: it takes the fix commit as input and traces back to the bug-introducing commit. An open question is whether the bug-introducing commit can also be identified from the outputs of bug oracles instead of fix commits—for example, from crash reports of a fuzzer, failing unit tests, or alerts from monitoring software. If so, agents could provide the bug-introducing changes as context for generating the fix itself, enabling agents to both locate and fix bugs. We leave this question for future work.

6.6 THREATS TO VALIDITY

We discuss threats to the internal, external, and construct validity of our study.

6.6.1 *Internal Validity*

Sample Size. Budget constraints limit our evaluation to 100–200 fix commits per dataset, which may not capture the full distribution of bug types and fix patterns. We mitigate this by using fixed random seeds for sampling, reporting statistical significance tests and effect sizes for all comparisons with previous approaches, and providing all sampling scripts for reproducibility.

Prompt Sensitivity. The instructions given to the agent may influence its behavior and effectiveness. We did not perform systematic prompt optimization; our prompts are straightforward task descriptions and are released with our artifact.

6.6.2 *External Validity*

Language and Project Scope. Our evaluation covers C/C++ and Java projects across four datasets derived from the Linux kernel and GitHub repositories. Results may not generalize to other languages or project types. We selected these datasets because they are the most popular in SZZ research and allow direct comparison with prior work.

LLM Evolution. Our results depend on current LLM capabilities and future models may yield different results. Our evaluation across six models and two agent frameworks (§6.4.3.4) demonstrates that the approach is not tied to a specific model.

6.6.3 *Construct Validity*

Ground Truth Quality. We rely on developer-annotated bug-introducing commits as ground truth. For DS_LINUX and DS_LINUX-26, these annotations come from `Fixes:` tags added by Linux kernel developers, which may occasionally be incorrect or incomplete. For DS_GITHUB-c and DS_GITHUB-j, the annotations were constructed by identifying fix commits where developers explicitly reference bug-introducing commits, followed by manual filtering [Ros+21]. Inaccuracies in these labels would affect all approaches equally, but could bias absolute performance estimates.

Single Bug-Introducing Commit Assumption. Our evaluation metrics assume that each fix commit has a well-defined set of bug-introducing commits. In practice, bugs can result from complex interactions across multiple commits, and the notion of a single introducing commit may oversimplify the true causal history. This limitation applies to

all SZZ approaches and is inherited from the standard evaluation methodology [Lyu+24].

6.7 CONCLUSION

We introduced SZZ-Agent, an agentic approach that uses binary search over source code versions to identify bug-introducing commits, outperforming all previous approaches by at least 12 percentage points in F1-score on three established datasets. Through five ablation studies, we established that this improvement stems from the agentic approach itself, not from data leakage or a more capable LLM backbone.

During our ablations, we discovered that binary search is unnecessary: Simple-SZZ-Agent, which gives a single agent the full candidate set, matches or outperforms SZZ-Agent on all datasets. It distills the fix commit into short greppable patterns to search the candidate set, explaining why cost and performance remain largely independent of the candidate set size. Simple-SZZ-Agent generalizes across agent frameworks and LLM backbones, and even cheap configurations outperform all previous non-agentic approaches.

Beyond identifying bug-introducing commits, the agent’s ability to compress bug knowledge into short patterns opens avenues for future work in root cause understanding, bug detection, and program repair. We believe that agents, equipped with simple developer tools like grep, offer a powerful and general strategy for navigating large search spaces in software engineering—and that the problems we studied here are only the beginning.

REFERENCES

- [Ant24] Anthropic. *Models Overview*. 2024. URL: <https://platform.claude.com/docs/en/about-claude/models/overview>.
- [Ant25a] Anthropic. *Claude Code*. 2025. URL: <https://github.com/anthropics/claude-code>.
- [Ant25b] Anthropic. *Claude Code Tools Reference*. 2025. URL: <https://code.claude.com/docs/en/tools-reference>.
- [Bao+22] Lingfeng Bao, Xin Xia, Ahmed E. Hassan, and Xiaohu Yang. “V-SZZ: automatic identification of version ranges affected by CVE vulnerabilities”. In: *Proceedings of the 44th IEEE/ACM International Conference on Software Engineering (ICSE)*. 2022. DOI: [10.1145/3510003.3510113](https://doi.org/10.1145/3510003.3510113).

- [Che+25] Xingchu Chen, Chengwei Liu, Jialun Cao, Yang Xiao, Xinyue Cai, Yeting Li, Jingyi Shi, Tianqi Sun, Haiming Chen, and Wei Huo. “Vulnerability-Affected Versions Identification: How Far Are We?” In: *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2025. DOI: [10.1109/ASE63991.2025.00244](https://doi.org/10.1109/ASE63991.2025.00244).
- [Cos+17] Daniel Alencar da Costa, Shane McIntosh, Weiyi Shang, Uirá Kulesza, Roberta Coelho, and Ahmed E. Hassan. “A Framework for Evaluating the Results of the SZZ Approach for Identifying Bug-Introducing Changes”. In: *IEEE Transactions on Software Engineering* (2017). DOI: [10.1109/TSE.2016.2616306](https://doi.org/10.1109/TSE.2016.2616306).
- [DRW14] Steven Davies, Marc Roper, and Murray Wood. “Comparing text-based and dependence-based approaches for determining the origins of bugs”. In: *Journal of Software: Evolution and Process* (2014). DOI: <https://doi.org/10.1002/smr.1619>.
- [Gu+24] Alex Gu, Baptiste Roziere, Hugh James Leather, Armando Solar-Lezama, Gabriel Synnaeve, and Sida Wang. “CRUX-Eval: A Benchmark for Code Reasoning, Understanding and Execution”. In: *Proceedings of the International Conference on Machine Learning (ICML)*. 2024. URL: <https://proceedings.mlr.press/v235/gu24c.html>.
- [Jin+25] Haolin Jin, Linghan Huang, Haipeng Cai, Jun Yan, Bo Li, and Huaming Chen. “From LLMs to LLM-based Agents for Software Engineering: A Survey of Current, Challenges and Future”. In: *arXiv preprint* (2025). DOI: [10.48550/arXiv.2408.02479](https://doi.org/10.48550/arXiv.2408.02479).
- [Ker14] Dave S. Kerby. “The Simple Difference Formula: An Approach to Teaching Nonparametric Correlation”. In: *Comprehensive Psychology* (2014). DOI: [10.2466/11.IT.3.1](https://doi.org/10.2466/11.IT.3.1).
- [Kim+06] Sunghun Kim, Thomas Zimmermann, Kai Pan, and E. James Jr. Whitehead. “Automatic Identification of Bug-Introducing Changes”. In: *Proceedings of the 21st IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2006. DOI: [10.1109/ASE.2006.23](https://doi.org/10.1109/ASE.2006.23).
- [Lin20] Linux Kernel. *drm/msm/a6xx: A640/A650 GMU firmware path*. 2020. URL: <https://github.com/torvalds/linux/commit/c6ed04f856a4>.
- [Lin22] Linux Kernel. *net/smc: optimize for smc_sndbuf_sync_sg_for_device and smc_rmb_sync_sg_for_cpu*. 2022. URL: <https://github.com/torvalds/linux/commit/0ef69e788411>.
- [Lin25a] Linux Kernel. *drm/amd/display: Fix ‘failed to blank crtc!’* 2025. URL: <https://github.com/torvalds/linux/commit/01f60348d8fb>.

- [Lin25b] Linux Kernel. *drm/amd/display: use udelay rather than fsleep*. 2025. URL: <https://github.com/torvalds/linux/commit/1d66c3f2b8c0>.
- [Lin25c] Linux Kernel. *drm/msm/a6xx: Fix GMU firmware parser*. 2025. URL: <https://github.com/torvalds/linux/commit/b4789aac9d34>.
- [Lin25d] Linux Kernel. *net/smc: fix one NULL pointer dereference in smc_ib_is_sg_need_sync()*. 2025. URL: <https://github.com/torvalds/linux/commit/ba1e9421cf1a>.
- [Lyu+24] Yunbo Lyu, Hong Jin Kang, Ratnadira Widayasari, Julia Lawall, and David Lo. “Evaluating SZZ Implementations: An Empirical Study on the Linux Kernel”. In: *IEEE Transactions on Software Engineering* (2024). DOI: [10.1109/TSE.2024.3406718](https://doi.org/10.1109/TSE.2024.3406718).
- [NCK18] Edmilson Campos Neto, Daniel Alencar da Costa, and Uirá Kulesza. “The impact of refactoring changes on the SZZ algorithm: An empirical study”. In: *Proceedings of the IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 2018. DOI: [10.1109/SANER.2018.8330225](https://doi.org/10.1109/SANER.2018.8330225).
- [RLB25] Niklas Risse, Jing Liu, and Marcel Böhme. “Top Score on the Wrong Exam: On Benchmarking in Machine Learning for Vulnerability Detection”. In: *Proceedings of the ACM on Software Engineering* (ISSTA 2025). **ACM SIGSOFT Distinguished Paper Award**. DOI: [10.1145/3728887](https://doi.org/10.1145/3728887).
- [Ros+21] Giovanni Rosa, Luca Pascarella, Simone Scalabrino, Rosalia Tufano, Gabriele Bavota, Michele Lanza, and Rocco Oliveto. “Evaluating SZZ Implementations Through a Developer-informed Oracle”. In: *Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering (ICSE)*. 2021. DOI: [10.1109/ICSE43902.2021.00049](https://doi.org/10.1109/ICSE43902.2021.00049).
- [Ros+23] Giovanni Rosa, Luca Pascarella, Simone Scalabrino, Rosalia Tufano, Gabriele Bavota, Michele Lanza, and Rocco Oliveto. “A comprehensive evaluation of SZZ Variants through a developer-informed oracle”. In: *Journal of Systems and Software* (2023). DOI: [10.1016/j.jss.2023.111729](https://doi.org/10.1016/j.jss.2023.111729).
- [Sch+23] Timo Schick, Jane Dwivedi-Yu, Roberto Dessí, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. “Toolformer: language models can teach themselves to use tools”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2023. URL: http://papers.nips.cc/paper_files/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html.

- [ŚZZ05] Jacek Śliwerski, Thomas Zimmermann, and Andreas Zeller. “When do changes induce fixes?” In: *ACM SIGSOFT Software Engineering Notes* (2005). DOI: [10.1145/1082983.1083147](https://doi.org/10.1145/1082983.1083147).
- [Tan+23] Lingxiao Tang, Lingfeng Bao, Xin Xia, and Zhongdong Huang. “Neural SZZ Algorithm”. In: *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2023. DOI: [10.1109/ASE56229.2023.00037](https://doi.org/10.1109/ASE56229.2023.00037).
- [Tan+25] Lingxiao Tang, Jiakun Liu, Zhongxin Liu, Xiaohu Yang, and Lingfeng Bao. “LLM4SZZ: Enhancing SZZ Algorithm with Context-Enhanced Assessment on Large Language Models”. In: *Proceedings of the ACM on Software Engineering* (2025). DOI: [10.1145/3728885](https://doi.org/10.1145/3728885).
- [Wan+25] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xian-gru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muenighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. “OpenHands: An Open Platform for AI Software Developers as Generalist Agents”. In: *arXiv preprint* (2025). DOI: [10.48550/arXiv.2407.16741](https://doi.org/10.48550/arXiv.2407.16741).
- [Wil45] Frank Wilcoxon. “Individual Comparisons by Ranking Methods”. In: *Biometrics Bulletin* (1945). DOI: [10.2307/3001968](https://doi.org/10.2307/3001968).
- [Yan+24] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. “SWE-agent: agent-computer interfaces enable automated software engineering”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2024. URL: http://papers.nips.cc/paper_files/paper/2024/hash/5a7c947568c1b1328ccc5230172e1e7c-Abstract-Conference.html.
- [Yao+23] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *Proceedings of the International Conference on Learning Representations (ICLR)*. 2023. URL: https://openreview.net/forum?id=WE_vluYUL-X.
- [Zha+24] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. “AutoCodeRover: Autonomous Program Improvement”. In: *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. 2024. DOI: [10.1145/3650212.3680384](https://doi.org/10.1145/3650212.3680384).

PART III

DISCUSSION AND CONCLUSIONS

DISCUSSION AND CONCLUSIONS

You should enjoy the little detours to the fullest. Because that's where you'll find the things more important than what you want.

— GING FREECSS

In this final chapter, I discuss the implications and impact of our research contributions, draw conclusions, and provide an outlook on the future of machine learning for software security (ML4SS).

7.1 OVERVIEW

When I started my PhD in November 2022, machine learning for software security (ML4SS) was receiving rapidly increasing attention, largely driven by the rise of deep learning for vulnerability detection. At the time, however, critical perspectives on the limits of these novel techniques were still hard to find. Figure 7.1 positions the four research contributions of this thesis on the Gartner hype cycle of machine learning for software security, which visualizes the visibility, or *hype*, of a new technology over time.

Testing ML4VD Adversarially. Our first contribution, published on arXiv in June 2023 and later at the USENIX Security Symposium 2024 [RB24], provided one of the earliest critical investigations of ML4VD. We showed that existing approaches classify functions by overfitting to features unrelated to vulnerabilities, such as identifier names. Moreover, the models were unable to distinguish vulnerable functions from their patched counterparts, indicating that they lacked a meaningful understanding of the code they classified. By challenging the reliability of the high scores commonly reported in the literature, this work *grounded* the growing hype surrounding ML4VD.

Top Score on the Wrong Exam. Our second contribution, published on arXiv in August 2024 and later at ISSTA 2025 [RLB25], revealed *why* the previous high scores could *only* be explained by relying on features unrelated to vulnerabilities: because the dominant benchmarking paradigm itself, treating vulnerability detection as a binary function-level classification task, was fundamentally flawed. In doing so, this work effectively invalidated the central evaluation setup used throughout much of the ML4VD literature and marked a turning point for the field at the peak of inflated expectations.

Pitfalls of LLMs for Security. Our third contribution, published on arXiv in December 2025 and later at NDSS 2026 [Eve+26], broadened the perspective from ML4VD specifically to the use of large language models in software security more generally. The work identified a wide range of pitfalls associated with applying LLMs to security tasks and analyzed their implications for research validity. Although critical

DISCUSSION AND CONCLUSIONS

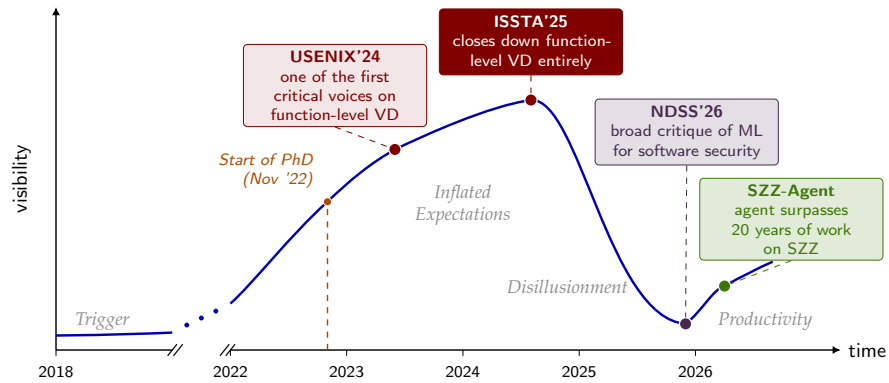


Figure 7.1: The four contributions of this thesis positioned on the Gartner hype cycle of machine learning for software security, by the date each preprint was first released.

perspectives had become more common by this point, this contribution was among the first to provide a comprehensive overview of what can go wrong when using LLMs in software security. Importantly, the work also shifted from critique toward guidance by offering concrete recommendations for mitigating these pitfalls and improving the reliability of future research.

Agentic Software Security. Our fourth contribution, published on arXiv in March 2026 and accepted at the 41st IEEE/ACM International Conference on Automated Software Engineering 2026 (ASE’26), explored how agentic systems can overcome many of the limitations of previous ML4SS approaches. By combining LLM reasoning with external tools such as search, the proposed agents surpassed more than two decades of prior work on identifying bug-introducing commits from fix commits. In contrast to earlier ML-based approaches, agents provide a substantially more flexible and powerful framework for software security tasks, extending the capabilities of machine learning for software security toward the broader limits of computation itself.

7.2 IMPLICATIONS AND IMPACT

7.2.1 USENIX’24: Adversarial Testing of ML4VD

While some prior work had noted the reliance of ML4VD techniques on unrelated artifacts such as variable names [Cha+21; BV20; Li+21], the proposed remedy was adversarial training: modifying unrelated code features during training to force the model to learn causal ones. Our USENIX’24 paper challenged this assumption.

Implications for ML4VD. Our USENIX’24 findings fundamentally challenged the validity of the dominant ML4VD benchmarking paradigm. High scores on function-level vulnerability detection had widely been interpreted as evidence that models learned to recognize vulnerabilities. Our results showed that this interpretation was

unwarranted: models could achieve state-of-the-art performance while relying primarily on features unrelated to vulnerability semantics. Moreover, adversarial training against selected artifacts did not resolve this issue, since models could simply shift to other unrelated features. As a result, benchmark improvements within the prevailing paradigm could no longer be assumed to reflect actual advances in vulnerability detection capability. Beyond our specific empirical findings, the adversarial evaluation methodology we introduced has since been adopted by subsequent work [Ull+24; Din+25; Li+25b; LLM25] to test whether ML4VD models truly rely on vulnerability-relevant features.

Implications for ML Benchmarking. Our findings expose a fundamental problem in using classification for benchmarking: features that correlate with the label but are causally unrelated to the capability we intend to measure. Worse, identifying such features and forcing the model to ignore them does not push it toward causal ones. Given any other unrelated features, the model will exploit them instead and still achieve high scores. How, then, can we be confident that *any* observed score in the classification paradigm is not explained by spurious correlations? I therefore argue that classification should be avoided for benchmarking capabilities where possible. Where it remains necessary, substantial effort must go into ruling out unrelated features, for example through adversarial evaluation methodologies such as ours.

Validity of our Findings. Our USENIX’24 evaluation relies on the function-level binary classification paradigm, which we now consider fundamentally flawed. However, this does not invalidate our findings. Our methodologies measure the extent to which ML4VD models rely on features unrelated to vulnerability semantics, and such features can still be exploited even in an ill-defined task. In fact, the paradigm’s invalidity reinforces our conclusions: if vulnerability labels cannot be determined from isolated functions, then high benchmark performance can only be achieved through reliance on unrelated features. The same reasoning explains why models were unable to reliably distinguish vulnerable functions from their patches. In other words, the invalidity of the function-level binary classification paradigm does not invalidate our USENIX’24 findings, but instead explains why these results were inevitable.

7.2.2 ISSTA’25: The End of Function-Level VD

The End of Function-Level Isolation. Before 2025, function-level binary classification was the de facto standard approach to ML4VD, used by 87.7% of papers published between 2020 and 2024 (see Chapter 4). Our ISSTA’25 paper revealed that this approach is fundamentally flawed: the vulnerability of a function cannot be decided in isolation. This is not a limitation of the technique, but a consequence of the nature of vulnerabilities themselves. Vulnerability is a property of a program. A function can *make* a program vulnerable, and this can sometimes be decided from the function alone under certain assump-

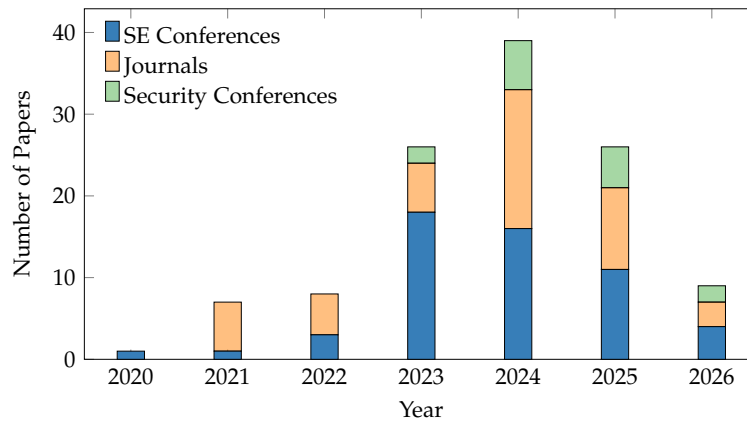


Figure 7.2: ML4VD papers per year until June 2026.

tions (e. g., that a given input is not sanitized outside of the function), but without such assumptions or knowledge about the surrounding context, the question “Is this function vulnerable?” is ill-defined. Our paper therefore marks the end of function-level binary classification as a valid evaluation paradigm for ML4VD.

Impact Analysis. To assess whether our findings actually ended function-level binary classification as the dominant benchmarking paradigm for ML4VD, I extended the literature survey presented in the ISSTA paper to the period after publication. The preprint was released on arXiv in August 2024, and the paper appeared at ISSTA in June 2025. I extended the survey to all papers published at the top software engineering conferences (IEEE/ACM ICSE, ACM FSE, ACM ISSTA, IEEE/ACM ASE), the top security conferences (IEEE S&P, NDSS, ACM CCS, USENIX Security), and the top journals (ACM TOSEM, IEEE TSE, IEEE TIFS, IEEE TDSC) between January 1, 2025 and May 20, 2026, using the same methodology as in the original survey (see Chapter 4).

The combined results (2020–2024 and 2025–2026) are shown in Figure 7.2 and Figure 7.3. Two observations stand out. First, our paper actually perfectly coincides with a turning point in ML4VD paper volume: the increasing trend of ML4VD papers from 2020–2024 reversed to a decrease in 2025. Second, the share of function-level papers dropped from 87.7% in 2020–2024 to 74.3% in 2025–2026, while alternative granularities such as repository-level (1.2% to 8.6%), file-level (4.9% to 8.6%), and program-level (3.7% to 5.7%) gained ground. The shift we hoped to initiate has thus begun, even though it may take further time to fully materialize.

Root Cause Analysis. The impact analysis shows that the decline of function-level binary classification correlates temporally with the release of our paper, but did our paper actually *cause* the change? It is inherently difficult to measure how many papers were *not* written because of our work. Moreover, there are plausible alternative explanations for the shift. For example, the decline of function-level binary classification also temporally correlates with the rise of software engi-

7.2 IMPLICATIONS AND IMPACT

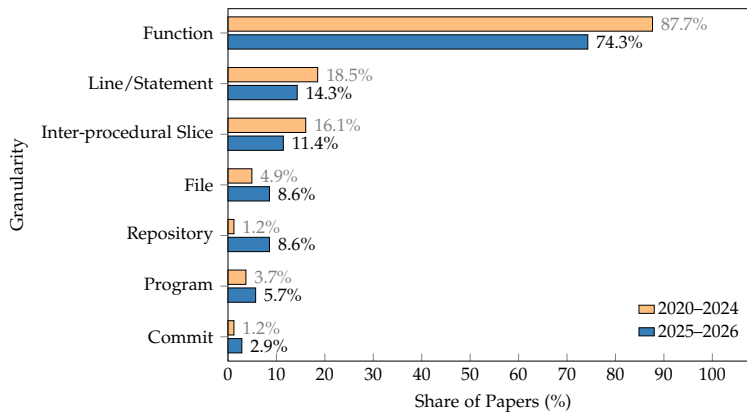


Figure 7.3: Share of ML4VD papers that use each granularity for the problem statement, comparing the period before our ISSTA’25 preprint (2020–2024) with the period after (2025–2026). Percentages are computed within each period. Papers using multiple granularities contribute to each.

neering agents. As our SZZ paper later demonstrated, agents excel at searching large codebases, making it far easier to extend them beyond function-level context compared to earlier LLMs, whose context window limitations made function-level analysis a natural choice. While we may never know the precise extent of our contribution to this development, our findings at least serve as a reference point for future researchers deciding how to approach the problem of vulnerability detection, and they motivate the alternative benchmarking paradigms I propose in Section 7.3.1.

7.2.3 NDSS’26: A Broader Critique of LLMs for Security

Any machine learning model introduces potential pitfalls to the research process that can invalidate research validity if they are not accounted for and no countermeasures are taken. In our NDSS paper, we identified nine of these pitfalls for LLMs in the security context, investigated the potential impact they might have, and provided guidance on how researchers can avoid them.

Implications for Research Validity. Our findings suggest that the LLM-for-Security field faces a validity crisis of which it is largely unaware. Our study covered every paper on LLMs for security and LLM security published at top SE and Security venues in 2023–2024, and *each one* was affected by at least one pitfall. Yet, only 15.7% of pitfalls were acknowledged, so the vast majority go unnoticed. Our impact analysis case studies also imply that the impact of being affected by such a pitfall is likely high, and can undermine the validity of a paper’s findings. The way out of this crisis is to increase awareness. Researchers in this area must become aware of these pitfalls to recognize, address, and mitigate them in the future, and to interpret the existing literature based on the presence or absence of these pitfalls. As our NDSS paper

DISCUSSION AND CONCLUSIONS

appeared only this year, however, it is still too early to tell whether this increased awareness will materialize.

7.2.4 SZZ-Agent: A New Horizon for Software Security

Agents represent the next avenue for ML-based software security. In our SZZ-Agent paper, we showed that they enable substantial progress on a difficult security task.

Implications for ML4SS. Our results suggest that external tools such as search substantially improve ML-based software security systems on tasks that require reasoning over large codebases. In our case, grep-based search enabled the agent to navigate repositories far beyond the standalone context window of an LLM, allowing it to retrieve and connect relevant code fragments across project history. This suggests that agentic workflows may be particularly valuable for repository-scale ML4SS tasks.

More broadly, our work highlights that combining LLMs with traditional software engineering tools is substantially more powerful than either component alone. The key strength of agents is that they can tackle a large and complex task by decomposing it into smaller subtasks that individual tools can solve: LLMs contribute the reasoning needed to plan this decomposition and to interpret intermediate results, while tools provide scalable retrieval, navigation, and execution capabilities. Together, they enable forms of analysis that neither standalone models nor classical pipelines can achieve as effectively. As our SZZ-Agent paper was released only this year and was only recently accepted, however, it is too early to assess its broader impact on the field.

7.3 FUTURE WORK

Based on our research contributions, I identify several directions for future work.

7.3.1 ML4VD Benchmarking

Due to the previously mentioned limitations of classification as a benchmarking paradigm for ML4VD (see Section 7.2.1), I believe the field should focus on fundamentally alternative benchmarking paradigms. Two of these paradigms seem to be particularly promising: *Bug-Triggering Test Case Generation* and *Exploit Generation*.

Bug-Triggering Test Case Generation. The first is the generation of bug-triggering test cases, also called *proofs of concept*, or PoCs, for potentially security-critical bug types. CyberGym [Wan+26b] (ICLR'26) benchmarks agents on producing such test cases for memory-safety bugs in C/C++ from textual descriptions of known vulnerabilities. Even the best agents reach only a 20% success rate, yet the study

uncovers 34 previously unknown vulnerabilities. Two limitations stand out. First, validating a PoC requires an oracle: sanitizers fill this role for memory-safety bugs, which is why CyberGym restricts itself to them, but no comparable oracle exists for other classes such as business-logic bugs. Second, the benchmark covers only known vulnerabilities, which humans have already confirmed to be exploitable; applied to new code, the approach cannot distinguish a security-critical exploitable bug from an ordinary one unless the agent also produces an exploit. These gaps motivate two questions for future work: how to verify PoCs for bug types beyond memory safety, and how to identify exploitable bugs from the bugs found.

Exploit Generation. The second paradigm is exploit generation. ExploitGym [Wan+26a], released as an arXiv preprint in May 2026, benchmarks agents on turning PoCs into full exploits. Success is verified automatically: a hidden flag is placed on the target, and an agent succeeds if it achieves remote code execution and retrieves the flag, with a second agent acting as a judge to confirm that the exploit indeed targets the intended vulnerability. Frontier agents exploit a non-trivial fraction of vulnerabilities, though far from all. Its main limitations, and corresponding avenues for future work, are a narrow range of target systems (e.g., no Windows, iOS, or Android), reliance on code execution as the sole success criterion, and the absence of ground truth, owing to the inherent difficulty of generating exploits.

Proof of Vulnerability. Together, a PoC with a working exploit is called a *proof of vulnerability*. However, should the two tasks be evaluated end-to-end, or separately, as in CyberGym and ExploitGym? Each option trades off the same two error types. End-to-end evaluation substantially reduces the number of security-irrelevant bugs found, since exploit generation against a sufficiently strong verifier is hard to shortcut; an agent that retrieves the flag has demonstrably exploited the target. The cost is more false negatives: it discards cases in which the agent produced a valid PoC for a security-relevant bug but could not weaponize it into a working exploit. Evaluating the tasks separately reverses this trade-off, admitting more false positives (i.e., security-irrelevant bugs) in exchange for fewer false negatives. Which of the two designs is preferable, and how to balance these competing error types, is therefore an important open question for future work on benchmarking ML4VD.

The Question of the Right Context Level. Our ISSTA’25 paper raised the question of which context level ML4VD benchmarking should use (e.g., function-level, file-level, or repository-level). Both CyberGym [Wan+26b] and ExploitGym [Wan+26a] address this by giving agents access to the entire repository and leaving context selection to the agent itself. This design aligns with our ISSTA’25 findings and effectively delegates the context-selection problem to the agent. However, our findings may apply even at the repository-level: just as a function cannot be judged in isolation, the context needed to decide whether a program is vulnerable may extend beyond the repository code itself for some vulnerabilities. For example, the compiler used to build the

program can determine whether a given construct is exploitable. The context should therefore not be constrained at all. The value of the repository-level is not that it bounds the context, but that it leaves the repository code unconstrained, which is the right principle. Pre-constraining the context, for example through interprocedural slicing, seems less promising: if restricting the context helps, agents could learn to do so autonomously. A more compelling direction for future work is understanding how agents can select the relevant parts from unbounded context efficiently and effectively.

7.3.2 *Agentic Software Security*

The second avenue for future work is understanding and improving *agents for software security*. Similar to 2022 when I started my PhD, we are in a phase where great results are reported, but the understanding of *how* these agents work, where their *limits* are, and whether we can trust the initial results is largely missing.

Vulnerability Detection. Recently, Anthropic reported that its agentic system Mythos discovered 23,019 vulnerabilities in open-source software, including 6,202 rated high or critical severity; partners additionally identified “more than ten thousand” vulnerabilities in proprietary software using the system [Ant26]. Mythos also achieves the best result on ExploitGym [Wan+26a] (see Section 7.3.1). These results suggest that vulnerability detection has entered a new phase in which autonomous agents can identify and exploit vulnerabilities at scale. Yet, despite these capabilities, we still understand little about how such agents reason about vulnerabilities, which classes of vulnerabilities they can reliably detect, and where their limitations lie. Developing a deeper understanding of these systems therefore represents an important direction for future work.

Vulnerability Triage. The large number of agent-generated vulnerability reports shifts the bottleneck for open-source maintainers and security teams from vulnerability detection to *vulnerability triage*: determining which reports correspond to real vulnerabilities that warrant fixes. In January 2026, the open-source project curl temporarily shut down its bug bounty program due to an overwhelming volume of low-quality AI-generated reports [Ste26c]. Although the program has since reopened without monetary rewards [Ste26b], triage remains a major challenge. This makes agentic vulnerability triage a promising direction for future work.

Program Repair. Similar to vulnerability triage, *repairing* vulnerabilities is becoming increasingly urgent for security teams. Historically, vulnerability discovery required significant expertise, limiting the number of capable attackers and defenders. Agentic systems are rapidly lowering this barrier, enabling both defenders and adversaries to identify vulnerabilities at unprecedented speed. As a result, the ability to deploy fixes quickly is becoming as critical as detection itself. Traditional disclosure windows of 90 days may no longer be appropriate in an era

of automated vulnerability discovery. Consequently, agentic program repair represents a promising direction for future research.

Program Reasoning. Another dimension of agentic software security that affects all of the previously mentioned directions for future work is the ability of agents to *reason* about program semantics. LLMs released before 2026 were shown to struggle with semantic program reasoning [LLM25; Wei+25a], consistent with our findings in USENIX’24 (see Chapter 3). However, the past six months have brought substantial advances across software security tasks, raising the possibility that agents may also have improved reasoning capabilities. Understanding how well agents can reason about programs, where their limits lie, how such reasoning can be improved, and whether stronger program reasoning can unlock progress in the previously discussed research directions therefore represents an important avenue for future work.

Secure Software Synthesis. As agents become increasingly central to software engineering, an additional important challenge is ensuring that synthesized software is secure by design, not only analyzing software for vulnerabilities or repairing them after discovery. BaxBench evaluates frontier LLMs on generating secure backend applications by challenging human security experts to develop exploits [Ver+25]. Across models, experts successfully exploited roughly half of the correctly generated applications, demonstrating that current frontier LLMs still frequently produce insecure code. This raises several important research questions closely connected to our prior work: How should we measure the security of synthesized software? Can agents make security-relevant decisions across different levels of abstraction, especially high-level choices such as system architecture and programming language, which have largely been absent from the literature? Can agents synthesize *provably* secure software with respect to certain security properties? These questions open another promising direction for future research.

Fundamental and Practical Limits. Beyond improving agents for software security tasks, it is equally important to understand their limits. Can sufficiently intelligent agents eventually discover all vulnerabilities in all software systems, or do fundamental trade-offs make this impossible? Since agents are themselves programs, they inherit the limits of computation (see Section 2.1.1). In particular, Rice’s theorem implies that no agent can perfectly decide many security-relevant properties of arbitrary programs. Whether these classical limits straightforwardly transfer to agents is, however, itself an open question: Tang and Böhme [TB25] argue that LLM-based systems perform a form of *empirical computation*, returning solutions that are empirically most likely rather than provably correct, and that the capabilities and limits of such computation cannot be fully understood within the classical, rationalist framework of computation. Characterizing the fundamental limits of agentic software security may therefore require new theoretical foundations. Yet the more practically important open question concerns the limits of intelligence in practice. As agents become increasingly capable, will software “approach zero bugs” [Ste26a], or

is the state space of modern software fundamentally too complex for even highly intelligent agents to exhaustively reason about? Understanding these theoretical and practical limits remains an important direction for future work.

7.4 CONCLUSIONS

When I started my PhD, the goal was to uncover the limits of machine learning for software security. The four contributions of this thesis each advanced that goal from a different angle.

We began by showing that the true capabilities of ML techniques for software security are only revealed when tested adversarially: high scores on benchmarks can hide a reliance on features entirely unrelated to the task at hand [RB24]. Investigating *why* these models relied on such features led to a deeper finding: vulnerability detection cannot be formulated as a function-level binary classification problem, because vulnerability is not a property that can be decided from a function in isolation [RLB25]. Stepping back, we found that the challenges are not confined to vulnerability detection. Using LLMs in security research introduces a broad range of pitfalls that, if left unchecked, can undermine the validity of research conclusions across the ML-for-security field [Eve+26]. Finally, looking ahead, we demonstrated that agents present a fundamentally more capable paradigm for ML-based software security, lifting its limits from those of individual models to the limits of computation itself [RB26].

Interestingly, today's situation mirrors what I encountered in 2022 when I began my PhD. Back then, early LLMs entered the software security field; today, it is agents. In both cases, impressive results are reported across the board, yet there is limited understanding of how these results are achieved, what the systems can truly do, where their limits lie, and which fundamental problems remain unsolved. While the field has advanced significantly, in many ways, the real work is only beginning.

APPENDIX

BIBLIOGRAPHY

-
- [Abb+76] R. P. Abbott, J. S. Chin, J. E. Donnelley, W. L. Konigsford, S. Tokubo, and D. A. Webb. *Security Analysis and Enhancements of Computer Operating Systems*. Technical report. Institute for Computer Sciences and Technology, National Bureau of Standards, 1976. DOI: [10.6028/NBS.IR.76-1041](https://doi.org/10.6028/NBS.IR.76-1041).
- [And72] James P. Anderson. *Computer Security Technology Planning Study*. Technical report. Electronic Systems Division, Air Force Systems Command, 1972. URL: <https://csrc.nist.gov/files/pubs/conference/1998/10/08/proceedings-of-the-21st-nissc-1998/final/docs/early-cs-papers/ande72a.pdf>.
- [Ant25] Anthropic. *Introducing Claude Code*. Blog post. 2025. URL: <https://www.anthropic.com/news/claude-3-7-sonnet>.
- [Ant26] Anthropic. *Project Glasswing: An initial update*. Blog post. 2026. URL: <https://www.anthropic.com/research/glasswing-initial-update>.
- [BBM96] V.R. Basili, L.C. Briand, and W.L. Melo. “A validation of object-oriented design metrics as quality indicators”. In: *IEEE Transactions on Software Engineering* (1996). DOI: [10.1109/32.544352](https://doi.org/10.1109/32.544352).
- [BCB15] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. “Neural Machine Translation by Jointly Learning to Align and Translate”. In: *arXiv preprint* (2015). DOI: [10.48550/arXiv.1409.0473](https://doi.org/10.48550/arXiv.1409.0473).
- [BCP26] Islem Bouzenia, Cristian Cadar, and Michael Pradel. “Evaluating LLM Agents on Automated Software Analysis Tasks”. In: *arXiv preprint* (2026). DOI: [10.48550/arXiv.2604.11270](https://doi.org/10.48550/arXiv.2604.11270).
- [Bis06] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006. URL: <https://www.microsoft.com/en-us/research/wp-content/uploads/2006/01/Bishop-Pattern-Recognition-and-Machine-Learning-2006.pdf>.
- [Bis18] Matt Bishop. *Computer Security: Art and Science*. Addison-Wesley, 2018. URL: <https://www.pearson.com/en-us/subject-catalog/p/computer-security-art-and-science/P200000000134/9780134097176>.
- [Bla18] Paul Black. “A Software Assurance Reference Dataset: Thousands of Programs With Known Bugs”. In: *Journal of Research of the National Institute of Standards and Technology* (2018). DOI: [10.6028/jres.123.005](https://doi.org/10.6028/jres.123.005).

BIBLIOGRAPHY

- [Böh24] Marcel Böhme. “Fundamental Challenges in Cybersecurity and a Philosophy of Vulnerability-Guided Hardening”. In: *arXiv preprint* (2024). DOI: [10.48550/arXiv.2402.01944](https://doi.org/10.48550/arXiv.2402.01944).
- [Bro+20] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. “Language Models are Few-Shot Learners”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2020. URL: <https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html>.
- [BRW13] Harold Booth, Doug Rike, and Gregory Witte. *The National Vulnerability Database (NVD): Overview*. Technical report. National Institute of Standards and Technology, 2013. URL: https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=915172.
- [BV20] Pavol Bielik and Martin Vechev. “Adversarial Robustness for Code”. In: *Proceedings of the International Conference on Machine Learning (ICML)*. 2020. URL: <http://proceedings.mlr.press/v119/bielik20a.html>.
- [Cas+23] Stephen Casper, Xander Davies, Claudia Shi, Thomas Krendl Gilbert, et al. “Open Problems and Fundamental Limitations of Reinforcement Learning from Human Feedback”. In: *arXiv preprint* (2023). DOI: [10.48550/arXiv.2307.15217](https://doi.org/10.48550/arXiv.2307.15217).
- [CBK23] Roland Croft, M. Ali Babar, and M. Mehdi Kholoosi. “Data Quality for Software Vulnerability Datasets”. In: *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE)*. 2023. DOI: [10.1109/ICSE48619.2023.00022](https://doi.org/10.1109/ICSE48619.2023.00022).
- [Cha+02] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. “SMOTE: synthetic minority over-sampling technique”. In: *Journal of Artificial Intelligence Research* (2002). DOI: [10.1613/jair.953](https://doi.org/10.1613/jair.953).
- [Cha+21] Saikat Chakraborty, Rahul Krishna, Yangruibo Ding, and Baishakhi Ray. “Deep Learning based Vulnerability Detection: Are We There Yet?”. In: *IEEE Transactions on Software Engineering* (2021). DOI: [10.1109/TSE.2021.3087402](https://doi.org/10.1109/TSE.2021.3087402).

- [Che+21] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. “Evaluating Large Language Models Trained on Code”. In: *arXiv preprint* (2021). DOI: [10.48550/arXiv.2107.03374](https://doi.org/10.48550/arXiv.2107.03374).
- [Che+23] Yizheng Chen, Zhoujie Ding, Lamya Alowain, Xinyun Chen, and David Wagner. “DiverseVul: A New Vulnerable Source Code Dataset for Deep Learning Based Vulnerability Detection”. In: *Proceedings of the International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*. 2023. DOI: [10.1145/3607199.3607242](https://doi.org/10.1145/3607199.3607242).
- [Cho19] François Chollet. “On the Measure of Intelligence”. In: *arXiv preprint* (2019). DOI: [10.48550/arXiv.1911.01547](https://doi.org/10.48550/arXiv.1911.01547).
- [Chr+17] Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. “Deep Reinforcement Learning from Human Preferences”. In: *Advances in Neural Information Processing Systems (NIPS)*. 2017. URL: <https://proceedings.neurips.cc/paper/2017/hash/d5e2c0adad503c91f91df240d0cd4e49-Abstract.html>.
- [Chu36] Alonzo Church. “An Unsolvable Problem of Elementary Number Theory”. In: *American Journal of Mathematics* (1936). DOI: [10.2307/2371045](https://doi.org/10.2307/2371045).
- [Cir+10] Dan Claudiu Ciresan, Ueli Meier, Luca Maria Gambardella, and Jürgen Schmidhuber. “Deep, Big, Simple Neural Nets for Handwritten Digit Recognition”. In: *Neural Computation* (2010). DOI: [10.1162/neco_a_00052](https://doi.org/10.1162/neco_a_00052).
- [Cyb89] George Cybenko. “Approximation by Superpositions of a Sigmoidal Function”. In: *Mathematics of Control, Signals, and Systems* (1989). DOI: [10.1007/bf02551274](https://doi.org/10.1007/bf02551274).
- [CZL23] Anton Cheshkov, Pavel Zadorozhny, and Rodion Levichev. “Evaluation of ChatGPT Model for Vulnerability Detection”. In: *arXiv preprint* (2023). DOI: [10.48550/arXiv.2304.07232](https://doi.org/10.48550/arXiv.2304.07232).

BIBLIOGRAPHY

- [Den+25] Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Vijay Bharadwaj, Jeff Holm, Raja Aluri, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. “SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks?” In: *arXiv preprint* (2025). DOI: [10.48550/arXiv.2509.16941](https://doi.org/10.48550/arXiv.2509.16941).
- [Dev+19] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*. 2019. DOI: [10.18653/v1/N19-1423](https://doi.org/10.18653/v1/N19-1423).
- [Din+25] Yangruibo Ding, Yanjun Fu, Omniyyah Ibrahim, Chawin Sitawarin, Xinyun Chen, Basel Alomair, David Wagner, Baishakhi Ray, and Yizheng Chen. “Vulnerability Detection with Code Language Models: How Far are We?” In: *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE)*. 2025. DOI: [10.1109/icse55347.2025.00038](https://doi.org/10.1109/icse55347.2025.00038).
- [Dwo+15] Cynthia Dwork, Vitaly Feldman, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Aaron Roth. “The Reusable Holdout: Preserving Validity in Adaptive Data Analysis”. In: *Science* (2015). DOI: [10.1126/science.aaa9375](https://doi.org/10.1126/science.aaa9375).
- [Eve+26] Jonathan Evertz*, Niklas Risse*, Nicolai Neuer, Andreas Müller, Philipp Normann, Gaetano Sapia, Srishti Gupta, David Pape, Soumya Shaw, Devansh Srivastav, Christian Wressnegger, Erwin Quiring, Thorsten Eisenhofer, Daniel Arp, and Lea Schönherr. “Chasing Shadows: Pitfalls in LLM Security Research”. In: *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. * **Equal Contribution**. 2026. DOI: [10.14722/ndss.2026.241749](https://doi.org/10.14722/ndss.2026.241749).
- [Fan+20] Jiahao Fan, Yi Li, Shaohua Wang, and Tien N. Nguyen. “A C/C++ Code Vulnerability Dataset with Code Changes and CVE Summaries”. In: *Proceedings of the 17th IEEE/ACM International Conference on Mining Software Repositories (MSR)*. 2020. DOI: [10.1145/3379597.3387501](https://doi.org/10.1145/3379597.3387501).
- [Fen+20] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. “CodeBERT: A Pre-Trained Model for Programming and Natural Languages”. In: *Findings of the Association for Computational Linguistics: EMNLP (EMNLP)*. 2020. DOI: [10.18653/v1/2020.findings-emnlp.139](https://doi.org/10.18653/v1/2020.findings-emnlp.139).

- [FT22] Michael Fu and Chakkrit Tantithamthavorn. “LineVul: A Transformer-based Line-Level Vulnerability Prediction”. In: *Proceedings of the 19th IEEE/ACM International Conference on Mining Software Repositories (MSR)*. 2022. DOI: [10.1145/3524842.3528452](https://doi.org/10.1145/3524842.3528452).
- [GBC16] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. URL: <http://www.deeplearningbook.org/>.
- [Göd31] Kurt Gödel. “Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I”. In: *Monatshefte für Mathematik und Physik* (1931). DOI: [10.1007/BF01700692](https://doi.org/10.1007/BF01700692).
- [Gri+16] Gustavo Grieco, Guillermo Luis Grinblat, Lucas Uzal, Sanjay Rawat, Josselin Feist, and Laurent Mounier. “Toward Large-Scale Vulnerability Discovery using Machine Learning”. In: *Proceedings of the ACM Conference on Data and Application Security and Privacy (CODASPY)*. 2016. DOI: [10.1145/2857705.2857720](https://doi.org/10.1145/2857705.2857720).
- [GSS14] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. “Explaining and Harnessing Adversarial Examples”. In: *arXiv preprint* (2014). DOI: [10.48550/arXiv.1412.6572](https://doi.org/10.48550/arXiv.1412.6572).
- [Guo+17] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. “On Calibration of Modern Neural Networks”. In: *Proceedings of the International Conference on Machine Learning (ICML)*. 2017. URL: <http://proceedings.mlr.press/v70/guo17a.html>.
- [Guo+22] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. “UniXcoder: Unified Cross-Modal Pre-training for Code Representation”. In: *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*. 2022. DOI: [10.18653/v1/2022.acl-long.499](https://doi.org/10.18653/v1/2022.acl-long.499).
- [Guo+25] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, et al. “DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning”. In: *Nature* (2025). DOI: [10.1038/s41586-025-09422-z](https://doi.org/10.1038/s41586-025-09422-z).
- [GV24] Tobias Groot and Matias Valdenegro-Toro. “Overconfidence is Key: Verbalized Uncertainty Evaluation in Large Language and Vision-Language Models”. In: *Proceedings of the Workshop on Trustworthy Natural Language Processing (TrustNLP)*. 2024. DOI: [10.18653/v1/2024.trustnlp-1.13](https://doi.org/10.18653/v1/2024.trustnlp-1.13).

BIBLIOGRAPHY

- [HM22] Hazim Hanif and Sergio Maffei. “VulBERTa: Simplified Source Code Pre-Training for Vulnerability Detection”. In: *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*. 2022. DOI: [10.1109/IJCNN55064.2022.9892280](https://doi.org/10.1109/IJCNN55064.2022.9892280).
- [Hov+12] Aram Hovsepyan, Riccardo Scandariato, Wouter Joosen, and James Walden. “Software vulnerability prediction using text analysis techniques”. In: *Proceedings of the International Workshop on Security Measurements and Metrics (MetriSec)*. 2012. DOI: [10.1145/2372225.2372230](https://doi.org/10.1145/2372225.2372230).
- [HS97] Sepp Hochreiter and Jürgen Schmidhuber. “Long Short-Term Memory”. In: *Neural Computation* (1997). DOI: [10.1162/neco.1997.9.8.1735](https://doi.org/10.1162/neco.1997.9.8.1735).
- [IEE90] IEEE. *IEEE Standard Glossary of Software Engineering Terminology*. Technical report. Institute of Electrical and Electronics Engineers, 1990. DOI: [10.1109/IEEESTD.1990.101064](https://doi.org/10.1109/IEEESTD.1990.101064).
- [Ji+23] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. “Survey of Hallucination in Natural Language Generation”. In: *ACM Computing Surveys* (2023). DOI: [10.1145/3571730](https://doi.org/10.1145/3571730).
- [Jim+24] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. “SWE-bench: Can Language Models Resolve Real-world Github Issues?” In: *Proceedings of the International Conference on Learning Representations (ICLR)*. 2024. URL: <https://openreview.net/forum?id=VTF8yNQm66>.
- [Kap+20] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. “Scaling Laws for Neural Language Models”. In: *arXiv preprint* (2020). DOI: [10.48550/arXiv.2001.08361](https://doi.org/10.48550/arXiv.2001.08361).
- [KPM92] T.M. Khoshgoftaar, A.S. Pandya, and H.B. More. “A neural network approach for predicting software development faults”. In: *Proceedings of the IEEE International Symposium on Software Reliability Engineering (ISSRE)*. 1992. DOI: [10.1109/ISSRE.1992.285855](https://doi.org/10.1109/ISSRE.1992.285855).
- [KS74] Paul A. Karger and Roger R. Schell. *Multics Security Evaluation: Vulnerability Analysis*. Technical report. HQ Electronic Systems Division, Air Force Systems Command, 1974. URL: <https://csrc.nist.gov/publications/history/karg74.pdf>.

- [KSH12] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. “ImageNet Classification with Deep Convolutional Neural Networks”. In: *Advances in Neural Information Processing Systems (NIPS)*. 2012. URL: <https://proceedings.neurips.cc/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html>.
- [Lam74] Butler W. Lampson. “Protection”. In: *ACM SIGOPS Operating Systems Review* (1974). URL: <https://dl.acm.org/doi/10.1145/775265.775268>.
- [Lan92] William Landi. “Undecidability of Static Analysis”. In: *ACM Letters on Programming Languages and Systems* (1992). DOI: [10.1145/161494.161501](https://doi.org/10.1145/161494.161501).
- [Lan95] William Landi. “Interprocedural Aliasing in the Presence of Pointers”. PhD thesis. 1995. DOI: [10.7282/t3-fyr2-a533](https://doi.org/10.7282/t3-fyr2-a533).
- [LeC+98] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. “Gradient-Based Learning Applied to Document Recognition”. In: *Proceedings of the IEEE* (1998). DOI: [10.1109/5.726791](https://doi.org/10.1109/5.726791).
- [Lew+20] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2020. URL: <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>.
- [LF03] Alexander Linden and Jackie Fenn. *Understanding Gartner’s Hype Cycles*. Technical report. Gartner, 2003.
- [LH07] Shane Legg and Marcus Hutter. “A Collection of Definitions of Intelligence”. In: *Proceedings of the Conference on Artificial General Intelligence (AGI)*. 2007. URL: <https://ebooks.iospress.nl/volumearticle/3471>.
- [Li+16] Yujia Li, Daniel Tarlow, Marc Brockschmidt, and Richard Zemel. “Gated Graph Sequence Neural Networks”. In: *arXiv preprint* (2016). DOI: [10.48550/arXiv.1511.05493](https://doi.org/10.48550/arXiv.1511.05493).
- [Li+18] Zhen Li, Deqing Zou, Shouhuai Xu, Xinyu Ou, Hai Jin, Sujuan Wang, Zhijun Deng, and Yuyi Zhong. “VulDeepP-ecker: A Deep Learning-Based System for Vulnerability Detection”. In: *Proceedings of the 25th Annual Network and Distributed System Security Symposium (NDSS)*. 2018. DOI: [10.14722/ndss.2018.23158](https://doi.org/10.14722/ndss.2018.23158).
- [Li+21] Zhen Li, Jing Tang, Deqing Zou, Qian Chen, Shouhuai Xu, Chao Zhang, Yichen Li, and Hai Jin. “Towards Making Deep Learning-based Vulnerability Detectors Robust”. In: *arXiv preprint* (2021). DOI: [10.48550/arXiv.2108.00669](https://doi.org/10.48550/arXiv.2108.00669).

BIBLIOGRAPHY

- [Li+24] Tianle Li, Ge Zhang, Quy Duc Do, Xiang Yue, and Wenhua Chen. “Long-context LLMs Struggle with Long In-context Learning”. In: *arXiv preprint* (2024). DOI: [10.48550/arXiv.2404.02060](https://doi.org/10.48550/arXiv.2404.02060).
- [Li+25a] Jia Li, Xuyuan Guo, Lei Li, Kechi Zhang, Ge Li, Jia Li, Zhengwei Tao, Fang Liu, Chongyang Tao, Yuqi Zhu, and Zhi Jin. “Longcodeu: Benchmarking long-context language models on long code understanding”. In: *arXiv preprint* (2025). DOI: [10.48550/arXiv.2503.04359](https://doi.org/10.48550/arXiv.2503.04359).
- [Li+25b] Youpeng Li, Weiliang Qi, Xuyu Wang, Fuxun Yu, and Xinda Wang. “Revisiting Pre-trained Language Models for Vulnerability Detection”. In: *arXiv preprint* (2025). DOI: [10.48550/arXiv.2507.16887](https://doi.org/10.48550/arXiv.2507.16887).
- [Lip18] Zachary C. Lipton. “The mythos of model interpretability”. In: *Communications of the ACM* (2018). DOI: [10.1145/3233231](https://doi.org/10.1145/3233231).
- [Liu+19] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. “RoBERTa: A Robustly Optimized BERT Pretraining Approach”. In: *arXiv preprint* (2019). DOI: [10.48550/arXiv.1907.11692](https://doi.org/10.48550/arXiv.1907.11692).
- [Liu+24] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. “Lost in the Middle: How Language Models Use Long Contexts”. In: *Transactions of the Association for Computational Linguistics* (2024). DOI: [10.1162/tacl_a_00638](https://doi.org/10.1162/tacl_a_00638).
- [LLM25] Thanh Le-Cong, Bach Le, and Toby Murray. “Can LLMs Reason About Program Semantics? A Comprehensive Evaluation of LLMs on Formal Specification Inference”. In: *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*. 2025. DOI: [10.18653/v1/2025.acl-long.1068](https://doi.org/10.18653/v1/2025.acl-long.1068).
- [Lu+21] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, Ge Li, Lidong Zhou, Linjun Shou, Long Zhou, Michele Tufano, Ming Gong, Ming Zhou, Nan Duan, Neel Sundaresan, Shao Kun Deng, Shengyu Fu, and Shujie Liu. “CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation”. In: *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS Datasets and Benchmarks)*. 2021. URL: <https://openreview.net/forum?id=61E4dQXaUcb>.
- [Luo+25] Yun Luo, Zhen Yang, Fandong Meng, Yafu Li, Jie Zhou, and Yue Zhang. “An Empirical Study of Catastrophic Forgetting in Large Language Models During Continual Fine-Tuning”. In: *IEEE Transactions on Audio, Speech and*

- Language Processing* (2025). DOI: [10.1109/TASLPRO.2025.3606231](https://doi.org/10.1109/TASLPRO.2025.3606231).
- [Mao+24] Zhenyy Mao, Jialong Li, Dongming Jin, Munan Li, and Kenji Tei. “Multi-Role Consensus Through LLMs Discussions for Vulnerability Detection”. In: *Proceedings of the IEEE International Conference on Software Quality, Reliability, and Security Companion (QRS-C)*. 2024. DOI: [10.1109/QRS-C63300.2024.00173](https://doi.org/10.1109/QRS-C63300.2024.00173).
- [Mao+25] Qiheng Mao, Zhenhao Li, Xing Hu, Kui Liu, Xin Xia, and Jianling Sun. “Towards Explainable Vulnerability Detection With Large Language Models”. In: *IEEE Transactions on Software Engineering* (2025). DOI: [10.1109/TSE.2025.3605442](https://doi.org/10.1109/TSE.2025.3605442).
- [McC+06] John McCarthy, Marvin L. Minsky, Nathaniel Rochester, and Claude E. Shannon. “A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence: August 31, 1955”. In: *AI Magazine* (2006). URL: <http://jmc.stanford.edu/articles/dartmouth/dartmouth.pdf>.
- [McC07] John McCarthy. *What is Artificial Intelligence?* Technical report. Stanford University, Computer Science Department, 2007. URL: <https://www-formal.stanford.edu/jmc/whatisai.pdf>.
- [McG04] Gary McGraw. “Software Security”. In: *IEEE Security & Privacy* (2004). DOI: [10.1109/MSECP.2004.1281254](https://doi.org/10.1109/MSECP.2004.1281254).
- [Mit97] Tom M. Mitchell. *Machine Learning*. McGraw-Hill, 1997. URL: <https://www.cs.cmu.edu/~tom/book.html>.
- [MP43] Warren S. McCulloch and Walter Pitts. “A Logical Calculus of the Ideas Immanent in Nervous Activity”. In: *The Bulletin of Mathematical Biophysics* (1943). DOI: [10.1007/BF02478259](https://doi.org/10.1007/BF02478259).
- [NDP17] Michael Nieves, Kelley Dempsey, and Victoria Yan Pilleri. *An Introduction to Information Security*. Technical report. National Institute of Standards and Technology, 2017. DOI: [10.6028/NIST.SP.800-12r1](https://doi.org/10.6028/NIST.SP.800-12r1).
- [Neu+07] Stephan Neuhaus, Thomas Zimmermann, Christian Holler, and Andreas Zeller. “Predicting Vulnerable Software Components”. In: *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2007. DOI: [10.1145/1315245.1315311](https://doi.org/10.1145/1315245.1315311).
- [Neu45] John von Neumann. *First Draft of a Report on the EDVAC*. Technical report. Moore School of Electrical Engineering, University of Pennsylvania, 1945. DOI: [10.5479/sil.538961.39088011475779](https://doi.org/10.5479/sil.538961.39088011475779).

BIBLIOGRAPHY

- [Non+23] Yu Nong, Yuzhe Ou, Michael Pradel, Feng Chen, and Haipeng Cai. “VulGen: Realistic Vulnerability Generation Via Pattern Mining and Deep Learning”. In: *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE)*. 2023. DOI: [10.1109/ICSE48619.2023.00211](https://doi.org/10.1109/ICSE48619.2023.00211).
- [Nor+26] Philipp Normann, Andreas Happe, Jürgen Cito, and Daniel Arp. “Post-Training Local LLM Agents for Linux Privilege Escalation with Verifiable Rewards”. In: *arXiv preprint* (2026). DOI: [10.48550/arXiv.2603.17673](https://doi.org/10.48550/arXiv.2603.17673).
- [Nun+24] Ana Nunez, Nafis Tanveer Islam, Sumit Kumar Jha, and Peyman Najafirad. “AutoSafeCoder: A Multi-Agent Framework for Securing LLM Code Generation through Static Analysis and Fuzz Testing”. In: *arXiv preprint* (2024). DOI: [10.48550/arXiv.2409.10737](https://doi.org/10.48550/arXiv.2409.10737).
- [OA96] N. Ohlsson and H. Alberg. “Predicting fault-prone software modules in telephone switches”. In: *IEEE Transactions on Software Engineering* (1996). DOI: [10.1109/32.553637](https://doi.org/10.1109/32.553637).
- [Ope23] OpenAI. *Function Calling and Other API Updates*. Blog post. 2023. URL: <https://openai.com/index/function-calling-and-other-api-updates/>.
- [Ope24] OpenAI. *Learning to Reason with LLMs*. Blog post. 2024. URL: <https://openai.com/index/learning-to-reason-with-llms/>.
- [Pha+21] Long Phan, Hieu Tran, Daniel Le, Hieu Nguyen, James Annibal, Alec Peltekian, and Yanfang Ye. “CoTexT: Multi-task Learning with Code-Text Transformer”. In: *Proceedings of the Workshop on Natural Language Processing for Programming (NLP4Prog)*. 2021. DOI: [10.18653/v1/2021.nlp4prog-1.5](https://doi.org/10.18653/v1/2021.nlp4prog-1.5).
- [PS18] Michael Pradel and Koushik Sen. “DeepBugs: A Learning Approach to Name-Based Bug Detection”. In: *Proceedings of the ACM on Programming Languages* 2.OOPSLA (2018). DOI: [10.1145/3276517](https://doi.org/10.1145/3276517).
- [PXW17] Yulei Pang, Xiaozhen Xue, and Huaying Wang. “Predicting Vulnerable Software Components through Deep Neural Network”. In: *Proceedings of the International Conference on Deep Learning Technologies (ICDLT)*. 2017. DOI: [10.1145/3094243.3094245](https://doi.org/10.1145/3094243.3094245).
- [Rad+18] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. *Improving Language Understanding by Generative Pre-Training*. Technical report. OpenAI, 2018. URL: https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf.

- [Rad+19] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. *Language Models are Unsupervised Multitask Learners*. Technical report. OpenAI, 2019. URL: [https://cdn.openai.com/better-language-models / language _ models _ are _ unsupervised _ multitask_learners.pdf](https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf).
- [RB24] Niklas Risse and Marcel Böhme. “Uncovering the Limits of Machine Learning for Automatic Vulnerability Detection”. In: *Proceedings of the 33rd USENIX Security Symposium (USENIX Security)*. 2024. URL: <https://www.usenix.org/system/files/usenixsecurity24-risse.pdf>.
- [RB26] Niklas Risse and Marcel Böhme. “How and Why Agents Can Identify Bug-Introducing Commits”. In: *arXiv preprint* (2026). **Accepted at the 41st IEEE/ACM International Conference on Automated Software Engineering 2026 (ASE’26)**. DOI: [10.48550/arXiv.2603.29378](https://doi.org/10.48550/arXiv.2603.29378).
- [Rec+19] Benjamin Recht, Rebecca Roelofs, Ludwig Schmidt, and Vaishaal Shankar. “Do ImageNet Classifiers Generalize to ImageNet?” In: *Proceedings of the International Conference on Machine Learning (ICML)*. 2019. URL: <http://proceedings.mlr.press/v97/recht19a.html>.
- [RGG21] Niklas Risse, Christina Göpfert, and Jan Philip Göpfert. “How to Compare Adversarial Robustness of Classifiers from a Global Perspective”. In: *Proceedings of the International Conference on Artificial Neural Networks (ICANN)*. 2021. DOI: [10.1007/978-3-030-86362-3_3](https://doi.org/10.1007/978-3-030-86362-3_3).
- [RHW86] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. “Learning Representations by Back-propagating Errors”. In: *Nature* (1986). DOI: [10.1038/323533a0](https://doi.org/10.1038/323533a0).
- [Ric53] Henry Gordon Rice. “Classes of Recursively Enumerable Sets and Their Decision Problems”. In: *Transactions of the American Mathematical Society* (1953). DOI: [10.1090/S0002-9947-1953-0053041-6](https://doi.org/10.1090/S0002-9947-1953-0053041-6).
- [Ris21] Niklas Risse. “Interpretability of Learned Models for User Activity Classification”. MA thesis. 2021.
- [RLB25] Niklas Risse, Jing Liu, and Marcel Böhme. “Top Score on the Wrong Exam: On Benchmarking in Machine Learning for Vulnerability Detection”. In: *Proceedings of the ACM on Software Engineering (ISSTA 2025)*. **ACM SIGSOFT Distinguished Paper Award**. DOI: [10.1145/3728887](https://doi.org/10.1145/3728887).
- [RMN09] Rajat Raina, Anand Madhavan, and Andrew Y. Ng. “Large-Scale Deep Unsupervised Learning Using Graphics Processors”. In: *Proceedings of the International Conference on Machine Learning (ICML)*. 2009. DOI: [10.1145/1553374.1553486](https://doi.org/10.1145/1553374.1553486).

BIBLIOGRAPHY

- [RN21] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 2021. URL: <https://www.pearson.com/us/higher-education/product/redirected-product/9780134610993.html>.
- [Roj98] R. Rojas. “How to make Zuse’s Z3 a universal computer”. In: *IEEE Annals of the History of Computing* (1998). DOI: [10.1109/85.707574](https://doi.org/10.1109/85.707574).
- [RSG16] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. ““Why Should I Trust You?”: Explaining the Predictions of Any Classifier”. In: *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. 2016. DOI: [10.1145/2939672.2939778](https://doi.org/10.1145/2939672.2939778).
- [Rud19] Cynthia Rudin. “Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead”. In: *Nature Machine Intelligence* (2019). DOI: [10.1038/s42256-019-0048-x](https://doi.org/10.1038/s42256-019-0048-x).
- [Rus+18] Rebecca L. Russell, Louis Y. Kim, Lei H. Hamilton, Tomo Lazovich, Jacob A. Harer, Onur Ozdemir, Paul M. Ellingwood, and Marc W. McConley. “Automated Vulnerability Detection in Source Code Using Deep Representation Learning”. In: *Proceedings of the 17th IEEE International Conference on Machine Learning and Applications (ICMLA)*. 2018. DOI: [10.1109/ICMLA.2018.00120](https://doi.org/10.1109/ICMLA.2018.00120).
- [Sam59] Arthur L. Samuel. “Some Studies in Machine Learning Using the Game of Checkers”. In: *IBM Journal of Research and Development* (1959). DOI: [10.1147/rd.33.0210](https://doi.org/10.1147/rd.33.0210).
- [SB18] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2018. URL: <http://incompleteideas.net/book/the-book-2nd.html>.
- [Sch+23] Timo Schick, Jane Dwivedi-Yu, Roberto Dessí, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. “Toolformer: language models can teach themselves to use tools”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2023. URL: http://papers.nips.cc/paper_files/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html.
- [Sch91] Jürgen Schmidhuber. “Learning Unambiguous Reduced Sequence Descriptions”. In: *Advances in Neural Information Processing Systems (NIPS)*. 1991. URL: <https://proceedings.neurips.cc/paper/1991/hash/2bb232c0b13c774965ef8558f0fbd615-Abstract.html>.

- [Ser+12] Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitry Vyukov. “AddressSanitizer: A Fast Address Sanity Checker”. In: *Proceedings of the USENIX Annual Technical Conference (USENIX ATC)*. 2012. URL: <https://www.usenix.org/conference/atc12/technical-sessions/presentation/serebryany>.
- [Sha+24] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y.K. Li, Y. Wu, and Daya Guo. “DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models”. In: *arXiv preprint* (2024). DOI: [10.48550/arXiv.2402.03300](https://doi.org/10.48550/arXiv.2402.03300).
- [Shi07] Robert W. Shirey. *Internet Security Glossary, Version 2*. Technical report. Internet Engineering Task Force (IETF), 2007. DOI: [10.17487/RFC4949](https://doi.org/10.17487/RFC4949).
- [Shu+24] Ilia Shumailov, Zakhar Shumaylov, Yiren Zhao, Nicolas Papernot, Ross Anderson, and Yarin Gal. “AI models collapse when trained on recursively generated data”. In: *Nature* (2024). DOI: [10.1038/s41586-024-07566-y](https://doi.org/10.1038/s41586-024-07566-y).
- [Sip13] Michael Sipser. *Introduction to the Theory of Computation*. Cengage Learning, 2013. URL: <https://www.cengage.com/c/student/9781133187790/>.
- [Sne+24] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. “Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters”. In: *arXiv preprint* (2024). DOI: [10.48550/arXiv.2408.03314](https://doi.org/10.48550/arXiv.2408.03314).
- [SP88] Richard W. Selby and Adam A. Porter. “Learning from Examples: Generation and Evaluation of Decision Trees for Software Resource Analysis”. In: *IEEE Transactions on Software Engineering* (1988). DOI: [10.1109/32.9061](https://doi.org/10.1109/32.9061).
- [Ste00] Robert J. Sternberg. *Handbook of Intelligence*. Cambridge University Press, 2000. DOI: [10.1017/CB09780511807947](https://doi.org/10.1017/CB09780511807947).
- [Ste26a] Daniel Sternberg. *APPROACHING ZERO BUGS*. Blog post. 2026. URL: <https://daniel.haxx.se/blog/2026/04/30/approaching-zero-bugs/>.
- [Ste26b] Daniel Sternberg. *CURL SECURITY MOVES AGAIN*. Blog post. 2026. URL: <https://daniel.haxx.se/blog/2026/02/25/curl-security-moves-again/>.
- [Ste26c] Daniel Sternberg. *THE END OF THE CURL BUG-BOUNTY*. Blog post. 2026. URL: <https://daniel.haxx.se/blog/2026/01/26/the-end-of-the-curl-bug-bounty/>.

BIBLIOGRAPHY

- [SW08] Yonghee Shin and Laurie Williams. “An empirical model to predict security vulnerabilities using code complexity metrics”. In: *Proceedings of the Second ACM-IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. 2008. DOI: [10.1145/1414004.1414065](https://doi.org/10.1145/1414004.1414065).
- [ŚZZ05] Jacek Śliwerski, Thomas Zimmermann, and Andreas Zeller. “When do changes induce fixes?”. In: *ACM SIGSOFT Software Engineering Notes* (2005). DOI: [10.1145/1082983.1083147](https://doi.org/10.1145/1082983.1083147).
- [Tan+23] Lingxiao Tang, Lingfeng Bao, Xin Xia, and Zhongdong Huang. “Neural SZZ Algorithm”. In: *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2023. DOI: [10.1109/ASE56229.2023.00037](https://doi.org/10.1109/ASE56229.2023.00037).
- [Tan+25] Lingxiao Tang, Jiakun Liu, Zhongxin Liu, Xiaohu Yang, and Lingfeng Bao. “LLM4SZZ: Enhancing SZZ Algorithm with Context-Enhanced Assessment on Large Language Models”. In: *Proceedings of the ACM on Software Engineering* (2025). DOI: [10.1145/3728885](https://doi.org/10.1145/3728885).
- [TB25] Eric Tang and Marcel Böhme. “Empirical Computation”. In: *arXiv preprint* (2025). DOI: [10.48550/arXiv.2503.10954](https://doi.org/10.48550/arXiv.2503.10954).
- [Tur+23] Miles Turpin, Julian Michael, Ethan Perez, and Samuel Bowman. “Language Models Don’t Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2023. URL: http://papers.nips.cc/paper_files/paper/2023/hash/ed3fea9033a80fea1376299fa7863f4a-Abstract-Conference.html.
- [Tur37] Alan M. Turing. “On Computable Numbers, with an Application to the Entscheidungsproblem”. In: *Proceedings of the London Mathematical Society* (1937). DOI: [10.1112/plms/s2-42.1.230](https://doi.org/10.1112/plms/s2-42.1.230).
- [Ull+24] Saad Ullah, Mingji Han, Saurabh Pujar, Hammond Pearce, Ayse Coskun, and Gianluca Stringhini. “LLMs Cannot Reliably Identify and Reason About Security Vulnerabilities (Yet?): A Comprehensive Evaluation, Framework, and Benchmarks”. In: *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*. 2024. DOI: [10.1109/SP54263.2024.00210](https://doi.org/10.1109/SP54263.2024.00210).
- [Vap95] Vladimir Naumovich Vapnik. *The Nature of Statistical Learning Theory*. Springer Verlag, 1995. DOI: [10.1007/978-1-4757-3264-1](https://doi.org/10.1007/978-1-4757-3264-1).

- [Vas+17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. “Attention is All You Need”. In: *Advances in Neural Information Processing Systems (NIPS)*. 2017. URL: <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>.
- [Ver+25] Mark Vero, Niels Mündler, Victor Chibotaru, Veselin Raychev, Maximilian Baader, Nikola Jovanović, Jingxuan He, and Martin Vechev. “BaxBench: Can LLMs Generate Correct and Secure Backends?” In: *Proceedings of the International Conference on Machine Learning (ICML)*. 2025. URL: <https://proceedings.mlr.press/v267/vero25a.html>.
- [VM01] John Viega and Gary McGraw. *Building Secure Software: How to Avoid Security Problems the Right Way*. Addison-Wesley, 2001. URL: https://openlibrary.org/books/OL18218463M/Building_secure_software.
- [Wan+25] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muenighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. “OpenHands: An Open Platform for AI Software Developers as Generalist Agents”. In: *arXiv preprint* (2025). DOI: [10.48550/arXiv.2407.16741](https://doi.org/10.48550/arXiv.2407.16741).
- [Wan+26a] Zhun Wang, Nico Schiller, Hongwei Li, Srijiith Sesha Narayana, Milad Nasr, Nicholas Carlini, Xiangyu Qi, Eric Wallace, Elie Bursztein, Luca Invernizzi, Kurt Thomas, Yan Shoshitaishvili, Wenbo Guo, Jingxuan He, Thorsten Holz, and Dawn Song. “ExploitGym: Can AI Agents Turn Security Vulnerabilities into Real Attacks?” In: *arXiv preprint* (2026). DOI: [10.48550/arXiv.2605.11086](https://doi.org/10.48550/arXiv.2605.11086).
- [Wan+26b] Zhun Wang, Tianneng Shi, Jingxuan He, Matthew Cai, Jialin Zhang, and Dawn Song. “CyberGym: Evaluating AI Agents’ Real-World Cybersecurity Capabilities at Scale”. In: *Proceedings of the International Conference on Learning Representations (ICLR)*. 2026. URL: <https://openreview.net/forum?id=2YvbLQEdYt>.
- [War70] Willis H. Ware. *Security Controls for Computer Systems: Report of Defense Science Board Task Force on Computer Security*. Technical report. The RAND Corporation, 1970. URL: <https://www.rand.org/pubs/reports/R609-1.html>.
- [Wei+22] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models”. In: *Advances in Neural*

BIBLIOGRAPHY

- Information Processing Systems (NeurIPS)*. 2022. URL: http://papers.nips.cc/paper_files/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html.
- [Wei+25a] Anjiang Wei, Jiannan Cao, Ran Li, Hongyu Chen, Yuhui Zhang, Ziheng Wang, Yuan Liu, Thiago S. F. X. Teixeira, Diyi Yang, Ke Wang, and Alex Aiken. “EquiBench: Benchmarking Large Language Models’ Reasoning about Program Semantics via Equivalence Checking”. In: *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2025. DOI: [10.18653/v1/2025.emnlp-main.1718](https://doi.org/10.18653/v1/2025.emnlp-main.1718).
- [Wei+25b] Yuxiang Wei, Olivier Duchenne, Jade Copet, Quentin Carbonneaux, Lingming Zhang, Daniel Fried, Gabriel Synnaeve, Rishabh Singh, and Sida I. Wang. “SWE-RL: Advancing LLM Reasoning via Reinforcement Learning on Open Software Evolution”. In: *arXiv preprint* (2025). DOI: [10.48550/arXiv.2502.18449](https://doi.org/10.48550/arXiv.2502.18449).
- [Wey82] Elaine J. Weyuker. “On Testing Non-Testable Programs”. In: *The Computer Journal* (1982). DOI: [10.1093/comjnl/25.4.465](https://doi.org/10.1093/comjnl/25.4.465).
- [Wu+25] Jiayun Wu, Jiashuo Liu, Zhiyuan Zeng, Tianyang Zhan, Tianle Cai, and Wenhao Huang. “Mitigating LLM Hallucination via Behaviorally Calibrated Reinforcement Learning”. In: *arXiv preprint* (2025). DOI: [10.48550/arXiv.2512.19920](https://doi.org/10.48550/arXiv.2512.19920).
- [Yan+24] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. “SWE-agent: agent-computer interfaces enable automated software engineering”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2024. URL: http://papers.nips.cc/paper_files/paper/2024/hash/5a7c947568c1b1328ccc5230172e1e7c-Abstract-Conference.html.
- [Yao+23] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *Proceedings of the International Conference on Learning Representations (ICLR)*. 2023. URL: https://openreview.net/forum?id=WE_vluYUL-X.
- [Zel24] Andreas Zeller. “Introduction to Debugging”. In: *The Debugging Book*. CISA Helmholtz Center for Information Security, 2024. URL: https://www.debuggingbook.org/html/Intro_Debugging.html.

- [Zha+24] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. “AutoCodeRover: Autonomous Program Improvement”. In: *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. 2024. DOI: [10.1145/3650212.3680384](https://doi.org/10.1145/3650212.3680384).
- [Zha+26] Cen Zhang, Younggi Park, Fabian Fleischer, Yu-Fu Fu, Jiho Kim, Dongkwan Kim, Youngjoon Kim, Qingxiao Xu, Andrew Chin, Ze Sheng, Hanqing Zhao, Brian J. Lee, Joshua Wang, Michael Pelican, David J. Musliner, Jeff Huang, Jon Silliman, Mikel Mcdaniel, Jefferson Casavant, Isaac Goldthwaite, Nicholas Vidovich, Matthew Lehman, and Taesoo Kim. “SoK: DARPA’s AI Cyber Challenge (AIxCC): Competition Design, Architectures, and Lessons Learned”. In: *arXiv preprint* (2026). DOI: [10.48550/arXiv.2602.07666](https://doi.org/10.48550/arXiv.2602.07666).
- [Zho+19] Yaqin Zhou, Shangqing Liu, Jingkai Siow, Xiaoning Du, and Yang Liu. “Devign: effective vulnerability identification by learning comprehensive program semantics via graph neural networks”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2019. URL: <https://proceedings.neurips.cc/paper/2019/hash/49265d2447bc3bbfe9e76306ce40a31f-Abstract.html>.
- [Zho+23] Chunting Zhou, Pengfei Liu, Puxin Xu, Srini Iyer, Jiao Sun, Yuning Mao, Xuezhe Ma, Avia Efrat, Ping Yu, Lili Yu, Susan Zhang, Gargi Ghosh, Mike Lewis, Luke Zettlemoyer, and Omer Levy. “LIMA: less is more for alignment”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2023. URL: http://papers.nips.cc/paper_files/paper/2023/hash/ac662d74829e4407ce1d126477f4a03a-Abstract-Conference.html.
- [Zou+23] Andy Zou, Zifan Wang, J. Zico Kolter, and Matt Fredrikson. “Universal and Transferable Adversarial Attacks on Aligned Language Models”. In: *arXiv preprint* (2023). DOI: [10.48550/arXiv.2307.15043](https://doi.org/10.48550/arXiv.2307.15043).

STATEMENT ON THE USAGE OF AI TOOLS

In this appendix, I describe which AI tools were used during the creation process of this thesis and how they were used.

Tools. The following AI tools were used: Claude Code v2.1.152, OpenAI Codex v0.133.0, Claude Opus 4.6 via web interface, OpenAI GPT 5.5 via web interface, Gemini 3.5 Flash via web interface, Gemini 3.1 Pro via web interface, and Grammarly. These tools were accessed between April 20 and June 15, 2026, and were used in the following ways:

1. **Grammar and Spelling:** The thesis was checked for grammar and spelling errors using AI tools. Most sections were reviewed with prompts such as “Fix grammar and spelling of this text”.
2. **Integrating the Original Publications:** The LaTeX sources of the original publications included in Chapter 3, Chapter 4, Chapter 5, and Chapter 6 were integrated using Claude Code.
3. **Editing for Readability and Length:** Some sections in the introduction, background, and discussion chapters were revised with the help of AI tools. I first wrote complete paragraph drafts and then asked the models to improve readability or shorten the text. A typical prompt looked like this: “Improve the flow of this paragraph while keeping it short.”
4. **Figures:** AI tools were used to create many of the visualizations in the thesis, especially margin-note figures. I described the intended figure and asked Claude Code to implement it in TikZ. Some main-text figures, including Figure 2.4, Figure 2.3, and Figure 7.1, were created in the same way.
5. **Title Page Image:** The title page image was generated with OpenAI GPT 5.5 and Gemini 3.1 Pro based on my description.
6. **Literature Research:** AI tools were also used to support literature research. For example, they helped identify historical publications, such as the first papers defining software security.
7. **Formatting:** AI tools assisted with LaTeX formatting tasks, such as fixing visual artifacts and adjusting figure sizes.
8. **Various Small Edits:** AI tools assisted with various small edits, such as turning a list of paragraphs into an enumeration, making sure a term is consistently spelled the same way, moving sections throughout the thesis, or transitioning from `autoref` to `cf`.